



Σχολή Θετικών Επιστημών & Τεχνολογίας

Πληροφορική

Πτυχιακή / Διπλωματική Εργασία

Ανάπτυξη συστήματος στο χώρο της αθλητικής πυγμαχίας για την παρακολούθηση της έντασης και πλήθους χτυπημάτων κατά τη διάρκεια αγώνα.

Νικόλαος Δημητρακαράκος

Επιβλέπων καθηγητής: Ιωάννης Κουρέτας

Πάτρα, Ιούνιος 2025

Η παρούσα εργασία αποτελεί πνευματική ιδιοκτησία του φοιτητή Νικόλαου Δημητρακαράκου που την εκπόνησε. Στο πλαίσιο της πολιτικής ανοικτής πρόσβασης, ο συγγραφέας παραχωρεί στο ΕΑΠ μη αποκλειστική άδεια να αναπαράγει, προσαρμόζει, δανείζει, παρουσιάζει στο κοινό και διαθέτει διεθνώς την εργασία σε ψηφιακή μορφή, μέσω οποιουδήποτε μέσου, αποκλειστικά για εκπαιδευτικούς και ερευνητικούς σκοπούς, χωρίς οικονομικό αντάλλαγμα, για όλη τη διάρκεια των δικαιωμάτων πνευματικής ιδιοκτησίας.

Η ανοικτή πρόσβαση στο πλήρες κείμενο για ανάγνωση και μελέτη δεν συνεπάγεται την παραχώρηση πνευματικών δικαιωμάτων. Απαγορεύεται η αναπαραγωγή, αναδημοσίευση, αντιγραφή, αποθήκευση, εμπορική εκμετάλλευση, μετάδοση, διανομή, έκδοση, εκτέλεση, «κατέβασμα» (downloading), «ανάρτηση» (uploading), μετάφραση ή τροποποίηση της εργασίας, εν όλω ή εν μέρει, χωρίς τη ρητή έγγραφη άδεια του συγγραφέα. Ο συγγραφέας διατηρεί ακέραια όλα τα ηθικά και περιουσιακά του δικαιώματα.

Ανάπτυξη συστήματος στο χώρο της αθλητικής πυγμαχίας για την παρακολούθηση της έντασης και πλήθους χτυπημάτων κατά τη διάρκεια αγώνα.

Νικόλαος Δημητρακαράκος

Επιτροπή Επίβλεψης Πτυχιακής / Διπλωματικής Εργασίας

Επιβλέπων Καθηγητής: Δρ. Ιωάννης Κουρέτας Συνεργαζόμενο Εκπαιδευτικό Προσωπικό (ΣΕΠ – ΕΑΠ), ΕΔΙΠ Πανεπιστήμιο Πατρών	Συν-Επιβλέπων Καθηγητής: Δρ. Κωνσταντίνος Γιαννακόπουλος Συνεργαζόμενο Εκπαιδευτικό Προσωπικό (ΣΕΠ – ΕΑΠ)
	Συν-Επιβλέπων Καθηγητής: Δρ. Ευάγγελος Τοπάλης Συνεργαζόμενο Εκπαιδευτικό Προσωπικό (ΣΕΠ – ΕΑΠ)

Πάτρα, Ιούνιος 2025

Ευχαριστίες

Με την ολοκλήρωση της παρούσας πτυχιακής εργασίας, θα ήθελα να εκφράσω τις θερμές μου ευχαριστίες σε όσους συνέβαλαν, με τον δικό τους τρόπο, στην υλοποίησή της. Πρωτίστως, οφείλω ένα μεγάλο ευχαριστώ στον επιβλέποντα καθηγητή μου, κύριο Ιωάννη Κουρέτα, για την πολύτιμη καθοδήγηση, την εμπιστοσύνη που μου έδειξε αναθέτοντάς μου το συγκεκριμένο θέμα, καθώς και για την αμέριστη υποστήριξή του καθ' όλη τη διάρκεια της εκπόνησης της εργασίας. Η συμβολή του υπήρξε καθοριστική για την επιτυχή έκβασή της. Επιπλέον, θα ήθελα να ευχαριστήσω θερμά τον Νίκο για τις εύστοχες και πολύτιμες συμβουλές του σε θέματα που αφορούν τα ηλεκτρονικά κυκλώματα, οι οποίες αποδείχθηκαν καίριες για την τεχνική αρτιότητα του συστήματος που αναπτύχθηκε. Ιδιαίτερες ευχαριστίες εκφράζω στη Μαρία για την ευγενική και καίρια συνεισφορά της με τους χειροποίητους αισθητήρες, οι οποίοι αποτέλεσαν θεμελιώδες στοιχείο της πειραματικής μας διάταξης. Τέλος, ευχαριστώ ειλικρινά τον Θέμη για την άμεση ανταπόκρισή του και την ευγενική παραχώρηση του χώρου του γυμναστηρίου, η οποία ήταν απαραίτητη για την πραγματοποίηση των δοκιμών του συστήματος και την επαλήθευση των αποτελεσμάτων μας.

1 Περίληψη

Η πρόοδος της τεχνολογίας σήμερα έχει **οδηγήσει** στην ανάπτυξη έξυπνων υφασμάτων που ονομάζονται φορετά υφάσματα (e-textiles, wearable textiles). Αυτά βρίσκουν χρήση σε πολλά αθλήματα. Ιδιαίτερη σημασία έχουν οι χρήσεις τους σε μαχητικά σπορ, όπως το Taekwondo, η πυγμαχία και το kick boxing, τα **οποία** περιλαμβάνουν **semi-contact και full-contact** μορφές αγώνων (**καλύτερα στα ελληνικά -επαφής και ημι-επαφής**). Αυτή η τεχνολογία υπόσχεται να φέρει επανάσταση στη διεξαγωγή των αγώνων αλλά και στη προπόνηση, **προσφέροντας** πιο σωστά δεδομένα για τις επιδόσεις του αθλητών. Η συγκεκριμένη εργασία εστιάζει στην ανάπτυξη ενός νέου συστήματος που χρησιμοποιεί αισθητήρες FSR (Πιεζοαντιστατικοί αισθητήρες). Οι αισθητήρες αυτοί **τοποθετούνται** σε φανέλες ή σάκους πυγμαχίας με στόχο να καταγράφουν τα χτυπήματα που δέχεται ο αθλητής. Η τεχνολογία αυτή φαίνεται ότι μπορεί να **βελτιώσει τις επιδόσεις τόσο στους αγώνες πυγμαχίας όσο και στις προπονήσεις στα γυμναστήρια, καταγράφοντας τα χτυπήματα που επιχειρούνται εναντίον του σάκου προπόνησης ή του σώματος του αθλητή κατά την διάρκεια του sparring**. Η συλλογή δεδομένων γίνεται μέσω μικροελεγκτών ESP32, οι οποίοι επεξεργάζονται και στέλνουν αυτές τις πληροφορίες σε μια εφαρμογή κινητού τηλεφώνου. Το αποτέλεσμα **αποτελεί έναν αξιόπιστο καταγραφέα** της ανάλυσης της έντασης και της συχνότητας των χτυπημάτων, δίνοντας σημαντικές πληροφορίες για τους προπονητές καθώς επίσης **και για τους** κριτές των αγώνων. Επίσης, το σύστημα αυτό βοηθά στην πιο δίκαιη εκτίμηση των επιδόσεων των αθλητών μειώνοντας τα λάθη **των διαιτητών** και αυξάνοντας τη **διαφάνεια των αγώνων**. Ταυτόχρονα, **επιτρέπει στους** προπονητές να **παρακολουθούν** καλύτερα την τεχνική και τη στρατηγική του αθλητή, βελτιώνοντας την διαδικασία προπόνησης. Αυτή η **εργασία** στοχεύει όχι μόνο στην **επίτευξη της καλύτερης δυνατής εμπειρίας του αθλητή αλλά και στη βελτίωση του τρόπου με τον οποίο πραγματοποιούνται** οι αγώνες και οι προπονήσεις στο άθλημα της πυγμαχίας. Συνδυάζοντας την απλότητα και τη χρήση των έξυπνων υφασμάτων με τη δύναμη της ψηφιακής τεχνολογίας, η εργασία αυτή ανοίγει νέους δρόμους στη σχέση της τεχνολογίας με το άθλημα της πυγμαχίας και να εισάγει τον όρο (ηλεκτρονικό γυμναστήριο ή e-gym ή smart-gym).

Λέξεις – Κλειδιά

ESP32, FSR, εφαρμογή κινητού, e-textiles, wearable textiles.

2 Abstract

The advancement of technology has led to the development of smart textiles, known as e-textiles or wearable textiles, which have found applications in various sports. Of particular interest are their uses in combat sports such as Taekwondo, boxing, and kickboxing, which involve semi-contact and full-contact formats. This technology promises to revolutionize both competition and training by providing more accurate performance data for athletes. This study focuses on the development of a novel system utilizing Force Sensitive Resistor (FSR) sensors integrated into boxing vests or punching bags. These sensors are designed to record the strikes received by an athlete. The system aims to enhance boxing matches and gym training by accurately capturing data on strikes during sparring sessions or training. Data collection is facilitated by ESP32 microcontrollers, which process and transmit the information to a mobile application. The system enables detailed analysis of the intensity and frequency of strikes, offering valuable insights for coaches and competition judges. Furthermore, it aids in fairer evaluations of athletes' performance by reducing human errors and increasing transparency during matches. Simultaneously, it provides coaches with a better understanding of athletes' techniques and strategies, improving the overall training process. This work aims to not only enhance the athlete's experience but also transform the way boxing matches, and training sessions are conducted. By combining the simplicity and practicality of smart textiles with the power of digital technology, this research opens new pathways in the integration of technology with the sport of boxing.

Keywords

ESP32, FSR, mobile application, e-textiles.

3 Περιεχόμενα

1	Περίληψη	v
2	Abstract	vi
3	Περιεχόμενα	vii
4	Κατάλογος Πινάκων	xii
5	Συντομογραφίες & Ακρωνύμια	xiii
1.	Εισαγωγή	1
1.1	Σκοπός.....	1
1.2	Δομή Εργασίας.....	6
2.	Θεωρητικό Υπόβαθρο	8
2.1	Μικροελεγκτές	8
2.1.1	Ορισμός και είδη	8
2.1.2	Arduino vs ESP32	15
2.2	Τεχνολογία IoT και Bluetooth.....	24
2.2.1	Ορισμός.....	24
2.2.2	Συνδεσιμότητα του IoT	31
2.2.3	Εφαρμογές του IoT στην καθημερινότητα	42
3.	Υλοποίηση συστήματος	45
3.1	Πλακέτες ανάπτυξης ESP32 και Arduino uno R4 Wi-Fi.....	45
3.2	Αισθητήρες	59
3.2.1	Αισθητήρας αντίστασης δύναμης FSR	59
3.2.2	Αισθητήρας Custom FSR – Υφάσματα - e-Textiles	63
3.2.3	Συνδεσιμότητα ESP32 με FSR	71
3.3	Εφαρμογή στο σάκο	73
3.3.1	Ενσωμάτωση του κυκλώματος στο σάκο	73
4.	Υλοποίηση Λογισμικού Συστήματος	75
4.1	Προγραμματισμός ESP32 – Arduino	75
4.2	Βιβλιοθήκες ESP32 – Arduino	80

4.3 Ανάπτυξη εφαρμογής Android.....	86
4.4 Βιβλιοθήκες εφαρμογής Android	103
5. Το σύστημα στην πράξη	109
5.1 Προετοιμασία.....	109
5.2 Εκτέλεση.....	115
5.3 Αποτελέσματα.....	116
6. Επίλογος.....	120
7. Βιβλιογραφία	123
8. Παράρτημα Α: «Πηγαίος κώδικας Arduino IoT Cloud.»	128
9. Παράρτημα Β: «Πηγαίος κώδικας ESP32.»	148
10. Παράρτημα Γ: «Πηγαίος κώδικας Android.»	162

Κατάλογος Εικόνων / Σχημάτων

Εικόνα 1. Ο μικροελεγκτής Beetle ESP32-C6.	55
Εικόνα 2. Υλοποίηση με κολλήσεις.....	55
Εικόνα 3. Ο Πηγαίος κώδικας μέσα από το thing.	58
Εικόνα 4. Το Dashboard Boxer Server με πληροφορίες από χτυπήματα.	58
Εικόνα 5. Η εφαρμογή IoT Remote και ο πίνακας (Dashboard Box Server).	59
Εικόνα 6. Ύφασμα αθλητικού είδους.	63
Εικόνα 7. Ρέλι μαύρο σατέν 100% PES.	63
Εικόνα 8. Ύφασμα χαλκού αγώγιμο.....	64
Εικόνα 9. Velostat – Lingstat.....	64
Εικόνα 10. Velcro – Χριτς Χρατς.....	65
Εικόνα 11. Spandex.	65
Εικόνα 12. Ρέλι μαύρο σατέν 100% PES	66
Εικόνα 13. Ύφασμα χαλκού αγώγιμο.....	66
Εικόνα 14. Velostat – Lingstat.....	67
Εικόνα 15. Velcro – Χριτς Χρατς.....	67
Εικόνα 16. Η κατασκευή τους.	68
Εικόνα 17. Η κατασκευή τους.	68

Εικόνα 18. Η κατασκευή τους.	68
Εικόνα 19. Ο μικρότερος αισθητήρας.	69
Εικόνα 20. Ο μεγαλύτερος αισθητήρας.	69
Εικόνα 21. Ο σάκος και ο αισθητήρας στο γυμναστήριο.	74
Εικόνα 22. Ο σάκος και ο αισθητήρας στο γυμναστήριο.	74
Εικόνα 23. Προσθήκη μιας διεύθυνσης URL στον Board Manager του Arduino IDE.....	75
Εικόνα 24. Επιλογή κατάλληλης πλακέτα ESP32.	76
Εικόνα 25. Μεταγλώττιση και ανέβασμα του κώδικα στο Μικροελεγκτή.	77
Εικόνα 26. Το IDE Visual Studio Code.....	88
Εικόνα 27. Η οθόνη που ζητά στο χρήστη να ενεργοποιήσει τα Location Service's.....	89
Εικόνα 28. Η οθόνη που ζητά στο χρήστη να ενεργοποιήσει η όχι την βελτιστοποίηση διαχείρισης ενέργειας της μπαταρίας για την εφαρμογή.	89
Εικόνα 29. Διάλογος για να την ενεργοποίηση του Bluetooth.	90
Εικόνα 30. Βλέπουμε τους μικροελεγκτές αποσυνδεδεμένους στο κινητό.	91
Εικόνα 31. Βλέπουμε τους μικροελεγκτές συνδεδεμένους στο κινητό.....	91
Εικόνα 32. Settings των αισθητήρων FSR.....	92
Εικόνα 33. Κουμπί Insert Sample Data.	92
Εικόνα 34. Battery Optimization Settings.	93
Εικόνα 35. Add Game.....	93
Εικόνα 36. Add Game.....	93
Εικόνα 37. Οι γύροι είναι εκτός επιτρεπόμενων ορίων.	94
Εικόνα 38. Το Game που δημιούργησε προστίθεται στη βάση δεδομένων.	94
Εικόνα 39. Κόκκινες επισημάνσεις για τις λανθασμένες επιλογές.....	94
Εικόνα 40. Η οθόνη Games.	95
Εικόνα 41. Η οθόνη Game Details.	95
Εικόνα 42. Η οθόνη Edit/Update Game.	96
Εικόνα 43. Εκτός ορίων τιμές στην οθόνη Edit/Update Game.	96
Εικόνα 44. Game Details.	97
Εικόνα 45. Η οθόνη του Match.....	97
Εικόνα 46. Η οθόνη του Match, κατά τη διάρκεια ενός παιχνιδιού.	98
Εικόνα 47. Η οθόνη Games	98
Εικόνα 48. Η οθόνη Games, Slidable List με επιλογές Events, Edit, Delete.	99

Εικόνα 49. Οθόνη Game Events.....	99
Εικόνα 50. Round 1.	100
Εικόνα 51. Round 2.	100
Εικόνα 52. Round 5.	100
Εικόνα 53. Themes.	101
Εικόνα 54. Light Mode.	101
Εικόνα 55. Dark Mode.....	101
Εικόνα 56. Ο διάλογος της επιλογής Exit.....	102
Εικόνα 57. Μεταγλώττιση του κώδικα μέσα από το περιβάλλον του Arduino IDE.	110
Εικόνα 58. Φόρτωση του παραγόμενου object μετά το compilation στον μικροελεγκτή.....	110
Εικόνα 59. Μεταγλώττιση του Thing μέσα από το περιβάλλον Arduino IoT Cloud.....	111
Εικόνα 60. Φόρτωση του παραγόμενου object του Thing μετά το compilation στον μικροελεγκτή.	111
Εικόνα 61. Το Thing που δημιούργησα στο Arduino IoT Cloud.	112
Εικόνα 62. Οι μεταβλητές που συνδέουν τον κώδικα του thing με το dashboard για να εμφανίσει τις τιμές.	113
Εικόνα 63. Εγκατάσταση της εφαρμογής σε κινητό από το Google Play Store.	114
Εικόνα 64. Το Dashboard που δημιούργησα στο Arduino IoT Cloud.	114
Εικόνα 65. Το πληροφοριακό σύστημα στην πράξη.	116
Εικόνα 66. Κατά την διάρκεια των δοκιμών.	118
Εικόνα 67. Κατά την διάρκεια των δοκιμών.	118
Εικόνα 68. Κατά την διάρκεια των δοκιμών.	118
Εικόνα 69. Κατά την διάρκεια των δοκιμών.	118
Εικόνα 70. Το κύκλωμα με τους αισθητήρες.....	119
Εικόνα 71. Μπροστινή όψη.	119
Εικόνα 72. Πίσω όψη.....	119
 Σχήμα 1. Βασική διάταξη 32bit μικροελεγκτή (Παπάζογλου & Λιωνής, 2020).....	13
Σχήμα 2. Πόλη όπου το Internet of Things (IoT) διασυνδέει συσκευές και υπηρεσίες.	25
Σχήμα 3. Χρησιμοποιώντας μια πύλη IoT για να συνδέσετε τα «πράγματα» στο cloud.	26
Σχήμα 4. Ασύρματο Δίκτυο Αισθητήρων.....	28
Σχήμα 5. Ταυτοποίηση μέσω Ραδιοσυχνοτήτων (RFID - Radio-Frequency Identification)...	29

Σχήμα 6. Οι τοπολογίες δικτύου.....	33
Σχήμα 7. Πρωτόκολλα Επικοινωνίας μοντέλο OSI.....	35
Σχήμα 8. Πρωτόκολλα Επικοινωνίας οργανωμένα σε ιεραρχικά επίπεδα.	36
Σχήμα 9. Υλοποίηση συστήματος.	46
Σχήμα 10. ESP32-C6 Functional Block Diagram (Διάγραμμα Λειτουργιών ESP32-C6)	47
Σχήμα 11. Διάταξη ακροδεκτών ESP32-C627 (QFN32, επάνω όψη).....	48
Σχήμα 12. Ο μικροελεγκτής Beetle ESP32-C6.	49
Σχήμα 13. Τα πρωτόκολλα επικοινωνίας που υποστηρίζει ο μικροελεγκτής Beetle ESP32-C6.	50
Σχήμα 14. Παρουσίαση της πλακέτας.	53
Σχήμα 15. Διάγραμμα ακροδεκτών.	54
Σχήμα 16. Σχεδιάγραμμα Tinker Cad.....	55
Σχήμα 17. Οι συνδέσεις των pins μπαταρίας, μικροελεγκτή και κυκλώματος αισθητήρα και αντίστασης.	56
Σχήμα 18. Arduino UNO R4 Wi-Fi Pinout. https://docs.arduino.cc/hardware/uno-r4-wifi/ ...57	57
Σχήμα 19. Interlink Electronics FSRTM 406	62
Σχήμα 20. Το σχεδιάγραμμα μας δείχνει πώς συνδέεται ο αισθητήρας FSR με το μικροελεγκτή ESP32.....	73
Σχήμα 21. Το διάγραμμα της βάσης δεδομένων SQLite που είναι ενσωματωμένη στην εφαρμογή.	105
Σχήμα 22. Σχεδιάγραμμα Tinker Cad.....	119

4 Κατάλογος Πινάκων

Πίνακας 1. Εικόνες μικροελεγκτών οικογένειας Arduino Uno	16
Πίνακας 2. Οικογένεια Arduino Uno.....	17
Πίνακας 3. Εικόνες μικροελεγκτών οικογένειας Arduino Nano	17
Πίνακας 4. Οικογένεια Arduino Nano	18
Πίνακας 5. Εικόνες μικροελεγκτών οικογένειας Arduino MKR.....	18
Πίνακας 6. Οικογένεια Arduino MKR	19
Πίνακας 7. Εικόνες μικροελεγκτών οικογένειας Arduino Mega.....	19
Πίνακας 8. Οικογένεια Arduino Mega	19
Πίνακας 9. Εικόνες μικροελεγκτών οικογένειας Arduino Portenta.....	20
Πίνακας 10. Οικογένεια Arduino Portenta	20
Πίνακας 11. Εικόνες μικροελεγκτών οικογενειών ESP32.....	23
Πίνακας 12. Οικογένειες του ESP32	23

5 Συντομογραφίες & Ακρωνύμια

6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks (Ασύρματα προσωπικά δίκτυα χαμηλής ισχύος με υποστήριξη IPv6)
ADC	Analog to Digital Converter (Μετατροπέας Αναλογικού σε Ψηφιακό Σήμα)
AI	Artificial Intelligence (Τεχνητή Νοημοσύνη)
API	Application Programming Interface
AOT	Ahead-Of-Time compilation
ARP	Address Resolution Protocol
BLE	Bluetooth Low Energy
CPU	Central Processing Unit (Μονάδα Κεντρικής Επεξεργασίας)
CRUD	Create, Read, Update, Delete
DAC	Digital to Analog Converter (Μετατροπέας Ψηφιακού σε Αναλογικό Σήμα)
DCDC	DC-to-DC converter
DHCP	Dynamic Host Configuration Protocol
DMA	Direct Memory Access
DNS	Domain Name System
EEPROM	Electrically Erasable Read Only Memory (Ηλεκτρικά Διαγράψιμη Μνήμη Μόνο για Ανάγνωση)
EPROM	Erasable Read Only Memory (Διαγράψιμη Μνήμη Μόνο για Ανάγνωση)
FSR	Force Sensing Resistors (Πιεζοαντιστατικοί αισθητήρες / Αισθητήρες δύναμης μεταβαλλόμενης αντίστασης)
FTP	File Transfer Protocol
GATT	Generic Attribute Profile
HTTP	Hypertext Transfer Protocol
I/O	Input/Output (Εισόδου/Εξόδου)

I2C	Inter-Integrated Circuit
I2S	Inter-IC Sound
ICMP	Internet Control Message Protocol
IDE	Integrated Development Environment (Ολοκληρωμένο Περιβάλλον Ανάπτυξης)
IEEE	Institute of Electrical and Electronics Engineers (Ινστιτούτο Ηλεκτρολόγων και Ηλεκτρονικών Μηχανικών)
IETF	Internet Engineering Task Force
IGMP	Internet Group Management Protocol
IMU	Inertial Measurement Unit (Αδρανειακές Μονάδες Μέτρησης)
IoT	Internet of Things (Διαδίκτυο των Πραγμάτων)
IP	Internet Protocol
IPv4	Internet Protocol version 4
IPv6	Internet Protocol version 6
ISM	Industrial, Scientific, Medical (Βιομηχανικές, Επιστημονικές και Ιατρικές εφαρμογές)
ITU-R	International Telecommunication Union Radiocommunication Sector (Διεθνής Τηλεπικοινωνιακή Ένωση Ραδιοεπικοινωνιών)
JIT	Just-In-Time compilation
JTAG	Joint Test Action Group
JSON	JavaScript Object Notation
LAN	Local Area Networks (Τοπικά Δίκτυα)
LDO	Low-Dropout Regulator
LPWAN	Low-Power Wide-Area Networks (Δίκτυα Ευρείας Περιοχής Χαμηλής Ισχύος)
MAC	Media Access Control

MTU	Maximum Transmission Unit
NAT	Network Address Translation
NB-IoT	Narrowband IoT
NFC	Near Field Communication (Επικοινωνία Κοντινού Πεδίου)
NTP	Network Time Protocol
OSI	Open Systems Interconnection
OTP	One-Time Programmable
PAN	Personal Area Networks (Προσωπικά Δίκτυα Περιοχής / Ασύρματα προσωπικά δίκτυα)
PCB	Printed Circuit Board (Τυπωμένο Κύκλωμα)
PES	Polyester (Πολυεστέρας)
QFN	Quad Flat No-leads
RAM	Random Access Memory (Μνήμη Τυχαίας Προσπέλασης)
RC Oscillator	Resistor-Capacitor Oscillator
RFC	Request for Comments
RFID	Radio-Frequency Identification (Ταυτοποίηση μέσω Ραδιοσυχνότητων)
ROM	Read Only Memory (Μνήμη Μόνο για Ανάγνωση)
RTC	Real Time Clock (Πολόι Πραγματικού Χρόνου)
SDK	Software Development Kit
SMTP	Simple Mail Transfer Protocol
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
SSH	Secure Shell
TCP	Transmission Control Protocol
TWT	Target Wake Time

UART	Universal Asynchronous Receiver-Transmitter
UDP	User Datagram Protocol
UI	User Interface (Διεπαφή Χρήστη)
USB	Universal Serial Bus
WAN	Wide Area Networks (Δίκτυα Ευρείας Κάλυψης)
Wi-Fi	Wireless Fidelity
WLAN	Wireless Local Area Network (Ασύρματα Τοπικά Δίκτυα)
WSNs	Wireless Sensor Networks (Ασύρματα Δίκτυα Αισθητήρων)

1. Εισαγωγή

1.1 Σκοπός

Η τεχνολογία έχει αλλάξει πολύ τα τελευταία χρόνια και επηρεάζει κάθε τομέα της ανθρώπινης δραστηριότητας. Ιδιαίτερα στον χώρο του αθλητισμού, η ένταξη τεχνολογικών λύσεων έχει φέρει μια νέα εποχή στην μέτρηση της απόδοσης και στη βελτίωση της εκπαίδευσης των αθλητών. Όμως, στο πεδίο της πυγμαχίας, η καταγραφή και αξιολόγησή των επιδόσεων συνεχίζει να βασίζεται πολύ σε υποκειμενικές μετρήσεις κι εκτιμήσεις είτε από τους προπονητές ή από κριτές. Αυτό φέρνει ασυμφωνίες και την έλλειψη αντικειμενικών δεδομένων που μπορούν να βοηθήσουν τον πρόοδο του αθλητή.

Η πυγμαχία είναι ένα σπορ δυνατής έντασης όπου η ταχύτητα και η δύναμη των κινήσεων κάνουν δύσκολη την παρατήρησή και την καταγραφή των χτυπημάτων. Τα κεντρικά προβλήματα που υπάρχουν είναι:

Υποκειμενικότητα και ανθρώπινα λάθη. Οι κριτές στηρίζονται στην δική τους παρατήρηση για να σημειώσουν τα χτυπήματα κάτι που μπορεί να φέρει σε λάθος ειδικά σε γρήγορες ταχύτητες.

Αμφισβήτηση αποτελεσμάτων: Όταν λείπουν τα αντικειμενικά στοιχεία φέρνει πολλές αμφισβητήσεις από τους αθλητές και τους προπονητές.

- Δυσκολία για αναγνώριση χτυπημάτων. Δεν υπάρχει τρόπος να καταγραφεί η δύναμη, το σημείο ή η συχνότητα των χτυπημάτων, αφήνοντας μεγάλο κενό στην ανάλυση της απόδοσης των αθλητών.

- Έλλειψη Πληροφόρησης για Βελτίωση: Οι αθλητές και οι προπονητές δεν έχουν πρόσβαση σε σωστά στοιχεία για να βελτιώσουν την τεχνική τους.

Η εμπειρία από αθλήματα όπως το Τάε Κβον Ντο δίνει ένα χρήσιμο πλαίσιο για την λύση προβλημάτων της πυγμαχίας:

- Ηλεκτρονικά Συστήματα στο Τάε Κβον Ντο: Η σύνθεση ηλεκτρονικών προστατευτικών στο σώμα και στα κράνη έχει βελτιώσει την ακρίβεια και την αντικειμενικότητα στους αγώνες, εξαλείφοντας τα περισσότερα σφάλματα που κάνουν οι άνθρωποι (Chi, 2005).

- Ξιφασκία: Οι ηλεκτρονικές συσκευές που είναι σε χρήση από το 1960 δίνουν ένα σύστημα για την αντικειμενική καταγραφή των χτυπημάτων.

Αυτές οι εφαρμογές δείχνουν ότι μια τεχνολογία μπορεί να βοηθήσει πολύ στην λύση δύσκολων προβλημάτων στα αθλήματα που χρειάζονται ακρίβεια (Tama Birkocak, Gomes and Carvalho, 2023).

Ο σκοπός αυτής της εργασίας είναι με την βοήθεια από τα αλλά αθλήματα που έχουν υλοποιήσει αντίστοιχα συστήματα η ανάπτυξη και χρήση αισθητήρων σε σάκους και γιλέκα πυγμαχίας για την καταγραφή χτυπημάτων των αθλητών. Η έρευνα αυτή θέλει να καλύψει το κενό που υπάρχει στην αξιολόγηση των επιδόσεων των αθλητών στην πυγμαχία, επειδή η σημερινή πρακτική βασίζεται κυρίως σε υποκειμενικές απόψεις. Η χρήση αισθητήρων στοχεύει να δώσει μετρήσιμα και αξιόπιστα δεδομένα.

Η μέθοδος αυτή προσπαθεί να κάνει πιο δίκαιη και διαφανή την εκπαίδευση και τους αγώνες, δίνοντας χρήσιμα εργαλεία στους αθλητές και στους προπονητές. Με τη χρήση αισθητήρων που μπορούν να καταγράψουν δεδομένα σε πραγματικό χρόνο οι αθλητές θα έχουν την ευκαιρία να βρουν την απόδοσή τους άμεσα μετά την προπόνηση ή τον αγώνα καθιστώντας δυνατή την γρήγορη αλλαγή της στρατηγικής. Επιπλέον οι αισθητήρες θα δίνουν χρήσιμη ανατροφοδότηση που θα βοηθήσει στην κατανόηση των αδυναμιών και των δυνατοτήτων κάθε αθλητή. Αυτό θα επιτρέψει την προσωπική προπόνηση τους ανάλογα με τις ειδικές ανάγκες τους. Έτσι η εργασία προσπαθεί όχι μόνο να βελτιώσει την αθλητική απόδοση αλλά και να προάγει την ασφάλεια των αθλητών μειώνοντας τον κίνδυνο τραυματισμού τους μέσω της παρακολούθησης της καταπόνησης σε κάθε προπόνηση. Το ενδιαφέρον για αυτή την έρευνα προέρχεται από την ανάγκη να βάλουμε τεχνολογικές λύσεις που θα επιτρέπουν την καλύτερη εκτίμηση των ικανοτήτων των αθλητών. Η προπόνηση (Song, Jeon, Park, An, Hwang, Lee, and Lee, 2010) συνήθως στηρίζεται σε υποκειμενικές αξιολογήσεις από τους προπονητές, οι οποίοι στηρίζονται στην εμπειρία και την παρατήρηση για να κρίνουν την απόδοσή των παικτών. Όμως, αυτές οι υποκειμενικές μέθοδοι δείχνουν αδυναμίες γιατί δεν επιτρέπουν πάντα τη σωστή και αντικειμενική καταγραφή της προόδου. Με τη χρήση των FSR αισθητήρων, η μέτρηση των αθλητικών επιδόσεων γίνεται πιο δίκαιη και σωστή. Στην πυγμαχία, αυτό είναι πολύ σημαντικό γιατί οι προσωπικές κρίσεις μπορεί να φέρνουν λάθος παρατηρήσεις για την πρόοδο του αθλητή ή και σε άδικες καταστάσεις στον αγώνα. Οι αισθητήρες αυτοί προσφέρουν μια λύση στο θέμα αυτό παρέχοντας στοιχεία που

μπορούμε να γράψουμε, να κοιτάξουμε και να χρησιμοποιήσουμε για καλύτερη απόδοση με ξεκάθαρο τρόπο. Επίσης, η χρήση τεχνολογίας για την καταγραφή της απόδοσης δεν περιορίζεται μόνο στη πυγμαχία αλλά ήδη εφαρμόζεται και σε άλλα αθλήματα. Οι προπονητές μπορούν να χρησιμοποιούν τα δεδομένα για να αναπτύξουν προγράμματα που ανταποκρίνονται στις ανάγκες των αθλητών βελτιώνοντας τη φυσική κατάσταση και την τεχνική τους. Αυτή η τεχνολογία είναι χρήσιμη όταν πρόκειται για επαγγελματίες και ερασιτέχνες αθλητές που στοχεύουν στην βελτιστοποίηση των επιδόσεων τους και την πρόληψη τραυματισμών.

Σημαντικό είναι ότι η τεχνολογία σε συνδυασμό με τους αισθητήρες μπορεί επίσης να βοηθήσει στην ανάπτυξη νέων τρόπων προπόνησης. Με την ικανότητα συνεχούς μέτρησης των επιδόσεων, οι προπονητές έχουν την ευκαιρία να αλλάξουν τις στρατηγικές τους σύμφωνα με τις άμεσες ανάγκες του αθλητή, βελτιώνοντας το αποτέλεσμα της προπόνησης και μειώνοντας το λάθος. Παρόμοιες τεχνολογίες έχουν χρησιμοποιηθεί σε άλλα σπορ, και τα αποτελέσματα που βγήκαν δείχνουν τη σημασία αυτών των τεχνολογιών για την αθλητική απόδοση. Στην ξιφασκία, για παράδειγμα οι αισθητήρες που μπήκαν στα σπαθιά επιτρέπουν την καταγραφή των επαφών και τη μελέτη των κινήσεων των αθλητών. Με αυτόν τον τρόπο οι προπονητές μπορούν να δούνε λάθη στην τεχνική των αθλητών και να αλλάξουν ανάλογα την προπόνηση τους. Στο Τάε Κβον Ντο, η χρήση γιλέκων με ενσωματωμένους αισθητήρες επιτρέπει την καταγραφή της δύναμης των χτυπημάτων και τον έλεγχο των κινήσεων των αθλητών. Αυτή η τεχνολογία είναι πολύ χρήσιμη κατά τη διάρκεια αγώνων καθώς επιτρέπει την σωστή καταγραφή των χτυπημάτων και την αντικειμενική αξιολόγηση της απόδοσης των αθλητών. Οι αισθητήρες αυτοί βοηθούν στην αύξηση της αντικειμενικότητας, στην αξιολόγηση, ενώ δίνουν σημαντικά δεδομένα που μπορεί να χρησιμοποιηθούν για ανάλυση της απόδοσης και τη βελτίωση της προπόνησης. Αυτές οι εφαρμογές δείχνουν ότι η χρήση της τεχνολογίας στον αθλητισμό δεν είναι κάτι καινούργιο, αλλά μια αναπόφευκτη ανάπτυξη που βοηθά τους αθλητές να γίνουν καλύτεροι. Η χρήση παρόμοιων εργαλείων στην πυγμαχία μπορεί να φέρει τα ίδια αποτελέσματα επιτρέποντας την σωστή μέτρηση της απόδοσης και την προσαρμογή προπόνησης στις ανάγκες κάθε αθλητή. Επίσης, τα δεδομένα που βγαίνουν από τέτοιες τεχνολογίες μπορούν να χρησιμοποιηθούν για τη δημιουργία αρχείων που δείχνουν την ανάπτυξη κάθε αθλητή με το πέρασμα του χρόνου κάτι που μπορεί να είναι κίνητρο για συνεχή βελτίωση.

Η καταγραφή στοιχείων σε άμεσο χρόνο είναι ένα από τα μεγαλύτερα οφέλη των πιεζοαντιστατικών αισθητήρων. Αυτό έχει αποδειχτεί πολύ χρήσιμο σε εφαρμογές άλλων αθλημάτων, όπως το Τάε Κβον Ντο, όπου έχει αναπτυχθεί σύστημα αξιολόγησής σε πραγματικό χρόνο για κορυφαίους αθλητές. Με την γρήγορη καταγραφή της απόδοσης του αθλητή κατά τη διάρκεια προπόνησης οι προπονητές μπορούν να αλλάξουν στρατηγική τους γρήγορα και αποτελεσματικά. Για παράδειγμα, αν ένας προπονητής δει μέσω των δεδομένων ότι η δύναμη των χτυπημάτων ενός αθλητή μειώνεται κατά την προπόνηση μπορεί να αποφασίσει να αλλάξει την ένταση ή την διάρκεια της προπόνησης ώστε να αποφευχθεί κόπωση και κίνδυνος τραυματισμού. Επιπλέον, η ικανότητα της παρακολούθησης της απόδοσης σε πραγματικό χρόνο μπορεί να χρησιμοποιηθεί για να βρεθούν μοτίβα και συνήθειες που επηρεάζουν την αποδοτικότητα του αθλητή. Για παράδειγμα, αν τα δεδομένα δείχνουν ότι η δύναμη των χτυπημάτων ενός αθλητή είναι πιο μεγάλη κατά τις πρώτες στιγμές της προπόνησής και πέφτει σιγά-σιγά, αυτό μπορεί να είναι σημάδι ότι ο αθλητής πρέπει να εστιάσει στη βελτίωση της αντοχής του. Με αυτό τον τρόπο, οι αισθητήρες αυτοί επιτρέπουν την άμεση και συνεχή προσαρμογή της προπόνησης παρέχοντας στους αθλητές τη δυνατότητα να γίνονται καλύτεροι συνεχώς. Επιπρόσθετα τα δεδομένα σε πραγματικό χρόνο μπορούν να χρησιμοποιηθούν για την προσομοίωση συνθηκών αγώνα. Οι αθλητές μπορούν να προπονηθούν με βάσει συγκεκριμένα σενάρια και να προσαρμόσουν τις αντιδράσεις τους στη βάση της ανάλυσης των δεδομένων σε πραγματικό χρόνο. Αυτό επιτρέπει τη βελτίωση των αντιδράσεων και μια τακτική κατά τη διάρκεια του αγώνα κάνοντας τους αθλητές πιο ανταγωνιστικούς και ικανούς να αντιμετωπίσουν οποιαδήποτε πρόκληση εμφανίζεται. Η καταγραφή και η ανάλυση των στοιχείων που μαζεύονται στη διάρκεια της προπόνησης δίνει χρήσιμες πληροφορίες για την δημιουργία ενός πλήρους προφίλ των αθλητών. Αυτά τα στοιχεία μπορούν να χρησιμοποιηθούν για να καθορίσουν τα δυνατά και τα αδύνατα σημεία των αθλητών, επιτρέποντας την προσωπική προσαρμογή του πλάνου προπόνησης σύμφωνα με τις ανάγκες τους. Η στατιστική μελέτη των στοιχείων αυτών μπορεί να δείξει πρότυπα που δεν φαίνονται στην αρχή. Για παράδειγμα, η σύγκρισή των στοιχείων από διάφορες προπονήσεις μπορεί να δείξει αν ο αθλητής βελτιώνεται με το πέρασμα του χρόνου ή αν υπάρχουν ειδικές περιοχές στις οποίες χρειάζεται περισσότερη φροντίδα. Αυτά τα δεδομένα μπορούν να χρησιμοποιηθούν για τον ορισμό των στόχων της προπόνησης και για την μέτρηση της προόδου του αθλητή ώστε να γίνουν εφικτοί.

Με την σωστή εξέταση των δεδομένων, οι προπονητές μπορούν να πάρουν πληροφορίες που θα τους βοηθήσουν να αλλάξουν την προπόνηση, έτσι για να βελτιώσουν τα σημεία που χρειάζονται φροντίδα οι αθλητές και να αναδείξουν εκείνα που υπερτερούν έναντι των αντιπάλων τους. Αυτή η μέθοδος είναι ιδανική για την ετοιμασία των αθλητών για αγώνες καθώς τους επιτρέπει να κοιτάζουν στις συγκεκριμένες περιοχές που θα τους δώσει πλεονέκτημα απέναντι στους αντιπάλους τους.

Επιπλέον, η στατιστική μελέτη μπορεί να βοηθήσει τους αθλητές να δουν πως η φυσική τους κατάσταση έχει επίδραση στην απόδοσή τους σε διαφορετικές φάσεις της προπόνησης ή του αγώνα. Με αυτό τον τρόπο οι προπονητές μπορούν να αλλάξουν το πρόγραμμα έτσι ώστε να κρατούν τους αθλητές σε καλή κατάσταση κατά τη διάρκεια των σημαντικών αγώνων. Η γνώση αυτών των μελετών είναι πολύ σημαντική για αθλητές που βρίσκονται σε επίπεδο πρωταθλητισμού όπου ακόμη και μικρές βελτιώσεις μπορεί να κάνουν την διαφορά ανάμεσα στη νίκη και την ήττα. Η ψυχολογία του αθλητή έχει μεγάλη σημασία στην απόδοσή του και η καταγραφή των δεδομένων μπορεί να βοηθήσει την αυτοπεποίθησή του. Οι αθλητές που έχουν καθαρή εικόνα των επιδόσεών τους μπορούν να δουν την πρόοδό τους με συγκεκριμένα στοιχεία αντί για υποκειμενικές εκτιμήσεις και εντυπώσεις. Αυτή είναι πολύ σημαντική στην πυγμαχία, όπου η αίσθηση της ισχύος και της ταχύτητας των χτυπημάτων μπορεί να επηρεαστεί από ψυχολογικούς παράγοντες. Οι αισθητήρες (Ki, Jeong and Lee, 2019) δίνουν δυνατότητα στον αθλητή να βλέπει με ακρίβεια τα αποτελέσματα της προπόνησης του γεγονός που μπορεί να ενισχύσει την αυτοπεποίθησή του και να τον βοηθήσει για να παραμείνει συγκεντρωμένος στους στόχους του. Σημειώνεται ότι η καταγραφή των στοιχείων μπορεί να βοηθήσει αθλητές για να δουν πότε είναι σε καλή φυσική κατάσταση και πότε πρέπει να ξεκουραστούν. Αυτό λιγοστεύει το άγχος που έρχεται με την υπερβολική προπόνηση και βοηθά τους αθλητές να διαχειριστούν καλύτερα το φορτίο τους. Έτσι, οι αθλητές είναι ικανοί για να βελτιώσουν την απόδοσή τους όχι μόνο σωματικά αλλά και ψυχολογικά. Επίσης, η χρήση αντικειμενικών δεδομένων μπορεί να βοηθήσει τους αθλητές για να κάνουν ρεαλιστικούς στόχους. Με τη δυνατότητα παρακολούθησης της προόδου τους, οι αθλητές μπορούν καθορίσουν συγκεκριμένα και εφικτά βήματα για τη βελτίωσή τους. Αυτό μειώνει το άγχος που είναι συνδεδεμένο με την αβεβαιότητα και αυτούς βοηθά να επικεντρωθούν εκεί που χρειάζεται, κάνοντας τη διαδικασία πιο αποτελεσματική και ψυχολογικά υποστηρικτική. Στο τέλος, η χρήση αισθητήρων στην πυγμαχία μπορεί να βοηθήσει την πιο σωστή καταγραφή και αξιολόγηση της προπόνησης. Αυτές οι τεχνολογίες κάνουν την μέτρηση πιο διαφανή και

αντικειμενική, συμβάλλοντας στην ανάπτυξη πιο επιτυχημένων στρατηγικών προπόνησης και στη βελτίωση των αθλητών. Έτσι, δημιουργείται μία ενιαία προσέγγιση της τεχνολογίας στον αθλητισμό όπου η τεχνολογία λειτουργεί σαν συνεργάτης κι όχι απλά σαν ένα εργαλείο, τονίζει την εικόνα και την αποτελεσματικότητα του αθλήματος και του προπονητή.

Το τελευταίο όραμα αυτής της πτυχιακής είναι η δημιουργία ενός ολοκληρωμένου συστήματος που θα ενσωματώνει την σύγχρονη τεχνολογία με τις καθημερινές ανάγκες των αθλητών, επιτρέποντας την ανάπτυξη ενός πλήρους και πάντα βελτιωμένου προγράμματος προπόνησης. Επίσης φιλοδοξεί να συμβάλει στην κατανόηση των δυνατοτήτων των πιεζοαντιστατικών αισθητήρων στον τομέα προπόνησης της πυγμαχίας, προσφέροντας μια μέθοδο που συνδυάζει την τεχνολογία με την πρακτική εμπειρία. Με κατάλληλη χρήση αυτών των τεχνολογιών ανοίγουν νέοι δρόμοι για ανάπτυξη του αθλητισμού και τη βελτίωση της απόδοσης αθλητών σε όλα επίπεδα, δημιουργώντας ένα μέλλον όπου η τεχνολογία και ο αθλητισμός ζούνε μαζί αρμονικά προς όφελος ανθρώπινης προόδου.

1.2 Δομή Εργασίας

Η δομή της εργασίας είναι η εξής:

Δεύτερο κεφάλαιο: Στο κεφάλαιο αυτό θα αναφέρουμε τα είδη των μικροελεγκτών και της τεχνολογίας IoT και Bluetooth που θα χρησιμοποιήσουμε για την σύνδεση τους, έτσι ώστε να γίνει κατανοητή η αρχιτεκτονική του μικροελεγκτή και ο τρόπος διασύνδεσης του με το πληροφοριακό σύστημα.

Τρίτο κεφάλαιο: Εδώ θα αναφερθούμε πιο συγκεκριμένα στα υλικά που θα χρησιμοποιήσουμε, όπως την πλακέτα ανάπτυξης ESP32, τους αισθητήρες τόσο του εμπορίου όσο και την προσωπική προσπάθεια δημιουργίας αισθητήρα. Θα μιλήσουμε για τον τρόπο σύνδεσης των αισθητήρων με την πλακέτα και τέλος θα αναφερθούμε στα είδη των υφασμάτων και πως μπορούν αυτά να χρησιμοποιηθούν, έτσι ώστε να έχει το σύστημα άμεση εφαρμογή στον αθλητή.

Τέταρτο κεφάλαιο: Στο κεφάλαιο αυτό θα παρουσιάσουμε το λογισμικό που αναπτύξαμε για το σύστημα, συγκεκριμένα τον προγραμματισμό του ESP32 με το Arduino IDE, τις βιβλιοθήκες που χρειαστήκαμε για αυτό και τέλος τον τρόπο δημιουργίας μιας εφαρμογής android η οποία θα κάνει συλλογή των δεδομένων από τους αισθητήρες, τα οποία θα

χρησιμοποιούνται στις συνεδρίες sparring (προπονητικό αγώνα) και στις συνεδρίες ατομικών προπονήσεων στους σάκους πυγμαχίας με σκοπό την βελτίωση της τεχνικής των αθλητών.

Πέμπτο κεφάλαιο: Τέλος σε αυτό το σημείο θα παρουσιάσουμε τις διάφορες εφαρμογές που μπορεί να έχει το σύστημα μας, στις πιο δημοφιλείς δραστηριότητες ενός γυμναστηρίου πυγμαχίας είτε χρησιμοποιώντας τους μεμονωμένους αισθητήρες στο σάκο πυγμαχίας είτε χρησιμοποιώντας τα ειδικά φανελάκια κατά την διάρκεια μίας συνεδρίας sparring.

2. Θεωρητικό Υπόβαθρο

2.1 Μικροελεγκτές

2.1.1 Ορισμός και είδη

Οι μικροελεγκτές αποτελούν βασικά στοιχεία της σύγχρονης τεχνολογίας και των συστημάτων αυτοματισμού, λειτουργώντας ως ολοκληρωμένα συστήματα με πολλαπλές λειτουργικές μονάδες σε ένα ενιαίο κύκλωμα. Περιλαμβάνουν επεξεργαστή (CPU), μνήμη (RAM και ROM/Flash), θύρες εισόδου/εξόδου (I/O) και περιφερειακά κυκλώματα, τα οποία επιτρέπουν την επικοινωνία με εξωτερικές συσκευές, όπως αισθητήρες (sensors) και ενεργοποιητές (actuators). Η ενσωμάτωση αυτών των στοιχείων καθιστά τους μικροελεγκτές ιδανικούς για εφαρμογές που απαιτούν εξοικονόμηση χώρου, χαμηλή κατανάλωση ενέργειας και αυξημένη λειτουργικότητα (R. Bannatyne and G. Viot, 2002).

Ένα από τα σημαντικότερα χαρακτηριστικά τους είναι η δυνατότητα αλληλεπίδρασης με το φυσικό περιβάλλον. Μέσω αισθητήρων (sensors), όπως θερμοκρασίας, πίεσης ή φωτεινότητας, συλλέγουν δεδομένα από το περιβάλλον και ελέγχουν φυσικές συσκευές, όπως λαμπτήρες LED, κινητήρες και οθόνες. Για παράδειγμα, σε ένα κλιματιστικό, ο μικροελεγκτής λαμβάνει την επιθυμητή θερμοκρασία από το τηλεχειριστήριο και παρακολουθεί συνεχώς την πραγματική θερμοκρασία μέσω αισθητήρων (sensors). Βάσει των δεδομένων που συλλέγει, ενεργοποιεί ή απενεργοποιεί τον συμπιεστή και τον ανεμιστήρα, εξασφαλίζοντας τη διατήρηση της θερμοκρασίας σε επιθυμητά επίπεδα.

Η λειτουργικότητά τους επεκτείνεται σε πολλές ακόμα εφαρμογές, όπως τα «έξυπνα σπίτια», όπου χρησιμοποιούνται για τη διαχείριση φωτισμού, θέρμανσης και συστημάτων ασφαλείας, προσφέροντας αυξημένη άνεση και έλεγχο στους χρήστες. Στις βιομηχανίες, ελέγχουν γραμμές παραγωγής, συμβάλλοντας στην αποδοτικότητα και τη μείωση του κόστους. Παράλληλα, οι μικροελεγκτές αποτελούν τον πυρήνα συσκευών IoT (Internet of Things), επιτρέποντας τη σύνδεση και την ανταλλαγή δεδομένων μεταξύ έξυπνων συσκευών. Αυτή η ικανότητα σύνδεσης του ψηφιακού με τον φυσικό κόσμο περιγράφεται ως φυσικός υπολογισμός (physical computing). Συνδυάζοντας το υλισμικό (hardware) με το λογισμικό (software), οι μικροελεγκτές επιλύουν πραγματικά προβλήματα σε τομείς όπως η αυτοκινητοβιομηχανία, η ιατρική τεχνολογία και ο βιομηχανικός αυτοματισμός. Καθώς η

τεχνολογία εξελίσσεται, οι μικροελεγκτές συνεχίζουν να διαδραματίζουν κεντρικό ρόλο στη διαμόρφωση της σύγχρονης τεχνολογικής επανάστασης.

Λειτουργικά Χαρακτηριστικά

Οι μικροελεγκτές σχεδιάζονται με έμφαση στη χαμηλή κατανάλωση ενέργειας, γεγονός που τους καθιστά κατάλληλους για συσκευές που λειτουργούν με μπαταρία. Οι προηγμένοι μικροελεγκτές διαθέτουν λειτουργίες αναμονής (standby mode) και ενεργοποιούνται από εξωτερικά συμβάντα, όπως η ενεργοποίηση ενός διακόπτη. Αυτή η δυνατότητα συμβάλλει στη σημαντική παράταση της διάρκειας ζωής της μπαταρίας στις εφαρμογές όπου χρησιμοποιούνται.

Είδη Μικροελεγκτών

1. Βασισμένοι στον τύπο της αρχιτεκτονικής:

- **8-bit:**
 - Χρησιμοποιούνται για απλές εφαρμογές με χαμηλές απαιτήσεις.
 - Παραδείγματα: AVR (π.χ. ATmega328), PIC16.
- **16-bit:**
 - Κατάλληλοι για εφαρμογές μέσης πολυπλοκότητας, όπως ελεγκτές συσκευών.
 - Παραδείγματα: MSP430, PIC24.
- **32-bit:**
 - Προσφέρουν υψηλές επιδόσεις για προηγμένες εφαρμογές, όπως IoT, επεξεργασία δεδομένων και έλεγχο μηχανών.
 - Παραδείγματα: ARM Cortex-M, ESP32.

2. Βασισμένοι στην οικογένεια/αρχιτεκτονική CPU:

- **AVR:**
 - Χρησιμοποιούνται συχνά στην πλατφόρμα Arduino.
 - Παραδείγματα: ATmega328, ATtiny85.
- **ARM Cortex:**
 - Υψηλή απόδοση και ευρεία υποστήριξη λογισμικού.
 - Παραδείγματα: STM32, LPC1768.
- **PIC:**
 - Δημοφιλής σε βιομηχανικές εφαρμογές.
 - Παραδείγματα: PIC16, PIC18.
- **8051:**
 - Παλαιότερη αρχιτεκτονική, ακόμα χρησιμοποιείται σε εφαρμογές χαμηλού κόστους.
 - Παραδείγματα: AT89C51.

- **RISC-V:**
 - Νέα ανοιχτού κώδικα αρχιτεκτονική, με αναπτυσσόμενη υποστήριξη.
 - Παραδείγματα: SiFive.

3. Βασισμένοι στις δυνατότητες επικοινωνίας:

- **Wi-Fi μικροελεγκτές:**
 - Για IoT εφαρμογές.
 - Παραδείγματα: ESP8266, ESP32.
- **Bluetooth μικροελεγκτές:**
 - Για επικοινωνία μικρής εμβέλειας.
 - Παραδείγματα: Nordic nRF52, ESP32.
- **LoRa/LoRaWAN μικροελεγκτές:**
 - Για εφαρμογές μεγάλης εμβέλειας χαμηλής ισχύος.
 - Παραδείγματα: STM32WL, ESP32-LORA.

4. Βασισμένοι στη χρήση τους:

- **Γενικής χρήσης:**
 - Για ευρεία γκάμα εφαρμογών.
 - Παραδείγματα: Arduino (ATmega328), STM32.
- **Ειδικού σκοπού:**
 - Σχεδιασμένοι για συγκεκριμένες εφαρμογές, π.χ. έλεγχος κινητήρα, αισθητήρες.
 - Παραδείγματα: TMS320 (ψηφιακή επεξεργασία σήματος), ESP32-CAM (κάμερες)

5. Βασισμένοι στις δυνατότητες μνήμης:

- **Μικροελεγκτές με μικρή μνήμη:**
 - Για βασικές εφαρμογές (π.χ. αισθητήρες).
 - Παραδείγματα: ATtiny85.
- **Μικροελεγκτές με μεγάλη μνήμη:**
 - Για περίπλοκες εφαρμογές με ανάγκη επεξεργασίας δεδομένων.
 - Παραδείγματα: STM32H7.

6. Βασισμένοι στη μέθοδο προγραμματισμού:

- **Μικροελεγκτές που υποστηρίζουν Assembly ή C:**
 - Παραδείγματα: PIC, AVR.
- **Μικροελεγκτές που υποστηρίζουν Python (π.χ. MicroPython):**
 - Παραδείγματα: ESP32, Raspberry Pi Pico.

7. Μικροελεγκτές Μικτού Σήματος

Μια ιδιαίτερη κατηγορία μικροελεγκτών είναι οι μικτού σήματος, οι οποίοι ενσωματώνουν τόσο αναλογικά όσο και ψηφιακά εξαρτήματα. Αυτή η κατηγορία μικροελεγκτών είναι εξαιρετικά χρήσιμη σε εφαρμογές όπου απαιτείται επεξεργασία σημάτων από αναλογικές πηγές, όπως αισθητήρες θερμοκρασίας, πίεσης ή φωτεινότητας.

Εφαρμογές Μικροελεγκτών

Οι μικροελεγκτές (Barrett and Pack, 2006) βρίσκουν εφαρμογή σε πληθώρα τομέων, όπως:

- **Αυτοματισμοί:** Έλεγχος βιομηχανικών γραμμών παραγωγής και συστημάτων οικιακού αυτοματισμού.
- **Ηλεκτρονικές συσκευές:** Ρύθμιση θερμοκρασίας σε φούρνους μικροκυμάτων, πλυντήρια και ψυγεία.
- **Ιατρικές συσκευές:** Παρακολούθηση και έλεγχος εξοπλισμού, όπως καρδιογράφοι και αντλίες έγχυσης φαρμάκων.
- **Εφαρμογές γυμναστηρίου και όργανα γυμναστικής:** Παρακολούθηση επιδόσεων των αθλητών στα όργανα και συλλογή δεδομένων από τις ασκήσεις τους.
- **Δίκτυα και IoT:** Ενσωματώνονται σε κόμβους δικτύου για τη συλλογή και ανταλλαγή δεδομένων

Προγραμματισμός Μικροελεγκτών

Οι μικροελεγκτές προγραμματίζονται με τη χρήση γλωσσών προγραμματισμού υψηλού επιπέδου, όπως η C και η C++, οι οποίες είναι ιδιαίτερα διαδεδομένες λόγω της ευκολίας και της ευελιξίας τους. Εναλλακτικά, ο προγραμματισμός μπορεί να γίνει σε γλώσσα Assembly, η οποία προσφέρει μεγαλύτερο έλεγχο στον χρήστη αλλά είναι πιο πολύπλοκη.

Κατασκευαστές Μικροελεγκτών (Metoree, 2025)

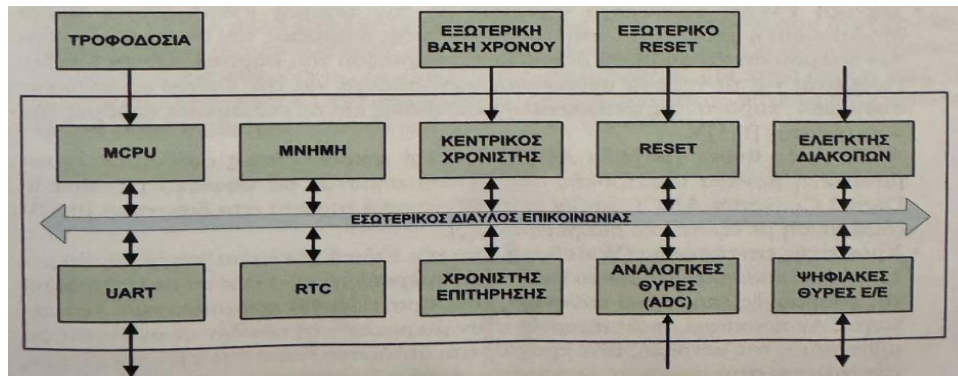
Οι κυριότερες εταιρείες που κατασκευάζουν μικροελεγκτές περιλαμβάνουν:

1. **Analog Devices:** Εξειδικεύεται σε μικροελεγκτές για αναλογικές εφαρμογές, προσφέροντας λύσεις υψηλής ακρίβειας για βιομηχανικούς και ιατρικούς τομείς.
2. **Cypress Semiconductor (τώρα μέρος της Infineon Technologies):** Γνωστή για τις εφαρμογές IoT και αυτοματισμών, με προϊόντα όπως οι σειρές PSoC που προσφέρουν ευελιξία και προσαρμοστικότητα.

3. **Infineon Technologies:** Διακρίνεται για τη χρήση της στους τομείς της αυτοκινητοβιομηχανίας και της ενέργειας, παρέχοντας μικροελεγκτές με έμφαση στην ασφάλεια και την αξιοπιστία.
4. **Maxim Integrated (τώρα μέρος της Analog Devices):** Επικεντρώνεται σε συσκευές χαμηλής κατανάλωσης ενέργειας, ιδανικές για φορητές και ιατρικές εφαρμογές.
5. **Microchip Technology:** Κατασκευάζει μικροελεγκτές γενικής χρήσης, όπως οι σειρές PIC και AVR, που είναι δημοφιλείς σε εκπαιδευτικές και βιομηχανικές εφαρμογές.
6. **NXP Semiconductors:** Προσφέρει λύσεις για IoT και ασφαλή δίκτυα, με έμφαση σε εφαρμογές αυτοκινητοβιομηχανίας και βιομηχανικού αυτοματισμού.
7. **STMicroelectronics:** Επικεντρώνεται σε εφαρμογές μικτού σήματος και IoT, με προϊόντα όπως η σειρά STM32 που χρησιμοποιείται ευρέως σε ενσωματωμένα συστήματα.
8. **Texas Instruments:** Προσφέρει μικροελεγκτές υψηλής απόδοσης για βιομηχανικές εφαρμογές, με έμφαση σε αναλογικές και ψηφιακές λύσεις.
9. **Renesas Electronics:** Ένας από τους κορυφαίους προμηθευτές μικροελεγκτών, με ισχυρή παρουσία σε εφαρμογές αυτοκινητοβιομηχανίας και βιομηχανικού ελέγχου.
10. **Qualcomm:** Γνωστή κυρίως για τους επεξεργαστές κινητών συσκευών, προσφέρει επίσης λύσεις μικροελεγκτών για εφαρμογές IoT και ασύρματης επικοινωνίας.
11. **Toshiba Electronic Devices & Storage:** Παρέχει μικροελεγκτές για ποικίλες εφαρμογές, συμπεριλαμβανομένων των καταναλωτικών ηλεκτρονικών και των βιομηχανικών συστημάτων.

Βασικά χαρακτηριστικά μικροελεγκτών

Οι μικροελεγκτές αποτελούν ένα κρίσιμο εργαλείο για την αυτοματοποίηση και την επεξεργασία δεδομένων, με μια σειρά από βασικά χαρακτηριστικά που καθορίζουν την απόδοσή τους. Επιπλέον διαθέτουν μια σειρά από κρίσιμα χαρακτηριστικά που τους καθιστούν ιδανικούς για πολλές εφαρμογές αυτοματισμού και ελέγχου. Τα κυριότερα χαρακτηριστικά τους περιγράφονται παρακάτω (σχήμα 1):



Σχήμα 1. Βασική διάταξη 32bit μικροελεγκτή (Παπάζογλου & Λιωνής, 2020).

1. Μονάδα Κεντρικής Επεξεργασίας (CPU)

Η μονάδα κεντρικής επεξεργασίας (Central Processing Unit, CPU) αποτελεί τον πυρήνα του μικροελεγκτή. Είναι υπεύθυνη για την ανάκληση (fetch) εντολών από τη μνήμη προγράμματος, την αποκωδικοποίησή τους και την εκτέλεση των εντολών. Περιλαμβάνει:

- **Registers:** Χρησιμοποιούνται για την προσωρινή αποθήκευση δεδομένων και διευθύνσεων.
- **ALU (Arithmetic Logic Unit):** Εκτελεί αριθμητικές και λογικές πράξεις.
- **Instruction Decoder:** Μεταφράζει τις εντολές σε σήματα ελέγχου.
- **Κυκλώματα Ελέγχου:** Ρυθμίζουν τη ροή δεδομένων και εντολών μεταξύ των μονάδων.

2. Μνήμη Προγράμματος

Η μνήμη προγράμματος αποθηκεύει τις εντολές που εκτελεί ο μικροελεγκτής. Είναι μη πτητική (non-volatile), δηλαδή τα δεδομένα της δεν χάνονται σε περίπτωση διακοπής ρεύματος. Είδη μνήμης προγράμματος περιλαμβάνουν:

- **EEPROM:** Electrically Erasable Read Only Memory. Μνήμη μόνο για ανάγνωση με ηλεκτρική διαγραφή. Μπορεί να επαναπρογραμματιστεί ηλεκτρικά.
- **Flash Memory:** Παρέχει μεγαλύτερη χωρητικότητα και ευελιξία στη διαγραφή δεδομένων ανά μπλοκ.
- **EPROM:** Erasable Read Only Memory. Διαγράψιμη μνήμη μόνο για ανάγνωση. Επαναπρογραμματίζεται μέσω υπεριώδους ακτινοβολίας.
- **Mask ROM και OTP:** Η Mask Read-Only Memory (Mask ROM) και η One-Time Programmable (OTP) μνήμη χαρακτηρίζονται από την αδυναμία επαναπρογραμματισμού, γεγονός που τις καθιστά κατάλληλες για εφαρμογές όπου οι δεδομένες πληροφορίες δεν απαιτούν αλλαγή.

3. Μνήμη RAM

Η μνήμη τυχαίας προσπέλασης (Random Access Memory, RAM) αποθηκεύει προσωρινά δεδομένα που απαιτούνται κατά την εκτέλεση των εντολών. Περιλαμβάνει τη **στοίβα (stack)**, όπου αποθηκεύονται διευθύνσεις επιστροφής από υπορουτίνες ή διακοπές (interrupts).

4. Ταλαντωτής Χρονισμού (Clock Oscillator)

Ο ταλαντωτής χρονισμού καθορίζει τη συχνότητα λειτουργίας του μικροελεγκτή, ρυθμίζοντας τον ρυθμό εκτέλεσης των εντολών. Μπορεί να είναι:

- **Εσωτερικός (RC Oscillator):** Χαμηλού κόστους αλλά με μειωμένη ακρίβεια.
- **Εξωτερικός (Quartz Oscillator):** Παρέχει υψηλή ακρίβεια για εφαρμογές υψηλών απαιτήσεων.

5. Κύκλωμα Επαναφοράς (Reset)

Το κύκλωμα επαναφοράς εξασφαλίζει την εκκίνηση του μικροελεγκτή σε μια προκαθορισμένη κατάσταση μετά την ενεργοποίηση της τροφοδοσίας, επιτρέποντας την αξιόπιστη λειτουργία του συστήματος.

6. Σειριακή Θύρα Επικοινωνίας

- **Σύγχρονες Θύρες (Synchronous):** Χρησιμοποιούν σήμα χρονισμού για να διασφαλίσουν τον συγχρονισμό κατά την ανταλλαγή δεδομένων.
- **Ασύγχρονες Θύρες (Asynchronous):** Χωρίς συγχρονισμό.

7. Ψηφιακές και Αναλογικές Θύρες Εισόδου/Εξόδου

Οι θύρες εισόδου/εξόδου χρησιμοποιούνται για την ανταλλαγή δεδομένων με το εξωτερικό περιβάλλον:

- **Ψηφιακές Θύρες (Digital I/O):** Ανταλλάσσουν δεδομένα δυαδικής μορφής (bits ή bytes).
- **Αναλογικές Θύρες (Analog I/O):** Χρησιμοποιούν μετατροπείς ADC (Analog to Digital Converter) και DAC (Digital to Analog Converter) για τη διαχείριση αναλογικών σημάτων.

8. Χρονιστές και Μετρητές

Οι χρονιστές (timers) χρησιμοποιούνται για την παρακολούθηση και τον συγχρονισμό γεγονότων. Ορισμένοι χρονιστές μπορούν να λειτουργήσουν και ως μετρητές (counters) για την καταμέτρηση εξωτερικών σημάτων ή παλμών.

9. Watchdog Timer

Ο Watchdog Timer είναι ένας ειδικός χρονιστής που ανιχνεύει πιθανές αστοχίες του συστήματος. Αν δεν επανεκκινηθεί από το πρόγραμμα σε τακτά διαστήματα, ενεργοποιεί επανεκκίνηση του μικροελεγκτή για την αποφυγή παρατεταμένης αστοχίας.

10. Ρολόι Πραγματικού Χρόνου (RTC)

Το Real Time Clock (RTC) διατηρεί την τρέχουσα ώρα, ημερομηνία και ημέρα, και χρησιμοποιείται για την καταγραφή χρονικών γεγονότων ή για προγραμματισμένες λειτουργίες.

2.1.2 Arduino vs ESP32

Ένα από τα σημαντικότερα χαρακτηριστικά τους είναι η δυνατότητα αλληλεπίδρασης με το φυσικό περιβάλλον. Μέσω αισθητήρων (sensors), όπως θερμοκρασίας, πίεσης ή φωτεινότητας, συλλέγουν δεδομένα από το περιβάλλον και ελέγχουν φυσικές συσκευές, όπως λαμπτήρες LED, κινητήρες και οθόνες. Για παράδειγμα, σε ένα κλιματιστικό, ο μικροελεγκτής λαμβάνει την επιθυμητή θερμοκρασία από το τηλεχειριστήριο και παρακολουθεί συνεχώς την πραγματική θερμοκρασία μέσω αισθητήρων (sensors). Βάσει των δεδομένων που συλλέγει, ενεργοποιεί ή απενεργοποιεί τον συμπιεστή και τον ανεμιστήρα, εξασφαλίζοντας τη διατήρηση της θερμοκρασίας σε επιθυμητά επίπεδα.

Arduino

Το Arduino¹ είναι μια πλατφόρμα μικροελεγκτών ανοιχτού υλικού, σχεδιασμένη για εύκολη χρήση από άτομα κάθε ηλικίας και επιπέδου και εξειδίκευσης. Με τη βοήθεια του λογισμικού **Arduino IDE**, οι χρήστες μπορούν να προγραμματίσουν τις πλακέτες Arduino για να αλληλεπιδρούν με το περιβάλλον. Ένα Arduino μπορεί να λαμβάνει σήματα εισόδου, όπως το πάτημα ενός κουμπιού ή την περιστροφή ενός ποτενσιόμετρου, και να τα μετατρέπει σε σήματα εξόδου, όπως το άναμμα ενός LED ή την κίνηση ενός μοτέρ. Το Arduino αποτελεί μία από τις πιο διαδεδομένες πλατφόρμες ανοιχτού υλικού παγκοσμίως, χάρη στην ευχρηστία του στη δημιουργία γρήγορων πρωτοτύπων. Είναι ιδιαίτερα δημοφιλές στην εκπαίδευση και την ανάπτυξη εφαρμογών που δεν απαιτούν προηγμένες γνώσεις ηλεκτρονικών ή προγραμματισμού.

¹ Arduino, 2025.

Arduino Hardware - Nano Family. [Ηλεκτρονικό]
Διαθέσιμο στο: <https://www.arduino.cc/en/hardware>

Οικογένειες Πλακετών Arduino

Η πλατφόρμα Arduino διαθέτει μια σειρά από οικογένειες πλακετών μικροελεγκτών ανοικτού κώδικα, σχεδιασμένες για να καλύπτουν τις ανάγκες διαφόρων τύπων εφαρμογών. Κάθε οικογένεια πλακετών περιλαμβάνει μοντέλα με συγκεκριμένα χαρακτηριστικά, που ενισχύουν τη λειτουργικότητά τους σε συγκεκριμένους τομείς, όπως εφαρμογές IoT, ρομποτική, βιομηχανικά έργα, και εκπαίδευση.

Οι κυριότερες οικογένειες πλακετών Arduino είναι οι εξής²:

Οικογένεια Arduino UNO

Η οικογένεια UNO (πίνακας 1) αποτελεί τη βασική σειρά πλακετών Arduino, ιδανική για αρχάριους και για εκπαιδευτική χρήση. Οι πλακέτες της οικογένειας αυτής βασίζονται στον μικροελεγκτή **ATmega328** και προσφέρουν βασικές δυνατότητες, όπως (πίνακας 2):

- **Χρήση σε εκπαιδευτικά περιβάλλοντα.**
- **Απλή σχεδίαση** για την υλοποίηση βασικών πρωτοτύπων.
Η οικογένεια αυτή είναι η πιο γνωστή και διαδεδομένη, αποτελώντας την ιδανική επιλογή για όσους εισέρχονται στον κόσμο του Arduino.

Η οικογένεια Arduino Uno.

			
Arduino UNO R4 Minima	Arduino UNO R4 WiFi	Arduino UNO R3	Arduino Leonardo
			
Arduino UNO Mini Limited Edition	Arduino Micro	Arduino Zero	Arduino UNO WiFi Rev2

Πίνακας 1. Εικόνες μικροελεγκτών οικογένειας Arduino Uno

² Arduino Pro, 2025.
Platform Hardware. [Ηλεκτρονικό]
Διαθέσιμο στο: <https://www.arduino.cc/pro/platform-hardware/>

Ο πίνακας της οικογένειας Arduino Uno.

Device Name	Number of Cores	Architecture	CPU Frequency (MHz)	RAM (KB)	Flash (KB/MB)	Buses	Digital I/O (PWM Pins)	Analog Pins	Operating Voltage (V)
Arduino UNO R4 Minima	1	ARM Cortex-M4	48	320	256 KB	I2C, SPI, UART	14 (PWM Pins: 6)	6	3.3
Arduino UNO R4 WiFi	1	ARM Cortex-M4	48	320	256 KB	I2C, SPI, UART	14 (PWM Pins: 6)	6	3.3
Arduino UNO R3	1	AVR	16	20	32 KB	I2C, SPI, UART	14 (PWM Pins: 6)	6	5.0
Arduino Leonardo	1	AVR	16	25	32 KB	I2C, SPI, UART	20 (PWM Pins: 7)	12	5.0
Arduino UNO Mini Limited Edition	1	AVR	16	20	32 KB	I2C, SPI, UART	14 (PWM Pins: 6)	6	5.0
Arduino Micro	1	AVR	16	25	32 KB	I2C, SPI, UART	20 (PWM Pins: 7)	12	5.0
Arduino Zero	1	ARM Cortex-M0+	48	320	256 KB	I2C, SPI, UART	20 (PWM Pins: 10)	6	3.3
Arduino UNO WiFi Rev2	1	AVR	16	60	48 KB	I2C, SPI, UART	14 (PWM Pins: 5)	6	5.0

Πίνακας 2. Οικογένεια Arduino Uno

Οικογένεια Arduino Nano

Η οικογένεια Nano (πίνακας 3) περιλαμβάνει πιο συμπαγείς πλακέτες, σχεδιασμένες για έργα όπου ο χώρος είναι περιορισμένος. Παρά το μικρό τους μέγεθος, προσφέρουν πληθώρα χαρακτηριστικών:

- Υποστήριξη Wi-Fi και Bluetooth σε ορισμένα μοντέλα (π.χ. **Nano 33 IoT** και **Nano ESP32**; πίνακας 4).
- Ενσωμάτωση δυνατοτήτων IoT.
- **Συμβατότητα με μικρές κατασκευές και φορητές εφαρμογές.**

Η οικογένεια Nano είναι δημοφιλής τόσο σε αρχάριους όσο και σε πιο προχωρημένους χρήστες, λόγω της ευελιξίας και της ποικιλίας των μοντέλων της.

Η οικογένεια Arduino Nano.

			
Arduino Nano 33 IoT	Arduino Nano RP2040 Connect	Arduino Nano ESP32	Arduino Nano 33 BLE Sense
			
Arduino Nano 33 BLE	Arduino Nano Every	Arduino Nano	

Πίνακας 3. Εικόνες μικροελεγκτών οικογένειας Arduino Nano

Ο πίνακας της οικογένειας Arduino Nano.

Device Name	Number of Core	Architecture	CPU Frequency (MHz)	RAM (KB)	Flash (KB/MB)	Buses	Digital I/O (PWM Pins)	Analog Pin	Operating Voltage (V)
Arduino Nano 33 IoT	1	ARM Cortex-M0+	48	32	256 KB	I2C, SPI, UART	14 (PWM Pins: 11)	8	3.3
Arduino Nano RP2040 Connect	2	ARM Cortex-M0+	133	264	16 MB	I2C, SPI, UART	20 (PWM Pins: 16)	4	3.3
Arduino Nano ESP32	2	Xtensa LX7	240	512	8 MB	I2C, SPI, UART	22 (PWM Pins: 16)	6	3.3
Arduino Nano 33 BLE Sense	1	ARM Cortex-M4F	64	256	1 MB	I2C, SPI, UART	14 (PWM Pins: 12)	8	3.3
Arduino Nano 33 BLE	1	ARM Cortex-M4F	64	256	1 MB	I2C, SPI, UART	14 (PWM Pins: 12)	8	3.3
Arduino Nano Every	1	AVR	20	6	48 KB	I2C, SPI, UART	14 (PWM Pins: 6)	8	5
Arduino Nano	1	AVR	16	2	32 KB	I2C, SPI, UART	14 (PWM Pins: 6)	8	5

Πίνακας 4. Οικογένεια Arduino Nano

Οικογένεια Arduino MKR

Η σειρά MKR (πίνακας 5) έχει σχεδιαστεί ειδικά για έργα **Internet of Things (IoT)**. Οι πλακέτες αυτής της οικογένειας είναι μικρές και ελαφριές, με χαμηλή κατανάλωση ενέργειας, και προσφέρουν εξειδικευμένα χαρακτηριστικά, όπως (πίνακας 6):

- Ενσωματωμένες δυνατότητες συνδεσιμότητας (Wi-Fi, GSM, LoRa).
- Υποστήριξη για εφαρμογές με μπαταρία.
- Ιδανική χρήση σε αισθητήρες, απομακρυσμένους ελέγχους, και συστήματα παρακολούθησης.

Η οικογένεια MKR είναι ιδιαίτερα δημοφιλής στους τομείς της έρευνας και ανάπτυξης IoT.

Η οικογένεια Arduino MKR.



Πίνακας 5. Εικόνες μικροελεγκτών οικογένειας Arduino MKR

Ο πίνακας της οικογένειας Arduino MKR.

Device Name	Number of Cores	Architecture	CPU Frequency (MHz)	RAM (KB)	Flash (KB/MB)	Buses	Digital I/O (PWM Pins)	Analog Pins	Operating Voltage (V)
Arduino MKR 1000 VIFI	1	ARM Cortex-M0+	48	32	256 KB	I2C, SPI, UART	8 (PWM Pins: 12)	7	3.3
Arduino MKR WiFi 1010	1	ARM Cortex-M0+	48	32	256 KB	I2C, SPI, UART	8 (PWM Pins: 12)	7	3.3
Arduino MKR FOX 1200	1	ARM Cortex-M0+	48	32	256 KB	I2C, SPI, UART	8 (PWM Pins: 12)	7	3.3
Arduino MKR WAN 1300	1	ARM Cortex-M0+	48	32	256 KB	I2C, SPI, UART	8 (PWM Pins: 12)	7	3.3
Arduino MKR WAN 1310	1	ARM Cortex-M0+	48	32	256 KB	I2C, SPI, UART	8 (PWM Pins: 12)	7	3.3
Arduino MKR GSM 1400	1	ARM Cortex-M0+	48	32	256 KB	I2C, SPI, UART	8 (PWM Pins: 12)	7	3.3
Arduino MKR NB 1500	1	ARM Cortex-M0+	48	32	256 KB	I2C, SPI, UART	8 (PWM Pins: 12)	7	3.3
Arduino MKR Vidor 4000	1	Intel Cyclone 10 FPGA + ARM Cortex-M0+	48	32	2 MB	I2C, SPI, UART	25 (PWM Pins: 12)	6	3.3
Arduino MKR Zero	1	ARM Cortex-M0+	48	32	256 KB	I2C, SPI, UART	8 (PWM Pins: 12)	7	3.3

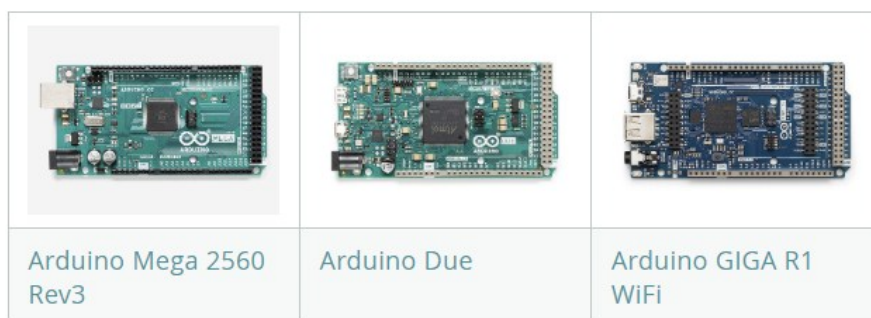
Πίνακας 6. Οικογένεια Arduino MKR

4. Οικογένεια Arduino Mega

Η οικογένεια Mega (πίνακας 7) προσφέρει αυξημένες δυνατότητες συνδεσιμότητας και απόδοσης, ιδανικές για έργα που απαιτούν περισσότερες εισόδους/εξόδους και μεγαλύτερη μνήμη. Κύρια χαρακτηριστικά (πίνακας 8):

- Μεγάλος αριθμός ψηφιακών εισόδων/εξόδων.
- Υψηλή απόδοση για έργα ρομποτικής και συστήματα αυτοματισμού.
- Καταλληλότητα για απαιτητικά έργα, όπως συστήματα πολλαπλών αισθητήρων.

Η οικογένεια Arduino Mega.



Πίνακας 7. Εικόνες μικροελεγκτών οικογένειας Arduino Mega

Ο πίνακας της οικογένειας Arduino Mega.

Device Name	Number of Cores	Architecture	CPU Frequency (MHz)	RAM (KB)	Flash (KB/MB)	Buses	Digital I/O (PWM Pins)	Analog Pins	Operating Voltage (V)
Arduino Mega 2560 Rev3	1	AVR	16	8	256 KB	I2C, SPI, UART	54 (PWM Pins: 15)	16	5.0
Arduino Due	1	ARM Cortex-M3	84	96	512 KB	I2C, SPI, UART	54 (PWM Pins: 12)	12	3.3
Arduino GIGA R1 WiFi	2	ARM Cortex-M7 + M4	480 (M7) / 240 (M4)	1024	2 MB	I2C, SPI, UART, CAN	76 (PWM Pins: 12)	16	3.3

Πίνακας 8. Οικογένεια Arduino Mega

5. Οικογένεια Arduino Portenta

Η οικογένεια Arduino Portenta (πίνακας 9) περιλαμβάνει πλακέτες υψηλών επιδόσεων, βιομηχανικής ποιότητας, σχεδιασμένες για προηγμένες εφαρμογές, όπως Τεχνητή Νοημοσύνη (AI), Μηχανική Μάθηση και Υπολογιστική στις Άκρες (Edge Computing). Παρακάτω παρατίθεται ένας συγκριτικός πίνακας με τα βασικά χαρακτηριστικά των κύριων πλακετών Portenta (πίνακας 10):

Οικογένεια Arduino Portenta



Πίνακας 9. Εικόνες μικροελεγκτών οικογένειας Arduino Portenta

Ο πίνακας της οικογένειας Arduino Portenta

Device Name	Microcontroller	Operating System	Memory	Connectivity	Digital I/O Pins	Analog Input Pins	Power Supply	Best for
Portenta H7	Dual-core STM32H747: Arm Cortex-M7 (480 MHz) and Cortex-M4 (240 MHz)	Mbed OS	16 MB Flash, 8 MB SDRAM	Wi-Fi, Bluetooth, Ethernet, USB-C	160 (via high-density connectors)	8	3.3V, 5V, USB-C, Li-Po battery	Dual-core processing for real-time and AI applications, industrial temperature range (-40 to 85°C)
Portenta X8	NXP i.MX 8M Mini (Quad-core Arm Cortex-A53, 1.8 GHz) and STM32MP1 (Dual-core Arm Cortex-A7, 650 MHz)	Linux-based OS	16 GB eMMC, 2 GB LPDDR4 RAM	Wi-Fi, Bluetooth, Ethernet, USB-C, Cellular (optional)	160 (via high-density connectors)	8	3.3V, 5V, USB-C, Li-Po battery	Heterogeneous multi-core processing for industrial IoT and edge computing
Portenta C33	Renesas RA6M5 (Arm Cortex-M33, 200 MHz)	Mbed OS	512 KB Flash, 128 KB RAM	Wi-Fi, Bluetooth, USB-C	160 (via high-density connectors)	8	3.3V, 5V, USB-C, Li-Po battery	Cost-effective module for accessible IoT development

Πίνακας 10. Οικογένεια Arduino Portenta

Πλεονεκτήματα του Arduino

Το Arduino υπερέχει σε αρκετούς τομείς σε σύγκριση με άλλες πλατφόρμες μικροελεγκτών, προσφέροντας:

1. **Χαμηλό κόστος πλακέτας:** Η αρχική ιδέα πίσω από το Arduino ήταν η δημιουργία ενός οικονομικού εργαλείου για την εκπαίδευση και την ανάπτυξη πρωτοτύπων. Τα εξαρτήματα που χρησιμοποιούνται είναι προσιτά και εύκολα διαθέσιμα.

2. **Διαπλατφορμικότητα:** Το λογισμικό **Arduino IDE** λειτουργεί σε όλα τα βασικά λειτουργικά συστήματα, όπως Windows, macOS και Linux, καθιστώντας το προσβάσιμο σε περισσότερους χρήστες.
3. **Απλό και ανοιχτού κώδικα περιβάλλον προγραμματισμού:** Το **Arduino IDE** είναι ιδανικό για αρχάριους, ενώ παρέχει και εργαλεία που μπορούν να αξιοποιηθούν από επαγγελματίες. Οι χρήστες μπορούν να δημιουργήσουν και να προσθέσουν δικές τους βιβλιοθήκες σε C++ ή ακόμη και να γράψουν κώδικα σε AVR-C για να επεκτείνουν τις δυνατότητές του.
4. **Υλικό ανοιχτής πηγής:** Τα σχέδια και οι τεχνικές προδιαγραφές των πλακετών Arduino είναι δημόσια διαθέσιμα. Αυτό επιτρέπει στους χρήστες να μελετήσουν το υλικό και να δημιουργήσουν τις δικές τους προσαρμοσμένες ή βελτιωμένες εκδόσεις.
5. **Ενεργή κοινότητα χρηστών:** Η διαδικτυακή κοινότητα γύρω από το Arduino είναι τεράστια και δραστήρια. Παρέχει βοήθεια στην επίλυση προβλημάτων υλικού και λογισμικού, ενώ παράλληλα συνεισφέρει στη συνεχή ανάπτυξη και βελτίωση των προϊόντων Arduino μέσω της ανταλλαγής ιδεών και εμπειριών.

ESP32

Το ESP32 είναι ένας ισχυρός και ευέλικτος μικροελεγκτής που αναπτύχθηκε από την Espressif Systems, σχεδιασμένος για εφαρμογές που απαιτούν συνδεσιμότητα μέσω Wi-Fi και Bluetooth. Είναι μια πλατφόρμα ανοιχτού υλικού, όπως το Arduino, αλλά προσφέρει περισσότερες δυνατότητες και ισχύ, καθιστώντας το ιδανικό για προηγμένες εφαρμογές IoT (Internet of Things).

Πλεονεκτήματα του ESP32

Το ESP32, σε σύγκριση με το Arduino, διαθέτει πιο προηγμένα χαρακτηριστικά, όπως:

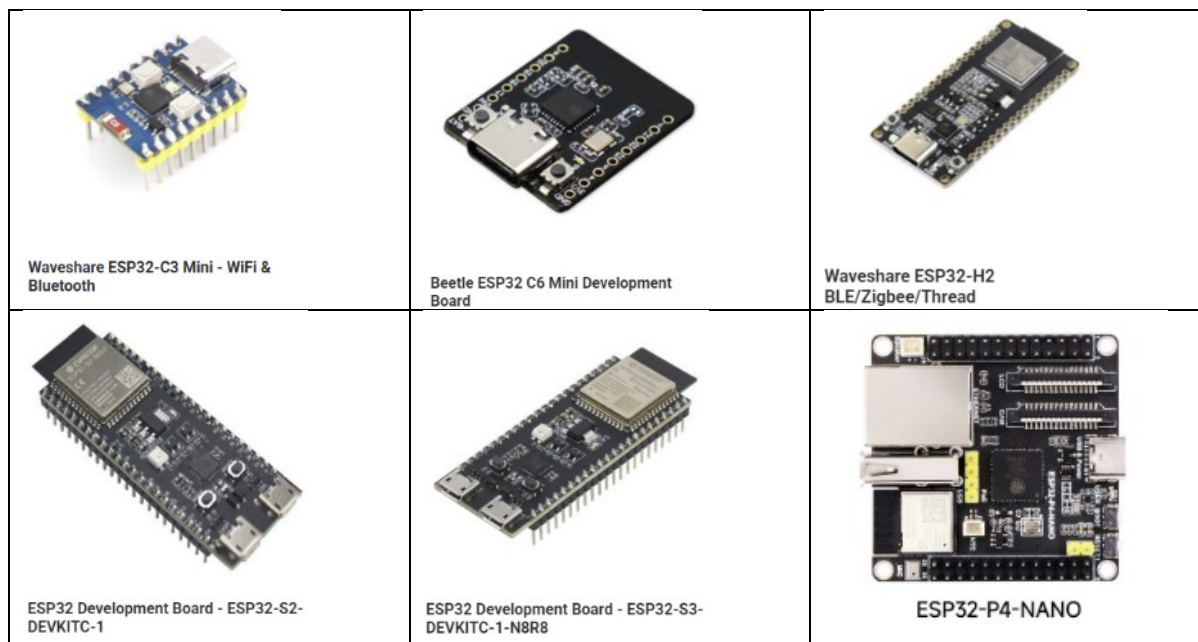
1. **Ασύρματη συνδεσιμότητα:** Το ESP32 ενσωματώνει Wi-Fi και Bluetooth, επιτρέποντας τη σύνδεση με ασύρματα δίκτυα και άλλες συσκευές, κάτι που είναι ιδανικό για εφαρμογές IoT (Internet of Things).
2. **Υψηλή ταχύτητα και επεξεργαστική ισχύς:** Ο διπύρηνος επεξεργαστής του ESP32 και η υψηλότερη ταχύτητα ρολογιού προσφέρουν αυξημένη υπολογιστική ισχύ για πιο απαιτητικές εφαρμογές.

3. **Μεγαλύτερη μνήμη:** Το ESP32 διαθέτει περισσότερη RAM και flash storage σε σχέση με τις περισσότερες πλακέτες Arduino, καθιστώντας το ικανό να χειρίζεται μεγαλύτερα προγράμματα και πιο σύνθετα δεδομένα.
4. **Ευέλικτες εισόδους/εξόδους (GPIO):** Οι ακίδες του ESP32 μπορούν να προσαρμοστούν σε πολλαπλές λειτουργίες, όπως PWM, ADC, DAC, και I2C, επιτρέποντας μεγαλύτερη ευελιξία στη σχεδίαση έργων.
5. **Χαμηλή κατανάλωση ενέργειας:** Το ESP32 είναι σχεδιασμένο με λειτουργίες χαμηλής κατανάλωσης, καθιστώντας το κατάλληλο για συσκευές που λειτουργούν με μπαταρία.
6. **Υποστήριξη πολλαπλών γλωσσών προγραμματισμού:** Εκτός από το Arduino IDE, το ESP32 υποστηρίζει εργαλεία όπως το MicroPython και το Espressif IDF, προσφέροντας ευελιξία στους προγραμματιστές.

Οικογένειες πλακετών ESP32

Η πλατφόρμα ESP32, αναπτυγμένη από την Espressif Systems, περιλαμβάνει μια ποικιλία οικογενειών μικροελεγκτών ανοικτού κώδικα, σχεδιασμένων για να καλύπτουν τις ανάγκες διαφόρων εφαρμογών. Κάθε οικογένεια μικροελεγκτών προσφέρει συγκεκριμένα χαρακτηριστικά και δυνατότητες, που ενισχύουν τη λειτουργικότητά τους σε τομείς όπως το Internet of Things (IoT), η αυτοματοποίηση, οι εφαρμογές μηχανικής μάθησης και η ενσωματωμένη συνδεσιμότητα. Οι οικογένειες αυτές είναι σχεδιασμένες για να προσφέρουν απόδοση, ευελιξία και οικονομία, καλύπτοντας τόσο απλές όσο και απαιτητικές εφαρμογές. Οι κυριότερες οικογένειες πλακετών ESP32 (ESP Boards, 2023) είναι οι εξής (πίνακες 11, 12): Η οικογένεια ESP32 περιλαμβάνει μια ποικιλία μικροελεγκτών με διαφορετικά χαρακτηριστικά και δυνατότητες. Ακολουθεί ένας πίνακας που συγκρίνει τις κύριες οικογένειες ESP32:

Οικογένειες πλακετών ESP32



Πίνακας 11. Εικόνες μικροελεγκτών οικογενειών ESP32

Ο πίνακας των οικογενειών του ESP32.

Family Name	Number of Cores	Architecture	CPU Frequency (MHz)	RAM (KB)	Flash (KB/MB)	Buses	Digital I/O (PWM Pins)	Analog Pins	Operating Voltage (V)
ESP32-WROOM	2	Xtensa LX6	240	520	4 MB	I2C, SPI, UART	34 (PWM Pins: 16)	18	3.3
ESP32-WROVER	2	Xtensa LX6	240	520	4 MB / 8 MB PSRAM	I2C, SPI, UART	34 (PWM Pins: 16)	18	3.3
ESP32-S2	1	Xtensa LX7	240	320	4 MB	I2C, SPI, UART	21 (PWM Pins: 14)	20	3.3
ESP32-C3	1	RISC-V	160	400	4 MB	I2C, SPI, UART	22 (PWM Pins: 6)	6	3.3
ESP32-S3	2	Xtensa LX7	240	512	16 MB	I2C, SPI, UART	44 (PWM Pins: 16)	20	3.3
ESP32-06	1	RISC-V	160	512	4 MB	I2C, SPI, UART	30 (PWM Pins: 10)	14	3.3
ESP32-H2	1	RISC-V	96	256	4 MB	I2C, SPI, UART	19 (PWM Pins: 8)	6	3.3
ESP32-P4	2	RISC-V	400	768	takes external flash	I2C, SPI, UART	40 (PWM Pins: 12)	16	3.3

Πίνακας 12. Οικογένειες του ESP32

Διαφορά Arduino και ESP32:

Παρόλο που το Arduino και το ESP32 έχουν κοινά χαρακτηριστικά, όπως η δυνατότητα χρήσης του Arduino IDE για τον προγραμματισμό τους, το ESP32 προσφέρει περισσότερες δυνατότητες. Το Arduino βασίζεται κυρίως σε μικροελεγκτές AVR ή ARM και είναι γνωστό για την απλότητα και την ευχρηστία του. Αντίθετα, το ESP32 είναι ένας ισχυρότερος μικροελεγκτής που ενσωματώνει ασύρματες δυνατότητες Wi-Fi και Bluetooth, καθιστώντας το ιδανικό για έργα που απαιτούν ασύρματη επικοινωνία. Επίσης, διαθέτει μεγαλύτερη ταχύτητα επεξεργασίας, περισσότερη μνήμη και περισσότερες δυνατότητες σύνδεσης με περιφερειακά, γεγονός που το καθιστά κατάλληλο για πιο απαιτητικές εφαρμογές.

Συμπέρασμα

Οι μικροελεγκτές αποτελούν αναπόσπαστο μέρος της σύγχρονης τεχνολογίας, παρέχοντας λύσεις που καλύπτουν ένα ευρύ φάσμα εφαρμογών, από την επιστήμη μέχρι τη βιομηχανία. Η συνεχής εξέλιξή τους σε επίπεδο απόδοσης, κατανάλωσης ενέργειας και ευελιξίας, καθώς και η ανάπτυξη τεχνολογιών IoT και έξυπνων συσκευών, ενισχύουν τη σημασία τους στο σύγχρονο τεχνολογικό τοπίο. Η επιλογή του κατάλληλου μικροελεγκτή εξαρτάται από τις ανάγκες της εφαρμογής, όπως η υπολογιστική ισχύς, η κατανάλωση ενέργειας, οι διαθέσιμες διεπαφές και το κόστος, με την τεχνολογική πρόοδο να οδηγεί σε συνεχείς βελτιώσεις και νέες λύσεις.

2.2 Τεχνολογία IoT και Bluetooth

2.2.1 Ορισμός

Το Διαδίκτυο των Πραγμάτων (IoT) είναι ένα σύστημα που συνδέει διάφορες συσκευές και αντικείμενα, όπως οικιακές συσκευές, αυτοκίνητα, αισθητήρες και συστήματα ασφαλείας, ώστε να επικοινωνούν μεταξύ τους ή με το διαδίκτυο. Η κύρια ιδέα πίσω από το IoT είναι η αλληλεπίδραση αυτών των συσκευών, διευκολύνοντας τη συλλογή δεδομένων και τον απομακρυσμένο έλεγχο από έναν χρήστη μέσω υπολογιστή ή κινητού (σχήμα 2).

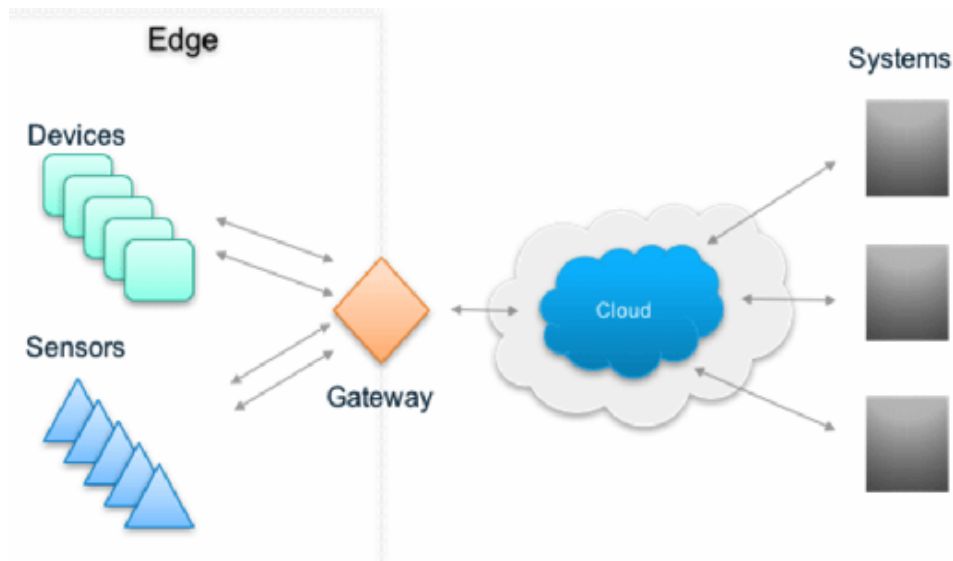
Ο όρος "πράγματα" στο IoT αναφέρεται σε κάθε είδους συσκευή που μπορεί να συνδεθεί και να λειτουργήσει αυτόνομα, είτε πρόκειται για ένα αυτοκίνητο με αισθητήρες είτε για ένα smartwatch. Αυτή η τεχνολογία αναπτύχθηκε στα τέλη της δεκαετίας του 1990, όταν ο Kevin Ashton, μαζί με την ομάδα του στο MIT, βρήκαν τρόπο να συνδέουν αντικείμενα στο διαδίκτυο μέσω τεχνολογίας RFID.

Το IoT θεωρείται μια από τις πιο σημαντικές τεχνολογικές επαναστάσεις του μέλλοντος, μαζί με το mobile internet και την αυτοματοποίηση, προσφέροντας νέες δυνατότητες στη διαχείριση δεδομένων, την ασφάλεια και την απομακρυσμένη παρακολούθηση.



Σχήμα 2. Πόλη όπου το Internet of Things (IoT) διασυνδέει συσκευές και υπηρεσίες.

Για να υλοποιηθεί το IoT, χρησιμοποιούνται διάφοροι τύποι δικτύων και τεχνολογιών, που λειτουργούν είτε αυτόνομα είτε συνδυαστικά (σχήμα 3).



Σχήμα 3. Χρησιμοποιώντας μια πύλη IoT για να συνδέσετε τα «πράγματα» στο cloud.

Υπάρχουν τέσσερα κύρια μοντέλα διασύνδεσης συσκευών IoT (Channelfutures.com, 2016):

- **Device to Device:** Σε αυτό το μοντέλο, δύο ή περισσότερες συσκευές συνδέονται απευθείας, χωρίς ενδιάμεσο Server εφαρμογών. Επικοινωνούν μέσω δικτύων IP ή διαδικτύου, χρησιμοποιώντας πρωτόκολλα όπως Bluetooth, Z-Wave ή ZigBee. Εφαρμογές αυτού του τύπου περιλαμβάνουν συστήματα οικιακού αυτοματισμού, που χρησιμοποιούν μικρά πακέτα δεδομένων για επικοινωνία με χαμηλό ρυθμό μετάδοσης.
- **Device to Cloud:** Οι συσκευές συνδέονται απευθείας σε διαδικτυακές υπηρεσίες cloud, αξιοποιώντας μηχανισμούς επικοινωνίας όπως Ethernet ή Wi-Fi. Αυτή η προσέγγιση χρησιμοποιείται από γνωστές καταναλωτικές συσκευές IoT, όπως οι Smart TVs.
- **Device to Gateway:** Σε αυτή τη μέθοδο, οι συσκευές συνδέονται μέσω μιας υπηρεσίας Application Layer Gateway που λειτουργεί ως διαμεσολαβητής μεταξύ της συσκευής και του cloud. Το μοντέλο αυτό παρέχει επιπλέον ασφάλεια και λειτουργίες, όπως η μετάφραση πρωτοκόλλων.
- **Back-End Data Sharing:** Επιτρέπει στους χρήστες να εξάγουν και να αναλύουν δεδομένα από τις συσκευές IoT, συνδυάζοντας τα με δεδομένα από άλλες πηγές.

Το 2009, μια ομάδα από περισσότερες από 20 εταιρείες και ερευνητικά κέντρα ανέπτυξε την κοινή αρχιτεκτονική IoT-A (Madakam and Tripathi., 2015) (IoT Architecture) για να διασφαλιστεί η διαλειτουργικότητα μεταξύ συσκευών και συστημάτων. Το IoT-A βασίζεται σε τέσσερα μέρη: το όραμα, τα επιχειρηματικά σενάρια, το IoT Reference Model και την IoT

Reference Architecture. Αυτά τα μέρη εξασφαλίζουν την οργανωτική συμβατότητα και την εκπαιδευτική χρησιμότητα του IoT.

Κατηγορίες Δικτύων IoT (Minew, 2024)

1. PAN (Personal Area Networks)

Τα PANs είναι δίκτυα μικρής εμβέλειας που επιτρέπουν τη σύνδεση συσκευών σε πολύ κοντινές αποστάσεις. Είναι συνήθως ασύρματα και καλύπτουν μια μικρή απόσταση, περίπου 10 μέτρων.

- Τεχνολογίες: Bluetooth, Zigbee, Z-Wave.
- Εφαρμογές: Σύνδεση έξυπνων συσκευών όπως φορητές συσκευές, έξυπνοι λαμπτήρες, και αισθητήρες κίνησης.

2. LAN (Local Area Networks)

Τα LANs καλύπτουν περιορισμένες γεωγραφικές περιοχές, όπως ένα σπίτι ή γραφείο. Αυτά μπορεί να είναι ενσύρματα, ασύρματα ή ένας συνδυασμός των δύο. Τα ασύρματα τοπικά δίκτυα (WLANs) καλύπτουν μεγαλύτερη απόσταση, περίπου 100 μέτρων..

- Τεχνολογίες: Wi-Fi, Ethernet.
- Εφαρμογές: Διαχείριση IoT συσκευών όπως έξυπνες κάμερες, θερμοστάτες, και συστήματα ασφαλείας.

3. LPWAN (Low-Power Wide-Area Networks)

Τα LPWANs προσφέρουν μεγάλη εμβέλεια επικοινωνίας με χαμηλή κατανάλωση ενέργειας, καθιστώντας τα ιδανικά για IoT συσκευές με περιορισμένη μπαταρία.

- Τεχνολογίες: LoRaWAN, Sigfox.
- Εφαρμογές: Παρακολούθηση αγροτικών καλλιεργειών, έξυπνοι μετρητές ενέργειας.

4. WAN (Wide Area Networks)

Τα WANs καλύπτουν μεγάλες γεωγραφικές περιοχές και υποστηρίζουν εφαρμογές IoT που απαιτούν παγκόσμια διασύνδεση.

- Τεχνολογίες: 4G, 5G, NB-IoT.
- Εφαρμογές: Συνδεδεμένα οχήματα, έξυπνες πόλεις.

5. Mesh Networks

Τα Mesh Networks επιτρέπουν σε κάθε συσκευή να λειτουργεί ως σημείο αναμετάδοσης δεδομένων, δημιουργώντας ένα ανθεκτικό και ευέλικτο δίκτυο.

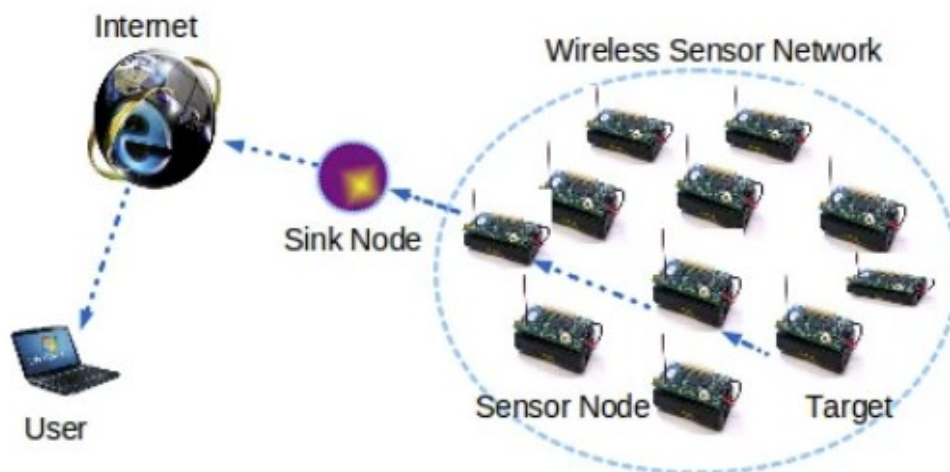
- Τεχνολογίες: Bluetooth Mesh, Thread.
- Εφαρμογές: Έξυπνος φωτισμός, βιομηχανικοί αυτοματισμοί.

6. Hybrid Networks

Τα Hybrid Networks συνδυάζουν διαφορετικές τεχνολογίες για τη βελτίωση της κάλυψης και της απόδοσης.

- Παραδείγματα: Ένα έξυπνο σπίτι που χρησιμοποιεί Wi-Fi για κάμερες και Zigbee για αισθητήρες.
- Εφαρμογές: Ενοποίηση πολλών IoT συσκευών με διαφορετικές απαιτήσεις.

Ασύρματα Δίκτυα Αισθητήρων³ (Wireless Sensor Networks - WSNs)



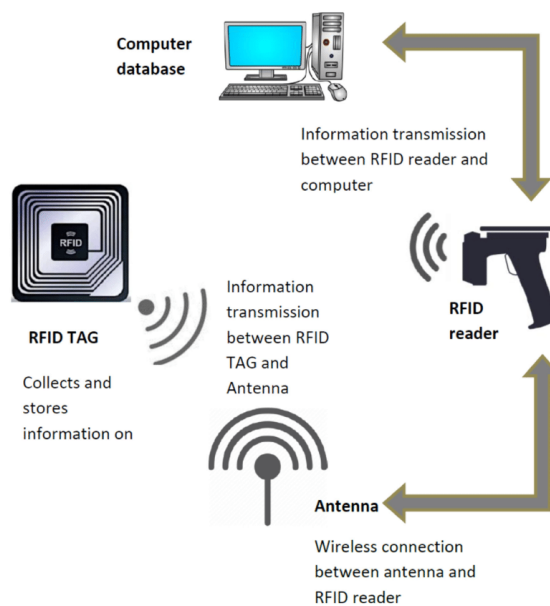
Σχήμα 4. Ασύρματο Δίκτυο Αισθητήρων.

³ Wikipedia, 2025. *Wireless Sensor Network*. [Ηλεκτρονικό] Διαθέσιμο στο: https://en.wikipedia.org/wiki/Wireless_sensor_network

Τα WSNs αποτελούν ένα σύνολο από αισθητήρες που επικοινωνούν ασύρματα για τη συλλογή και μετάδοση δεδομένων. Είναι θεμελιώδη στο IoT⁴, καθώς επιτρέπουν τη σύνδεση του φυσικού κόσμου με τον ψηφιακό (σχήμα 4).

- Χαρακτηριστικά:
 - Ασύρματη επικοινωνία μέσω Zigbee, LoRaWAN, ή Bluetooth.
 - Χαμηλή κατανάλωση ενέργειας και αυτονομία.
 - Κατανεμημένη συλλογή δεδομένων.
- Εφαρμογές:
 - Περιβαλλοντική παρακολούθηση (π.χ. θερμοκρασία, υγρασία).
 - Έξυπνη γεωργία (π.χ. αισθητήρες εδάφους).
 - Βιομηχανική παρακολούθηση.

Ταυτοποίηση μέσω Ραδιοσυχνοτήτων (RFID - Radio-Frequency Identification)⁵



Σχήμα 5. Ταυτοποίηση μέσω Ραδιοσυχνοτήτων (RFID - Radio-Frequency Identification)

⁴ Wikipedia, 2025. *Internet of Things*. [Ηλεκτρονικό] Διαθέσιμο στο: https://en.wikipedia.org/wiki/Internet_of_things

⁵ Wikipedia, 2023. *RFID - Radio-Frequency Identification*. [Ηλεκτρονικό] Διαθέσιμο στο: <https://el.wikipedia.org/wiki/RFID>

Η RFID είναι τεχνολογία που χρησιμοποιείται για την αναγνώριση αντικειμένων, ζώων ή ανθρώπων μέσω ραδιοσυχνότητων. Αποτελεί βασικό στοιχείο του IoT, ειδικά σε εφαρμογές παρακολούθησης και ταυτοποίησης (σχήμα 5).

- Τύποι RFID:
 - Παθητικά Tags: Χρησιμοποιούν την ενέργεια του αναγνώστη για τη μετάδοση δεδομένων.
 - Ενεργά Tags: Έχουν δική τους πηγή ενέργειας και προσφέρουν μεγαλύτερη εμβέλεια.
 - Ημι-ενεργά Tags: Συνδυάζουν χαρακτηριστικά των δύο παραπάνω.
- Εφαρμογές:
 - Εφοδιαστική αλυσίδα (logistics).
 - Αυτόματα συστήματα πληρωμών (π.χ. διόδια).
 - Υγειονομική περίθαλψη (π.χ. παρακολούθηση φαρμάκων).

Συνολική Εικόνα του IoT

Οι παραπάνω τεχνολογίες και τύποι δικτύων συνεργάζονται για να δημιουργήσουν το οικοσύστημα του IoT:

- Τα PAN⁶, LAN, LPWAN, WAN, Mesh, και Hybrid Networks υποστηρίζουν την επικοινωνία μεταξύ συσκευών σε διαφορετικές αποστάσεις και εφαρμογές.
- Τα WSNs συλλέγουν δεδομένα από το περιβάλλον, ενώ η RFID παρέχει δυνατότητες ταυτοποίησης.
- Το Cloud Computing λειτουργεί ως το κεντρικό σημείο αποθήκευσης και επεξεργασίας δεδομένων, υποστηρίζοντας την έξυπνη ανάλυση και λήψη αποφάσεων.

⁶ Wikipedia, 2025. *Wireless Network*. [Ηλεκτρονικό] Διαθέσιμο στο: https://en.wikipedia.org/wiki/Wireless_network

Το IoT συνεχίζει να εξελίσσεται, με τις τεχνολογίες αυτές να παίζουν πρωταγωνιστικό ρόλο στην έξυπνη διαχείριση πόρων, την αυτοματοποίηση και τη βελτίωση της καθημερινότητάς μας.

Συχνότητες Επικοινωνίας και Παγκόσμιοι Κανονισμοί

Υπάρχουν οργανισμοί που είναι υπεύθυνοι για τη διαχείριση των συχνοτήτων σε παγκόσμιο επίπεδο. Οι οργανισμοί αυτοί διαθέτουν τις συχνότητες για συγκεκριμένες χρήσεις, καθορίζοντας παράλληλα τα χαρακτηριστικά και τα σχέδια πιστοποίησης. Στις περισσότερες περιοχές, το φάσμα συχνοτήτων που διατίθεται υπόκειται σε υποχρέωση αδειοδότησης, γεγονός που απαιτεί την αγορά άδειας για τη χρήση συγκεκριμένων καναλιών συχνοτήτων. Η **Διεθνής Τηλεπικοινωνιακή Ένωση Ραδιοεπικοινωνιών (ITU-R)** είναι ο κύριος οργανισμός που συντονίζει τη διαμοιρασμένη χρήση του τηλεπικοινωνιακού φάσματος σε παγκόσμιο επίπεδο. Έχουν καθοριστεί ειδικές ζώνες συχνοτήτων για βιομηχανικές, επιστημονικές και ιατρικές εφαρμογές (Industrial, Scientific, Medical - ISM). Οι ζώνες αυτές μπορούν να χρησιμοποιηθούν χωρίς την ανάγκη άδειας, αν και οι κανονισμοί μπορεί να διαφέρουν ελαφρώς ανάλογα με τη χώρα.

Οι πιο κοινές και διαδεδομένες ζώνες συχνοτήτων ISM περιλαμβάνουν:

- **433 MHz**
- **868 MHz**
- **915 MHz**
- **2.4 GHz**
- **5 GHz**

Η ζώνη συχνοτήτων των 2.4 GHz έχει γίνει η πιο διαδεδομένη παγκοσμίως, διότι η χρήση της επιτρέπεται χωρίς την απαίτηση άδειας σε όλες σχεδόν τις χώρες (Nick Lethaby - IoT Ecosystem Manager Texas Instruments, 2017).

2.2.2 Συνδεσιμότητα του IoT

Η δικτύωση αναφέρεται στη σύνδεση δύο ή περισσότερων υπολογιστικών συστημάτων ή συσκευών μεταξύ τους, με κύριο στόχο την ανταλλαγή δεδομένων και πόρων. Μέσω της

δικτύωσης, επιτυγχάνεται η επικοινωνία μεταξύ συσκευών, η κοινή χρήση πληροφοριών και η πρόσβαση σε υπηρεσίες και πόρους μέσω ενός δικτύου (Koosha Paridel, 2010). Ορισμένες βασικές έννοιες και τεχνολογίες που σχετίζονται με τη δικτύωση περιλαμβάνουν:

- **Δίκτυα Υπολογιστών (Computer Networks):**

Αναφέρονται σε συστήματα που συνδέουν υπολογιστές και άλλες συσκευές, επιτρέποντας την ανταλλαγή δεδομένων και πόρων. Τα δίκτυα αυτά μπορεί να είναι ενσύρματα ή ασύρματα.

- **Διαδίκτυο (Internet):**

Πρόκειται για ένα παγκόσμιο δίκτυο υπολογιστών, το οποίο διευκολύνει την επικοινωνία και την ανταλλαγή πληροφοριών μέσω ενός συνόλου κανόνων επικοινωνίας, γνωστών ως πρωτόκολλα.

- **Πρωτόκολλα Επικοινωνίας (Communication Protocols):**

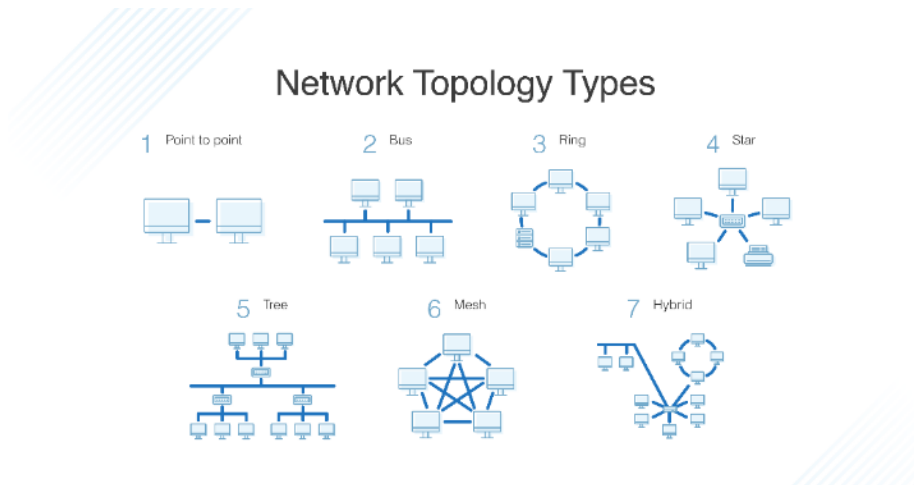
Τα πρωτόκολλα επικοινωνίας αποτελούν κανόνες που καθορίζουν τον τρόπο με τον οποίο διαφορετικές συσκευές συνδέονται, ανταλλάσσουν δεδομένα και διασφαλίζουν την ομαλή λειτουργία των δικτύων.

Τοπολογία Δικτύου⁷ (Network Topology)

Η τοπολογία δικτύου αφορά τον τρόπο με τον οποίο οργανώνονται και συνδέονται μεταξύ τους οι συσκευές μέσα σε ένα δίκτυο. Οι βασικές τοπολογίες περιλαμβάνουν (σχήμα 6):

- **Σημείο-προς-Σημείο (Point to Point):** Αφορά σύνδεση μεταξύ δύο κόμβων.
- **Διαύλου (Bus):** Όλες οι συσκευές συνδέονται σε ένα κοινό κανάλι μεταφοράς.
- **Αστέρα (Star):** Οι συσκευές συνδέονται σε έναν κεντρικό κόμβο.
- **Δακτυλίου (Ring):** Οι συσκευές συνδέονται σε κυκλικό σχήμα.
- **Δέντρου (Tree):** Οι συνδέσεις οργανώνονται σε ιεραρχική δομή.
- **Πλέγματος (Mesh):** Κάθε συσκευή συνδέεται με άλλες πολλές.

⁷ Wikipedia, 2025. Τοπολογία Δικτύου (Network Topology). [Ηλεκτρονικό] Διαθέσιμο στο: https://el.wikipedia.org/wiki/Τοπολογία_δικτύου



Σχήμα 6. Οι τοπολογίες δικτύου.

Δικτύωση Επιχειρήσεων (Enterprise Networking)

Η δικτύωση επιχειρήσεων αναφέρεται στη σύνδεση υπολογιστών και συσκευών ενός οργανισμού με στόχο την εσωτερική επικοινωνία, την ανταλλαγή πληροφοριών και τη διαχείριση επιχειρησιακών διαδικασιών.

Ασφάλεια Δικτύου (Network Security)

Η ασφάλεια δικτύου περιλαμβάνει μέτρα και πρακτικές για την προστασία του δικτύου από απειλές, όπως κακόβουλο λογισμικό (malware) ή μη εξουσιοδοτημένη πρόσβαση.

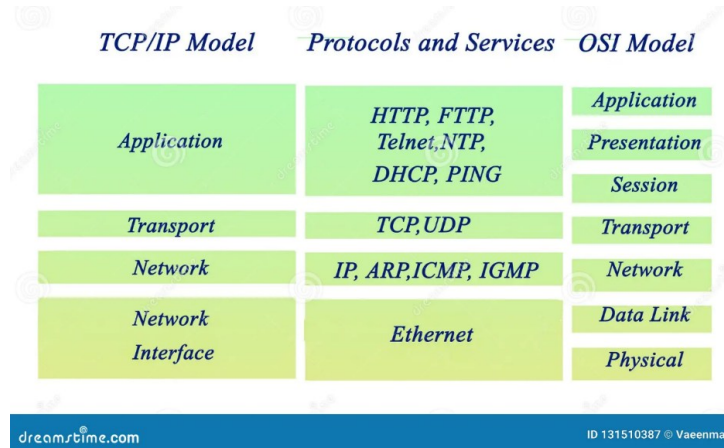
Συστήματα Διαχείρισης Δικτύου (Network Management Systems)

Πρόκειται για εργαλεία που διευκολύνουν τη διαχείριση, την παρακολούθηση και την αποκατάσταση βλαβών στο δίκτυο. Αυτά τα συστήματα είναι απαραίτητα για την εύρυθμη λειτουργία των υποδομών δικτύου (S. Sicari, 2015).

Πρωτόκολλα Επικοινωνίας (GeeksforGeeks, 2024)

Τα πρωτόκολλα επικοινωνίας αποτελούν σύνολα κανόνων που καθορίζουν τη διαδικασία μετάδοσης δεδομένων μεταξύ υπολογιστών και άλλων συσκευών σε ένα δίκτυο. Αυτοί οι κανόνες ορίζουν τον τρόπο αποστολής και λήψης δεδομένων, τον τρόπο κωδικοποίησης των μηνυμάτων και τις διαδικασίες που εξασφαλίζουν την ασφαλή και αξιόπιστη ανταλλαγή πληροφοριών. Είναι ζωτικής σημασίας για τη σωστή λειτουργία ενός δικτύου και την εξασφάλιση της επικοινωνίας μεταξύ συσκευών και εφαρμογών. Ορισμένα από τα σημαντικότερα πρωτόκολλα επικοινωνίας είναι (σχήμα 7):

- **TCP (Transmission Control Protocol):** Το βασικό πρωτόκολλο για τη διασφάλιση αξιόπιστης μεταφοράς δεδομένων στο Διαδίκτυο. Χρησιμοποιείται για τη διαχείριση της μετάδοσης πακέτων δεδομένων, καθώς και για την αντιστοίχιση διευθύνσεων IP και θυρών.
- **HTTP (Hypertext Transfer Protocol):** Εφαρμόζεται στην ανταλλαγή δεδομένων μέσω του Παγκόσμιου Ιστού (World Wide Web) και είναι απαραίτητο για την πρόσβαση σε ιστοσελίδες.
- **FTP (File Transfer Protocol):** Χρησιμοποιείται για τη μεταφορά αρχείων μεταξύ υπολογιστών, παρέχοντας έναν απλό τρόπο ανταλλαγής δεδομένων.
- **SMTP (Simple Mail Transfer Protocol):** Απαραίτητο για τη μεταφορά μηνυμάτων ηλεκτρονικού ταχυδρομείου (e-mails) από μία συσκευή σε άλλη.
- **DNS (Domain Name System):** Εξυπηρετεί τη μετάφραση ονομάτων domain σε διευθύνσεις IP, διευκολύνοντας την πρόσβαση σε ιστοσελίδες μέσω ονόματος αντί αριθμητικής διεύθυνσης.
- **UDP (User Datagram Protocol):** Επιτρέπει τη μεταφορά δεδομένων χωρίς να απαιτείται δημιουργία σύνδεσης, παρέχοντας μεγαλύτερη ταχύτητα, αλλά χαμηλότερη αξιοπιστία σε σύγκριση με το TCP.
- **SSH (Secure Shell):** Εξασφαλίζει την ασφαλή μεταφορά δεδομένων μεταξύ συσκευών, χρησιμοποιείται συχνά για απομακρυσμένη πρόσβαση σε υπολογιστές.
- **DHCP (Dynamic Host Configuration Protocol):** Αυτοματοποιεί τη διαδικασία διαμόρφωσης διευθύνσεων IP και άλλων παραμέτρων δικτύου για τις συσκευές που συνδέονται στο δίκτυο.



Σχήμα 7. Πρωτόκολλα Επικοινωνίας μοντέλο OSI.

Η αρχιτεκτονική του Διαδικτύου σχεδιάστηκε αρχικά ώστε να επιτρέπει τη χρήση λιγότερων από 4,3 δισεκατομμυρίων μοναδικών διευθύνσεων IP. Ωστόσο, η ταχεία ανάπτυξη της τεχνολογίας και του διαδικτύου κατά τις τελευταίες δεκαετίες είχε ως αποτέλεσμα την εξάντληση αυτών των διευθύνσεων μέχρι το τέλος της δεκαετίας του 1980. Το πρόβλημα αυτό είναι γνωστό ως εξάντληση διευθύνσεων IPv4 (Huawei Technologies Co., 2022) (Todd Lammle, 2020). Η τεχνολογία NAT προτάθηκε ως λύση σε αυτό το ζήτημα. Πρόκειται για μία διαδικασία κατά την οποία πολλές συσκευές σε ένα ιδιωτικό δίκτυο χρησιμοποιούν μία κοινή δημόσια IP για την επικοινωνία τους με το Διαδίκτυο, εξοικονομώντας έτσι διευθύνσεις. Παράλληλα, η χρήση της Δρομολόγησης μεταξύ τομέων χωρίς κατηγορίες (Classless Inter-Domain Routing - CIDR) αποτέλεσε ένα ακόμα μέσο διαχείρισης του προβλήματος εξάντλησης διευθύνσεων IP. Οι τεχνολογίες αυτές απέκτησαν ευρεία εφαρμογή στις αρχές της δεκαετίας του 2010, ενώ η μετάβαση σε διευθύνσεις IPv6 συνεχίζεται έως σήμερα.

Internet Protocol Suite

Εισαγωγή

Η σουίτα πρωτοκόλλων διαδικτύου (Internet Protocol Suite), γνωστή και ως TCP/IP, αποτελεί το εννοιολογικό πλαίσιο και το σύνολο πρωτοκόλλων επικοινωνίας που χρησιμοποιούνται στο διαδίκτυο και σε δίκτυα υπολογιστών. Παρέχει από άκρο σε άκρο επικοινωνία δεδομένων, καθορίζοντας τη διαδικασία πακετοποίησης, μετάδοσης και δρομολόγησης των δεδομένων μέσω τεσσάρων επιπέδων: Link, Internet, Transport και Application (σχήμα 8).

Link Layer

Το Link Layer είναι υπεύθυνο για την τοποθέτηση και λήψη πακέτων δεδομένων στο μέσο δικτύου. Είναι ανεξάρτητο από την τεχνολογία δικτύου και προσαρμόζεται σε νέες τεχνολογίες, όπως η λειτουργία ATM.

Internet Layer

Το Internet Layer χειρίζεται τη διεύθυνσιδοτήση, τη συσκευασία και τη δρομολόγηση δεδομένων. Περιλαμβάνει τα πρωτόκολλα IP, ARP, ICMP και IGMP, τα οποία εξυπηρετούν λειτουργίες όπως η ανάλυση διευθύνσεων και η αναφορά σφαλμάτων.

Transport Layer

Το Transport Layer προσφέρει υπηρεσίες επικοινωνίας μέσω των πρωτοκόλλων TCP και UDP. Το TCP διασφαλίζει αξιόπιστη επικοινωνία, ενώ το UDP παρέχει ταχύτερη αλλά μη αξιόπιστη επικοινωνία για εφαρμογές που απαιτούν χαμηλό φορτίο δεδομένων.

Application Layer (GeeksforGeeks, 2024)

Το Application Layer περιλαμβάνει πρωτόκολλα όπως HTTP, FTP, SMTP, DNS, τα οποία επιτρέπουν την επικοινωνία εφαρμογών και τη διαχείριση δεδομένων στο διαδίκτυο.



Σχήμα 8. Πρωτόκολλα Επικοινωνίας οργανωμένα σε ιεραρχικά επίπεδα.

Πρότυπα και Διαλειτουργικότητα

Μία από τις βασικές προκλήσεις στον τομέα της επικοινωνίας συστημάτων είναι η επίτευξη διαλειτουργικότητας μεταξύ συσκευών από διαφορετικούς κατασκευαστές. Διαλειτουργικότητα σημαίνει ότι οι συσκευές μπορούν να ανταλλάσσουν δεδομένα χωρίς δυσκολίες επικοινωνίας, ανεξάρτητα από τον κατασκευαστή τους. Η αντιμετώπιση αυτής της πρόκλησης αποτελεί βασικό στόχο οργανισμών προτύπων, οι οποίοι θέτουν τις απαιτούμενες προδιαγραφές και αναπτύσσουν διαδικασίες δοκιμών. Βάσει του μοντέλου OSI, ορισμένα πρότυπα εστιάζουν σε συγκεκριμένα επίπεδα του δικτύου, ενώ άλλα καλύπτουν ολόκληρη την αρχιτεκτονική (Lethaby, 2017).

Ένας από τους πιο σημαντικούς οργανισμούς στον τομέα αυτό είναι το Ινστιτούτο Ηλεκτρολόγων και Ηλεκτρονικών Μηχανικών (IEEE), το οποίο ιδρύθηκε το 1963 και είναι μη κερδοσκοπικός φορέας. Η σειρά προτύπων IEEE 802.x αποτελεί μία από τις σημαντικότερες συνεισφορές του, με τα εξής κύρια πρότυπα:

- IEEE 802.11: Προδιαγραφές για ασύρματα τοπικά δίκτυα (WLAN), τα οποία αποτελούν τη βάση του Wi-Fi.
- IEEE 802.15.4: Προδιαγραφές για ασύρματα προσωπικά δίκτυα (PAN), τα οποία χρησιμοποιούνται από πρωτόκολλα όπως Zigbee, 6LoWPAN και WirelessHART.

Η σειρά προτύπων 802.x επικεντρώνεται κυρίως στο επίπεδο σύνδεσης δικτύου (Lethaby, 2017), Παράλληλα, η Internet Engineering Task Force (IETF), που ιδρύθηκε το 1986, εστιάζει στη δημιουργία προτύπων για το διαδίκτυο, με ιδιαίτερη έμφαση στη σουίτα πρωτοκόλλων TCP/IP. Οι προδιαγραφές της IETF δημοσιεύονται σε έγγραφα με τον τίτλο "Request for Comments" (RFCs). Αυτά υπόκεινται σε διαδοχικές αναθεωρήσεις έως ότου αποκτήσουν το στάδιο "best current practice". Ορισμένα γνωστά RFCs είναι:

- RFC 791: Περιγραφή του πρωτοκόλλου IPv4.
- RFC 793: Ορισμός του πρωτοκόλλου TCP.
- RFC 2616: Προδιαγραφές του πρωτοκόλλου HTTP/1.1.

(Nick Lethaby - IoT Ecosystem Manager, Texas Instruments, 2017)

Ωστόσο, ούτε το IEEE ούτε η IETF υλοποιούν προγράμματα πιστοποίησης. Αντίθετα, οργανισμοί όπως η Wi-Fi Alliance, η Bluetooth SIG, η Thread Group και η Zigbee Alliance βασίζονται σε αυτά τα πρότυπα για να αναπτύξουν προγράμματα πιστοποίησης. Αυτοί οι

οργανισμοί προσφέρουν τη δυνατότητα στις εταιρείες-μέλη να υποβάλουν τα προϊόντα τους σε ελέγχους διαλειτουργικότητας. Εάν τα προϊόντα περάσουν επιτυχώς τις διαδικασίες αυτές, αποκτούν το δικαίωμα χρήσης των αντίστοιχων λογοτύπων, διασφαλίζοντας τη συμμόρφωση με τα πρότυπα διαλειτουργικότητας (Lethaby, 2017).

Wi-Fi

Η τεχνολογία Wi-Fi, η οποία βασίζεται στο πρότυπο IEEE 802.11, αναπτύχθηκε ως ασύρματη εναλλακτική λύση στο δημοφιλέστερο ενσύρματο πρότυπο Ethernet IEEE 802.3, με κύριο στόχο την προσαρμογή της στη σύνδεση στο διαδίκτυο. Σήμερα, αποτελεί βασικό χαρακτηριστικό όλων των σύγχρονων φορητών υπολογιστών, tablets, smartphones και έξυπνων τηλεοράσεων. Τα περισσότερα δίκτυα Wi-Fi λειτουργούν στη ζώνη ISM των 2.4 MHz, ενώ σε πολλές περιπτώσεις χρησιμοποιείται και η ζώνη των 5 MHz, η οποία προσφέρει περισσότερα διαθέσιμα κανάλια και μεγαλύτερους ρυθμούς μεταφοράς δεδομένων.

Το λογισμικό που απαιτείται για την υποστήριξη Wi-Fi και τη χρήση του πρωτοκόλλου TCP/IP είναι αρκετά εκτεταμένο και πολύπλοκο. Αυτό δεν αποτελεί πρόβλημα για συσκευές όπως φορητοί υπολογιστές και έξυπνα τηλέφωνα, τα οποία διαθέτουν ισχυρούς μικροεπεξεργαστές και μεγάλη μνήμη. Ωστόσο, η ενσωμάτωση Wi-Fi σε συσκευές με περιορισμένη υπολογιστική ισχύ ήταν, μέχρι πρόσφατα, είτε τεχνικά δύσκολη είτε οικονομικά ασύμφορη. Σήμερα, χάρη στους σύγχρονους επεξεργαστές και τις ολοκληρωμένες μονάδες που περιλαμβάνουν το λογισμικό Wi-Fi και TCP/IP, αυτή η πρόκληση έχει ξεπεραστεί.

Τα νεότερα ολοκληρωμένα κυκλώματα Wi-Fi έχουν υιοθετήσει καινοτόμα πρωτόκολλα για τη διαχείριση της κατάστασης αδράνειας, εξασφαλίζοντας γρήγορους χρόνους ενεργοποίησης και απενεργοποίησης. Αυτές οι εξελίξεις έχουν ως στόχο τη σημαντική μείωση της κατανάλωσης ισχύος. Δεδομένου ότι οι συσκευές IoT (Internet of Things) δεν χρειάζονται τους μέγιστους ρυθμούς δεδομένων που μπορεί να προσφέρει το Wi-Fi, η έξυπνη διαχείριση ενέργειας επιτρέπει τη βέλτιστη χρήση των μπαταριών. Με αυτήν τη διαχείριση, συσκευές IoT μπορούν να παραμένουν συνδεδεμένες στο διαδίκτυο για περισσότερο από έναν χρόνο, χρησιμοποιώντας μόνο δύο αλκαλικές μπαταρίες AA (Lethaby, 2017).

Bluetooth

Το Bluetooth, που αναπτύχθηκε από την Ericsson το 1994, σχεδιάστηκε αρχικά ως ασύρματο πρότυπο επικοινωνίας για τη σύνδεση τηλεφώνων και υπολογιστών. Η τεχνολογία αυτή λειτουργεί στη ζώνη ISM των 2,4 GHz και αρχικά ενσωματώθηκε στο πρότυπο IEEE 802.15.1,

το οποίο πλέον δεν διατηρείται. Σήμερα, το Bluetooth Special Interest Group (Bluetooth SIG) έχει αναλάβει τη διαχείριση και την εξέλιξη της τεχνολογίας Bluetooth, καθιστώντας το ένα από τα πιο ευρέως χρησιμοποιούμενα πρότυπα ασύρματης επικοινωνίας (Lethaby, 2017).

Η πρώτη έκδοση του Bluetooth, γνωστή ως «κλασικό Bluetooth», σχεδιάστηκε για μικρές αποστάσεις και αντικατέστησε τη χρήση καλωδίων σε τοπολογίες σημείου-προς-σημείο (point-to-point) και αστέρα (star). Το κλασικό Bluetooth υποστηρίζει κυρίως εφαρμογές όπου απαιτείται συνεχής σύνδεση, όπως τα ασύρματα ακουστικά και οι μεταφορές αρχείων. Με την εισαγωγή του Bluetooth 4.0 το 2010, προστέθηκε το Bluetooth Low Energy (BLE), που αναπτύχθηκε αρχικά από τη Nokia και ενσωματώθηκε από το Bluetooth SIG. Το BLE σχεδιάστηκε ειδικά για εφαρμογές Διαδικτύου των Πραγμάτων (IoT), καθώς επιτρέπει τη χαμηλή κατανάλωση ενέργειας και την υποστήριξη μακροχρόνιας λειτουργίας με τη χρήση μικρών μπαταριών τύπου «coin-cell».

Το Bluetooth 4.2, που κυκλοφόρησε το 2014, έφερε σημαντικές βελτιώσεις στην απόδοση, την εμβέλεια και την ασφάλεια, με τη δυνατότητα αυξημένης ταχύτητας διακίνησης δεδομένων έως και 800 Kbps. Παράλληλα, προστέθηκαν χαρακτηριστικά όπως η βελτιωμένη προστασία ιδιωτικότητας και η υποστήριξη για IPv6 μέσω GATT (Generic Attribute Profile), καθιστώντας δυνατή τη σύνδεση συσκευών BLE στο διαδίκτυο μέσω προφίλ IP. Η αυξημένη χωρητικότητα των πακέτων από 27 σε 251 bytes βελτίωσε περαιτέρω τη διακίνηση δεδομένων, ενώ εισήχθη κρυπτογράφηση βιομηχανικής κλάσης για ενισχυμένη ασφάλεια.

Το πιο πρόσφατο πρότυπο, Bluetooth 5, που κυκλοφόρησε το 2016, παρουσίασε ακόμα μεγαλύτερες δυνατότητες. Τετραπλασίασε την εμβέλεια, διπλασίασε την ταχύτητα μετάδοσης και αύξησε την ικανότητα μετάδοσης δεδομένων κατά οκτώ φορές, κάνοντάς το ιδανικό για εφαρμογές IoT μεγάλης κλίμακας, όπως έξυπνα σπίτια και βιομηχανικές εγκαταστάσεις. Επιπλέον, το Bluetooth 5 υποστηρίζει λειτουργίες αναμετάδοσης και εντοπισμού θέσης, επεκτείνοντας τις εφαρμογές του στον τομέα της υγειονομικής περίθαλψης, της ασφάλειας και των υπηρεσιών logistics.

Η τεχνολογία Bluetooth έχει εξελιχθεί σε ένα αναπόσπαστο εργαλείο για την ασύρματη σύνδεση συσκευών, ενισχύοντας τις δυνατότητες επικοινωνίας τόσο στις καταναλωτικές όσο και στις βιομηχανικές εφαρμογές. Με τη συνεχή ανάπτυξη νέων εκδόσεων, η τεχνολογία παραμένει πρωτοπόρος στην υποστήριξη του οικοσυστήματος IoT.

Zigbee

Το Zigbee είναι ένα ανοιχτό πρότυπο που έχει αναπτυχθεί, υποστηρίζεται και προωθείται από τη Zigbee Alliance, έναν οργανισμό που αποτελείται από περισσότερες από 400 εταιρείες διαφόρων βιομηχανιών. Έχει σχεδιαστεί για να παρέχει μια ολοκληρωμένη λύση συνδεσιμότητας, με έμφαση στη διαλειτουργικότητα των συσκευών και στη δυνατότητα σύνδεσης με υπηρεσίες νέφους (cloud connectivity). Το Zigbee λειτουργεί στη ζώνη ISM στα 2.4 GHz και βασίζεται στο πρότυπο IEEE 802.15.4.

Ένα από τα βασικά πλεονεκτήματα του Zigbee είναι η δυνατότά του να εξασφαλίζει ουσιαστική διαλειτουργικότητα μεταξύ συσκευών διαφορετικών κατασκευαστών. Το πρωτόκολλο περιλαμβάνει μια προκαθορισμένη βιβλιοθήκη τύπων συσκευών, μοντέλων δεδομένων και προτύπων συμπεριφοράς, τα οποία έχουν αναπτυχθεί από έμπειρους κατασκευαστές του χώρου. Παράλληλα, η σουίτα πρωτοκόλλου Zigbee ενσωματώνει λειτουργίες ασφάλειας, δικτύωσης και διαχείρισης συσκευών.

Οι εφαρμογές του Zigbee εστιάζουν κυρίως στους τομείς του έξυπνου σπιτιού και των έξυπνων κτηρίων, όπου χρησιμοποιείται για τη διαχείριση φωτισμού, τον έλεγχο οικιακών συστημάτων και την ενίσχυση της ασφάλειας, προσφέροντας αποδοτικές και ευέλικτες λύσεις.

6LoWPAN

Το 6LoWPAN, το οποίο αποτελεί ακρωνύμιο για τα ασύρματα προσωπικά δίκτυα χαμηλής ισχύος με υποστήριξη IPv6, επιτρέπει σε μικρού μεγέθους συσκευές με χαμηλή κατανάλωση ενέργειας να επικοινωνούν απευθείας με άλλες συσκευές ή διακομιστές που χρησιμοποιούν το πρωτόκολλο IP, όπως μέσω Ethernet ή Wi-Fi. Η επίσημη καθιέρωση του 6LoWPAN πραγματοποιήθηκε το 2011 μέσω του RFC6282.

Το πρότυπο αυτό καθορίζει έναν αποδοτικό μηχανισμό προσαρμογής μεταξύ του επιπέδου σύνδεσης 802.15.4 και μιας στοίβας TCP/IP. Λόγω της ποικιλομορφίας στους ρυθμούς λειτουργίας του επιπέδου 802.15.4, οι λύσεις 6LoWPAN που αναπτύσσονται από διαφορετικούς κατασκευαστές δεν είναι απαραίτητα συμβατές σε τοπικό επίπεδο δικτύου. Οι εφαρμογές του 6LoWPAN βρίσκουν χρήση στις ζώνες ISM των 2.4 GHz, 868 MHz και 915 MHz.

Μεταξύ των πλεονεκτημάτων του 6LoWPAN περιλαμβάνονται χαρακτηριστικά όπως η υποστήριξη τοπολογίας πλέγματος, η δυνατότητα μεγάλου μεγέθους δικτύου, η αξιόπιστη

επικοινωνία και η χαμηλή κατανάλωση ενέργειας. Επιπλέον, το 6LoWPAN αξιοποιεί τα οφέλη της επικοινωνίας μέσω του πρωτοκόλλου IP, καθιστώντας το ιδανικό για εφαρμογές που απαιτούν ευελιξία και ελαχιστοποίηση της ενεργειακής κατανάλωσης.

Thread

Η Google, η Samsung και αρκετοί κατασκευαστές ολοκληρωμένων κυκλωμάτων, που στόχευαν στη σύνδεση κόμβων IoT χαμηλής ενεργειακής κατανάλωσης απευθείας σε δίκτυα IP, δημιούργησαν την ομάδα Thread Group. Ο κύριος στόχος της ομάδας ήταν η ανάπτυξη ενός προτύπου που να εξασφαλίζει τη διαλειτουργικότητα στον τομέα των εφαρμογών έξυπνου σπιτιού.

Το πρωτόκολλο Thread είναι βασισμένο στα χαρακτηριστικά του προτύπου IEEE 802.15.4 του 2006 και λειτουργεί στη ζώνη ISM των 2.4 GHz, όπως και άλλες τεχνολογίες όπως το Wi-Fi, το Bluetooth και το Zigbee. Το Thread προσφέρει ταχύτητα μεταφοράς δεδομένων έως 250 Kbps και υποστηρίζει μέχρι 250 συσκευές σε ένα τοπικό δίκτυο πλέγματος.

Αξιοποιώντας τα πλεονεκτήματα των προτύπων 802.15.4 και 6LoWPAN, όπως η χαμηλή κατανάλωση ενέργειας, η διευθυνσιοδότηση IPv6 και η δυνατότητα δημιουργίας δικτύου πλέγματος, το Thread αποτελεί μια ιδανική λύση για εφαρμογές IoT στον τομέα του έξυπνου σπιτιού. (Nick Lethaby - IoT Ecosystem Manager Texas Instruments, 2017)

WirelessHART

Το WirelessHART και το Wireless IO Link είναι πρωτόκολλα που λειτουργούν στη ζώνη ISM των 2.4 GHz και έχουν σχεδιαστεί με έμφαση στη χρήση τους σε βιομηχανικά περιβάλλοντα, ιδιαίτερα σε περιπτώσεις με αυξημένα επίπεδα ηλεκτρομαγνητικών παρεμβολών. Και τα δύο πρωτόκολλα απαιτούν την ύπαρξη πυλών για τη σύνδεσή τους με δίκτυα IP.

Το Wireless IO Link έχει περιορισμένη εμβέλεια, φτάνοντας μέχρι τα 20 μέτρα, και επικεντρώνεται στη χαμηλή καθυστέρηση, καθώς οι εφαρμογές που το χρησιμοποιούν, όπως οι αισθητήρες και οι ενεργοποιητές, απαιτούν ταχεία απόκριση. Αντίθετα, το WirelessHART, το οποίο βασίζεται στο πρότυπο 802.15.4, υποστηρίζει τοπολογίες τόσο αστέρα όσο και πλέγματος, γεγονός που το καθιστά κατάλληλο για εφαρμογές όπου απαιτείται υψηλή αξιοπιστία και ευελιξία στη δικτύωση. (Nick Lethaby - IoT Ecosystem Manager Texas Instruments, 2017)

Sub-1 GHz

Στις περισσότερες χώρες, πέρα από την ευρέως γνωστή ζώνη ISM των 2.4 GHz, υπάρχουν διαθέσιμες ζώνες χαμηλότερων συχνοτήτων, όπως τα 433 MHz, 868 MHz και 915 MHz, οι οποίες ονομάζονται Sub-1 GHz. Οι ζώνες αυτές χρησιμοποιούνται σε διάφορες βιομηχανικές εφαρμογές, όπου αξιοποιούνται ιδιόκτητα πρωτόκολλα που λειτουργούν σε ασύρματα συστήματα αυτών των συχνοτήτων. Για την ενσωμάτωση των συστημάτων Sub-1 GHz στο IoT, απαιτείται η χρήση εφαρμογών μέσω πύλης διαδικτύου. Οι συγκεκριμένες λύσεις ξεχωρίζουν για την ιδιαίτερα χαμηλή κατανάλωση ισχύος και τη μεγάλη εμβέλεια που προσφέρουν, ενώ εφαρμόζουν μια απλή τοπολογία αστέρα. (Nick Lethaby - IoT Ecosystem Manager Texas Instruments, 2017)

NFC

Η Επικοινωνία Κοντινού Πεδίου (NFC - Near Field Communication) επιτρέπει την ασύρματη και ανέπαφη επικοινωνία μεταξύ συσκευών. Αυτή η τεχνολογία λειτουργεί δημιουργώντας ένα ρεύμα ραδιοσυχνοτήτων, μέσω του οποίου μια συσκευή, όπως ένας αναγνώστης ή μια ενεργή συσκευή, μπορεί να συνδεθεί με άλλη συμβατή συσκευή NFC ή με μια μικρή ετικέτα NFC που περιέχει τις απαραίτητες πληροφορίες (Madlmayr, Langer, Kantner and Scharinger, 2015)

Το NFC υποστηρίζει δύο βασικούς τύπους συσκευών. Οι παθητικές συσκευές, όπως οι ετικέτες, περιλαμβάνουν δεδομένα που μπορούν να αναγνωστούν από έναν αναγνώστη, ενώ οι ενεργές συσκευές επιτρέπουν αμφίδρομη επικοινωνία (peer-to-peer) ανάμεσα σε δύο NFC συσκευές, διευκολύνοντας την ανταλλαγή δεδομένων. (NearFieldCommunication Organization, 2017)

2.2.3 Εφαρμογές του IoT στην καθημερινότητα

Τομείς Εφαρμογών IoT (Μαλούτα, & Μίχου, 2019)

Οι τομείς όπου μπορεί να βρει εφαρμογή το Internet of Things (IoT) περιλαμβάνουν τις παρακάτω κατηγορίες:

- **Υγεία:** Το IoT χρησιμοποιείται για την παρακολούθηση της κατάστασης υγείας ασθενών, επιτρέποντας γρηγορότερη και πιο αποτελεσματική θεραπεία. Επιπλέον, διευκολύνει τη δημιουργία λεπτομερών ιατρικών προφίλ ασθενών και την απομακρυσμένη διαχείριση της υγείας.

- **Βιομηχανία:** Στον βιομηχανικό τομέα, το IoT βοηθά στη διαχείριση των γραμμών παραγωγής, στη βελτίωση της ασφάλειας του προσωπικού και στην εξοικονόμηση ενέργειας, αυξάνοντας τη συνολική αποδοτικότητα.
- **Μεταφορές:** Περιλαμβάνει την παρακολούθηση της θέσης και της κατάστασης οχημάτων, καθώς και της ταχύτητας. Επίσης, συμβάλλει στη διαχείριση της υγείας των οδηγών και της σωστής λειτουργίας του οχήματος.
- **Γεωργία:** Συμβάλλει στη βελτιστοποίηση της άρδευσης και της λίπανσης των καλλιεργειών, ενώ διευκολύνει την παρακολούθηση κρίσιμων παραμέτρων, όπως η θερμοκρασία και η υγρασία, ειδικά σε θερμοκήπια. Επιπλέον, το IoT μπορεί να εφαρμοστεί για την ανίχνευση και την αντιμετώπιση ασθενειών που επηρεάζουν τα φυτά.
- **Έξυπνα σπίτια:** Στην κατηγορία αυτή περιλαμβάνονται εφαρμογές όπως η ρύθμιση της θερμοκρασίας και της υγρασίας, ο έξυπνος φωτισμός, καθώς και ο έλεγχος φίτρων αέρα ή ιονιστών. Επίσης, χρησιμοποιείται για απομακρυσμένο έλεγχο οικιακών συσκευών.
- **Έξυπνες πόλεις:** Βελτιώνει τη διαχείριση του φωτισμού, την καθαριότητα και την αποκομιδή απορριμμάτων, προσφέροντας βελτιστοποιημένες λύσεις για τον αστικό σχεδιασμό.
- **Ενέργεια:** Χρησιμοποιείται για τη συλλογή δεδομένων από τους καταναλωτές, με στόχο τη βελτιστοποίηση της διανομής και της παραγωγής ενέργειας. Επιπλέον, βοηθά στον εντοπισμό διαρροών και άλλων απωλειών.
- **Έξυπνα γυμναστήρια :** Στον τομέα των έξυπνων γυμναστηρίων, το IoT φέρνει επανάσταση στον τρόπο που οι χρήστες αθλούνται και διαχειρίζονται την υγεία και την φυσική τους κατάσταση. Οι συσκευές IoT συνδυάζουν τεχνολογία, δεδομένα και αυτοματισμούς για να προσφέρουν εξατομικευμένες και αποτελεσματικές εμπειρίες άσκησης.

Συμπεράσματα

Όπως αναλύθηκε προηγουμένως, υπάρχουν πολυάριθμες τεχνολογίες ασύρματης επικοινωνίας, καθεμία από τις οποίες διαθέτει συγκεκριμένα πλεονεκτήματα και

μειονεκτήματα. Η επιλογή της καταλληλότερης τεχνολογίας εξαρτάται από τις ιδιαίτερες απαιτήσεις κάθε εφαρμογής. Αυτή η επιλογή πρέπει να λαμβάνει υπόψη παράγοντες, όπως η πολυπλοκότητα, η συνδεσιμότητα, η διαλειτουργικότητα, η εμβέλεια, η κατανάλωση ενέργειας και το επίπεδο ασφάλειας που χρειάζεται να διασφαλιστεί, ανάλογα με τις προδιαγραφές της εφαρμογής.

Σχέση με τον Κανονισμό GDPR

Ο Γενικός Κανονισμός Προστασίας Δεδομένων της ΕΕ στοχεύει στη διεύρυνση της προστασίας των δεδομένων στην εποχή των big data και του cloud computing, εξασφαλίζοντας ότι η προστασία των δεδομένων αποτελεί θεμελιώδες βασικό δικαίωμα, το οποίο θα ρυθμίζεται με συνέπεια σε όλη την Ευρώπη. Στόχος του είναι να διευκολύνει τη ροή δεδομένων προσωπικού χαρακτήρα σε όλα τα 28 κράτη μέλη της ΕΕ ενδυναμώνοντας παράλληλα τα θεμελιώδη δικαιώματα και τις ελευθερίες των φυσικών προσώπων καθώς και το νομικό πλαίσιο σε όλα τα κράτη – μέλη. Διαθέσιμο: <https://www.addicted.gr/gdpr-kanonismos-ti-einai-kai-pos-tha-epireasei-tin-epicheirisi-sas/#more-3302>

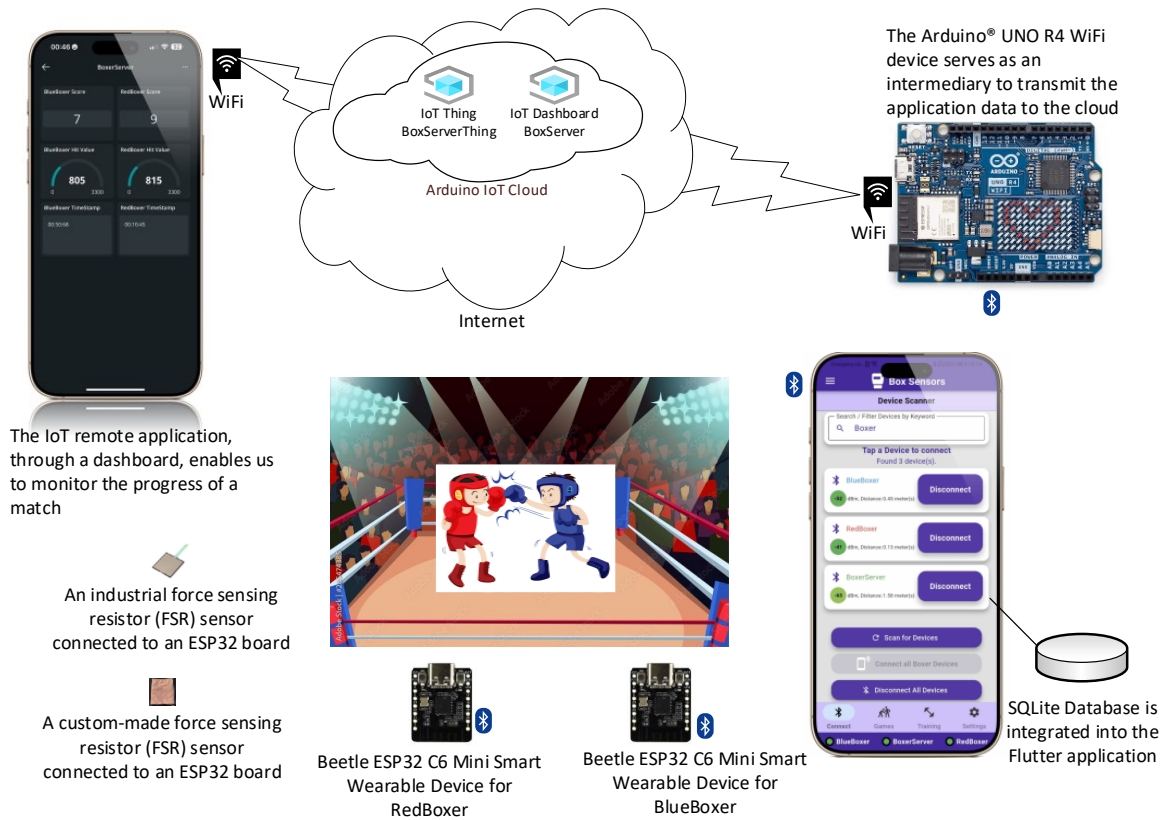
3. Υλοποίηση συστήματος

3.1 Πλακέτες ανάπτυξης ESP32 και Arduino uno R4 Wi-Fi

Υλοποίηση συστήματος.

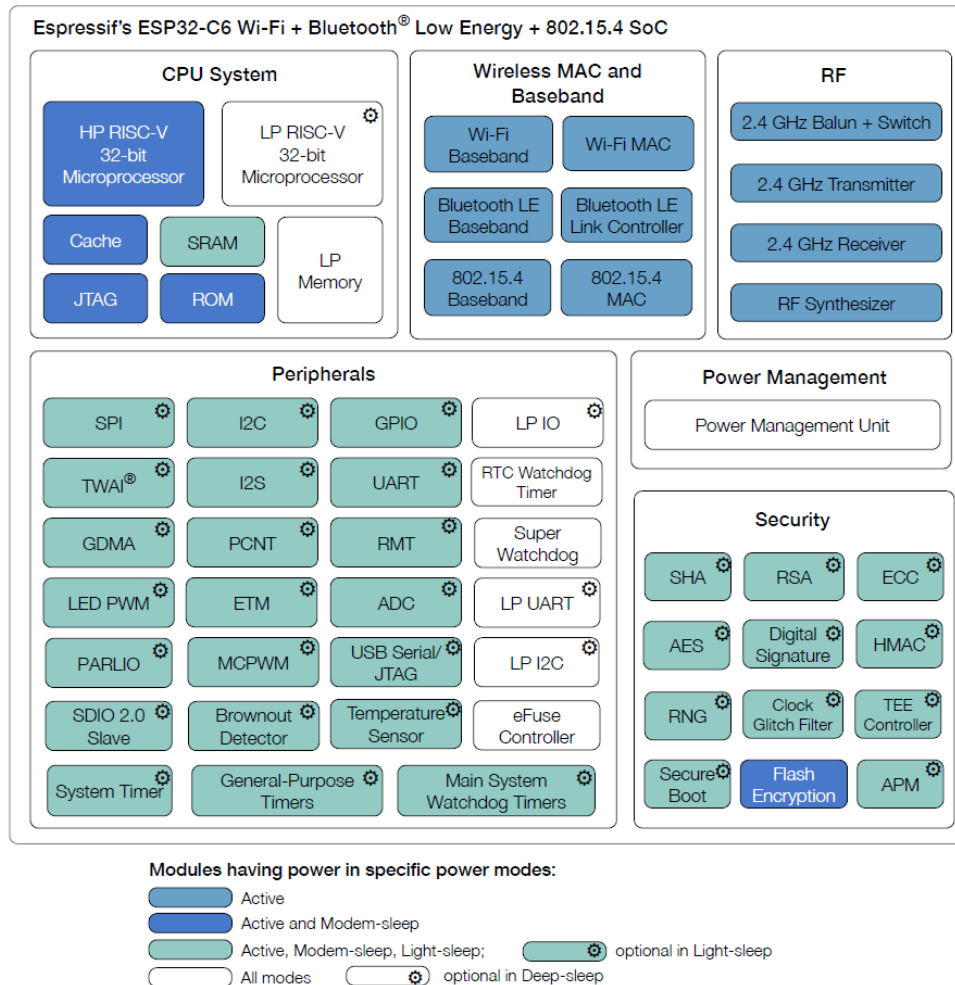
Για να μπορέσουμε να υλοποιήσουμε το σύστημα καταμέτρησης χτυπημάτων κατά τη διάρκεια ενός πυγμαχικού αγώνα ή μιας προπόνησης θα χρησιμοποιήσουμε διάφορες συσκευές, οι οποίες θα μας επιτρέψουν να καταγράψουμε την απαραίτητη πληροφορία. Η βασική συσκευή που μπορεί να καταγράφει τα χτυπήματα ενός πυγμάχου είναι μια φορητή συσκευή ESP32 (μικροελεγκτής) που έχει συνδεδεμένο έναν ή πολλούς αισθητήρες καταγραφής χτυπημάτων, που για τη δική μας περίπτωση είναι πιεζοηλεκτρικοί αισθητήρες FSR. Για το σύστημα μας χρησιμοποιούμε δύο μικροελεγκτές που ονομάζονται Blue Boxer και Red Boxer. Όταν ο μικροελεγκτής ESP32 λαμβάνει πληροφορίες χτυπημάτων τις στέλνει μέσω σύνδεσης Bluetooth σε ένα κινητό τηλέφωνο τύπου Android που έχει εγκατεστημένη την εφαρμογή Box Sensors την οποία κατασκευάσαμε και θα παρουσιάσουμε σε επόμενο κεφάλαιο αναλυτικά. Η συγκεκριμένη εφαρμογή λαμβάνει την πληροφορία που στέλνουν οι μικροελεγκτές ESP32 είτε πρόκειται για αγώνα πυγμαχίας ή για προπόνηση ενός πυγμάχου. Την πληροφορία την αποθηκεύει σε μια βάση δεδομένων SQLite που βρίσκεται ενσωματωμένη στην εφαρμογή. Εκεί αποθηκεύεται η πληροφορία για τον αγώνα, τους γύρους τη διάρκεια των γύρων και του διαλείμματος μεταξύ τους. Αφού ολοκληρωθεί ένας αγώνας μπορεί ο χρήστης της εφαρμογής να ανατρέξει και να δει τις λεπτομέρειες και τα χτυπήματα του αγώνα ανά γύρο. Η εφαρμογή αναλαμβάνει να στείλει την πληροφορία που περιέχει τα χτυπήματα των αθλητών και σε έναν άλλον μικροελεγκτή τον Arduino Uno R4 Wi-Fi μέσω σύνδεσης Bluetooth, τον ονομάζουμε Boxer Server. Το Arduino αναλαμβάνει να στείλει τα μηνύματα στο Arduino IoT Cloud μέσω σύνδεσης Wi-Fi. Εκεί παρουσιάζονται σε έναν πίνακα ανακοινώσεων(Dashboard). Αν το Arduino δεν είναι συνδεδεμένο δεν πηγαίνουν στον πίνακα ανακοινώσεων(Dashboard) τα χτυπήματα. Αυτό το ελέγχει αυτόματα η εφαρμογή του κινητού ώστε να μην στέλνονται άσκοπα τα χτυπήματα όταν δεν υπάρχει σύνδεση μεταξύ της εφαρμογής του κινητού και του μικροελεγκτή Arduino Uno R4 Wi-Fi. Με αυτό τον τρόπο δίνεται η δυνατότητα στους διαχειριστές του συστήματος να χρησιμοποιήσουν ή όχι την σύνδεση με το Arduino IoT Cloud γιατί υπάρχει περίπτωση τα δεδομένα να θεωρούνται κρίσιμα. Το όλο σύστημα μπορεί να επεκταθεί μελλοντικά με διάφορες νέες ιδέες, να

προστεθούν νέοι αισθητήρες με διαφορετικές δυνατότητες που προσφέρουν επιπρόσθετη πληροφορία για τους αθλητές (σχήμα 9).



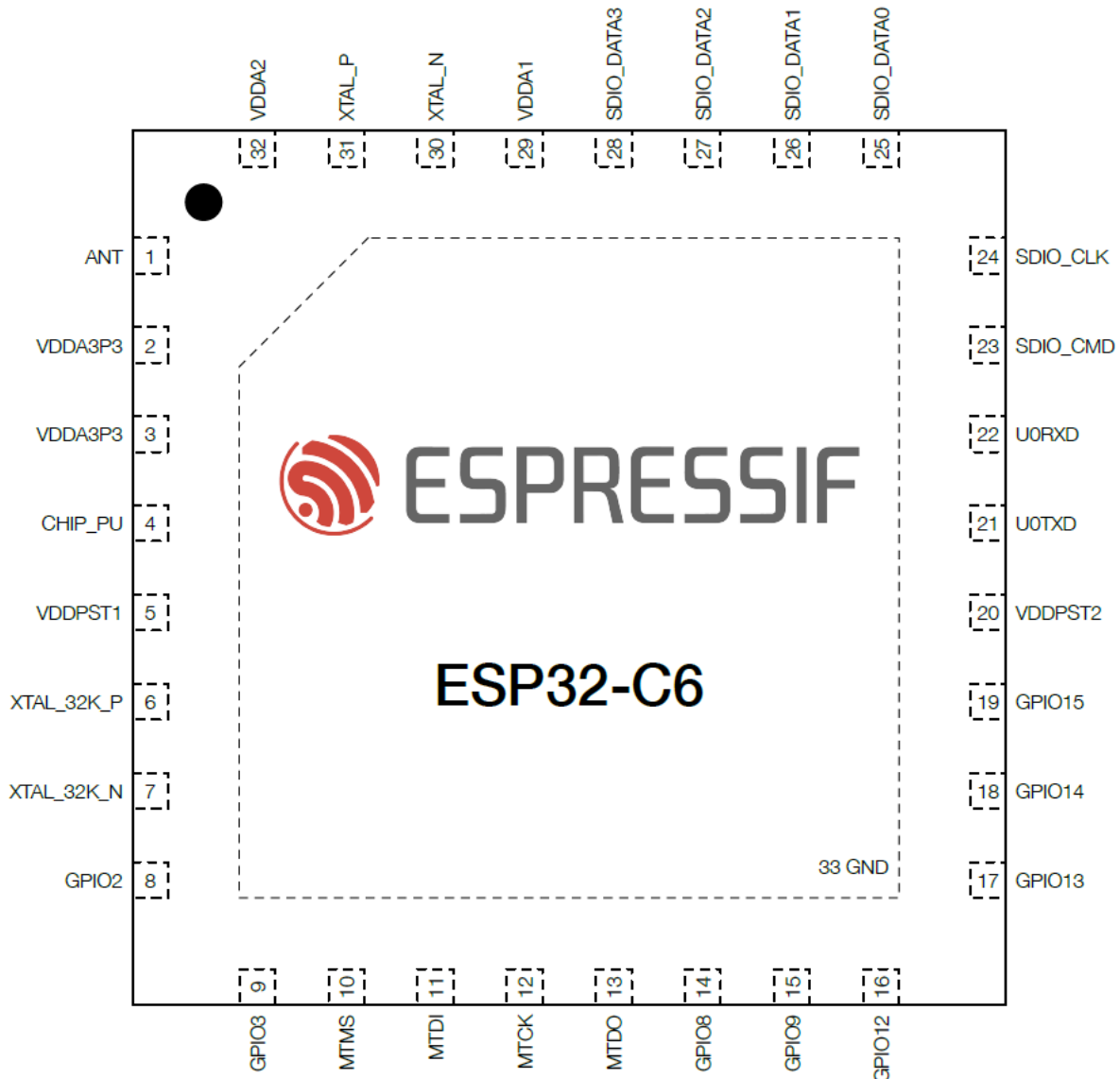
Σχήμα 9. Υλοποίηση συστήματος.

ESPRESSIF ESP32-C6⁸



Σχήμα 10. ESP32-C6 Functional Block Diagram (Διάγραμμα Λειτουργιών ESP32-C6)

⁸ <https://dfimg.dfrobot.com/60c1e008bddfc41c3293de80/wiki/bd5febb16dd4b86eb34cb387815080ae.pdf>

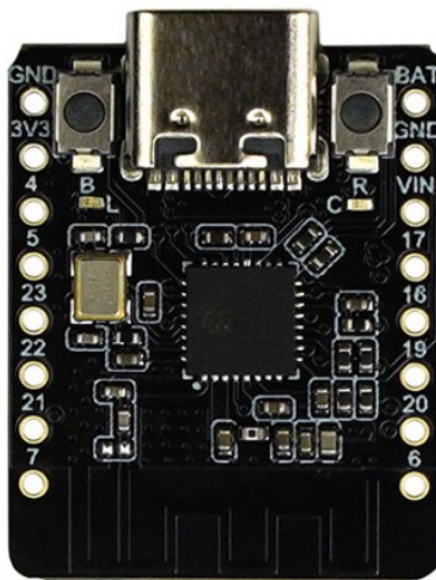


Σχήμα 11. Διάταξη ακροδεκτών ESP32-C627 (QFN32, επάνω όψη).

Το Beetle ESP32-C6⁹ (σχήμα 10, 11). αποτελεί μια εξαιρετικά μικρή πλακέτα ανάπτυξης για εφαρμογές Arduino IoT, η οποία βασίζεται στο τσιπ ESP32-C6 (σχήμα 12). Χαρακτηρίζεται από την πολύ χαμηλή κατανάλωση ενέργειας και έχει διαστάσεις περίπου ίδιες με αυτές ενός νομίσματος (25 x 20,5 mm). Ο επεξεργαστής RISC-V 32-bit με 160 MHz του εξασφαλίζει υψηλές επιδόσεις, ενώ υποστηρίζει πρωτόκολλα όπως Wi-Fi 6, Bluetooth 5, Zigbee 3.0 και Thread 1.3, επιτρέποντάς του να ενσωματώνεται εύκολα σε διάφορα δίκτυα IoT. Επιπλέον, η ενσωματωμένη μονάδα διαχείρισης φόρτισης μπαταριών λιθίου εξαλείφει την ανάγκη για

⁹ https://wiki.dfrobot.com/SKU_DFR1117_Beetle_ESP32_C6

επιπλέον εξαρτήματα, συμβάλλοντας σε μια ιδιαίτερα συμπαγή σχεδίαση. Επιλέχτηκε ως πρότυπο για την πτυχιακή γιατί είναι το μοναδικό που υπάρχει στην αγορά μικροελεγκτών που παρέχει μικρές διαστάσεις και ενσωματωμένο κύκλωμα διαχείρισης φόρτισης μπαταρίας ώστε να μπορεί να φορεθεί χωρίς να εμποδίζει στις κινήσεις του. Στην πτυχιακή εργασία χρησιμοποιώ το πρωτόκολλο Bluetooth για να επικοινωνήσει ο μικροελεγκτής Beetle ESP32-C6 με την εφαρμογή του κινητού που δημιούργησα. Διαμέσου του πρωτόκολλου Bluetooth μεταφέρετε η πληροφορία από τους αισθητήρες στην εφαρμογή του κινητού και αποθηκεύεται σε με μια βάση δεδομένων.



Σχήμα 12. Ο μικροελεγκτής Beetle ESP32-C6.

Παρά το μικροσκοπικό του μέγεθος που συγκρίνεται με ένα νόμισμα, το Beetle ESP32-C6 διαθέτει έως και 13 θύρες εισόδου/εξόδου, εξαλείφοντας κάθε ανησυχία για πιθανή έλλειψη IO κατά την υλοποίηση έργων. Ενσωματώνει επίσης κύκλωμα για την ασφαλή φόρτιση μπαταριών λιθίου και υποστηρίζει την παρακολούθηση της τάσης, επιτρέποντας τη λήψη κατάλληλων μέτρων όταν το ενεργειακό επίπεδο δεν επαρκεί για τη συνεχή λειτουργία της συσκευής. Το Beetle ESP32-C6 είναι συμβατό με τα πρωτόκολλα Wi-Fi και Thread, καθιστώντας το κατάλληλο για την υλοποίηση συσκευών τελικής χρήσης Matter είτε μέσω Wi-Fi είτε μέσω Thread, εξασφαλίζοντας ομαλή επικοινωνία και συνεργασία μεταξύ έξυπνων οικιακών συσκευών ανεξαρτήτως συστήματος ή πλατφόρμας. Επιπλέον, υποστηρίζει τα πρωτόκολλα BLE και Zigbee, ενώ σε συνδυασμό με άλλα MCU μπορεί να λειτουργήσει ως δρομολογητής ορίων για το Thread, πύλη Matter και γέφυρα Zigbee (σχήμα 13).



Σχήμα 13. Τα πρωτόκολλα επικοινωνίας που υποστηρίζει ο μικροελεγκτής Beetle ESP32-C6.

Το ESP32-C6 αποτελεί το πρώτο τσιπ της Espressif που ενσωματώνει το πρωτόκολλο Wi-Fi 6 (802.11ax), επιτρέποντας μια βελτιωμένη διαχείριση του δικτύου με υψηλή απόδοση και ελάχιστη καθυστέρηση. Επιπλέον, η ενσωματωμένη τεχνολογία Target Wake Time (TWT) συμβάλλει ουσιαστικά στη μείωση της κατανάλωσης ενέργειας, παρατείνοντας έτσι τη διάρκεια ζωής της μπαταρίας και εξασφαλίζοντας αξιόπιστη μακροχρόνια λειτουργία της συσκευής.

Χαρακτηριστικά

- Διαθέτοντας εξαιρετικά συμπαγές μέγεθος, οι διαστάσεις είναι απλώς 25*20,5 mm.
- Εξοπλισμένο με το τσιπ ESP32-C6, υποστηρίζει πρωτόκολλα επικοινωνίας Wi-Fi, BLE, Zigbee και Thread.
- Υποστηρίζει το πρωτόκολλο Wi-Fi 6, εξασφαλίζοντας χαμηλότερη καθυστέρηση και μειωμένη κατανάλωση ενέργειας.
- Εξαιρετικά χαμηλή κατανάλωση ενέργειας
 - Έκδοση 1.0: Ρεύμα βαθύ ύπνου 14μΑ
 - Έκδοση 1.1: Ρεύμα βαθύ ύπνου 37μΑ
- Ενσωματώνει λειτουργία φόρτισης μπαταρίας λιθίου.
- Υποστηρίζει ανίχνευση τάσης μπαταρίας, παρέχοντας πληροφορίες για τα επίπεδα ισχύος της συσκευής.

Πιθανά σενάρια εφαρμογής

- Έργα έξυπνου φωτισμού.
- Έξυπνα έργα περιβαλλοντικής παρακολούθησης.
- Έργα έξυπνου διακόπτη.

Προδιαγραφές

Βασικές Παράμετροι

- Τάση λειτουργίας: 3,3V
- Τάση εισόδου Type-C: 5V DC
- Μέγιστο ρεύμα φόρτισης: 0,5A
- Ρεύμα αδράνειας: (Λειτουργία βαθιάς αδράνειας, τροφοδοτείται από μπαταρία)
 - Έκδοση 1.0: 14μΑ
 - Έκδοση 1.1: 37μΑ
- Θερμοκρασία λειτουργίας: -10~60°C
- Διαστάσεις: 20,5x25mm

Πληροφορίες υλικού

- Επεξεργαστής: RISC-V μονοπύρηνος επεξεργαστής
- Κύρια συχνότητα: 160 MHz
- SRAM: 512 KB
- ROM: 320 KB
- Flash: 4MB
- RTC SRAM: 16 KB
- USB: USB 2.0 CDC

WIFI

- Πρωτόκολλο WIFI: IEEE 802.11b/g/n
 - IEEE 802.11ax (λειτουργία μη AP μόνο για 20 MHz)
- Εύρος ζώνης: Υποστήριξη 20 MHz και 40 MHz στη ζώνη 2,4 GHz
- Λειτουργία WIFI: Σταθμός, SoftAP, συνδυασμένη λειτουργία SoftAP+Station
- Συχνότητα WIFI: 2,4 GHz
- Συνάθροιση πλαισίων: TX/RX A-MPDU, TX/RX A-MSDU

Bluetooth

- Πρωτόκολλο Bluetooth: Bluetooth 5, πλέγμα Bluetooth
- Συχνότητα Bluetooth: 125 Kbps, 500 Kbps, 1 Mbps, 2 Mbps

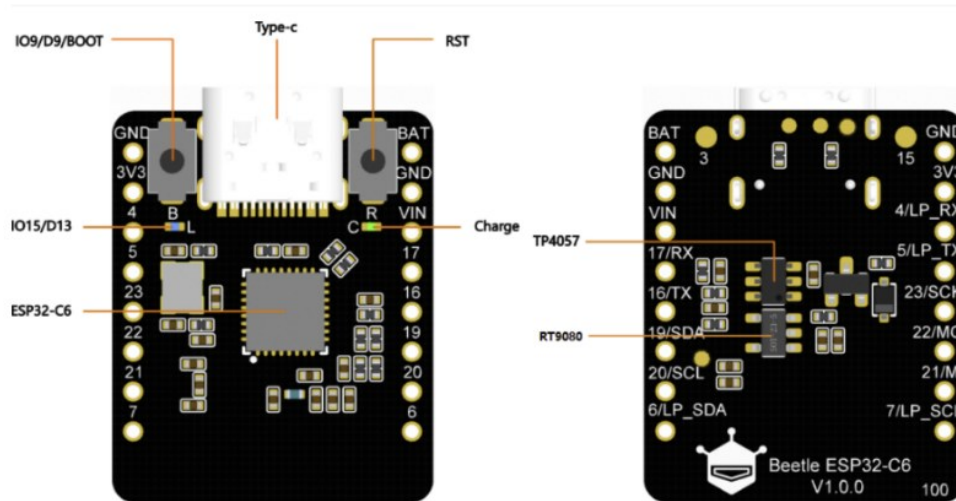
IEEE 802.15.4

- Συμβατό με το πρωτόκολλο IEEE 802.15.4-2015
- Ζώνη συχνοτήτων: 2,4 GHz
- Ρυθμός δεδομένων: 250 Kbps
- Υποστηρίζει Thread 1.3 και Zigbee 3.0

Θύρες εισόδου εξόδου

- Ψηφιακές I/O x13
- LED PWM 6 καναλιών
- SPI x1
- UART x3 (LP UART x1)
- I2C x2 (LP I2C x1)
- I2S x1
- Πομποδέκτης υπέρυθρων: κανάλι εκπομπής x5, κανάλι λήψης x5
- 1 × 12-bit SAR ADC, 7 κανάλια
- Ελεγκτής DMA: μετάδοση καναλιού x3, λήψη καναλιού x3

Γενική παρουσίαση της πλακέτας¹⁰ (σχήμα 14)



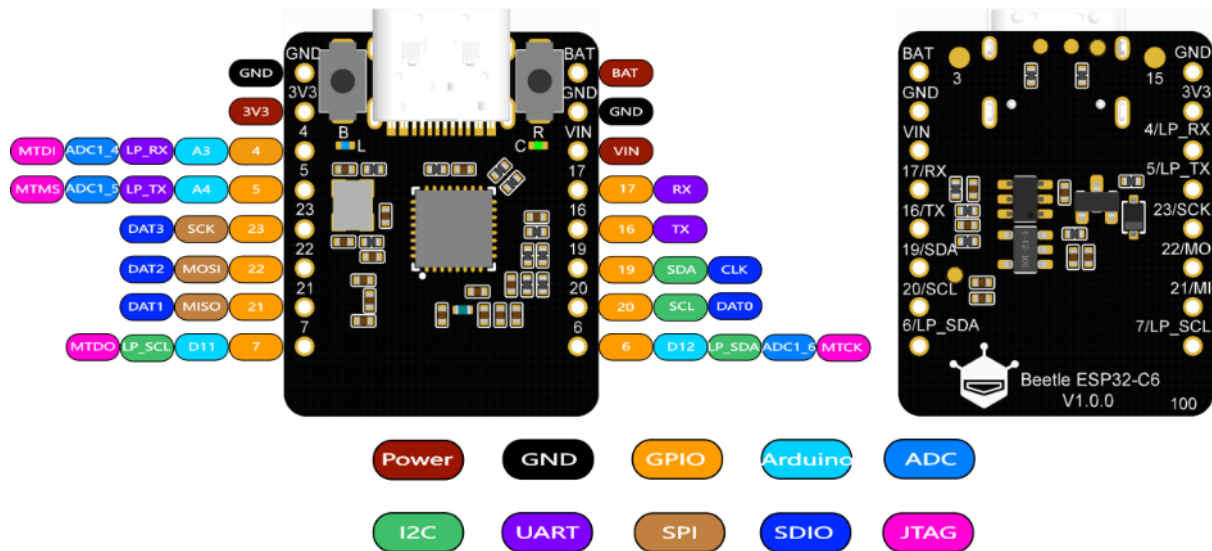
Σχήμα 14. Παρουσίαση της πλακέτας.

- **Type-C:** Θύρα USB Type-C
- **IO15/D13:** Ενσωματωμένο LED.
- **Φόρτιση:** Ένδειξη φόρτισης
 - Σβηστό: δεν είναι συνδεδεμένο στην πρίζα ή πλήρως φορτισμένο
 - Ενεργό: φόρτιση
 - Αναβοσβήνει: Η μπαταρία δεν είναι συνδεδεμένη
- **RST:** Κουμπί επαναφοράς
- **IO9/D9/BOOT:** GPIO9 / Κουμπί εκκίνησης
- **ESP32-C6:** Τσιπ ESP32-C6FH4
- **TP4057:** Τσιπ διαχείρισης φόρτισης μπαταρίας λιθίου TP4057
- **RT9080¹¹:** 3,3 V LDO χαμηλής ισχύος (V1.0)
- **TPS62A02DRLR:** 3,3V DCDC (V1.1)

¹⁰ <https://dfimg.dfrobot.com/60c1e008bddfc41c3293de80/wiki/68dbe234c15e4e218d4c9b54a674ea61.pdf>

¹¹ <https://dfimg.dfrobot.com/5d57611a3416442fa39bffc/wiki/d310c343a276135955547d238c122064.pdf>

Διάγραμμα ακροδεκτών¹²

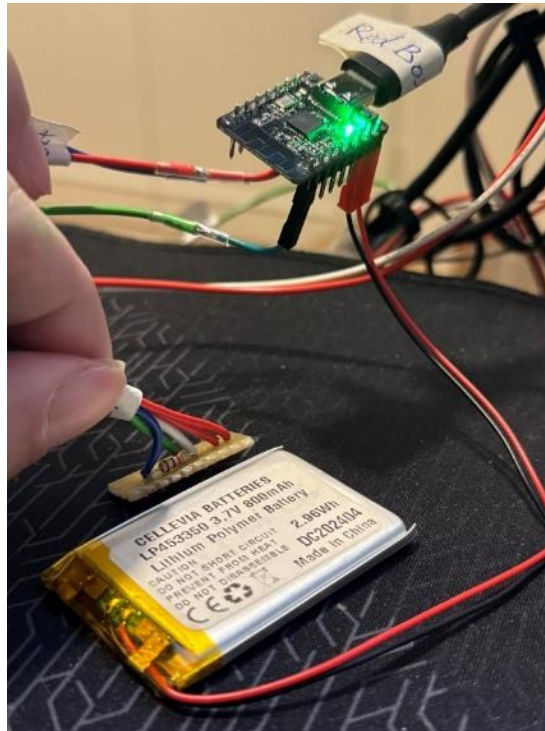


Σχήμα 15. Διάγραμμα ακροδεκτών.

Περιγραφή ακροδεκτών (σχήμα 15)

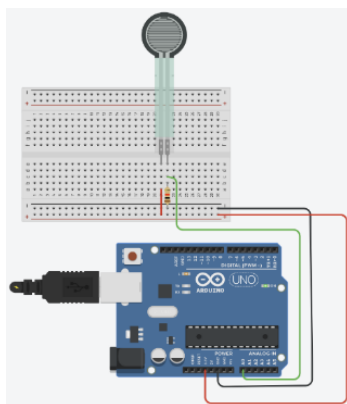
- Τροφοδοσία: Πείρο τροφοδοσίας
 - VIN: 5V DC
 - 3V3: Σταθερή έξοδος 3,3V
- GND: πείρος κοινής γείωσης
- GPIO: ESP32 προεπιλεγμένος αριθμός GPIO
- Arduino: Αντιστοίχιση GPIO του Beetle ESP32-C6 στο Arduino
- ADC: ESP32 προεπιλεγμένη ακίδα μετατροπής αναλογικού σε ψηφιακό
- I2C: Διεπαφή I2C
 - Χαρτογράφηση Beetle ESP32-C6 I2C στο Arduino
 - LP_SDA/SCL: Καρφίτσα I2C χαμηλής ισχύος
- UART: Διεπαφή UART
 - LP_TX/RX: Καρφίτσα UART χαμηλής ισχύος
- SPI: Αντιστοίχιση SPI του Beetle ESP32-C6 στο Arduino
- SDIO: προεπιλεγμένη καρφίτσα SDIO ESP32
- JTAG: Διεπαφή εντοπισμού σφαλμάτων

¹²<https://dfimg.dfrobot.com/60c1e008bddfc41c3293de80/wiki/b6ccaa7b1c039a85403ae8ec6a33c98a.png>

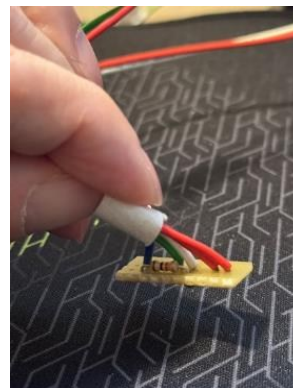


Εικόνα 1. Ο μικροελεγκτής Beetle ESP32-C6.

Στην παραπάνω (εικόνα 1) βλέπουμε το μικροελεγκτή, την μπαταρία λιθίου καθώς τη φορτίζει ο μικροελεγκτής Beetle ESP32-C6 από το κύκλωμα διαχείρισης φόρτισης TP4057. Στην παρακάτω (σχήμα 16) παρουσιάζεται ο τρόπος διασύνδεσης του αισθητήρα με το μικροελεγκτή με το Tinker cad και στην (εικόνα 2) το ίδιο κύκλωμα πάνω σε μια πολύ μικρή πλακέτα όπου κολλήσαμε όλες τις συνδέσεις με κολλήσεις ώστε να είναι σταθερό για να μπορέσει να αντέχει τα χτυπήματα.

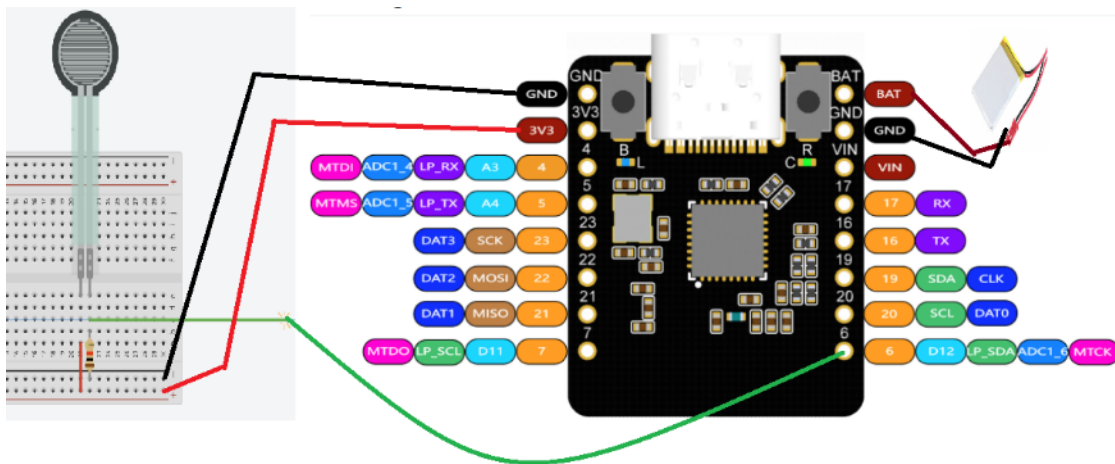


Σχήμα 16. Σχεδιάγραμμα Tinker Cad.



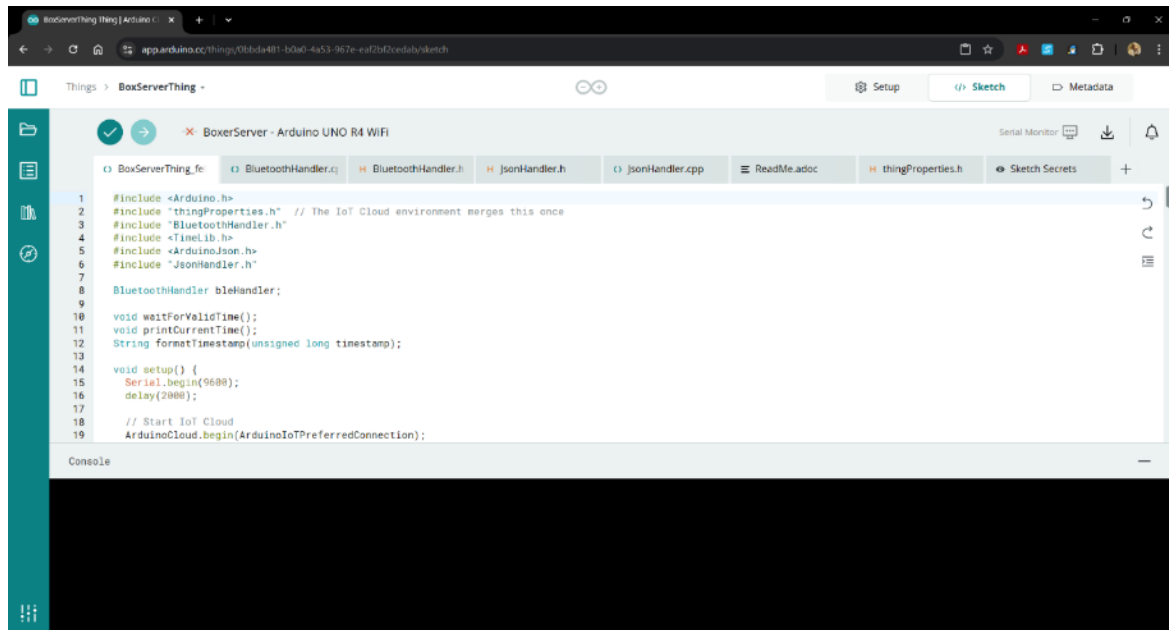
Εικόνα 2. Υλοποίηση με κολλήσεις.

Για το σύστημα που υλοποιώ έχω χρησιμοποιήσει δύο μικροελεγκτές Beetle ESP32 C6 Mini Smart Wearable Device, έναν σε κάθε αθλητή που ονομάζονται και αναγνωρίζονται από το σύστημα ως Blue Boxer και Red Boxer οι οποίοι δέχονται από τον αισθητήρα ή τους αισθητήρες την πληροφορία και την μεταφέρουν μέσω σύνδεσης Bluetooth στην εφαρμογή του κινητού. Ο κώδικας των μικροελεγκτών είναι γραμμένος σε C και C++. Τα pin που χρησιμοποιώ για να γίνει η σύνδεση της μπαταρίας είναι το BAT και GND από τα δεξιά όπως κοιτάμε τον μικροελεγκτή. Τα pin GND και 3V3 στα αριστερά για να παρέχει ρεύμα στο κύκλωμα της αντίστασης και του αισθητήρα. Το pin 6 στα δεξιά που συνδέεται ο αισθητήρας για να δίνει την αναλογικές τιμές στον μικροελεγκτή. Στο παρακάτω (σχήμα 17) φαίνονται οι συνδέσεις.

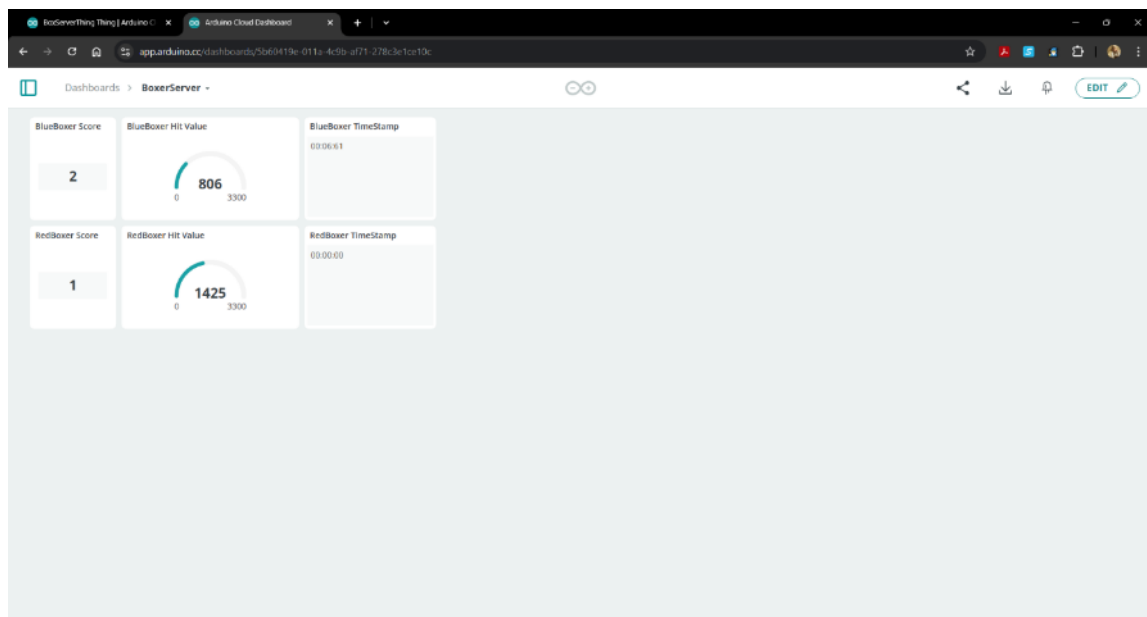


Σχήμα 17. Οι συνδέσεις των pins μπαταρίας, μικροελεγκτή και κυκλώματος αισθητήρα και αντίστασης.

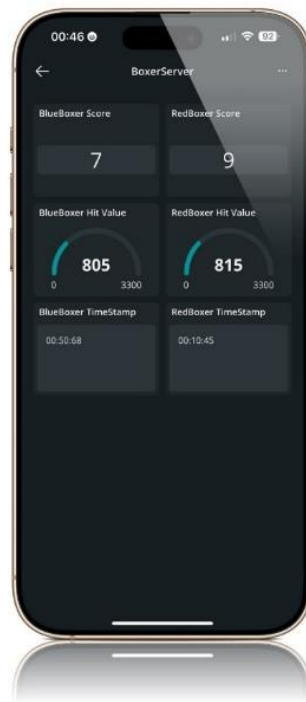
χρήστης από μια ειδική εφαρμογή στα κινητά τηλέφωνα για Android ή Ios που ονομάζεται IoT Remote, μπορεί όμως και να το δει από κάποιον Web browser. Η παρακάτω (εικόνα 5) δείχνει την εφαρμογή IoT Remote σε κινητό τηλέφωνο. Ο ελεγκτής για να λειτουργήσει χρειάζεται μόνο να τροφοδοτείται με ρεύμα. Και τα PIN που έχει πάνω δεν χρησιμοποιούνται καθόλου, γιατί δεν μας χρειάζονται για τη λειτουργία που τον χρησιμοποιούμε στο σύστημα μας (σχήμα).



Εικόνα 3. Ο Πηγαίος κώδικας μέσα από το thing.



Εικόνα 4. Το Dashboard Boxer Server με πληροφορίες από χτυπήματα.



Εικόνα 5. Η εφαρμογή IoT Remote και ο πίνακας (Dashboard Box Server).

3.2 Αισθητήρες

3.2.1 Αισθητήρας αντίστασης δύναμης FSR

Οι **αισθητήρες δύναμης μεταβαλλόμενης αντίστασης** (Force Sensing Resistors, FSR) είναι λεπτές, εύκαμπτες διατάξεις πολυμερικού φιλμ, οι οποίες μειώνουν την αντίστασή τους όταν εφαρμόζεται δύναμη στην ενεργή επιφάνειά τους. Σε αντίθεση με πιο κλασικούς αισθητήρες μέτρησης δύναμης (π.χ. strain gauges, load cells), οι FSR δεν προορίζονται για απόλυτα ακριβείς ή γραμμικές μετρήσεις του φορτίου, αλλά αποτελούν μια προσιτή και εύχρηστη λύση για την ποιοτική ή ημι-ποσοτική αποτίμηση δυνάμεων και πιέσεων.

Ιστορική Αναδρομή και Επιστημονική Τεκμηρίωση

Οι FSR άρχισαν να αναπτύσσονται εμπορικά τη δεκαετία του 1980–1990, με γνωστές εταιρείες (π.χ. Interlink Electronics) να παράγουν τυποποιημένες πλακέτες. Επιστημονικές εργασίες άρχισαν να διερευνούν τη χρήση τους σε εφαρμογές όπως η ανάλυση βηματισμού, η ρομποτική και οι φορετές συσκευές (wearables) (Park, Kim, & Jeong, 2020) (Seyfried & Schmalz, 2019).

Σήμερα χρησιμοποιούνται εκτενώς σε συστήματα αλληλεπίδρασης ανθρώπου-μηχανής, σε φορητά ιατρικά όργανα, καθώς και σε συσκευές αθλητικών μετρήσεων, όπου η ακρίβεια της δύναμης δεν χρειάζεται να είναι «εργαστηριακού» επιπέδου αλλά πρέπει να είναι επαρκής για την καταγραφή τάσεων, συγκριτικών μετρήσεων και ενδείξεων κρούσης.

Θεωρία Λειτουργίας FSR

Βασική Αρχή

Η βασική αρχή λειτουργίας ενός FSR στηρίζεται στην αλλαγή της ηλεκτρικής αντίστασης ανάλογα με τη δύναμη ή την πίεση που ασκείται επάνω του. Συγκεκριμένα:

- Όταν δεν ασκείται δύναμη, η αντίσταση του FSR είναι πολύ υψηλή (μπορεί να φτάσει και τα 10 MΩ ή περισσότερο).
- Με την αύξηση της δύναμης, η ενεργή περιοχή συμπιέζει το ειδικό πολυμερές, μειώνοντας την αντίσταση ακόμα και σε μερικές εκατοντάδες Ω (ανάλογα με το εύρος του αισθητήρα).

Η σχέση δύναμης-αντίστασης προσεγγίζει συνήθως μια αντίστροφη εκθετική (power-law) καμπύλη, ενώ για δυνάμεις εντός συγκεκριμένου εύρους (π.χ. 0–10 kg) μπορούμε να θεωρήσουμε ότι ο αισθητήρας παρουσιάζει αρκετά επαναλήψιμα αποτελέσματα³⁵.

Πλεονεκτήματα και Περιορισμοί

- **Πλεονεκτήματα:** Μικρό πάχος, ευελιξία, χαμηλό κόστος, εύκολη διασύνδεση με μικροελεγκτές (Arduino, ESP32 κ.λπ.), γρήγορος χρόνος απόκρισης (συνήθως μερικά msec).
- **Περιορισμοί:** Περιορισμένη ακρίβεια σε απόλυτες τιμές δύναμης, ευαισθησία σε συνθήκες περιβάλλοντος (θερμοκρασία, υγρασία), μη γραμμική έξοδος και φαινόμενα υστέρησης (hysteresis) και «drift» σε στατικές φορτίσεις.

Χαρακτηριστικά και Τεχνικές Προδιαγραφές

1. **Δυναμικό εύρος:** Συνήθως 0–10 kg ή και περισσότερο, ανάλογα με το μοντέλο.
2. **Ευαισθησία πίεσης:** Στα 100–200 psi (pounds per square inch) επέρχεται κορεσμός, δηλαδή περαιτέρω αύξηση της δύναμης δεν μεταβάλλει αισθητά την αντίσταση.

3. **Επαναληψιμότητα:** Περίπου $\pm 5\%$ έως $\pm 25\%$ σε τυπικές εφαρμογές (ανάλογα με τη γεωμετρία του αισθητήρα, το πάχος και την ακρίβεια της μηχανικής κατασκευής).
4. **Χρόνος απόκρισης:** 1–2 msec σε δυναμικές μετρήσεις.
5. **Διάρκεια ζωής:** Συνήθως >10 εκατομμύρια ενεργοποιήσεις, εφόσον δεν ξεπερνιούνται οι μηχανικοί και ηλεκτρικοί περιορισμοί (Interlink Electronics, 2021).

Εφαρμογές των Αισθητήρων FSR

Οι FSR βρίσκουν εφαρμογή σε ένα ευρύ φάσμα πεδίων:

- **Ανάλυση βηματισμού (Gait Analysis):** Τοποθέτηση FSR σε πέλματα ή παπούτσια για την καταγραφή πιέσεων³⁵.
- **Ρομποτική:** Χρήση σε ρομποτικούς βραχίονες για την ανίχνευση της δύναμης αφής.
- **Ιατρικές συσκευές:** Ανίχνευση πίεσης σε αναπηρικά αμαξίδια, στρώματα ασθενών, με σκοπό την πρόληψη κατακλίσεων.
- **Παιχνίδια & Διεπαφές Χρήστη:** Πλήκτρα αφής, χειριστήρια μουσικών οργάνων κ.λπ.
- **Αθλητικές Εφαρμογές:** Μέτρηση δύναμης κρούσης σε διάφορα αθλήματα (π.χ. πυγμαχία, kick-boxing).

Χρήση των FSR για Καταγραφή Δύναμης Πυγμάχων

Στην παρούσα πτυχιακή εργασία, οι FSR τοποθετούνται σε σημεία όπου προσκρούει η γροθιά του πυγμάχου. Η διάταξη μπορεί να είναι είτε:

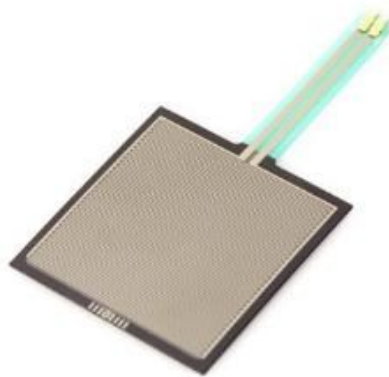
1. **Πάνω σε σάκο πυγμαχίας:** Ο αισθητήρας ενσωματώνεται στην εξωτερική επιφάνεια ή εσωτερικά του σάκου, με ένα προστατευτικό στρώμα (π.χ. αφρώδες υλικό) ώστε να αποφεύγονται μηχανικές βλάβες.
2. **Πάνω σε γάντια:** Ένα ή περισσότερα FSR μπορούν να ενσωματωθούν στο σημείο των αρθρώσεων του χεριού, όπου επιτυγχάνεται η κρούση.

Σημεία-Κλειδιά για τη Μέτρηση Κρούσης

- **Υψηλές Δυνάμεις:** Σε αντίθεση με τη μέτρηση βάρους (π.χ. 0–12 kg σε παιδικό κάθισμα), η πυγμαχία μπορεί να παράγει αιχμές δύναμης εκατοντάδων Newton σε πολύ σύντομο χρονικό διάστημα.

- **Δυναμική Μέτρηση:** Η μέτρηση δεν είναι στατική (όπως το βάρος), αλλά παροδική (transient). Η δύναμη ανεβαίνει απότομα και πέφτει σε διάστημα λίγων χιλιοστών του δευτερολέπτου.
- **Συχνότητα δειγματοληψίας:** Χρειάζεται αρκετά υψηλή (τουλάχιστον >1 kHz) για να καταγράψουμε με σχετική ακρίβεια την αιχμή της κρούσης (Kim & Park, 2018).
- **Συστήματα Απόσβεσης:** Προτείνεται η χρήση λεπτού ελαστικού ή αφρώδους υλικού ανάμεσα στην επιφάνεια πρόσκρουσης και τον FSR, ώστε να διαχέονται οι ακραίες τάσεις που θα μπορούσαν να καταστρέψουν τον αισθητήρα ή να επηρεάσουν τη μέτρηση.

Σε πρόσφατες εργασίες (Freedman, Freedman, Freedman & Freedman, 2021) (Chang, Yang, Guo & Zhang, 2021) έχουν αναπτυχθεί πρωτότυπα συστήματα που τοποθετούνται είτε στον σάκο είτε στον βραχίονα του αθλητή, χρησιμοποιώντας δίκτυο (array) FSR αισθητήρων. Παρόλο που η γραμμικότητα και η απόλυτη ακρίβεια είναι περιορισμένες, οι συγγραφείς επιτυγχάνουν ικανοποιητική σχετική μέτρηση της δύναμης κρούσης (π.χ. σύγκριση ανάμεσα σε διαφορετικούς πυγμάχους ή μεταξύ διαφορετικών χτυπημάτων του ίδιου αθλητή). Στο παρακάτω (σχήμα 19) παρουσιάζεται ο αισθητήρας FSR 406.



Σχήμα 19. Interlink Electronics FSRTM 406¹⁵

¹⁵ Datasheet Interlink Electronics FSRTM 406

<https://cdn.sparkfun.com/assets/c/4/6/8/b/2010-10-26-DataSheet-FSR406-Layout2.pdf>

3.2.2 Αισθητήρας Custom FSR – Υφάσματα - e-Textiles ¹⁶

Τα υλικά που χρησιμοποιήθηκαν για την κατασκευή του αισθητήρα FSR:

1. Στο μικρό μέγεθος:



Ύφασμα αθλητικού είδους¹⁷ (εικόνα 6):

- Ύφασμα με micro mesh από πολυεστέρα spandex, με αριθμό αντικειμένου HS5953, είναι πλεγμένο με 92% πολυεστέρα και 8% spandex.
- Το ύφασμα στεγνώνει γρήγορα, έχει σχεδιαστεί να απομακρύνει την υγρασία από το σώμα.
- Είναι το ιδανικό υλικό για κάθε αθλητικό ένδυμα που χρησιμοποιείται σε αθλήματα.
- Πληρούν αυστηρά πρότυπα ποιότητας, αυτά τα υφάσματα που στεγνώνουν γρήγορα παράγονται από τις προηγμένες κυκλικές πλεκτομηχανές.

Εικόνα 6. Ύφασμα αθλητικού είδους.



Ρέλι μαύρο σατέν 100% PES¹⁸ (εικόνα 7):

- Το σατέν είναι ένα είδος υφάσματος που χρησιμοποιείται ευρέως για διάφορες χρήσεις. Είναι επίσης γνωστό ως ένα χοντρό λείο μεταξωτό ύφασμα, το οποίο χαρακτηρίζεται από λεία επιφάνεια και συνήθως λαμπερό πρόσωπο και θαμπή πλάτη. Αν και αρχικά ήταν μεταξωτό ύφασμα, τώρα κατασκευάζεται από νήματα άλλων ινών.

Εικόνα 7. Ρέλι μαύρο σατέν 100% PES.

¹⁶ Sorna Mugi Viswanathan, E-Textile and its Applications DOI:10.1002/adma.200990013.PMID

¹⁷ <https://ifasmata.com/portal/yfasmatologia>

¹⁸ <https://el.lunss.com/blog/satin-fabric-knowledge-11.html>



Εικόνα 8. Ύφασμα χαλκού αγωγίμο.

- Ύφασμα χαλκού αγωγίμο και αντιοξειδωτικό είναι εύκολο στο ράψιμο και κατάλληλο για DIY projects¹⁹ (εικόνα 8):

Τα αγωγίμα υφάσματα χαλκού και νικελίου είναι ελαφριά ανθεκτικά προϊόντα και παρέχουν το υψηλότερο επίπεδο απόδοσης στην θωράκιση και την αγωγιμότητα.

Υλικό βάσης: Πολυεστέρας

Επίστρωση: Χαλκός-Νικέλιο

Περιεχόμενα υλικού: Πολυεστέρας / Χαλκός / Νικέλιο 70:16:14

Στυλ υφάσματος: Απλή ύφανση και επικάλυψη.

Αποτελεσματικότητα θωράκισης:
10Mhz -3Ghz:> 60dB

Αντοχή στην επιφάνεια: $\leq 0,05 \text{ Ohm} / \text{M}^2$

Περιγραφή πλεονεκτήματος:

- Ελαφρύ και απαλό
- Πολύ χαμηλή αντίσταση, ανώτερη αγωγιμότητα
- Πολύ καλό εφέ προστασίας.



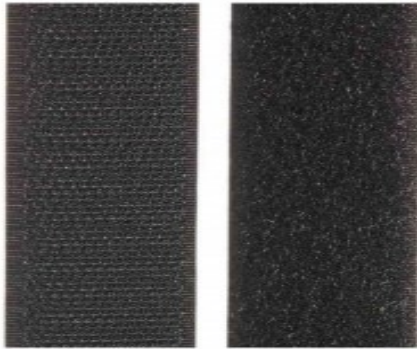
Εικόνα 9. Velostat – Linqstat.

- Velostat – Linqstat²⁰ (εικόνα 9):

Το Velostat, γνωστό και ως Linqstat, είναι ένα υλικό συσκευασίας κατασκευασμένο από πολυμερές φύλλο (πολυολεφίνες) εμποτισμένο με αιθάλη για να το κάνει κάπως ηλεκτρικά αγωγίμο. Χρησιμοποιείται για την προστασία αντικειμένων ή συσκευών που είναι επιρρεπείς σε ζημιές από ηλεκτροστατική εκφόρτιση. Το Velostat είναι πιεζοαντιστικό. Η αντίστασή του αλλάζει με κάμψη ή πίεση. Για παράδειγμα, 25 mm 2 φρέσκου Velostat 0,2 mm τοποθετημένο σε σάντουιτς ανάμεσα σε δύο ηλεκτρόδια έχει αντίσταση περίπου 9kΩ χωρίς να ασκηθεί καμία δύναμη, αλλά μόνο 1 kΩ όταν εφαρμόζεται δύναμη 3 Newton.

¹⁹ <http://el.3ltechtextile.com/conductive-fabric-product/>

²⁰ <https://en.wikipedia.org/wiki/Velostat>

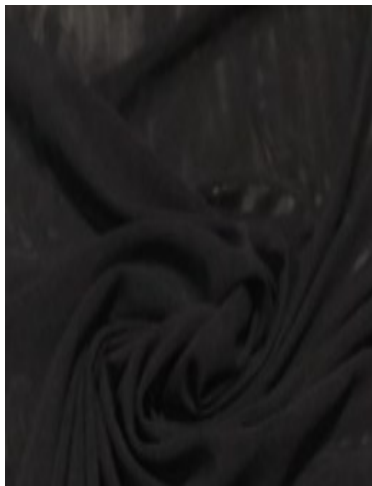


- **Velcro – Χριτς Χρατς** (εικόνα 10):

Το Velcro (επίσης γνωστό ως Βελκρο ή Κριτς Κρατς) είναι ένα σύστημα κλεισίματος με αυτοκόλλητες ταινίες, που αποτελείται από δύο μέρη: το ένα με πολύ μικρά πλαστικά γαντζάκια ενώ η άλλη πλευρά είναι σαν σφουγγαράκι. Όταν πιέζονται μαζί προσφέρουν ένα σταθερό και ανθεκτικό κλείσιμο.²¹

Εικόνα 10. Velcro – Χριτς Χρατς.

2. Τα υλικά που χρησιμοποιήθηκαν για την κατασκευή του αισθητήρα FSR που είναι μεγαλύτερος σε μέγεθος είναι τα εξής:



- **Spandex^{22,23}** (εικόνα 11).

Επίσης γνωστό ως Lycra ή elastane, το Spandex είναι μια συνθετική ίνα που χαρακτηρίζεται από την εξαιρετική ελαστικότητά του. Το Spandex αναμειγνύεται με διάφορους τύπους ίνας για να προσθέσει ελαστικότητα και χρησιμοποιείται για τα πάντα από αθλητικά ρούχα, ως φόδρα για τα ρούχα σας ή ακόμα και για το μαγιό σας, μέχρι και κάλτσες. Η ευελιξία του spandex επιτρέπει την χρήση του σε μια πληθώρα ενδυμάτων, από αθλητικά έως καθημερινά ενδύματα. Η ιδιότητά του να επανέρχεται στην αρχική του μορφή ακόμα και μετά από συχνή χρήση, εγγυάται τη διατήρηση της ποιότητας.

Εικόνα 11. Spandex.

²¹ <https://ligaapola.gr/product-category/velcro/>

²² https://www.velona.gr/28-%CE%B5%CE%AF%CE%B4%CE%B7-%CF%85%CF%86%CE%AC%CF%83%CE%BC%CE%B1%CF%84%CE%BF%CF%82-%CE%BA%CE%B1%CE%B9-%CE%BF%CE%B9-%CF%87%CF%81%CE%AE%CF%83%CE%B5%CE%B9%CF%82-%CF%84%CE%BF%CF%85%CF%82/?srsltid=AfmBOoqyURrXDceYYiNqu_EEr1rCWI84QYkvVWQRC6P0_QX2U7KdGozd.

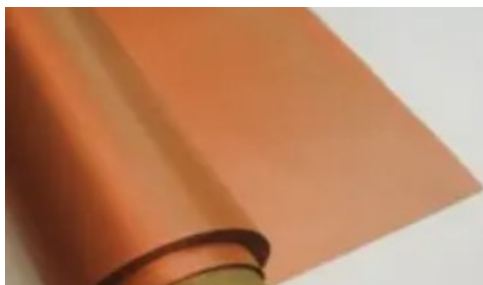
²³ https://shaperfit.gr/blogs/news/%CF%84%CE%BF-%CF%84%CE%AD%CE%BB%CE%B5%CE%B9%CE%BF-%CF%8D%CF%86%CE%B1%CF%83%CE%BC%CE%B1-%CF%85%CF%88%CE%B7%CE%BB%CE%AE%CF%82-%CF%80%CE%BF%CE%B9%CF%8C%CF%84%CE%B7%CF%84%CE%B1%CF%82?srsltid=AfmBOopSmONLVaBMW19fpGgOK3Etp8ipbHJUxX8IGqg_PVPaSe1lyC1k



Εικόνα 12. Ρέλι μαύρο σατέν 100% PES

• **Ρέλι μαύρο σατέν 100% PES²⁴** (εικόνα 12):

Το σατέν είναι ένα είδος υφάσματος που χρησιμοποιείται ευρέως για διάφορες χρήσεις. Είναι επίσης γνωστό ως ένα χοντρό λείο μεταξωτό ύφασμα, το οποίο χαρακτηρίζεται από λεία επιφάνεια και συνήθως λαμπερό πρόσωπο και θαμπή πλάτη. Αν και αρχικά ήταν μεταξωτό ύφασμα, τώρα κατασκευάζεται από νήματα άλλων ινών.



Εικόνα 11. Ύφασμα χαλκού αγωγίμο.

• **Ύφασμα χαλκού αγωγίμο και αντιοξειδωτικό** είναι εύκολο στο ράψιμο και κατάλληλο για DIY projects²⁵ (εικόνα 13):

Τα αγωγίμα υφάσματα χαλκού και νικελίου είναι ελαφριά ανθεκτικά προϊόντα και παρέχουν το υψηλότερο επίπεδο απόδοσης στην θωράκιση και την αγωγιμότητα.

Υλικό βάσης: Πολυεστέρας

Επίστρωση: Χαλκός-Νικέλιο

Περιεχόμενα υλικού: Πολυεστέρας / Χαλκός / Νικέλιο 70:16:14

Στυλ υφάσματος: Απλή ύφανση και επικάλυψη.

Αποτελεσματικότητα θωράκισης: 10Mhz -3Ghz:> 60dB

Αντοχή στην επιφάνεια: $\leq 0,05 \text{ Ohm} / \text{M}^2$

Περιγραφή πλεονεκτήματος:

- Ελαφρύ και απαλό
- Πολύ χαμηλή αντίσταση, ανώτερη αγωγιμότητα
- Πολύ καλό εφέ προστασίας.

²⁴ <https://el.lunss.com/blog/satin-fabric-knowledge-11.html>

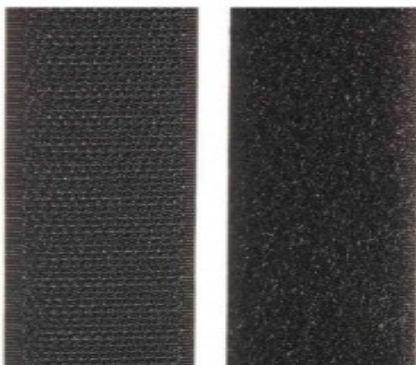
²⁵ <http://el.3ltechtextile.com/conductive-fabric-product/>



Εικόνα 12. Velostat – Linqstat.

• **Velostat – Linqstat²⁶** (εικόνα 14):

Το Velostat, γνωστό και ως Linqstat, είναι ένα υλικό συσκευασίας κατασκευασμένο από πολυμερές φύλλο (πολυολεφίνες) εμποτισμένο με αιθάλη για να το κάνει κάπως ηλεκτρικά αγωγίμο. Χρησιμοποιείται για την προστασία αντικειμένων ή συσκευών που είναι επιρρεπείς σε ζημιές από ηλεκτροστατική εκφόρτιση. Το Velostat είναι πιεζοαντιστικό. Η αντίστασή του αλλάζει με κάμψη ή πίεση. Για παράδειγμα, 25 mm 2 φρέσκου Velostat 0,2 mm τοποθετημένο σε σάντουιτς ανάμεσα σε δύο ηλεκτρόδια έχει αντίσταση περίπου 9kΩ χωρίς να ασκηθεί καμία δύναμη, αλλά μόνο 1 kΩ όταν εφαρμόζεται δύναμη 3 Newton.



Εικόνα 13. Velcro – Χριτς Χρατς.

• **Velcro²⁷ – Χριτς Χρατς** (εικόνα 15):

Το Velcro (επίσης γνωστό ως Βελκρο ή Κριτς Κρατς) είναι ένα σύστημα κλεισίματος με αυτοκόλλητες ταινίες, που αποτελείται από δύο μέρη: το ένα με πολύ μικρά πλαστικά γαντζάκια ενώ η άλλη πλευρά είναι σαν σφουγγαράκι. Όταν πιέζονται μαζί προσφέρουν ένα σταθερό και ανθεκτικό κλείσιμο.

²⁶ <https://en.wikipedia.org/wiki/Velostat>

²⁷ <https://ligaapola.gr/product-category/velcro/>

Παραθέτω τις εικόνες 16,17,18, 19, 20 από την δημιουργία των custom made αισθητήρων.



Εικόνα 14.
Η κατασκευή τους.

Εικόνα 15.
Η κατασκευή τους.

Εικόνα 16.
Η κατασκευή τους.

Δημητρακαράκος Νικόλαος, 'Ανάπτυξη συστήματος στο χώρο της αθλητικής πυγμαχίας για την παρακολούθηση της έντασης και πλήθους χτυπημάτων κατά τη διάρκεια αγώνα.'



Εικόνα 17. Ο μικρότερος αισθητήρας.



Εικόνα 18. Ο μεγαλύτερος αισθητήρας.

Πιεζοηλεκτρικό Αγώγιμο Υλικό (Velostat/Linqstat)²⁸

Το Velostat, επίσης γνωστό ως Linqstat, είναι ένα μαύρο πολυμερικό φύλλο εμποτισμένο με αιθάλη (άνθρακα), το οποίο χαρακτηρίζεται από πιέζω-ευαίσθητες αγώγιμες ιδιότητες. Αρχικά αναπτύχθηκε για εφαρμογές στη συσκευασία ηλεκτρονικών εξαρτημάτων, όμως σήμερα χρησιμοποιείται ευρέως σε ηλεκτρονικά έργα DIY και φορετές τεχνολογίες, εξαιτίας των ξεχωριστών ιδιοτήτων του.

Βασικά Χαρακτηριστικά

Πιεζο-αντιστατικές ιδιότητες:

Το Velostat μεταβάλλει την ηλεκτρική του αντίσταση όταν δέχεται πίεση ή άλλη μηχανική τάση. Αυτή η ιδιότητα επιτρέπει τη χρήση του ως αισθητήρα που μπορεί να καταγράψει μεταβολές στην πίεση ή παραμόρφωση, καθιστώντας το ιδανικό για κατασκευή αισθητήρων δύναμης, πίεσης ή κάμψης.

Φυσικές Ιδιότητες:

Συνήθως διατίθεται σε φύλλα μεγέθους περίπου 28cm x 28cm με πάχος περίπου 0,1mm. Έχει επιφανειακή αντίσταση κάτω από 31 KΩ και πάχος γύρω στα 500 μικρόμετρα. Είναι εύκαμπτο, θερμοκρασιακά ανθεκτικό από περίπου -45°C έως 65°C, και διαθέτει καλές μηχανικές ιδιότητες, όπως ευκαμψία και ανθεκτικότητα.

Εφαρμογές

Εύκαμπτοι Αισθητήρες:

Η ικανότητα του Velostat να αλλάζει αντίσταση με την εφαρμογή πίεσης το καθιστά ιδιαίτερα χρήσιμο στη δημιουργία προσαρμοσμένων αισθητήρων για διάφορες εφαρμογές, όπως αισθητήρες που ενσωματώνονται σε υποδήματα, οι οποίοι μπορούν να ενεργοποιούν φωτισμό όταν ανιχνεύουν πίεση.

²⁸ Giovanelli, D., Farella, E., & Castellini, C. (2021). A Velostat-Based Pressure-Sensitive Mat for Center-of-Pressure Measurement. *International Journal of Environmental Research and Public Health*, 18(11). DOI: 10.3390/ijerph18115958.

Φορετές Τεχνολογίες (Wearables):

Η ευελιξία και η ευαισθησία του στις μηχανικές τάσεις καθιστούν το Velostat ιδανικό για ενσωμάτωση σε φορετές συσκευές, όπου μπορεί να παρακολουθεί παραμέτρους όπως κινήσεις, στάση σώματος και βιομετρικά δεδομένα.

Περιορισμοί:

Παρά τις σημαντικές δυνατότητές του, η ακρίβεια της σχέσης πίεσης-αντίστασης ενδέχεται να διαφέρει ανάλογα με το δείγμα, καθιστώντας αναγκαία τη βαθμονόμηση πριν από ειδικές εφαρμογές.

Συνολικά, το Velostat αποτελεί ένα εξαιρετικά χρήσιμο και ευέλικτο υλικό για ένα ευρύ φάσμα εφαρμογών στους τομείς της ηλεκτρονικής και της φορετής τεχνολογίας.

Στους αισθητήρες επίσης που κατασκευάζω υπάρχουν και δύο καλώδια, ένα από κάθε μεριά του χάλκινου υφάσματος που είναι κολλημένα με το χάλκινο ύφασμα με αγωγήμη ταινία χαλκού.

3.2.3 Συνδεσιμότητα ESP32 με FSR

Όταν ο Μικροελεγκτής παίρνει εντολή να ξεκινήσει η αντιστροφή μέτρηση του χρόνου από την εφαρμογή του κινητού, ο αισθητήρας μπορεί να καταγράψει κτυπήματα που δέχεται από τον πυγμάχο. Μόλις εφαρμοστεί δύναμη στον αισθητήρα, ενεργοποιείται η διαδικασία ανάγνωσης και καταγραφής των δεδομένων και αποστολή τους από τον μικροελεγκτή ESP32 στην εφαρμογή του κινητού ώστε να αποθηκευτούν σε μια βάση δεδομένων. Ο αισθητήρας FSR406²⁹ λειτουργεί προσομοιώνοντας την πίεση που ασκείται πάνω του, μετατρέποντας τη δύναμη των γροθιών του πυγμάχου σε τάση. Όσο μεγαλύτερη είναι η δύναμη του χτυπήματος που δέχεται ο αισθητήρας, τόσο μικρότερη γίνεται η εσωτερική αντίσταση του FSR. Αντίθετα, όταν η δύναμη είναι μικρή ή μηδενική, η αντίσταση μπορεί να φτάσει έως και 10 MΩ. Στη συγκεκριμένη πτυχιακή, ο αισθητήρας FSR406 τοποθετείται στο εξωτερικό μέρος του σάκου πυγμαχίας ή στην ειδικά σχεδιασμένη πυγμαχική επιφάνεια προπόνησης, ώστε να ανιχνεύει χωρίς να έχει την δυνατότητα να μετρά την πραγματική δύναμη των χτυπημάτων των

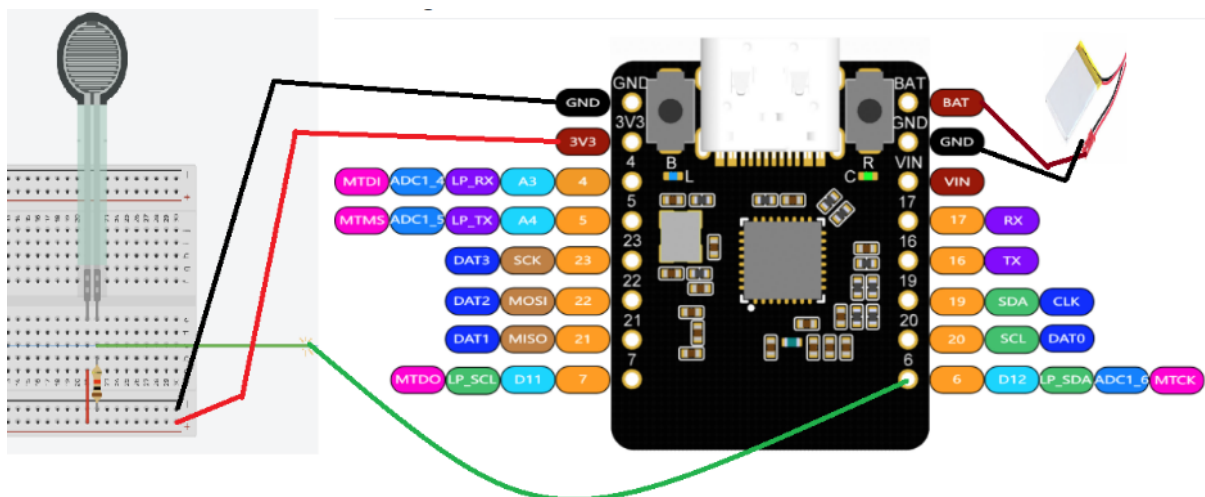
²⁹ <https://cdn.sparkfun.com/assets/c/4/6/8/b/2010-10-26-DataSheet-FSR406-Layout2.pdf>

πυγμάχων κατά τη διάρκεια προπόνησης ή των αγώνων. Επειδή η επιφάνεια τους είναι μικρή χρειάζεται να τοποθετηθούν αρκετοί αισθητήρες. Η σύνδεση του αισθητήρα με το ESP32 πραγματοποιήθηκε σε αναλογικό pin 6(ADC), προκειμένου να καταγράφεται με ακρίβεια η αναλογική τιμή της τάσης που προκύπτει από τα χτυπήματα, μάλιστα η τιμή αυτή είναι σε millivolt. Τη δυνατότητα αυτή μας την δίνει μια συνάρτηση του API του ESP32 η **analogReadMilliVolts**³⁰. Για τη διασφάλιση της σωστής και σταθερής λειτουργίας του αισθητήρα χρησιμοποιήθηκε μία αντίσταση pull-down 10 KΩ, η οποία τοποθετήθηκε μεταξύ του αισθητήρα και της γείωσης (ground). Θα γίνουν δοκιμές και με αντίσταση 1KΩ. Η αντίσταση αυτή είναι απαραίτητη ώστε να εξασφαλίζεται η επαναφορά του αισθητήρα στην αρχική κατάσταση μετά από κάθε ανάγνωση για να αποφεύγονται λανθασμένες μετρήσεις λόγω υπολειμματικής τάσης στον αισθητήρα. Επιπλέον θα γίνουν δοκιμές με αισθητήρες που κατασκεύασα με την βοήθεια της κυρίας Μαρίας Λάμπρου που είναι σχεδιάστρια μύδας. Η **αντίσταση pull-down 10 KΩ** είναι μια αντίσταση που συνδέεται ανάμεσα στην είσοδο του αισθητήρα (ή του μικροελεγκτή) και τη γείωση (ground). Ο ρόλος της είναι να διατηρεί την τάση του σήματος κοντά στο μηδέν (λογικό LOW ή 0 V), όταν ο αισθητήρας δεν δέχεται πίεση ή όταν δεν υπάρχει άλλο σήμα. Με αυτόν τον τρόπο, εξασφαλίζεται η σταθερότητα του σήματος και αποφεύγονται λανθασμένες ή τυχαίες αναγνώσεις, λόγω θορύβου ή υπολειμματικής τάσης. Η τιμή της αντίστασης (10 KΩ) είναι τυπική, καθώς προσφέρει έναν ισορροπημένο συνδυασμό ανάμεσα σε σταθερότητα και κατανάλωση ενέργειας. Σε περίπτωση που χρησιμοποιηθεί αντίσταση pull-down μικρότερης τιμής, όπως 1 KΩ αντί της προτεινόμενης 10 KΩ, μεταβάλλεται σημαντικά η συμπεριφορά του κυκλώματος. Η μικρότερη αυτή τιμή αντίστασης συνεπάγεται ότι το ρεύμα που θα διαρρέει τον αισθητήρα FSR³¹ και το κύκλωμα θα είναι μεγαλύτερο, κάτι που έχει ως αποτέλεσμα τη μείωση της ευαισθησίας του αισθητήρα, ειδικά σε χαμηλές τιμές πίεσης ή σε χτυπήματα μικρότερης έντασης. Πρακτικά, αυτό σημαίνει ότι ο αισθητήρας θα απαιτεί μεγαλύτερη δύναμη για να δημιουργήσει αξιοσημείωτη μεταβολή τάσης, κάνοντας πιο δύσκολη τη διάκριση μεταξύ ελαφριών και μεσαίων χτυπημάτων κάτι που είναι σημαντικό επειδή τα χτυπήματα θα έχουν λογικά μεγάλη δύναμη.

³⁰ <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html#analogreadmillivolts>

³¹ https://cdn.sparkfun.com/assets/e/3/b/3/8/force_sensing_resistor_guide.pdf

Από την άλλη πλευρά, μια μικρότερη αντίσταση pull-down (1 KΩ) παρέχει ταχύτερη επαναφορά του αισθητήρα στην αρχική του κατάσταση μετά από κάθε χτύπημα, καθώς «αδειάζει» πιο γρήγορα η τάση που παραμένει. Αυτό μπορεί να είναι επιθυμητό σε εφαρμογές όπου απαιτείται γρήγορη εναλλαγή ανάμεσα σε διαδοχικές μετρήσεις, όπως για παράδειγμα σε προπόνηση υψηλής έντασης με γρήγορα και συνεχή χτυπήματα. Ωστόσο, είναι σημαντικό να ληφθεί υπόψη πως με τη μικρότερη αντίσταση, η συνολική κατανάλωση ενέργειας στο κύκλωμα αυξάνεται, γεγονός που μπορεί να είναι επιβαρυντικό σε εφαρμογές που τροφοδοτούνται από μπαταρίες ή απαιτούν χαμηλή κατανάλωση ενέργειας. Συνεπώς, η επιλογή της αντίστασης pull-down πρέπει να γίνεται με βάση τα χαρακτηριστικά και τις ειδικές απαιτήσεις της εφαρμογής. Το σχήμα 20 μας δείχνει τον τρόπο διασύνδεσης του αισθητήρα FSR με τον μικροελεγκτή ESP32.



Σχήμα 20. Το σχεδιάγραμμα μας δείχνει πώς συνδέεται ο αισθητήρας FSR με το μικροελεγκτή ESP32.

3.3 Εφαρμογή στο σάκο

3.3.1 Ενσωμάτωση του κυκλώματος στο σάκο

Θα τοποθετήσω το κύκλωμα στο πάνω μέρος του σάκου μαζί με την μπαταρία που το τροφοδοτεί. Τα δύο καλώδια που φεύγουν από τον αισθητήρα συνδέονται σε δύο καλώδια που καταλήγουν στο μικροκύκλωμα που έχει πάνω την αντίσταση. Από το μικροκύκλωμα φεύγουν δυο καλώδια που πάνε το ένα στα 3,3 volt του μικροελεγκτή ESP32 και το άλλο στην διπλανή γείωση (GND) ώστε να τροφοδοτηθεί με ρεύμα το μικροκύκλωμα. Τέλος ένα καλώδιο που

είναι πάνω στο μικροκύκλωμα που πάει πάνω στο pin 6 του μικροελεγκτή ESP32 όπου παίρνουμε τιμές σε millivolt όταν γίνεται κάποιο χτύπημα. Παραθέτω δύο (εικόνες 21, 22) από το αποτέλεσμα.



Εικόνα 19. Ο σάκος και ο αισθητήρας στο γυμναστήριο.



Εικόνα 20. Ο σάκος και ο αισθητήρας στο γυμναστήριο.

4. Υλοποίηση Λογισμικού Συστήματος

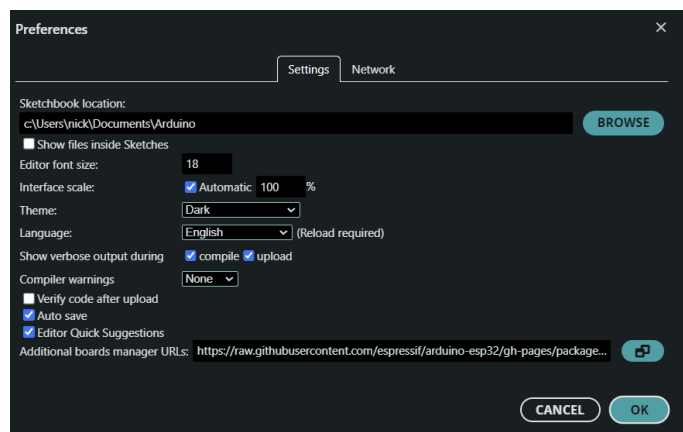
4.1 Προγραμματισμός ESP32 – Arduino

Στο κεφάλαιο αυτό θα αναλυθεί η διαδικασία προγραμματισμού του μικροελεγκτή ESP32 και του Arduino Uno R4 Wi-Fi αξιοποιώντας το περιβάλλον Arduino.

Ο προγραμματισμός του ESP32 μέσω του Arduino IDE (Integrated Development Environment) αποτελεί μια δημοφιλή προσέγγιση, ιδιαίτερα για χρήστες που είναι ήδη εξοικειωμένοι με την πλατφόρμα Arduino. Το ESP32 είναι ένας ισχυρός μικροελεγκτής χαμηλού κόστους με ενσωματωμένες δυνατότητες Wi-Fi και Bluetooth, καθιστώντας το ιδανική επιλογή για ένα ευρύ φάσμα εφαρμογών, συμπεριλαμβανομένων των έργων Internet of Things (IoT).

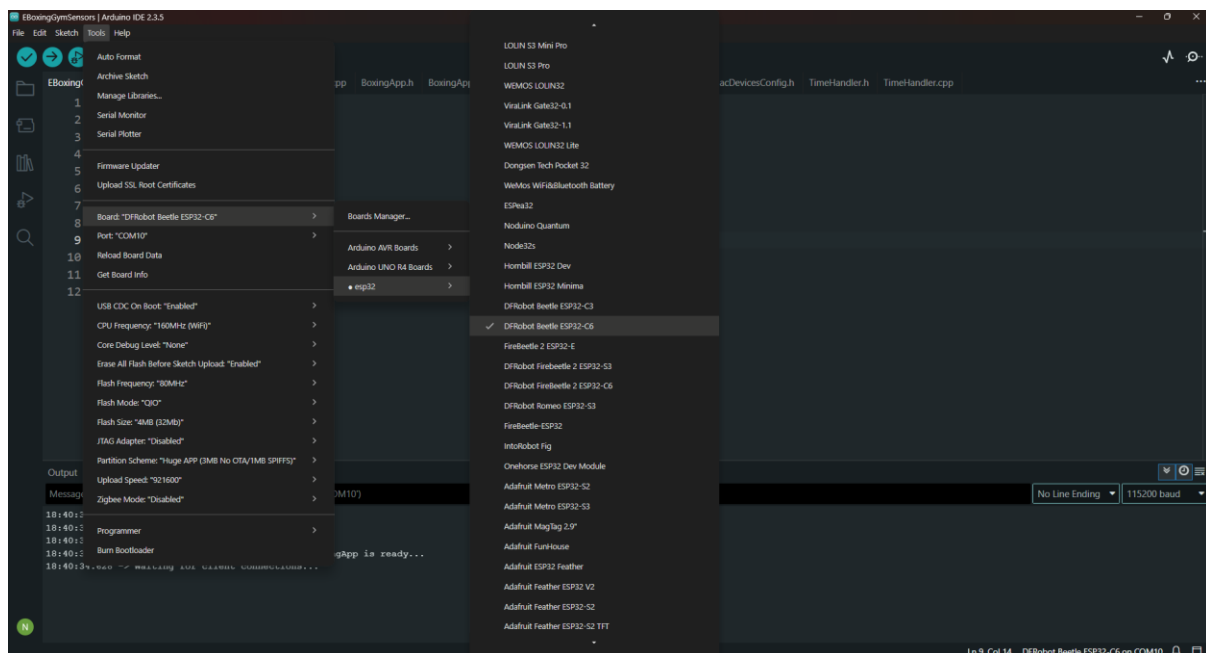
Η χρήση του Arduino IDE για τον προγραμματισμό του ESP32 προσφέρει μια απλοποιημένη διαδικασία ανάπτυξης. Οι προγραμματιστές μπορούν να αξιοποιήσουν την πληθώρα των διαθέσιμων βιβλιοθηκών και παραδειγμάτων κώδικα που έχουν αναπτυχθεί από την κοινότητα του Arduino. Η διαδικασία εγκατάστασης του πυρήνα ESP32 για το Arduino IDE είναι σχετικά απλή και περιλαμβάνει την προσθήκη μιας διεύθυνσης URL στον Board Manager του Arduino IDE, επιτρέποντας την εύκολη εγκατάσταση των απαραίτητων εργαλείων και του λογισμικού για την υποστήριξη του ESP32.

Η διεύθυνση αυτή για τον μικροελεγκτή που χρησιμοποιήσα είναι ή εξής: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json (εικόνα 23). Όπως ήδη γνωρίζουμε ο μικροελεγκτής που χρησιμοποιήθηκε για την πτυχιακή είναι ο Beetle ESP32-C6, και έχει τους αισθητήρες FSR συνδεδεμένους πάνω του.



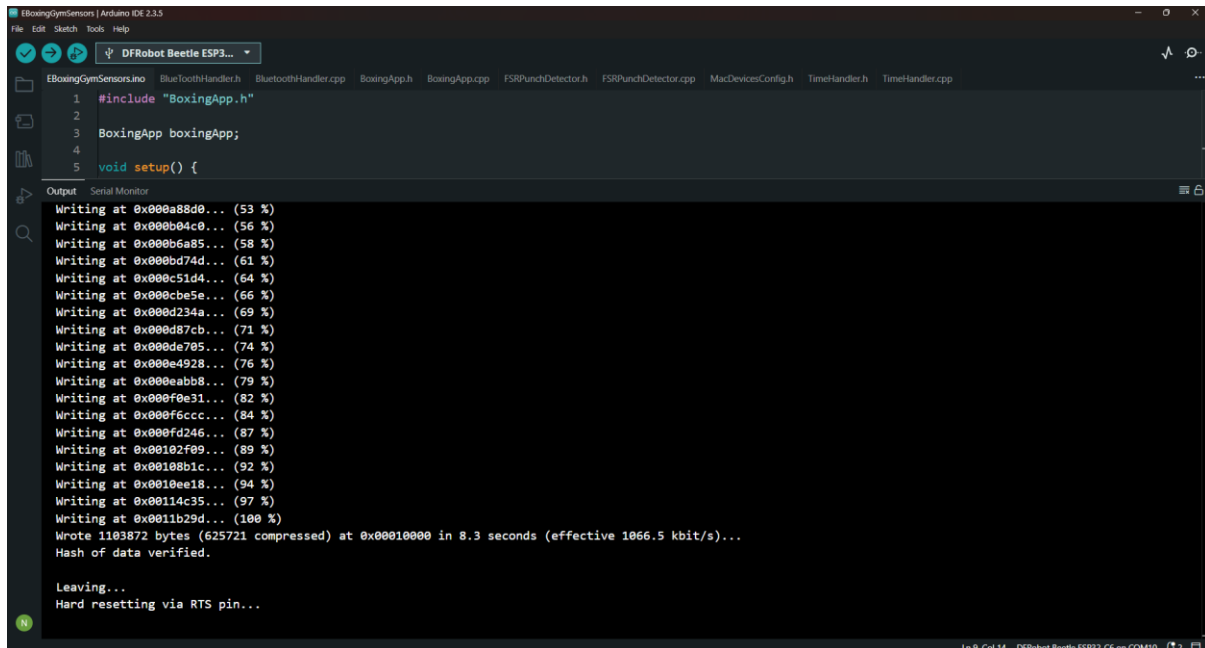
Εικόνα 21. Προσθήκη μιας διεύθυνσης URL στον Board Manager του Arduino IDE.

Μόλις ολοκληρωθεί η εγκατάσταση, οι χρήστες μπορούν να επιλέξουν την κατάλληλη πλακέτα ESP32 (εικόνα 24) από το μενού "Tools > Board" και να ξεκινήσουν τον προγραμματισμό χρησιμοποιώντας τη γλώσσα προγραμματισμού C/C++. Το περιβάλλον παρέχει βασικές λειτουργίες όπως η μεταγλώττιση του κώδικα και η μεταφόρτωσή του στον μικροελεγκτή.



Εικόνα 22. Επιλογή κατάλληλης πλακέτα ESP32.

Η διαδικασία που μεταγλωττίζεται ο κώδικας και ανεβαίνει στο μικροελεγκτή (εικόνα 25).



Εικόνα 23. Μεταγλώττιση και ανέβασμα του κώδικα στο Μικροελεγκτή.

Η ευκολία χρήσης, η εκτεταμένη υποστήριξη από την κοινότητα και η πληθώρα των διαθέσιμων πόρων καθιστούν τον προγραμματισμό του ESP32 με το Arduino IDE μια ελκυστική επιλογή τόσο για αρχάριους όσο και για έμπειρους προγραμματιστές.

Η Λειτουργία των κλάσεων καταγραφής επιδόσεων πυγμαχίας στο Beetle ESP32-C6.

Η παρούσα εφαρμογή έχει σχεδιαστεί για την παρακολούθηση και καταγραφή των επιδόσεων ενός αθλητή πυγμαχίας, αξιοποιώντας αισθητήρες FSR και το πρωτόκολλο επικοινωνίας Bluetooth Low Energy (BLE). Η κεντρική λογική του συστήματος ενσωματώνεται στην κλάση BoxingApp, η οποία αναλαμβάνει τον συντονισμό όλων των επιμέρους λειτουργιών. Κατά την εκκίνηση, η BoxingApp αρχικοποιεί τις απαραίτητες παραμέτρους, όπως η διάρκεια του γύρου, καθώς και οι τιμές ευαισθησίας και κατωφλίου για τους αισθητήρες. Επιπλέον, διαχειρίζεται την επικοινωνία με τον χρήστη μέσω εντολών που λαμβάνονται από εξωτερική συσκευή, επιτρέποντας την έναρξη, παύση, συνέχιση και επαναφορά των γύρων αγώνων ή προπόνησης.

Η επικοινωνία με εξωτερικές συσκευές, όπως κινητά τηλέφωνα ή tablets, πραγματοποιείται μέσω της κλάσης BluetoothHandler. Αυτή η κλάση είναι υπεύθυνη για την εγκαθίδρυση της σύνδεσης BLE, τη δημιουργία των απαραίτητων υπηρεσιών και χαρακτηριστικών για την αποστολή και λήψη δεδομένων. Μέσω αυτής, η εφαρμογή μπορεί να στέλνει πληροφορίες,

όπως τα δεδομένα των χτυπημάτων, και να λαμβάνει εντολές για τον έλεγχο της προπόνησης ή του αγώνα. Η αναγνώριση και η ονομασία της συσκευής κατά τη διαδικασία της σύζευξης BLE υποστηρίζεται από τις πληροφορίες που παρέχονται στο αρχείο MacDevicesConfig.h, το οποίο περιέχει αντιστοιχίσεις διευθύνσεων MAC με συγκεκριμένα ονόματα συσκευών.

Για την ανίχνευση των χτυπημάτων χρησιμοποιείται η κλάση FSRPunchDetector. Η κλάση αυτή αρχικοποιεί τους αισθητήρες FSR και παρακολουθεί συνεχώς τις τιμές τους. Όταν ένα χτύπημα ξεπεράσει μια προκαθορισμένη τιμή ευαισθησίας, καταγράφεται ως έγκυρο. Η FSRPunchDetector αναλαμβάνει επίσης την καταμέτρηση των χτυπημάτων και τη μορφοποίηση των σχετικών δεδομένων, όπως η χρονική στιγμή και η τάση του ρεύματος του αισθητήρα όταν καταγράφηκε το χτύπημα, ώστε να είναι έτοιμα προς αποστολή. Πρέπει να τονίσουμε ότι δεν υπάρχει κάποια συνάρτηση στον κώδικα η οποία μας δίνει τη δυνατότητα να καταγράψουμε την πραγματική δύναμη. Αφού οι αισθητήρες δεν παρέχουν τη δυνατότητα για μια ρεαλιστική προσέγγιση της δύναμης, δεν δημιουργήθηκε κάποια συνάρτηση ώστε να την καταγράψει.

Τέλος, η διαχείριση του χρόνου κατά τη διάρκεια της προπόνησης ή του αγώνα γίνεται από την κλάση TimeHandler. Αυτή η κλάση παρέχει λειτουργίες για την έναρξη, παύση, επαναφορά και συνέχιση χρονομέτρων, επιτρέποντας την ακριβή παρακολούθηση της διάρκειας των γύρων και των διαλειμμάτων. Η BoxingApp χρησιμοποιεί την TimeHandler για να ελέγχει την εξέλιξη του χρόνου και να τερματίζει τους γύρους όταν συμπληρωθεί η προκαθορισμένη διάρκεια.

Συνολικά, το πρόγραμμα συνδυάζει τη λειτουργικότητα αυτών των κλάσεων για να παρέχει ένα ολοκληρωμένο σύστημα παρακολούθησης προπόνησης πυγμαχίας ή αγώνα, από την ανίχνευση των χτυπημάτων και τη διαχείριση του χρόνου μέχρι την ασύρματη επικοινωνία των δεδομένων. Ο κώδικας που δημιουργήθηκε φορτώνεται στους δύο μικροελεγκτές που ονομάζονται BlueBoxer και RedBoxer στο μικρό αυτό πληροφοριακό σύστημα.

Λειτουργία κώδικα του Arduino Uno R4 Wi-Fi ως κεντρικού κόμβου Δεδομένων.

Ο κώδικας που αναπτύχθηκε για τον μικροελεγκτή Arduino Uno R4 Wi-Fi εστιάζει στον ρόλο του ως κεντρικού κόμβου επικοινωνίας και επεξεργασίας δεδομένων στο σύστημα παρακολούθησης πυγμαχίας. Η κύρια λειτουργικότητά του περιλαμβάνει την παραλαβή συγκεντρωτικών δεδομένων αγώνα από μια εξωτερική συσκευή (κινητό τηλέφωνο) μέσω

Bluetooth Low Energy (BLE), την ανάλυση αυτών των δεδομένων, και την ασφαλή προώθησή τους στην πλατφόρμα Arduino IoT Cloud μέσω σύνδεσης Wi-Fi.

Κατά την αρχικοποίηση (φάση setup), το σύστημα πρώτα εγκαθιστά την επικοινωνία με το Arduino IoT Cloud. Αυτό περιλαμβάνει την ενεργοποίηση της σύνδεσης Wi-Fi, χρησιμοποιώντας τα διαπιστευτήρια που παρέχονται, και την επαλήθευση της επιτυχούς σύνδεσης με την πλατφόρμα Cloud. Παράλληλα, ρυθμίζονται οι "Cloud Variables", οι οποίες αποτελούν τις γέφυρες δεδομένων μεταξύ του μικροελεγκτή και του Cloud. Μια σημαντική διαδικασία κατά την εκκίνηση είναι ο συγχρονισμός της τοπικής ώρας του μικροελεγκτή με την ώρα που παρέχεται από το Arduino IoT Cloud, διασφαλίζοντας την ακρίβεια των χρονικών σημάνσεων. Τέλος, ενεργοποιείται η λειτουργία Bluetooth Low Energy, καθιστώντας τον Arduino Uno R4 Wi-Fi ανιχνεύσιμο και έτοιμο να δέχεται συνδέσεις από την εφαρμογή του κινητού τηλεφώνου.

Στην κύρια επαναληπτική λειτουργία (φάση loop), ο μικροελεγκτής εκτελεί δύο βασικές διεργασίες συνεχώς. Πρώτον, διατηρεί ενεργή την επικοινωνία με το Arduino IoT Cloud, επιτρέποντας τον αμφίδρομο συγχρονισμό των Cloud Variables. Δεύτερον, παρακολουθεί για εισερχόμενα δεδομένα από τη συνδεδεμένη συσκευή Bluetooth.

Όταν δεδομένα σε μορφή JSON λαμβάνονται από το κινητό τηλέφωνο, ενεργοποιείται η διαδικασία επεξεργασίας. Μια εξειδικευμένη λογική, ενσωματωμένη ως κλάση JsonHandler εντός του σκίτσου, αναλαμβάνει την ανάλυση (parsing) του εισερχόμενου JSON string. Από αυτό εξάγονται οι επιμέρους πληροφορίες, όπως η ταυτότητα του πυγμάχου που δέχτηκε το χτύπημα, ο πυγμάχος που σημείωσε τον πόντο, η τιμή του αισθητήρα (όπως μεταφέρθηκε από το σύστημα καταγραφής) και η σχετική χρονική σήμανση. Αυτές οι πληροφορίες ενημερώνουν τις αντίστοιχες Cloud Variables. Οποιαδήποτε αλλαγή στις τιμές αυτών των μεταβλητών προωθείται αυτόματα από τον μικροελεγκτή στην πλατφόρμα Arduino IoT Cloud. Επιπλέον, ο κώδικας περιλαμβάνει βοηθητικές συναρτήσεις για τη διαχείριση και τη μορφοποίηση της ώρας, ώστε οι χρονικές σημάνσεις να είναι ακριβείς και ευανάγνωστες.

Είναι κρίσιμο να τονιστεί ότι ο Arduino Uno R4 Wi-Fi σε αυτή τη διαμόρφωση δεν είναι απευθείας συνδεδεμένος με τους φυσικούς αισθητήρες καταγραφής χτυπημάτων. Ο ρόλος του είναι να λειτουργεί ως ένας αξιόπιστος δέκτης και επεξεργαστής των δεδομένων που έχουν ήδη συλλεχθεί και πιθανώς προ-επεξεργαστεί από το σύστημα του κινητού τηλεφώνου. Αναλύει τα δομημένα δεδομένα JSON και τα προωθεί στο Cloud, λειτουργώντας ως η γέφυρα

μεταξύ της τοπικής συλλογής δεδομένων (μέσω κινητού) και της απομακρυσμένης πλατφόρμας παρακολούθησης και ανάλυσης. Αυτή η αρθρωτή αρχιτεκτονική επιτρέπει τον σαφή διαχωρισμό των λειτουργιών ανίχνευσης από την κεντρική διαχείριση και προώθηση των δεδομένων.

4.2 Βιβλιοθήκες ESP32 – Arduino

Βιβλιοθήκες και προσαρμοσμένες δομές κώδικα για το Beetle ESP32-C6

Για την υλοποίηση της λειτουργικότητας ανίχνευσης χτυπημάτων και επικοινωνίας στους μικροελεγκτές Beetle ESP32-C6, οι οποίοι λειτουργούν ως οι περιφερειακές μονάδες "BlueBoxer" και "RedBoxer", χρησιμοποιήθηκαν οι παρακάτω βασικές βιβλιοθήκες και αναπτύχθηκαν εξειδικευμένες κλάσεις:

ESP32 Arduino Core³² (esp32 by Espressif Systems)

Παρέχει το θεμελιώδες περιβάλλον εκτέλεσης και τις βασικές λειτουργίες για τον μικροελεγκτή ESP32-C6, συμπεριλαμβανομένης της διαχείρισης των GPIO για την ανάγνωση των αισθητήρων, των περιφερειακών επικοινωνίας και των χρονομετρητών. Είναι η βάση πάνω στην οποία χτίζεται όλη η εφαρμογή Arduino για το ESP32.

(BLEDevice, BLEServer, BLEUtils, BLE2902)^{33,34,35}

(μέρος του ESP32 Arduino Core για Bluetooth Low Energy).

Αυτές οι βιβλιοθήκες παρέχουν ένα ολοκληρωμένο API για τη λειτουργία του ESP32-C6 ως συσκευής Bluetooth Low Energy (BLE). Στο συγκεκριμένο έργο, ο ESP32-C6 λειτουργεί ως BLE server (peripheral), διαφημίζοντας τις υπηρεσίες του και επιτρέποντας τη σύνδεση από μια κεντρική συσκευή (το κινητό τηλέφωνο "BoxSensors"). Οι βιβλιοθήκες αυτές χρησιμοποιούνται για τη δημιουργία υπηρεσιών (services) και χαρακτηριστικών (characteristics) BLE, τη διαχείριση των συνδέσεων, την αποστολή ειδοποιήσεων (notifications) και τη λήψη δεδομένων. Ειδικότερα, το BLE2902 χρησιμοποιείται για την προσθήκη περιγραφών (descriptors) στα χαρακτηριστικά, επιτρέποντας στις συνδεδεμένες

³² Βλ. ESP32 Arduino Core, <https://github.com/espressif/arduino-esp32>.

³³ Βλ. GATT Server API, https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/bluetooth/esp_gatts.html.

³⁴ Βλ. GATT Client API, https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/bluetooth/esp_gattc.html

³⁵ Βλ. BLE examples, <https://github.com/espressif/arduino-esp32/tree/master/libraries/BLE/examples>

συσκευές να εγγραφούν για ειδοποιήσεις. Η ρύθμιση του MTU (Maximum Transmission Unit) γίνεται επίσης για βελτιωμένη συμβατότητα.

ArduinoJson³⁶ (έκδοση 7.4.1 του Benoit Blanchon)

Αυτή η δημοφιλής βιβλιοθήκη χρησιμοποιείται για την ανάλυση (deserialization) των εντολών που λαμβάνονται σε μορφή JSON μέσω BLE από την κεντρική εφαρμογή. Επιτρέπει την εύκολη εξαγωγή τιμών από τα μηνύματα JSON, όπως εντολές για την έναρξη/παύση γύρων ή την ενημέρωση ρυθμίσεων των αισθητήρων.

Standard C++ Libraries

Standard C++ Libraries (π.χ., <vector>, <string.h> στο πλαίσιο του ESP32 Arduino Core): Χρησιμοποιούνται για βασικές δομές δεδομένων και λειτουργίες, όπως το std::vector στην κλάση BluetoothHandler για τη διαχείριση των συνδεδεμένων clients (αν και στον παρεχόμενο κώδικα φαίνεται να έχει απλοποιηθεί η διαχείριση των clients).

Προσαρμοσμένες Κλάσεις (Custom Classes)

Για την οργάνωση και τη διαχείριση της πολυπλοκότητας της εφαρμογής στους ESP32-C6, αναπτύχθηκαν οι ακόλουθες εξειδικευμένες κλάσεις:

BoxingApp (BoxingApp.h, BoxingApp.cpp)

Αποτελεί την κεντρική κλάση ενορχήστρωσης της εφαρμογής σε κάθε Beetle ESP32-C6. Διαχειρίζεται τη συνολική λογική, αρχικοποιεί και συντονίζει τις άλλες κλάσεις (BluetoothHandler, FSRPunchDetector, TimeHandler). Είναι υπεύθυνη για την επεξεργασία των εντολών που λαμβάνονται μέσω Bluetooth (π.χ., έναρξη/παύση γύρου, ενημέρωση ρυθμίσεων), την ενεργοποίηση της ανίχνευσης χτυπημάτων και την αποστολή των δεδομένων των χτυπημάτων.

BluetoothHandler (BlueToothHandler.h, BluetoothHandler.cpp για ESP32)

Δημιουργήθηκε ως ένα εξειδικευμένο "περίβλημα" (wrapper) γύρω από τις βιβλιοθήκες BLE του ESP32 Arduino Core. Σκοπός της είναι η απλοποίηση της υλοποίησης της επικοινωνίας Bluetooth Low Energy. Αναλαμβάνει τη ρύθμιση του ESP32-C6 ως BLE server, τη δημιουργία των απαραίτητων υπηρεσιών (service) και χαρακτηριστικών (characteristics) για τη λήψη εντολών (RX) και την αποστολή δεδομένων (TX). Διαχειρίζεται επίσης τις συνδέσεις και τις

³⁶ Βλ. ArduinoJson Library by Benoit Blanchon, <https://arduinojson.org/>.

αποσυνδέσεις των client συσκευών και παρέχει μεθόδους για την αποστολή και λήψη μηνυμάτων.

FSRPunchDetector (FSRPunchDetector.h, FSRPunchDetector.cpp)

Η κλάση αυτή είναι υπεύθυνη για την ανίχνευση των χτυπημάτων μέσω των αισθητήρων FSR. Αρχικοποιεί τα pins των αισθητήρων και παρακολουθεί συνεχώς τις αναλογικές τους τιμές (σε millivolts). Εφαρμόζει λογική debounce για την αποφυγή πολλαπλών καταγραφών από ένα μόνο χτύπημα και χρησιμοποιεί προκαθορισμένες τιμές ευαισθησίας (sensitivity) και κατωφλίου (threshold) για την επικύρωση ενός χτυπήματος. Όταν ανιχνευτεί έγκυρο χτύπημα, αυξάνει τον μετρητή χτυπημάτων και μορφοποιεί τις σχετικές πληροφορίες (όπως η χρονική στιγμή και η ένταση του σήματος του αισθητήρα) σε ένα string έτοιμο προς αποστολή

TimeHandler (TimeHandler.h, TimeHandler.cpp)

Αυτή η κλάση παρέχει λειτουργίες για τη διαχείριση του χρόνου κατά τη διάρκεια των γύρων προπόνησης ή αγώνα. Επιτρέπει την έναρξη, παύση, συνέχιση και επαναφορά χρονομέτρων, δίνοντας τη δυνατότητα στην BoxingApp να παρακολουθεί με ακρίβεια τη διάρκεια των γύρων και να τερματίζει έναν γύρο όταν συμπληρωθεί ο προκαθορισμένος χρόνος

MacDevicesConfig.h

Αυτό το αρχείο κεφαλίδας δεν είναι κλάση, αλλά παίζει σημαντικό ρόλο στην αναγνώριση της συσκευής. Περιέχει πίνακες με προκαθορισμένες διευθύνσεις MAC και τα αντίστοιχα ονόματα ("BlueBoxer", "RedBoxer"). Κατά την εκκίνηση, η BoxingApp διαβάζει την BLE MAC address του ESP32-C6 και τη συγκρίνει με τις λίστες σε αυτό το αρχείο για να προσδιορίσει το όνομα με το οποίο θα διαφημιστεί η συσκευή μέσω BLE. Αυτό επιτρέπει την αυτόματη ανάθεση του σωστού ονόματος σε κάθε μικροελεγκτή.

Όλες οι προαναφερθείσες βιβλιοθήκες και οι προσαρμοσμένες κλάσεις συνεργάζονται για να επιτρέψουν σε κάθε Beetle ESP32-C6 να ανιχνεύει χτυπήματα, να διαχειρίζεται την κατάσταση του γύρου και να επικοινωνεί ασύρματα με την κεντρική εφαρμογή στο κινητό τηλέφωνο, συμβάλλοντας στην ολοκληρωμένη λειτουργία του συστήματος παρακολούθησης πυγμαχίας.

Βιβλιοθήκες για το Arduino Uno R4 Wi-Fi.

Παρέχει το runtime περιβάλλον για τον Renesas RA4M1 (GPIO, timers, UART, SPI, I²C κ.λπ.) και τον χειρισμό των βασικών επιπέδων του board.

Arduino Core for Arduino Uno R4 Wi-Fi³⁷.

Παρέχει το runtime περιβάλλον για τον Renesas RA4M1 (GPIO, timers, UART, SPI, I²C κ.λπ.) και τον χειρισμό των βασικών επιπέδων του board.

WiFi (έκδοση 1.9.1)³⁸

Αποτελεί τη θεμελιώδη βιβλιοθήκη του Arduino core για την πλακέτα Uno R4, παρέχοντας τις βασικές λειτουργίες για την αλληλεπίδραση με την ενσωματωμένη μονάδα Wi-Fi (u-blox NINA-W102).

ArduinoIoTCloud³⁹ (έκδοση 2.5.1)

Υψηλού επιπέδου API που αναλαμβάνει την αυθεντικοποίηση, το synchronization των cloud μεταβλητών και τη διαχείριση των events με την Arduino IoT Cloud.

Arduino_ConnectionHandler

Παρέχει τον WiFiConnectionHandler μέσω του αρχείου thingProperties.h, ο οποίος χρησιμοποιεί το SSID και το PASS από το arduino_secrets.h για να διαχειριστεί τη σύνδεση Wi-Fi. Αποτελεί μέρος του **ArduinoIoTCloud**.

ArduinoBLE⁴⁰ (έκδοση 1.4.0)

Υλοποιεί API για λειτουργία central/peripheral σε Bluetooth Low Energy. Στο έργο αυτό, χρησιμοποιείται για να επιτρέψει στον UNO R4 Wi-Fi να λειτουργεί ως BLE peripheral (Server), λαμβάνοντας μηνύματα JSON από το συνδεδεμένο κινητό τηλέφωνο.

ArduinoJson⁴¹ (έκδοση 7.4.1)

Επιτρέπει την ανάλυση (deserialization) και δημιουργία (serialization) δεδομένων JSON. Στο έργο, χρησιμοποιείται για την ανάλυση των μηνυμάτων JSON που λαμβάνονται μέσω BLE από το κινητό, μετατρέποντάς τα σε αντικείμενα StaticJsonDocument για εύκολη εξαγωγή των δεδομένων.

TimeLib⁴² (έκδοση 1.6.0)

Η κλάση BluetoothHandler δημιουργήθηκε ως ένα εξειδικευμένο "περίβλημα" (wrapper) γύρω από την κεντρική βιβλιοθήκη ArduinoBLE. Σκοπός της είναι να απλοποιήσει την υλοποίηση

³⁷ Βλ. Arduino Core, <https://github.com/arduino/ArduinoCore-renesas>

³⁸ Βλ. WiFi library, <https://github.com/arduino/ArduinoCore-renesas/tree/main/libraries/WiFi>

³⁹ Βλ. ArduinoIoTCloud Library, <https://docs.arduino.cc/arduino-cloud/>

⁴⁰ Βλ. ArduinoBLE Library, <https://www.arduino.cc/reference/en/libraries/arduinoable/>

⁴¹ Βλ. ArduinoJson Library by Benoit Blanchon, <https://arduinojson.org/>

⁴² Βλ. TimeLib Library by Paul Stoffregen, <https://github.com/PaulStoffregen/Time>

της επικοινωνίας Bluetooth Low Energy (BLE) στον Arduino Uno R4 Wi-Fi, διαχειριζόμενη τη ρύθμιση της συσκευής ως BLE Server και παρέχοντας απλές μεθόδους για τη λήψη (και δυνητικά αποστολή) δεδομένων από και προς το συνδεδεμένο κινητό τηλέφωνο.

Τα αρχεία `thingProperties.h` & `arduino_secrets.h` ορίζουν τις cloud μεταβλητές και τα Wi-Fi credentials.

Κάθε μία από αυτές τις βιβλιοθήκες εξυπηρετεί κρίσιμες λειτουργίες: από τον έλεγχο της ασύρματης επικοινωνίας, την επεξεργασία JSON, μέχρι τον συγχρονισμό χρόνου και τη μόνιμη αποθήκευση ρυθμίσεων, διασφαλίζοντας αξιοπιστία και επεκτασιμότητα του project.

Για την εξειδικευμένη λειτουργικότητα του Arduino Uno R4 Wi-Fi, αναπτύχθηκαν προσαρμοσμένες δομές κώδικα. Συγκεκριμένα, η κλάση `BluetoothHandler` (υλοποιημένη στα αρχεία `BluetoothHandler.h` και `BluetoothHandler.cpp`) δημιουργήθηκε ως ένα "περίβλημα" (wrapper) γύρω από την κεντρική βιβλιοθήκη `ArduinoBLE`. Σκοπός της είναι η απλοποίηση της διαχείρισης της επικοινωνίας Bluetooth Low Energy (BLE), ρυθμίζοντας τη συσκευή ως BLE Server (peripheral) και παρέχοντας εύχρηστες μεθόδους για τη λήψη δεδομένων από το συνδεδεμένο κινητό τηλέφωνο. Επιπλέον, η κλάση `JsonHandler`, που ορίζεται εντός του κυρίως αρχείου (σκίτσο `.ino`), περιλαμβάνει τη στατική μέθοδο `parseIncoming` για την αποτελεσματική ανάλυση των μηνυμάτων JSON που λαμβάνονται μέσω BLE και την ενημέρωση των αντίστοιχων Cloud Variables. Τέλος, το αρχείο `thingProperties.h` (αυτόματης παραγωγής από το Arduino IoT Cloud) είναι αυτό που ορίζει τις μεταβλητές που είναι υπεύθυνες για την ανταλλαγή δεδομένων μεταξύ του Thing και του Dashboard. Επίσης το `arduino_secrets.h` που περιέχει τα διαπιστευτήρια του χρήστη είναι θεμελιώδες για την σύνδεση Wi-Fi που αρχικοποιεί τον χειριστή της σύνδεσης. Όλες οι προαναφερθείσες βιβλιοθήκες και αυτές οι προσαρμοσμένες δομές κώδικα, συνεργάζονται για να επιτελέσουν κρίσιμες λειτουργίες, όπως ο έλεγχος της ασύρματης επικοινωνίας (Wi-Fi και Bluetooth), η επεξεργασία των μηνυμάτων JSON, ο συγχρονισμός των δεδομένων με την πλατφόρμα Arduino IoT Cloud και η διαχείριση του χρόνου, διασφαλίζοντας έτσι την αξιοπιστία και την επεκτασιμότητα του έργου.

Custom Modules

Η κλάση `BluetoothHandler` δημιουργήθηκε ως ένα εξειδικευμένο "περίβλημα" (wrapper) γύρω από την κεντρική βιβλιοθήκη `ArduinoBLE`. Σκοπός της είναι να απλοποιήσει την υλοποίηση της επικοινωνίας Bluetooth Low Energy (BLE) στον Arduino Uno R4 Wi-Fi, διαχειριζόμενη

τη ρύθμιση της συσκευής ως BLE Server και παρέχοντας απλές μεθόδους για τη λήψη (και δυνητικά αποστολή) δεδομένων από και προς το συνδεδεμένο κινητό τηλέφωνο.

Τα αρχεία `thingProperties.h` & `arduino_secrets.h` ορίζουν τις cloud μεταβλητές και τα Wi-Fi credentials.

Κάθε μία από αυτές τις βιβλιοθήκες εξυπηρετεί κρίσιμες λειτουργίες: από τον έλεγχο της ασύρματης επικοινωνίας, την επεξεργασία JSON, μέχρι τον συγχρονισμό χρόνου και τη μόνιμη αποθήκευση ρυθμίσεων, διασφαλίζοντας αξιοπιστία και επεκτασιμότητα του project.

Για την εξειδικευμένη λειτουργικότητα του Arduino Uno R4 Wi-Fi, αναπτύχθηκαν προσαρμοσμένες δομές κώδικα. Συγκεκριμένα, η κλάση `BluetoothHandler` (υλοποιημένη στα αρχεία `BluetoothHandler.h` και `BluetoothHandler.cpp`) δημιουργήθηκε ως ένα "περίβλημα" (wrapper) γύρω από την κεντρική βιβλιοθήκη `ArduinoBLE`. Σκοπός της είναι η απλοποίηση της διαχείρισης της επικοινωνίας Bluetooth Low Energy (BLE), ρυθμίζοντας τη συσκευή ως BLE Server (peripheral) και παρέχοντας εύχρηστες μεθόδους για τη λήψη δεδομένων από το συνδεδεμένο κινητό τηλέφωνο. Επιπλέον, η κλάση `JsonHandler`, που ορίζεται εντός του κυρίως αρχείου (σκίτσο .ino), περιλαμβάνει τη στατική μέθοδο `parseIncoming` για την αποτελεσματική ανάλυση των μηνυμάτων JSON που λαμβάνονται μέσω BLE και την ενημέρωση των αντίστοιχων Cloud Variables. Τέλος, το αρχείο `thingProperties.h` (αυτόματης παραγωγής από το Arduino IoT Cloud) είναι αυτό που ορίζει τις μεταβλητές που είναι υπεύθυνες για την ανταλλαγή δεδομένων μεταξύ του Thing και του Dashboard. Επίσης το `arduino_secrets.h` που περιέχει τα διαπιστευτήρια του χρήστη είναι θεμελιώδες για την σύνδεση Wi-Fi που αρχικοποιεί τον χειριστή της σύνδεσης. Όλες οι προαναφερθείσες βιβλιοθήκες και αυτές οι προσαρμοσμένες δομές κώδικα, συνεργάζονται για να επιτελέσουν κρίσιμες λειτουργίες, όπως ο έλεγχος της ασύρματης επικοινωνίας (Wi-Fi και Bluetooth), η επεξεργασία των μηνυμάτων JSON, ο συγχρονισμός των δεδομένων με την πλατφόρμα Arduino IoT Cloud και η διαχείριση του χρόνου, διασφαλίζοντας έτσι την αξιοπιστία και την επεκτασιμότητα του έργου.

4.3 Ανάπτυξη εφαρμογής Android

Ανάπτυξη εφαρμογής Android.

Για να αναπτυχθεί η εφαρμογή χρησιμοποιήθηκε το Flutter⁴³ το οποίο αποτελεί ένα ανοιχτού κώδικα framework για την ανάπτυξη διεπαφών χρήστη, το οποίο έχει σχεδιάσει η Google. Μέσω μίας ενιαίας βάσης κώδικα, επιτρέπει τη δημιουργία εφαρμογών για τον Παγκόσμιο Ιστό, το Fuchsia, το Android, το iOS, καθώς και για desktop λειτουργικά όπως Linux, macOS και Windows. Για να δημιουργηθεί η εφαρμογή στο λειτουργικό σύστημα Android, χρησιμοποιήθηκε το Android Studio⁴⁴ (εικόνα 26) όπου περιλαμβάνει το Android SDK⁴⁵ το οποίο είναι η βάση την οποία χρειάζεται το Flutter SDK⁴⁶ για να λειτουργήσει. Η γλώσσα προγραμματισμού του Flutter framework είναι η Dart⁴⁷ οποία είναι μια αντικειμενοστραφής γλώσσα προγραμματισμού, βασισμένη σε κλάσεις και με αυτόματη διαχείριση μνήμης (garbage collection), η οποία υιοθετεί σύνταξη τύπου C. Η μεταγλώττισή της μπορεί να παράγει είτε κώδικα μηχανής, είτε JavaScript, είτε WebAssembly. Επιπλέον, υποστηρίζει διασυνδέσεις (interfaces), αφηρημένες κλάσεις, παραμετρικές κλάσεις με πραγματική αναπαράσταση τύπου (reified generics) και συμπερασμό τύπων (type inference). Το IDE περιβάλλον που χρησιμοποιήθηκε είναι το Visual Studio Code⁴⁸ ώστε να γραφτεί ο κώδικας. Το κινητό τηλέφωνο που χρησιμοποίησα είναι το Honor10 Lite με λειτουργικό Android 10. Το όνομα της εφαρμογής είναι Box Sensors.

Το Flutter, πέραν του ότι είναι ένα UI toolkit ανοιχτού κώδικα από την Google, διακρίνεται για την πολυεπίπεδη αρχιτεκτονική του, η οποία αποτελεί τη βάση για την ανάπτυξη εφαρμογών υψηλής απόδοσης και αισθητικής από μία ενιαία βάση κώδικα. Αυτή η αρχιτεκτονική περιλαμβάνει τρία κύρια επίπεδα: το Dart Framework, με το οποίο αλληλεπιδρούν οι προγραμματιστές και το οποίο παρέχει πλούσιες βιβλιοθήκες, widgets και εργαλεία για τη δημιουργία της διεπαφής χρήστη, την C++ Engine, η οποία είναι υπεύθυνη για την απόδοση γραφικών χαμηλού επιπέδου, χρησιμοποιώντας μηχανισμούς όπως η Skia⁴⁹ ή η νεότερη Impeller⁵⁰ για βελτιστοποιημένη απόδοση και ομαλά κινούμενα γραφικά και τον

⁴³ Βλ. Google, *Flutter*, <https://flutter.dev/>.

⁴⁴ Βλ. Google, *Android Studio*, <https://developer.android.com/studio>.

⁴⁵ Βλ. Google, *Develop for Android*, <https://developer.android.com/develop>.

⁴⁶ Βλ. Google, *Flutter SDK overview*, <https://docs.flutter.dev/tools/sdk>.

⁴⁷ Βλ. Google, *Dart*, <https://dart.dev/>.

⁴⁸ Βλ. Microsoft, *Visual Studio Code*, <https://code.visualstudio.com/>.

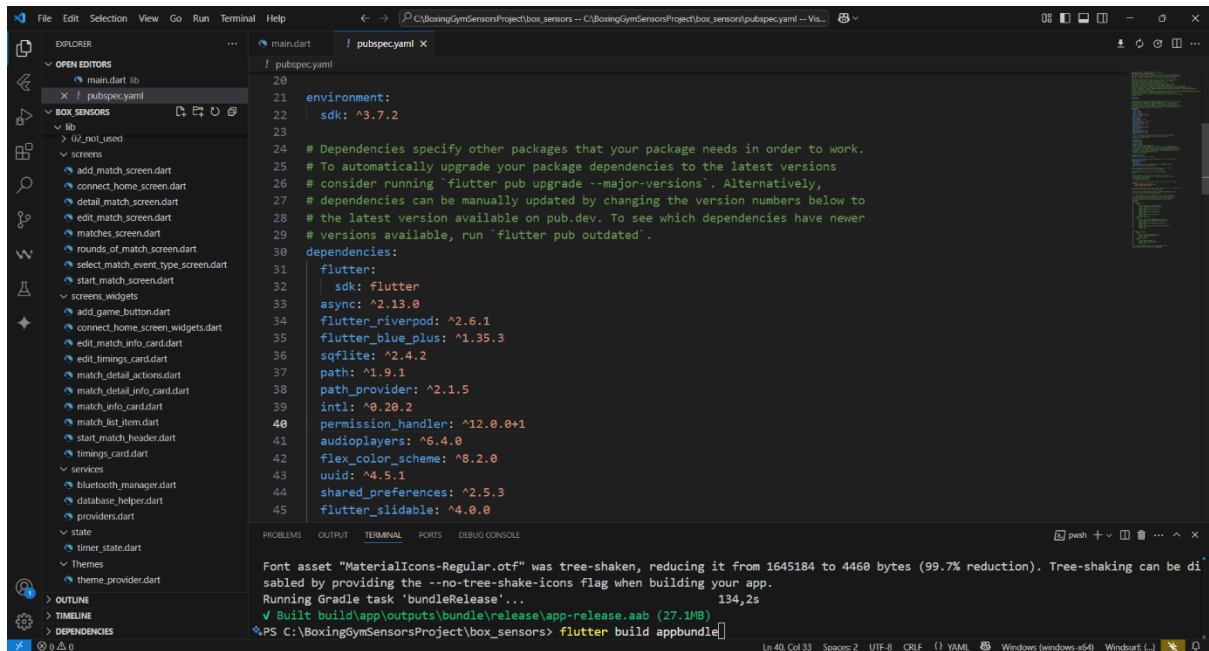
⁴⁹ Βλ. See The Skia Project, *Skia Graphics Engine*, <https://skia.org/>.

⁵⁰ Βλ. Google, *Impeller*, <https://docs.flutter.dev/perf/impeller>.

Embedder, ένα τμήμα κώδικα προσαρμοσμένο στην εκάστοτε πλατφόρμα, το οποίο λειτουργεί ως μεσολαβητής, επιτρέποντας στην εφαρμογή Flutter να αξιοποιεί τις εγγενείς υπηρεσίες και τους πόρους του λειτουργικού συστήματος. Η ανάπτυξη στο Flutter επικεντρώνεται γύρω από τα widgets, τα οποία είναι αμετάβλητα δομικά στοιχεία που περιγράφουν την εμφάνιση και τη διάταξη της διεπαφής. Η φιλοσοφία του Flutter ότι "η UI είναι συνάρτηση της κατάστασης" (UI as a function of state⁵¹) οδηγεί σε ένα δηλωτικό μοντέλο προγραμματισμού, όπου η διεπαφή χρήστη ανακατασκευάζεται αυτόματα και αποδοτικά ως απόκριση σε αλλαγές της κατάστασης της εφαρμογής, απλοποιώντας τη διαχείριση πολύπλοκων διεπαφών.

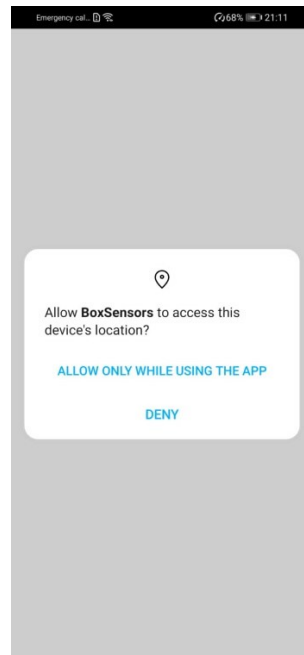
Η επιλογή της Dart ως γλώσσας προγραμματισμού για το Flutter δεν είναι τυχαία, καθώς η Dart είναι βελτιστοποιημένη από την Google ειδικά για την ανάπτυξη διεπαφών χρήστη που απαιτούν υψηλή απόδοση και ταχεία ανάπτυξη. Ένα από τα ισχυρότερα πλεονεκτήματά της είναι ο διττός μηχανισμός μεταγλώττισης: η Just-In-Time (JIT) μεταγλώττιση όπου κατά την ανάπτυξη επιτρέπει την επαναστατική λειτουργία του stateful hot reload, προσφέροντας άμεση οπτικοποίηση των αλλαγών, ενώ η Ahead-Of-Time (AOT) μεταγλώττιση σε εγγενή κώδικα μηχανής για τις εκδόσεις παραγωγής εξασφαλίζει ταχύτατη εκκίνηση και ομαλή εκτέλεση της εφαρμογής. Επιπλέον των χαρακτηριστικών που ήδη αναφέρθηκαν, η Dart ενσωματώνει την ισχυρή ασφάλεια έναντι τιμών null (sound null safety), η οποία προλαμβάνει σφάλματα χρόνου εκτέλεσης ανιχνεύοντάς τα κατά τη μεταγλώττιση και συμβάλλει στη δημιουργία πιο αξιόπιστου κώδικα. Η γλώσσα παρέχει επίσης εξελιγμένη υποστήριξη για ασύγχρονο προγραμματισμό μέσω των async/await, Futures και Streams, απαραίτητη για να εξασφαλίζεται η υψηλή ταχύτητα απόκρισης της διεπαφής χρήστη κατά την εκτέλεση παρασκηνιακών διεργασιών. Η προηγμένη και αποδοτική αυτόματη διαχείριση μνήμης (garbage collection) της Dart εξασφαλίζει ομαλή απόδοση, ακόμα και σε διεπαφές χρήστη με υψηλές απαιτήσεις πόρων και σύνθετα κινούμενα γραφικά.

⁵¹ Βλ. Google, *UI State production*, <https://developer.android.com/topic/architecture/ui-layer/state-production>.



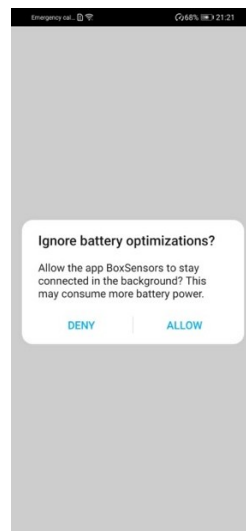
Εικόνα 24. Το IDE Visual Studio Code.

Στην παρούσα ενότητα, θα αναλυθεί ο τρόπος λειτουργίας της εφαρμογής που αναπτύχθηκε: Όταν ο χρήστης ξεκινήσει την εφαρμογή, του ζητάει να ενεργοποιήσει τα Location Service's. Στο κινητό που έγινε η ανάπτυξη της εφαρμογής εμφανίζεται η παρακάτω εικόνα 27. Ο χρήστης πρέπει να επιλέξει “ALLOW ONLY WHILE USING THE APP” το μήνυμα μπορεί να διαφέρει σε άλλες συσκευές και άλλες εκδόσεις Android, γιατί αλλιώς δεν θα μπορεί να βρει τους τρεις μικροελεγκτές που έχουν πάνω τους Bluetooth Low Energy.



Εικόνα 25. Η οθόνη που ζητά στο χρήστη να ενεργοποιήσει τα Location Service's.

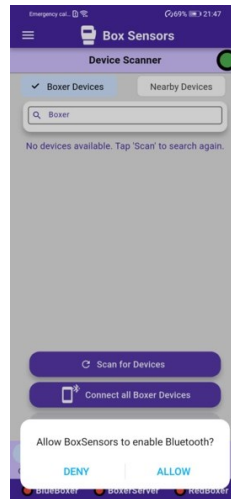
Όταν ο χρήστης κάνει την επιλογή εμφανίζεται μια νέα οθόνη η οποία του ζητά αν θέλει να κάνει Ignore battery optimizations (εικόνα 28), μπορεί να επιλέξει ALLOW οπότε η εφαρμογή θα καταναλώνει περισσότερη ενέργεια, μπορεί όμως να επιλέξει και DENY οπότε ενεργοποιείται η δυνατότητα του Android να διαχειρίζεται την ενέργεια της εφαρμογής.



Εικόνα 26. Η οθόνη που ζητά στο χρήστη να ενεργοποιήσει η όχι την βελτιστοποίηση διαχείρισης ενέργειας της μπαταρίας για την εφαρμογή.

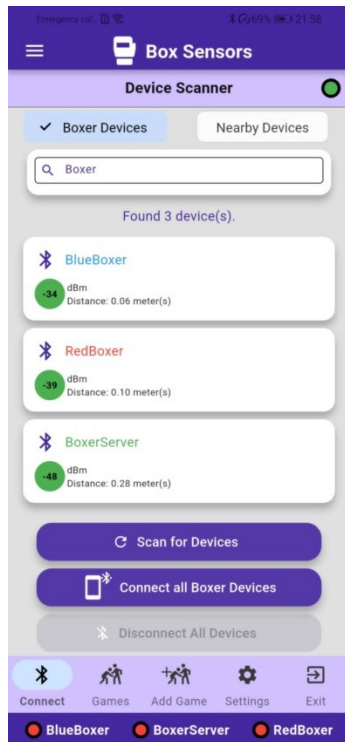
Όταν ο χρήστης κάνει την επιλογή του τότε εμφανίζεται η κεντρική οθόνη της εφαρμογής και αν το Bluetooth του κινητού δεν είναι ήδη ενεργοποιημένο εμφανίζεται ένας διάλογος για να

το ενεργοποιήσει (εικόνα 29). Εδώ ο χρήστης πρέπει να επιλέξει ALLOW ώστε να έχει την δυνατότητα να συνδεθεί στους μικροελεγκτές.

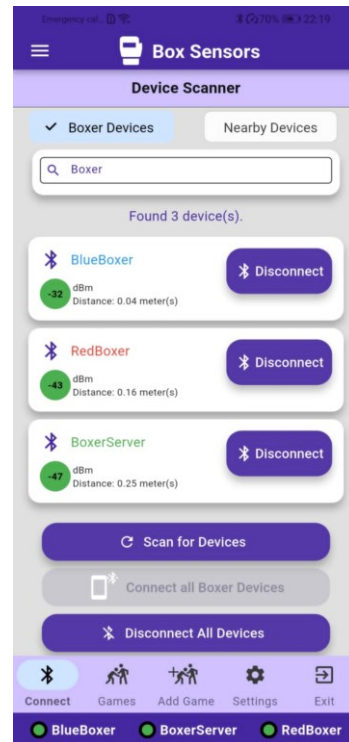


Εικόνα 27. Διάλογος για να την ενεργοποίηση του Bluetooth.

Αμέσως μετά εμφανίζεται η κεντρική οθόνη με τούς τρεις μικροελεγκτές, BlueBoxer, RedBoxer, BoxerServer. Οι δύο πρώτοι είναι οι μικροελεγκτές που συλλέγουν δεδομένα από τους αισθητήρες και τα στέλνουν στην εφαρμογή και ο τρίτος στέλνει τα δεδομένα στο Arduino IoT Cloud αφού τα λάβει από το κινητό τηλέφωνο. Στη οθόνη υπάρχουν κάτω 3 κουμπιά τα οποία μας διευκολύνουν να ανιχνεύσουμε συσκευές Bluetooth, να συνδεθούμε σε αυτές και να αποσυνδεθούμε. Μπορούμε όμως και να συνδεθούμε χωριστά στην κάθε συσκευή, πατώντας πάνω στην κάρτα με το όνομα της συσκευής όπως και να αποσυνδεθούμε με κουμπί που εμφανίζεται όταν οι συσκευές είναι συνδεδεμένες. Αν ο χρήστης πατήσει πάνω στην επιλογή Nearby Devices μπορεί να δει όλες τις συσκευές που είναι στην εμβέλεια του κινητού χωρίς να μπορεί να συνδεθεί σε αυτές αφού δεν ανήκουν στο πληροφοριακό σύστημα. Υπάρχει η δυνατότητα να φιλτράρει ο χρήστης τις συσκευές με όποια συμβολοσειρά θέλει αλλά δεν μπορεί να συνδεθεί πάνω τους, καθότι επιτρέπεται να συνδεθούν στο κινητό μόνο οι τρεις συγκεκριμένοι μικροελεγκτές (εικόνες 30, 31).

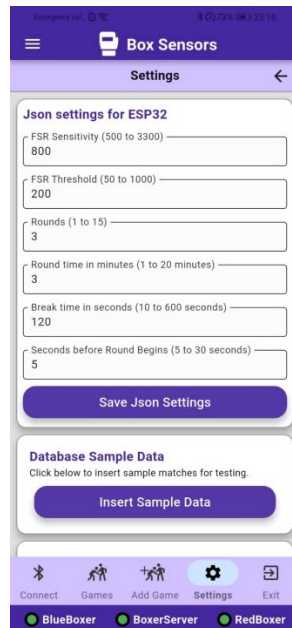


Εικόνα 30. Βλέπουμε τους μικροελεγκτές αποσυνδεδεμένους στο κινητό.



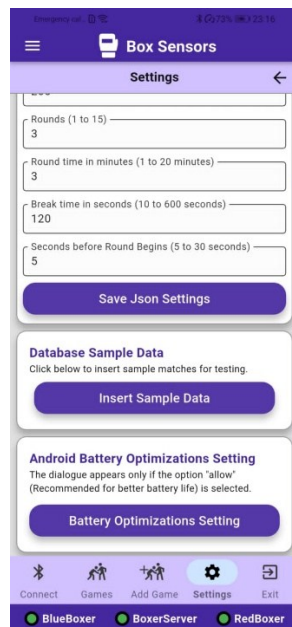
Εικόνα 31. Βλέπουμε τους μικροελεγκτές συνδεδεμένους στο κινητό.

Αν επιλέξει ο χρήστης την οθόνη Settings έχει την δυνατότητα να κάνει ρυθμίσεις που αφορούν τους δύο μικροελεγκτές BlueBoxer και RedBoxer. Οι ρυθμίσεις αυτές αφορούν τούς αισθητήρες FSR των μικροελεγκτών και συγκεκριμένα την ευαισθησία τους. Η ρύθμιση FSR Sensitivity μπορεί να πάρει τιμές από 500 έως 3300. Με τη μέγιστη να μην ενδείκνυται να επιλεγεί. Δηλώνει από ποια τιμή και πάνω καταμετράται το χτύπημα. Η ρύθμιση FSR Threshold 50 έως 1000, ειδική ρύθμιση ώστε ο αισθητήρας να καταμετρήσει ξανά ένα χτύπημα, αφού το ρεύμα που πέρνα από την αντίσταση έχει μειωθεί στην τιμή αυτή. Με αυτόν τον τρόπο αποτρέπει να μετρούνται πολλαπλά χτυπήματα κατά λάθος. Ρύθμιση Rounds για το πόσους γύρους μπορεί να κρατά ένα παιχνίδι. Ρύθμιση για την διάρκεια του γύρου σε λεπτά. Ρύθμιση για τη διάρκεια του διαλείμματος μεταξύ των γύρων. Ρύθμιση για την αντίστροφη μέτρηση όταν ξεκινά ένας αγώνας. Το κουμπί Save Json Settings αποθηκεύει στη βάση δεδομένων τις ρυθμίσεις (εικόνα 32),



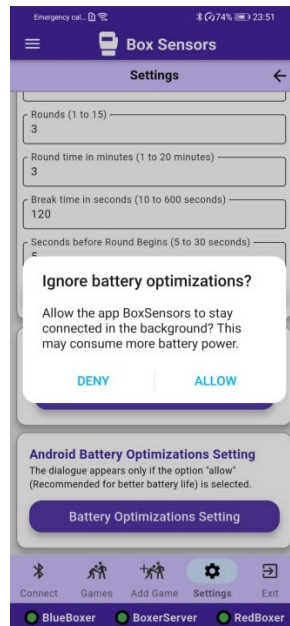
Εικόνα 28. Settings των αισθητήρων FSR

Κουμπί Insert Sample Data για πιο γρήγορη εισαγωγή ενός δείγματος από αγώνες για τις δοκιμές (εικόνα 33).



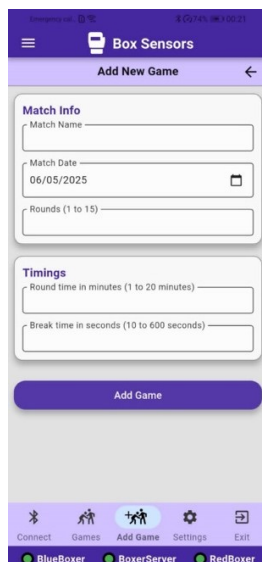
Εικόνα 29. Κουμπί Insert Sample Data.

Κουμπί Battery Optimization Setting (εικόνα 34) το οποίο εμφανίζει το παράθυρο διαλόγου μόνο στη περίπτωση που το Ignore battery optimizations δεν έχει ήδη επιλεγεί. Η συγκεκριμένη οθόνη έχει δυνατότητα κύλισης (scrolling).

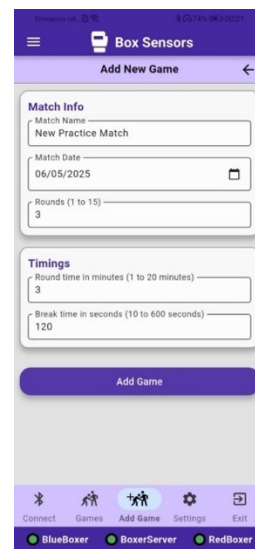


Εικόνα 30. Battery Optimization Settings.

Όταν ο χρήστης επιλέξει την οθόνη Add Game (εικόνες 35, 36) έχει την δυνατότητα να εισάγει ένα νέο Game στη βάση δεδομένων. Τα πεδία που μπορεί να συμπληρώσει είναι τα Match Name, Match Date, Rounds, Round time in minutes, Break time in seconds. Σε κάποια από τα πεδία επιτρέπεται να εισάγει ένα εύρος τιμών.

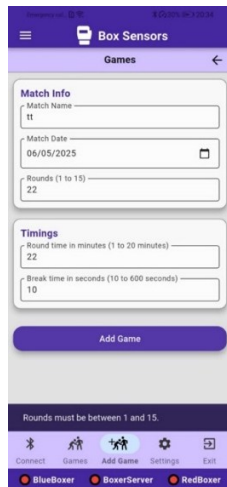


Εικόνα 31. Add Game

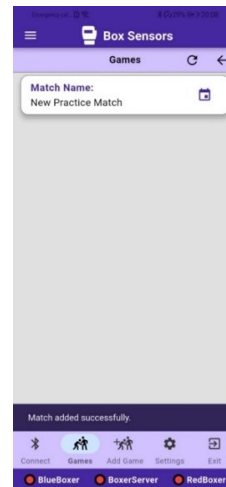


Εικόνα 32. Add Game

Όταν ολοκληρώσει την εισαγωγή και πατήσει το κουμπί Add Game εισάγεται το Game που δημιούργησε στη βάση δεδομένων (εικόνες 37, 38).

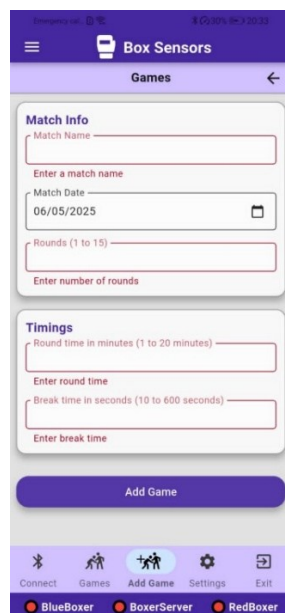


Εικόνα 33. Οι γύροι είναι εκτός επιτρεπόμενων ορίων.



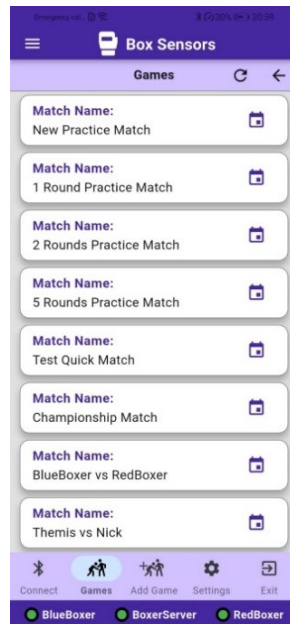
Εικόνα 34. Το Game που δημιούργησε προστίθεται στη βάση δεδομένων.

Αν υπάρχουν λάθη στη εισαγωγή στην οθόνη βγαίνουν κόκκινες επισημάνσεις (εικόνα 39), πχ αν δεν δώσει ένα Match Name, που του εξηγούν τι πρέπει να διορθωθεί, με το εύρος τιμών που επιτρέπονται σε καθε πεδίο ή βγαίνει ένα Snackbar που ενημερώνει παλι με το εύρος τιμών που επιτρέπονται.



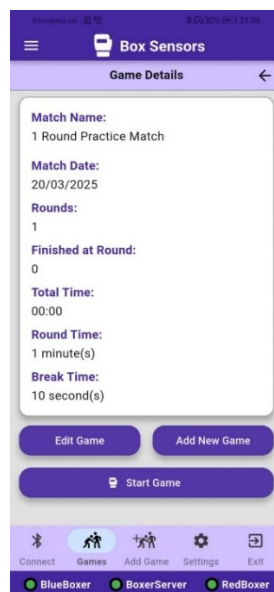
Εικόνα 35. Κόκκινες επισημάνσεις για τις λανθασμένες επιλογές.

Όταν ο χρήστης πατήσει στο Navigation Bar την επιλογή Games εμφανίζεται η οθόνη Games που περιέχει τα αποθηκευμένα Games της εφαρμογής όπως φαίνεται στην εικόνα 40.



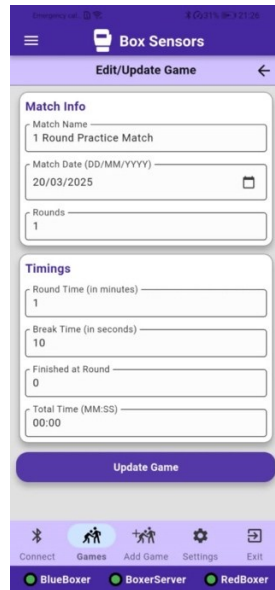
Εικόνα 36. Η οθόνη Games.

Εάν πατήσει ο χρήστης πάνω σε μια κάρτα που περιέχει το όνομα ενός Game μεταφέρεται σε μια νέα οθόνη την Game Details (εικόνα 41). Στο πάνω μέρος της περιέχει μια κάρτα με τα πεδία ενός Game. Τα πεδία είναι τα εξής: Match name, Match Date, Rounds, Finished at Round, Total Time, Round Time, Break Time. Η οθόνη περιέχει τρία κουμπιά τα Edit Game, Add New Game και Start Game. Δεν θα αναφερθούμε ξανά στο κουμπί Add New Game επειδή το αναπτύξαμε προηγουμένως.



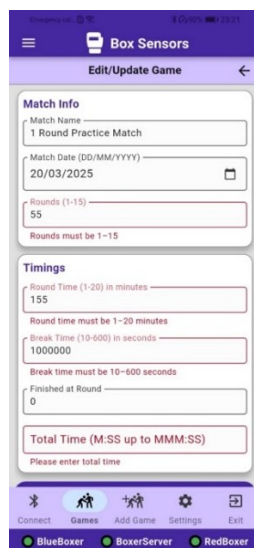
Εικόνα 37. Η οθόνη Game Details.

Εάν ο χρήστης πατήσει το κουμπί Edit Game μεταφέρεται σε μια νέα οθόνη την Edit/Update Game (εικόνα 42), όπου μπορεί να αλλάξει τα επτά πεδία, Match name, Match Date, Rounds, Finished at Round, Total Time, Round Time, Break Time και να πατήσει το κουμπί Update Game για να αποθηκεύσει τις αλλαγές στη βάση δεδομένων.



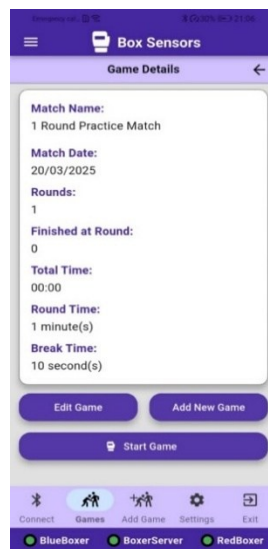
Εικόνα 38. Η οθόνη Edit/Update Game.

Στην αλλαγή των πεδίων βγαίνουν κόκκινες επισημάνσεις στην περίπτωση που οι τιμές είναι εκτός ορίων (εικόνα 43).



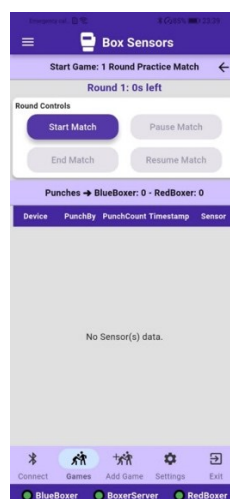
Εικόνα 39. Εκτός ορίων τιμές στην οθόνη Edit/Update Game.

Όταν ο χρήστης πατήσει το κουμπί Start Game μεταφέρεται σε μια καινούρια οθόνη την Start Game από την οποία μπορεί να ξεκινήσει έναν αγώνα ή μια προπόνηση που θα έχει διάρκεια την οποία μπορούμε να δούμε από την προηγούμενη οθόνη Games Details(εικόνα 44) και να την αλλάξουμε αν χρειάζεται από το κουμπί Edit Game.



Εικόνα 40. Game Details.

Πατώντας το κουμπί Start Match ξεκινά ο αγώνας και η αντίστροφη μέτρηση του γύρου (εικόνα 45). Μπορούμε να κάνουμε παύση του γύρου και της αντίστροφης μέτρησης με το κουμπί Pause Match και να αρχίσουμε ξανά την αντίστροφη μέτρηση από εκεί που σταμάτησε με το κουμπί Resume Match.



Εικόνα 41. Η οθόνη του Match.

Τέλος μπορούμε να πατήσουμε το κουμπί End Match για να σταματήσει η προπόνηση ή ο αγώνας, πριν τελειώσει ο χρόνος, για διάφορους λόγους όπως για παράδειγμα πρόωρη διακοπή

της προπόνησης ή του αγώνα. Επίσης μπορούμε να δούμε στην ίδια οθόνη την εξέλιξη των χτυπημάτων, το γενικό σύνολο, αλλά και σε έναν πίνακα το χρονικό σημείο που έγινε ένα συγκεκριμένο χτύπημα ποιος το έκανε και ποιος το δέχτηκε και μια τιμή που μας δίνουν οι αισθητήρες. Ο χρήστης της εφαρμογής δεν έχει δυνατότητα να πάει σε άλλη οθόνη κατά την διάρκεια της αντίστροφης μέτρησης παρά μόνο αν τελειώσει ο αγώνας ή τον διακόψει ο ίδιος πατώντας το κουμπί End Match (εικόνα 46).

Device	PunchBy	PunchCount	Timestamp	Sensor
RedBoxer	BlueBoxer	9	00:20:75	809
RedBoxer	BlueBoxer	8	00:20:34	826
RedBoxer	BlueBoxer	7	00:19:71	809
RedBoxer	BlueBoxer	6	00:19:10	810
BlueBoxer	RedBoxer	3	00:18:47	828
RedBoxer	BlueBoxer	5	00:17:10	809
RedBoxer	BlueBoxer	4	00:14:20	826
BlueBoxer	RedBoxer	2	00:08:83	844
RedBoxer	BlueBoxer	3	00:07:13	809
RedBoxer	BlueBoxer	2	00:06:05	810
RedBoxer	BlueBoxer	1	00:04:65	809
BlueBoxer	RedBoxer	1	00:03:85	843

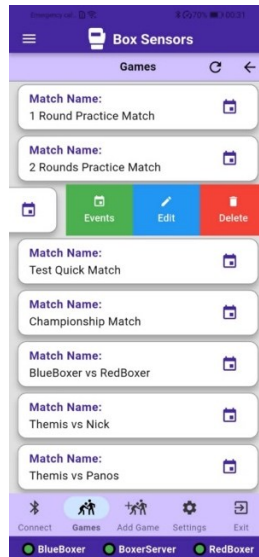
Εικόνα 42. Η οθόνη του Match, κατά τη διάρκεια ενός παιχνιδιού.

Από την οθόνη Games μπορεί ο χρήστης να πατήσει πάνω στο εικονίδιο του ημερολογίου για να μεταφερθεί στην Οθόνη Game Events (εικόνα 47) η οποία περιέχει τους ολοκληρωμένους αγώνες ή τις προπονήσεις.

Match Name
1 Round Practice Match
2 Rounds Practice Match
5 Rounds Practice Match
Test Quick Match
Championship Match
BlueBoxer vs RedBoxer
Themis vs Nick
Themis vs Panos

Εικόνα 43. Η οθόνη Games

Στη κάρτα κάθε ολοκληρωμένου αγώνα βλέπουμε επίσης το όνομα του νικητή και το αποτέλεσμα των σημείων του αγώνα. Μπορούμε επίσης να κάνουμε swiπε στα αριστερά και να εμφανιστεί ένα Slidable List με επιλογές Events, Edit, Delete (εικόνα 48).



Εικόνα 44. Η οθόνη Games, Slidable List με επιλογές Events, Edit, Delete.

Με την επιλογή Events οδηγούμαστε στην Οθόνη Game Events (εικόνα 49), με την επιλογή Edit οδηγούμαστε στην Οθόνη Edit/Update Game, με την επιλογή Delete διαγράφουμε το συγκεκριμένο Game μαζί με όλες τις εγγραφές που το συνδέουν στους πίνακες της βάσης δεδομένων.



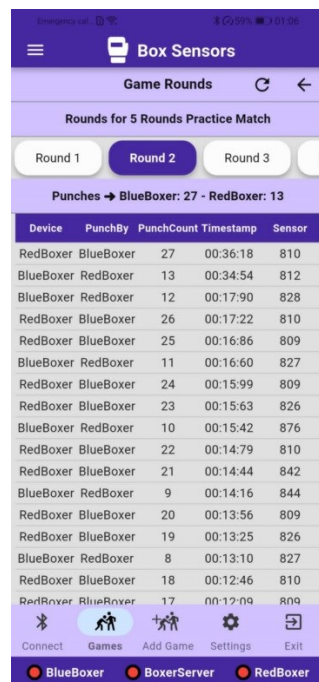
Εικόνα 45. Οθόνη Game Events.

Εάν πατήσει ο χρήστης κάποιο από τα ολοκληρωμένα Game Events μεταφέρεται σε μια νέα οθόνη που περιέχει αναλυτικές πληροφορίες για του γύρους του παιχνιδιού ή της προπόνησης. Σε αυτήν την οθόνη με όνομα Game Rounds βλέπουμε τους γύρους με τα σημεία του κάθε γύρου, αναλυτικά όπως δείχνουν οι παρακάτω εικόνες (Εικόνες 50, 51, 52).



Device	PunchBy	PunchCount	Timestamp	Sensor
RedBoxer	BlueBoxer	11	00:59:80	810
BlueBoxer	RedBoxer	3	00:59:02	812
RedBoxer	BlueBoxer	10	00:58:39	810
RedBoxer	BlueBoxer	9	00:57:63	809
RedBoxer	BlueBoxer	8	00:57:40	809
RedBoxer	BlueBoxer	7	00:57:19	810
RedBoxer	BlueBoxer	6	00:56:87	809
RedBoxer	BlueBoxer	5	00:56:46	809
RedBoxer	BlueBoxer	4	00:56:13	825
BlueBoxer	RedBoxer	2	00:55:92	812
RedBoxer	BlueBoxer	3	00:55:38	809
RedBoxer	BlueBoxer	2	00:54:91	809
RedBoxer	RedBoxer	1	00:54:55	812
RedBoxer	BlueBoxer	1	00:53:85	809

Εικόνα 46. Round 1.



Device	PunchBy	PunchCount	Timestamp	Sensor
RedBoxer	BlueBoxer	27	00:36:18	810
BlueBoxer	RedBoxer	13	00:34:54	812
BlueBoxer	RedBoxer	12	00:17:90	828
RedBoxer	BlueBoxer	26	00:17:22	810
RedBoxer	BlueBoxer	25	00:16:86	809
BlueBoxer	RedBoxer	11	00:16:60	827
RedBoxer	BlueBoxer	24	00:15:99	809
RedBoxer	BlueBoxer	23	00:15:63	826
BlueBoxer	RedBoxer	10	00:15:42	876
RedBoxer	BlueBoxer	22	00:14:79	810
RedBoxer	BlueBoxer	21	00:14:44	842
BlueBoxer	RedBoxer	9	00:14:16	844
RedBoxer	BlueBoxer	20	00:13:56	809
RedBoxer	BlueBoxer	19	00:13:25	826
RedBoxer	RedBoxer	8	00:13:10	827
RedBoxer	BlueBoxer	18	00:12:46	810
RedBoxer	BlueBoxer	17	00:12:09	809

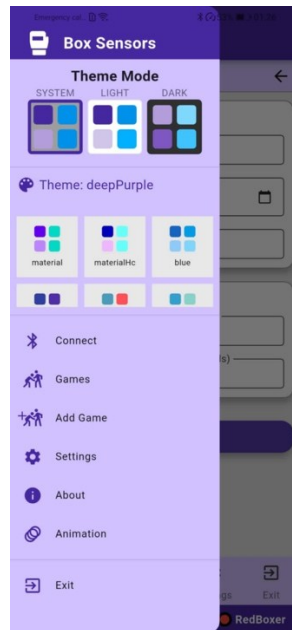
Εικόνα 47. Round 2.



Device	PunchBy	PunchCount	Timestamp	Sensor
BlueBoxer	RedBoxer	12	00:20:74	828
RedBoxer	BlueBoxer	14	00:19:95	810
RedBoxer	BlueBoxer	13	00:19:22	810
BlueBoxer	RedBoxer	11	00:18:77	827
BlueBoxer	RedBoxer	10	00:18:20	827
RedBoxer	BlueBoxer	12	00:14:49	810
BlueBoxer	RedBoxer	9	00:14:10	811
RedBoxer	BlueBoxer	11	00:13:52	809
BlueBoxer	RedBoxer	8	00:13:16	811
RedBoxer	BlueBoxer	10	00:12:61	809
RedBoxer	BlueBoxer	9	00:12:31	810
RedBoxer	BlueBoxer	8	00:11:94	810
BlueBoxer	RedBoxer	7	00:11:05	811
RedBoxer	BlueBoxer	7	00:10:55	809
BlueBoxer	RedBoxer	6	00:10:21	812
BlueBoxer	RedBoxer	5	00:09:31	812
RedBoxer	BlueBoxer	6	00:08:57	810

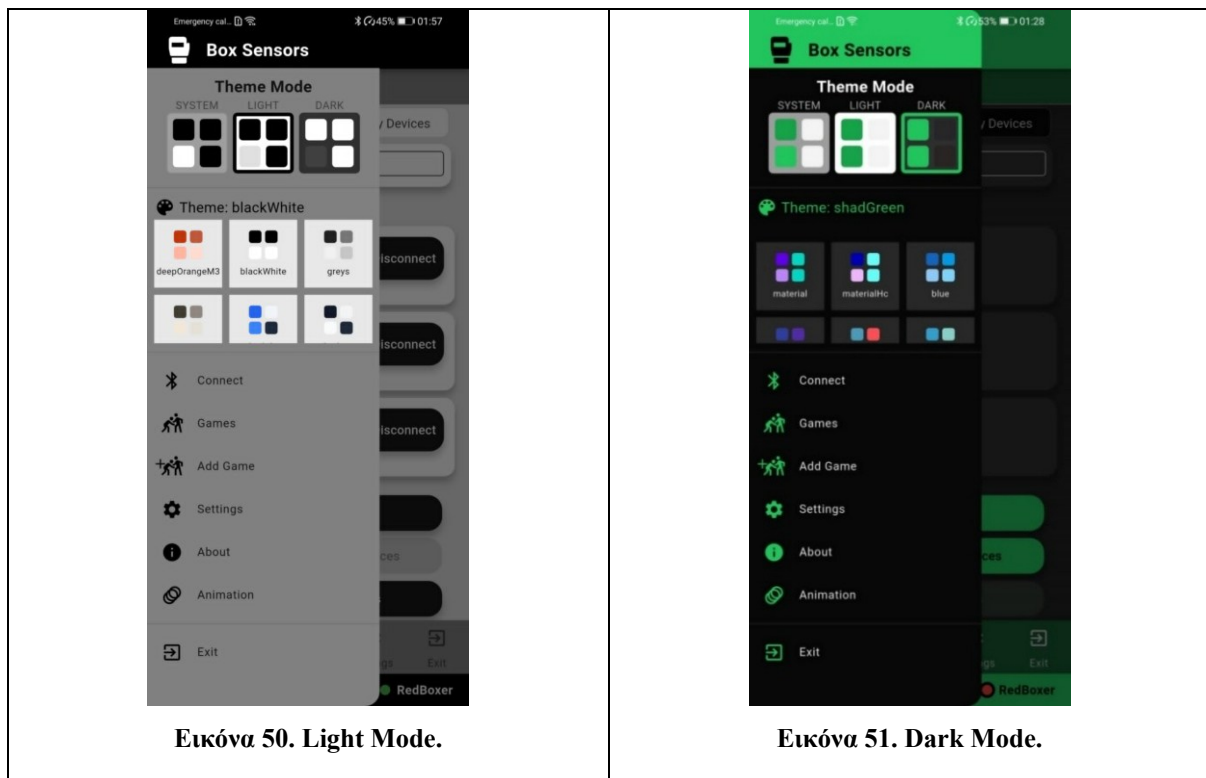
Εικόνα 48. Round 5.

Όπως βλέπουμε στην εφαρμογή αριστερά πάνω, υπάρχει ένα ειδικό εικονίδιο, ονομάζεται hamburger icon, το οποίο είναι ένα ευρέως διαδεδομένο σύμβολο σε πολλές εφαρμογές για κινητά και ιστοσελίδες και ανοίγει ένα μενού. Στην εφαρμογή εδώ χρησιμοποιείται για να ανοίξει ένα αντικείμενο Drawer, για να εμφανίσει επιλογές πλοήγησης και διάφορες άλλες λειτουργίες. Όταν ανοίξει αυτό το μενού μπορούμε να αλλάξουμε το θέμα της εφαρμογής. Έχουμε δηλαδή την δυνατότητα διαχείρισης θεμάτων (themes) στην εφαρμογή (εικόνα 53) προσφέροντας έτοιμα πρότυπα χρωματικών συνδυασμών. Είναι στην ευχέρεια του χρήστη να επιλέξει αυτό που του αρέσει και αν κλείσει την εφαρμογή, αποθηκεύει την επιλογή του, ώστε αν την ξαναανοίξει θα ανοίξει με την δική του χρωματική επιλογή.



Εικόνα 49. Themes.

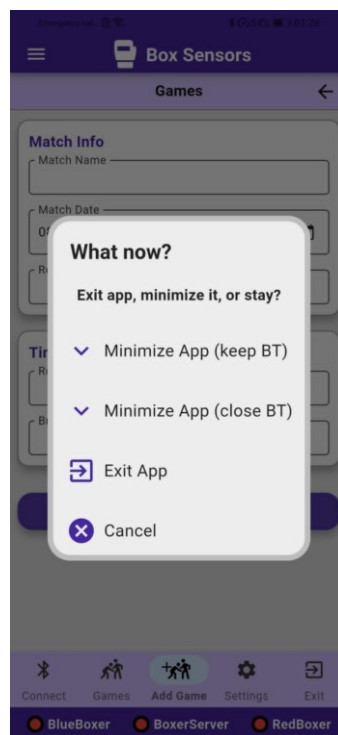
Παρέχεται επίσης η υποστήριξη των λειτουργιών Light Mode (εικόνα 54) και Dark Mode (εικόνα 55). Στις παρακάτω εικόνες βλέπουμε διάφορα θέματα που μπορεί να επιλέξει ο χρήστης. Οι άλλες επιλογές του menu είναι οι ίδιες, όπως και στο Navigation Bar.



Εικόνα 50. Light Mode.

Εικόνα 51. Dark Mode.

Τέλος, παρέχεται η δυνατότητα με την επιλογή Exit ο χρήστης να μπορεί να κλείσει την εφαρμογή (εικόνα 56), να την κάνει minimize, ή να κλείσει το διάλογο. Εάν πατήσει την επιλογή close, κλείνει ο διάλογος και επιστρέφει στην εφαρμογή. Σε αντίθεση με τα επιτραπέζια λειτουργικά συστήματα (π.χ. Windows), στο Android δεν υπάρχει η κλασική εντολή «ελαχιστοποίησης» (minimize). Αντί γι' αυτό, οι εφαρμογές παραμένουν ενεργές στο παρασκήνιο και ο χρήστης τις διαχειρίζεται μέσω της οθόνης «Πρόσφατες εφαρμογές» (Recents). Από εκεί μπορεί να επανεκκινήσει ή να τερματίσει την εκάστοτε εφαρμογή, χωρίς να την κλείσει εντελώς.



Εικόνα 52. Ο διάλογος της επιλογής Exit.

Έξοδος (Exit) Με την επιλογή «Έξοδος», η εφαρμογή τερματίζεται πλήρως, απελευθερώνονται οι πόροι που χρησιμοποιούσε, κλείνουν όλα τα ενεργά threads και ο χρήστης επιστρέφει στην κύρια οθόνη της συσκευής είτε σε άλλη ενεργή εφαρμογή.

Ελαχιστοποίηση (Minimize) Η «Ελαχιστοποίηση» δεν τερματίζει την εφαρμογή, αλλά τη μεταφέρει σε κατάσταση παρασκήνιου (background). Το Android δεν διαθέτει ξεχωριστό πλήκτρο «Minimize»· αντιθέτως, η εφαρμογή παραμένει ενεργή στη μνήμη και είναι προσβάσιμη από την οθόνη πρόσφατες εφαρμογές (Recents). Με αυτόν τον τρόπο, ο χρήστης μπορεί να επιστρέψει γρήγορα σε αυτή χωρίς να απαιτείται πλήρης επανεκκίνηση

Άκυρο (Cancel) Η επιλογή «Άκυρο» κλείνει τον διάλογο χωρίς να εκτελεί καμία ενέργεια. Η εφαρμογή παραμένει στην τρέχουσα κατάσταση και ο χρήστης επιστρέφει στην προηγούμενη οθόνη χωρίς διακοπή της λειτουργίας.

4.4 Βιβλιοθήκες εφαρμογής Android

Οι βιβλιοθήκες που χρησιμοποίησα για την ανάπτυξη της εφαρμογής:

Box sensors.

Το κύριο πακέτο της εφαρμογής, που ονομάζεται box sensors, αποτελεί το σημείο εκκίνησης (entry point) του έργου Flutter. Περιέχει τον βασικό φάκελο lib/ με τον κώδικα της εφαρμογής, ρυθμίζει την έκδοση (1.0.3+4) και ορίζει το περιβάλλον ανάπτυξης (Dart SDK $\geq 3.7.2$). Επιπλέον, εδώ καθορίζονται όλα τα dependencies και dev dependencies που χρησιμοποιούνται.

Async⁵²

Το πακέτο async παρέχει βοηθητικά εργαλεία για καλύτερο χειρισμό ασύγχρονου κώδικα, όπως τα StreamGroup, CancelableOperation και διάφορα extensions για Future/Stream. Χρησιμοποιείται για να οργανώνει και να συντονίζει πολλαπλές ασύγχρονες διεργασίες, π.χ. όταν διαβάζονται ταυτόχρονα δεδομένα από αισθητήρες FSR και BLE. Διευκολύνει την ακύρωση, την καθυστέρηση ή τον συνδυασμό streams με δομημένο τρόπο.

Flutter Riverpod⁵³

Το flutter riverpod (έκδοση 2.6.1) είναι ένα σύστημα διαχείρισης κατάστασης (state management) βασισμένο σε providers, που προωθεί τη συνοχή, τις καλές πρακτικές και τον ανεξάρτητο από widgets σχεδιασμό. Παρέχει StateProviders, FutureProviders και StreamProviders, επιτρέποντας την εύκολη διαχείριση και ανανέωση της κατάστασης της εφαρμογής. Στο έργο της πτυχιακής, χρησιμοποιήθηκε για να διαχειρίζεται την κατάσταση σύνδεσης Bluetooth, τα δεδομένα από τους αισθητήρες FSR και τις ρυθμίσεις χρήστη με καθαρότητα και φιλικό τρόπο. Την κλάση TimerState, την αντίστροφη μέτρηση του αγώνα ή της προπόνησης. Την κλάση FlexColorScheme διαχειρίζεται τη χρωματική παλέτα των

⁵² Βλ. Google, Flutter - Dart API docs, <https://api.flutter.dev/flutter/dart-async/>.

⁵³ Βλ. Remi Rousselet, Flutter Riverpod, <https://riverpod.dev/>.

θεμάτων. Την κλάση DatabaseHelper η οποία υλοποιείται ως singleton pattern, ώστε να υπάρχει μία μόνο ενεργή σύνδεση στη βάση δεδομένων και να αποφεύγονται συγκρούσεις ή περιττή κατανάλωση πόρων. Με τη μετατροπή της σε Riverpod Provider, κάθε widget ή service που την καλεί λαμβάνει πάντα την ίδια instance, χωρίς πολλαπλές αρχικοποιήσεις. Με τους “χειριστές συμβάντων κύκλου ζωής” (lifecycle hooks) του Riverpod, η εφαρμογή ανανεώνει αυτόματα τα δεδομένα της, ενώ παράλληλα διευκολύνεται η αντιμετώπιση σφαλμάτων κατά την εκτέλεση και η διεξαγωγή δοκιμών με δεδομένα προσομοίωσης (mock testing).

Flutter blue plus⁵⁴

Το flutter blue plus (έκδοση 1.35.3) είναι ένα πακέτο για επικοινωνία μέσω Bluetooth Low Energy (BLE) σε Flutter. Υποστηρίζει αναζήτηση συσκευών, σύνδεση, ανάγνωση/εγγραφή characteristics και notifications. Στην εφαρμογή χρησιμοποιείται για να συνδέσει με ESP32 μικροελεγκτές που φέρουν τους αισθητήρες FSR, ώστε να διαβάζει σε πραγματικό χρόνο την πίεση που ανιχνεύεται σε αυτούς με το κινητό τηλέφωνο. Επίσης το πρωτόκολλο Bluetooth το χρησιμοποίησα και στο Arduino Uno R4 Wi-Fi για να λαμβάνω τα δεδομένα του κινητού τηλεφώνου ώστε να τα λαμβάνει το Arduino IoT Cloud.

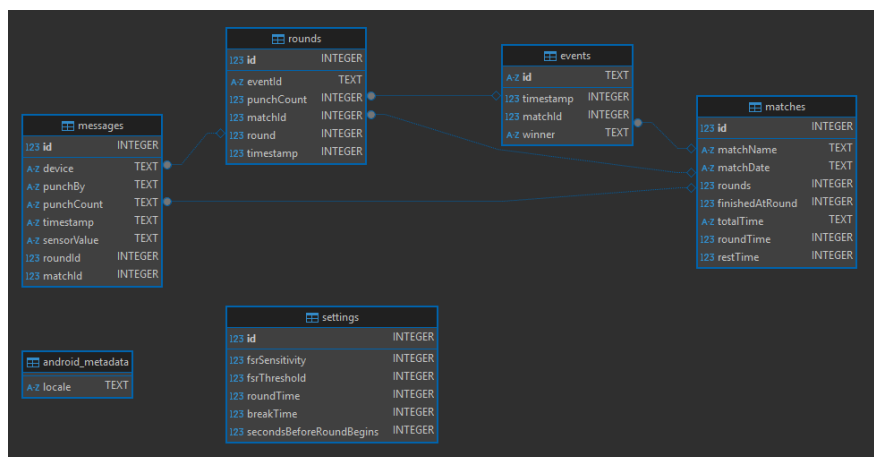
Sqflite^{55, 56}

Το sqflite (έκδοση 2.4.2) προσφέρει SQLite βάση δεδομένων τοπικά στη συσκευή, επιτρέποντας δημιουργία πινάκων, CRUD λειτουργίες (βασικές πράξεις διαχείρισης δεδομένων σε οποιαδήποτε βάση) και transactions. Χρησιμοποιείται για αποθήκευση ιστορικών μετρήσεων από τους αισθητήρες, ρυθμίσεις χρήστη ή ημερολόγια συμβάντων χωρίς την ανάγκη εξωτερικού server. Η σταθερότητα και η δοκιμασμένη απόδοση του το καθιστούν αξιόπιστη λύση για διαρκή αποθήκευση δεδομένων σε κινητές συσκευές. Οι πίνακες της βάσης που δημιουργήθηκε για την εφαρμογή είναι εξής : Matches, Events, Rounds, Messages, Settings (σχήμα 21).

⁵⁴ Βλ. flutter blue plus | Flutter package, https://pub.dev/packages/flutter_blue_plus.

⁵⁵ Βλ. Flutter Team, Persistent storage architecture: SQL, <https://docs.flutter.dev/app-architecture/design-patterns/sql>

⁵⁶ Sqflite Flutter module, <https://pub.dev/packages/sqflite>.



Σχήμα 21. Το διάγραμμα της βάσης δεδομένων SQLite που είναι ενσωματωμένη στην εφαρμογή.

Path & path provider:

Path⁵⁷: Βοηθάει στη διαχείριση και κατασκευή μονοπατιών αρχείων (file paths) με ανεξάρτητο τρόπο (σε Android, iOS κ.λπ.), π.χ. συγχώνευση φακέλων, εξαγωγή ονόματος αρχείου κ.ά.

Path provider⁵⁸: Παρέχει access σε ειδικούς καταλόγους του συστήματος, όπως application documents ή temporary directories.

Σε συνδυασμό, επιτρέπουν τη δημιουργία και πρόσβαση σε τοπικά αρχεία δεδομένων (π.χ. εξαγωγή logs ή αποθήκευση backup του SQLite).

Intl⁵⁹

Το intl (έκδοση 0.20.2) υποστηρίζει διεθνοποίηση (i18n) και γλωσσική τοπικοποίηση (l10n) κειμένων, ημερομηνιών, νομισματικών μονάδων και αριθμών. Στην πτυχιακή χρησιμοποιείται για τη σωστή μορφοποίηση ημερομηνιών/ωρών των μετρήσεων FSR σύμφωνα με το ελληνικό locale (el_GR), καθώς και για την υποστήριξη για πολύγλωσση διεπαφή (interface).

Permission handler⁶⁰

Το permission handler (έκδοση 12.0.0+1) διευκολύνει τον έλεγχο και την αίτηση δικαιωμάτων (permissions) στο Android/iOS, όπως Bluetooth ή μικρόφωνο. Στην εφαρμογή, ζητάει στον χρήστη την άδεια χρήσης BLE πριν ξεκινήσει την επικοινωνία με τους μικροελεγκτές. Επίσης

⁵⁷ Βλ. Path Flutter module, <https://pub.dev/packages/path>.

⁵⁸ Βλ. Path provider | Flutter package, https://pub.dev/packages/path_provider.

⁵⁹ Βλ. intl Dart package, <https://pub.dev/packages/intl>.

⁶⁰ Βλ. PermissionHandler package, https://pub.dev/packages/permission_handler.

η εφαρμογή ζητά άδεια πρόσβασης στις υπηρεσίες τοποθεσίας (Location Services) του κινητού διότι διαφορετικά δεν θα μπορούσαμε με κανένα τρόπο να ανιχνεύσουμε τα Bluetooth Low energy devices δηλαδή τους μικροελεγκτές. Το απαιτεί το λειτουργικό σύστημα Android. Επίσης, εμφανίζει ένα διάλογο ο οποίος μας ρωτά αν θέλουμε να απενεργοποιήσουμε την ρύθμιση Battery optimization settings.

Audioplayers⁶¹

Το audioplayers (έκδοση 6.4.0) επιτρέπει αναπαραγωγή ήχων και μουσικής από τοπικά αρχεία ή URL. Χρησιμοποιείται για audio feedback σε συμβάντα, π.χ. ηχητικό σήμα όταν ξεκινά ή σταματά ή μέτρηση.

Flex color scheme⁶²

Το flex color scheme (έκδοση 8.2.0) προσφέρει έτοιμες, ευέλικτες θεματικές (themes) Material Design, με υποστήριξη light/dark mode και προσαρμοσμένα χρώματα. Στην εφαρμογή διευκολύνει την γρήγορη και συνεπή σχεδίαση του UI, εξασφαλίζοντας οπτική συνοχή και προσβασιμότητα (accessibility).

Uuid⁶³

Το πακέτο uuid (έκδοση 4.5.1) παράγει μοναδικούς κωδικούς (UUID), οι οποίοι είναι χρήσιμοι για την επισήμανση και αποθήκευση εγγραφών δεδομένων.

Shared preferences⁶⁴

Το shared preferences (έκδοση 2.5.3) παρέχει δυνατότητες αποθήκευσης μικρών δεδομένων (π.χ. Boolean, int, String). Χρησιμοποιείται για αποθήκευση ρυθμίσεων χρήστη για την παραμετροποίηση της εφαρμογής. Εδώ το χρησιμοποιώ για να αποθηκεύσω την επιλογή του χρήστη, το χρωματικό θέμα της εφαρμογής.

Flutter slidable⁶⁵

Το flutter slidable (έκδοση 4.0.0) επιτρέπει να ενσωματώσω συρόμενες ενέργειες σε λίστες (swipe-to-action), όπως διαγραφή ή επεξεργασία στοιχείων. Στην εφαρμογή χρησιμοποιείται στην οθόνη Games στον πίνακα Matches, ώστε με swipe να εμφανίζονται λειτουργίες όπως “Διαγραφή”, “Edit”, “Event”. Δηλαδή αν σύρουμε την κάρτα προς τα αριστερά.

⁶¹ Βλ. Audioplayers package, <https://pub.dev/packages/audioplayers>.

⁶² Βλ. FlexColorScheme, <https://docs.flexcolorscheme.com/>.

⁶³ Βλ. Uuid Flutter Package, <https://pub.dev/packages/uuid>.

⁶⁴ Βλ. Shared Preferences Flutter package, https://pub.dev/packages/shared_preferences.

⁶⁵ Βλ. Flutter slidable package, https://pub.dev/packages/flutter_slidable.

Gap⁶⁶

Το gap (έκδοση 3.0.1) είναι ένα μικρό βοηθητικό πακέτο για εύκολη εισαγωγή σταθερών κενών (SizedBox) μεταξύ widgets, χωρίς να γράφουμε επαναλαμβανόμενο κώδικα.

Animated text kit⁶⁷

Το animated text kit (έκδοση 4.2.3) προσφέρει έτοιμα animation για κείμενο—typing, fade, colorize κ.ά. Χρησιμοποιείται σε οθόνες εκκίνησης ή παρουσίασης με σύντομα μηνύματα, δίνοντας δυναμικό χαρακτήρα στο UI.

Sentry flutter⁶⁸

Το sentry flutter (έκδοση 8.14.2) ενσωματώνει υπηρεσίες παρακολούθησης σφαλμάτων (crash reporting) και performance monitoring. Η εφαρμογή στέλνει exceptions και πληροφορίες καταρρεύσεων στο Sentry dashboard, επιτρέποντας έγκαιρη ανίχνευση και διόρθωση σφαλμάτων. Το χρησιμοποιήθηκε αρκετά κατά τη δοκιμαστική περίοδο.

Package info plus⁶⁹

Το package info plus (έκδοση 8.3.0) διαβάζει δυναμικά πληροφορίες της εφαρμογής, όπως version, build number και package name. Χρησιμοποιείται στις ρυθμίσεις ή στο “Για την εφαρμογή” για εμφάνιση της τρέχουσας έκδοσης προς τον χρήστη.

Logging⁷⁰ (dev dependency)

Το logging (έκδοση 1.3.0) διευκολύνει την καταγραφή (logging) μηνυμάτων σε κονσόλα ή αρχείο κατά την ανάπτυξη. Το χρησιμοποιούμε για debug prints, δοκιμές και ανάλυση ροής εκτέλεσης.

Flutter launcher icons⁷¹ (dev dependency).

Το flutter launcher icons (έκδοση 0.14.3) αυτοματοποιεί τη δημιουργία launcher icons για Android/iOS από ένα αρχικό αρχείο εικόνας. Αυτό απλοποιεί σημαντικά την ενσωμάτωση των εικονιδίων, καθώς το πακέτο δημιουργεί αυτόματα όλες τις απαιτούμενες παραλλαγές μεγέθους και τις τοποθετεί στους σωστούς φακέλους του project, χωρίς να χρειάζονται από τον προγραμματιστή χειροκίνητες ενέργειες παραμετροποίησης.

⁶⁶ Βλ. Flutter gap Flutter package, https://pub.dev/packages/flutter_gap.

⁶⁷ Βλ. Animated text kit Flutter package, https://pub.dev/packages/animated_text_kit.

⁶⁸ Βλ. Sentry for Flutter, <https://docs.sentry.io/platforms/dart/guides/flutter/>.

⁶⁹ Βλ. Package info plus, Dart package, https://pub.dev/packages/package_info_plus.

⁷⁰ Βλ. Flutter login package, https://pub.dev/packages/flutter_login.

⁷¹ Βλ. Flutter launcher icons, Dart package, https://pub.dev/packages/flutter_launcher_icons.

Flutter lints⁷² (dev dependency)

Το flutter lints (έκδοση 5.0.0) παρέχει σύνολο καθιερωμένων lint κανόνων για βελτίωση ποιότητας κώδικα. Βοηθά στην τήρηση βέλτιστων πρακτικών, στην αποφυγή σφαλμάτων και στην ενιαία μορφοποίηση. Η προτεινόμενη μορφοποίηση κώδικα δεν μου είναι ιδιαίτερα οικεία, ωστόσο στο περιβάλλον του Flutter αναγνωρίζεται ως βέλτιστη πρακτική.

Material color utilities⁷³ (dependency override)

Το πακέτο material color utilities (έκδοση 0.12.0) είναι ένα εργαλείο που βοηθάει να δημιουργηθούν αυτόματα ομάδες χρωμάτων που ταιριάζουν μεταξύ τους, καθώς και οι διαφορετικοί τόνοι αυτών των χρωμάτων. Αυτό γίνεται σύμφωνα με τις οδηγίες του Material Design⁷⁴, κάνοντας την εφαρμογή να φαίνεται πιο όμορφη και οργανωμένη. Μέσω του dependency override εξασφαλίζεται ότι χρησιμοποιείται ακριβώς αυτή η έκδοση του πακέτου, διατηρώντας πλήρη συμβατότητα με το flex_color scheme που βασίζεται σ' αυτό.

Κάθε ένα από τα παραπάνω modules εξυπηρετεί συγκεκριμένες ανάγκες της εφαρμογής από τη διαχείριση κατάστασης και BLE επικοινωνίας, μέχρι το θεματικό styling, την αποθήκευση δεδομένων και την παρακολούθηση σφαλμάτων. Η ολοκληρωμένη χρήση τους διασφαλίζει την αξιοπιστία, ευελιξία και επεκτασιμότητα του έργου.

⁷² Βλ. Flutter lints, Dart package, https://pub.dev/packages/flutter_lints.

⁷³ Βλ. Flutter Team, Flutter material library, <https://api.flutter.dev/flutter/material/material-library.html>.

⁷⁴ Βλ. Google, *Material Design 3*, <https://m3.material.io/>.

5. Το σύστημα στην πράξη

Στο παρόν κεφάλαιο, αναλύεται η πρακτική εφαρμογή και η λειτουργία του ολοκληρωμένου συστήματος παρακολούθησης πυγμαχίας που αναπτύχθηκε. Περιγράφονται τα στάδια προετοιμασίας των επιμέρους υλικών και λογισμικών συνιστωσών, η διαδικασία εκτέλεσης ενός τυπικού σεναρίου χρήσης κατά τη διάρκεια μιας προπόνησης ή ενός αγώνα, καθώς και τα είδη των αποτελεσμάτων που συλλέγονται, επεξεργάζονται και παρουσιάζονται στον τελικό χρήστη.

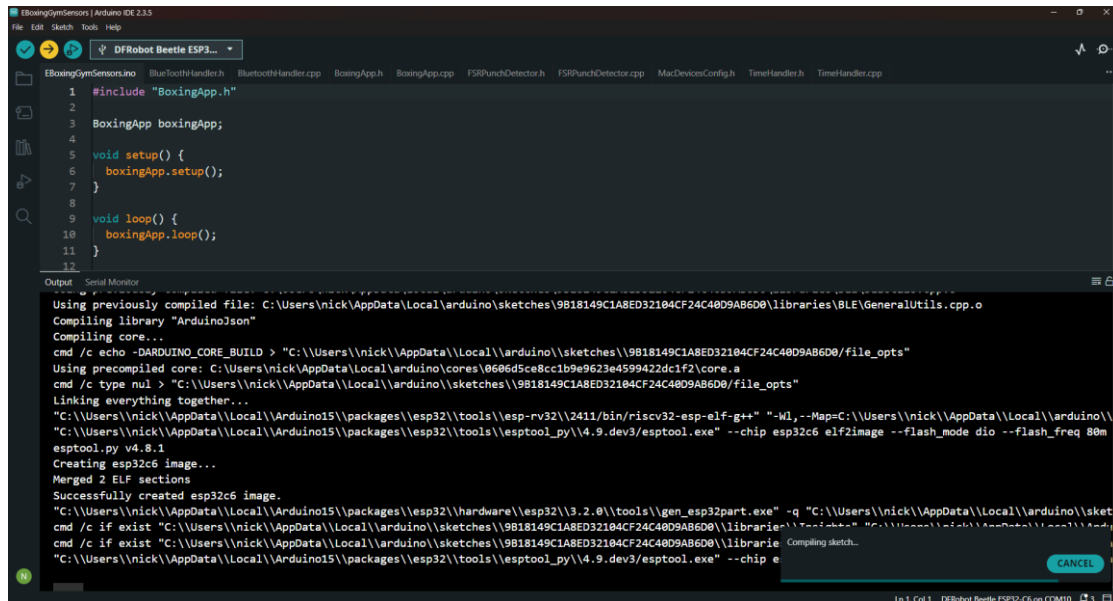
5.1 Προετοιμασία

Για την εύρυθμη και αξιόπιστη λειτουργία του συστήματος, είναι απαραίτητη η ολοκλήρωση συγκεκριμένων βημάτων προετοιμασίας για κάθε τμήμα της αρχιτεκτονικής:

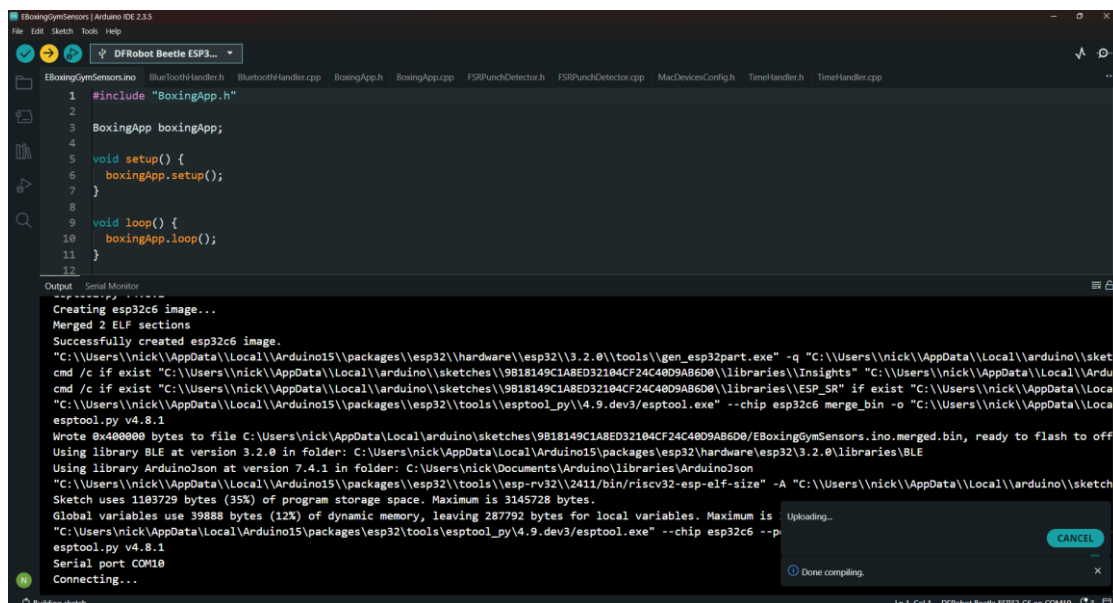
Προετοιμασία Περιφερειακών Μονάδων Ανίχνευσης (Beetle ESP32-C6):

Υλική Διάταξη: Πραγματοποιείται η σύνδεση των αισθητήρων FSR (Force Sensitive Resistors) στις καθορισμένες αναλογικές εισόδους των μικροελεγκτών Beetle ESP32-C6. Ιδιαίτερη μέριμνα λαμβάνεται για τη σταθερή και ασφαλή τοποθέτησή τους στον εξοπλισμό των αθλητών (π.χ., γάντια πυγμαχίας ή στόχοι προπόνησης), ώστε να διασφαλίζεται η ακριβής καταγραφή των κρούσεων.

Εγκατάσταση Λογισμικού (Firmware): Το εξειδικευμένο firmware, το οποίο ενσωματώνει τη λογική ανίχνευσης χτυπημάτων, διαχείρισης χρόνου και επικοινωνίας Bluetooth Low Energy (μέσω των κλάσεων BoxingApp, FSRPunchDetector, TimeHandler και BluetoothHandler για ESP32), μεταγλωττίζεται (εικόνα 57) και φορτώνεται (εικόνα 58) σε κάθε μονάδα ESP32-C6. Κάθε μικροελεγκτής ταυτοποιείται αυτόματα ως "BlueBoxer" ή "RedBoxer" κατά την εκκίνηση, βάσει της μοναδικής του διεύθυνσης MAC και των αντιστοιχίσεων που ορίζονται στο αρχείο MacDevicesConfig.h.



Εικόνα 53. Μεταγλώττιση του κώδικα μέσα από το περιβάλλον του Arduino IDE.



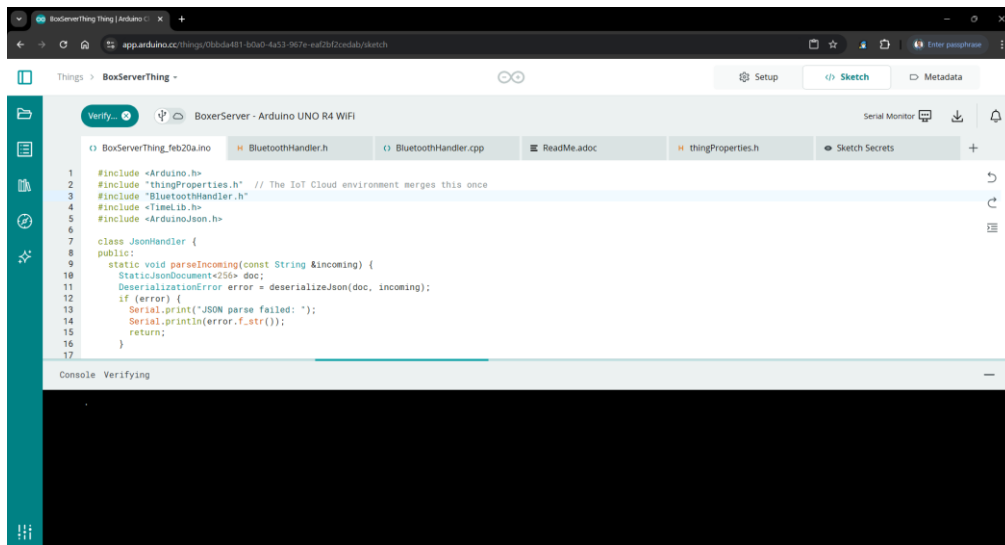
Εικόνα 54. Φόρτωση του παραγόμενου object μετά το compilation στον μικροελεγκτή.

Ενεργοποίηση: Οι μονάδες ESP32-C6 τίθενται σε λειτουργία μέσω κατάλληλης πηγής τροφοδοσίας της μπαταρίας Li-Po⁷⁵ - 3.7v 800mAh.

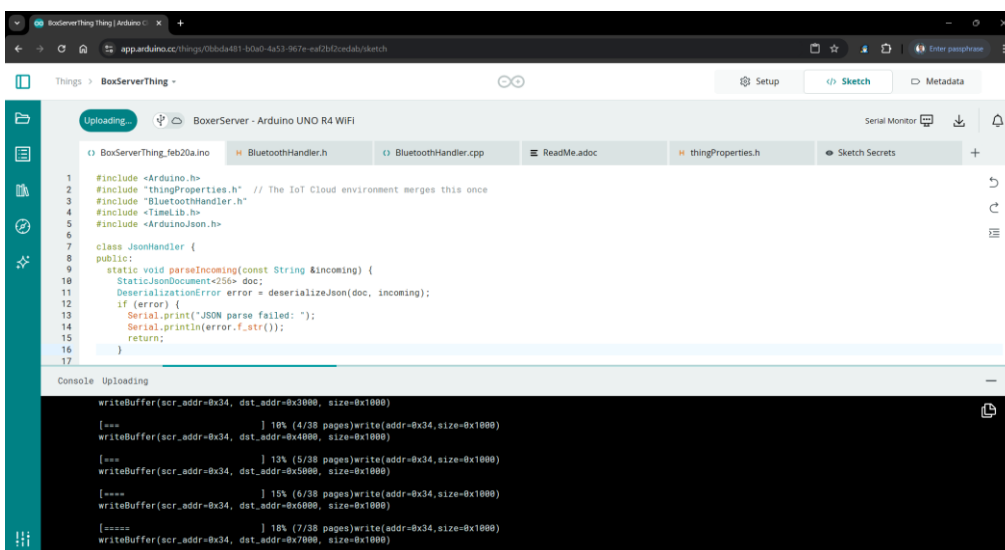
⁷⁵ Βλ. Specification Approval Sheet PDF, https://grobotronics.com/images/companies/1/datasheets/lp453350_800mah_pcm_2.54mmkonektor.pdf?1506785727349.

Προετοιμασία Κεντρικού Κόμβου Δεδομένων (Arduino Uno R4 Wi-Fi):

Εγκατάσταση Λογισμικού (Firmware): Το αντίστοιχο firmware για τον κεντρικό κόμβο, υπεύθυνο για τη λήψη δεδομένων μέσω BLE, την ανάλυση JSON (με την κλάση JsonHandler) και την προώθηση στο Arduino IoT Cloud, φορτώνεται στον μικροελεγκτή Arduino Uno R4 Wi-Fi (εικόνες 59, 60). Πρέπει να τονιστεί ότι μπορεί να παρακαμφθεί ο συγκεκριμένος μικροελεγκτής, στην περίπτωση που δεν χρειαζόμαστε καθόλου κάποιον πίνακα ανακοινώσεων. Η βάση δεδομένων βρίσκεται στην εφαρμογή κινητού τηλεφώνου, οπότε εκεί.

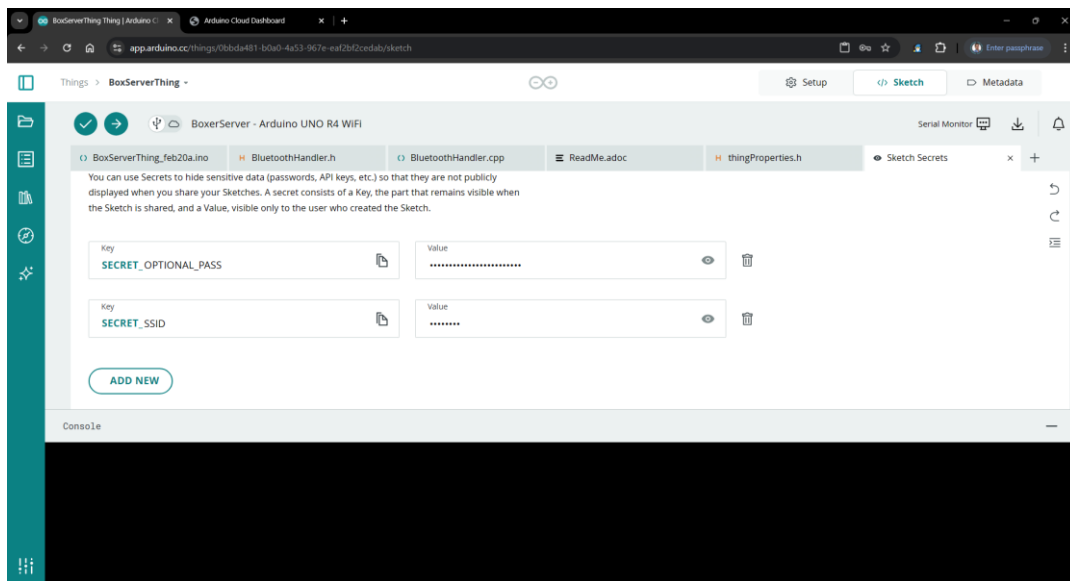


Εικόνα 55. Μεταγλώττιση του Thing μέσα από το περιβάλλον Arduino IoT Cloud



Εικόνα 60. Φόρτωση του παραγόμενου object του Thing μετά το compilation στον μικροελεγκτή.

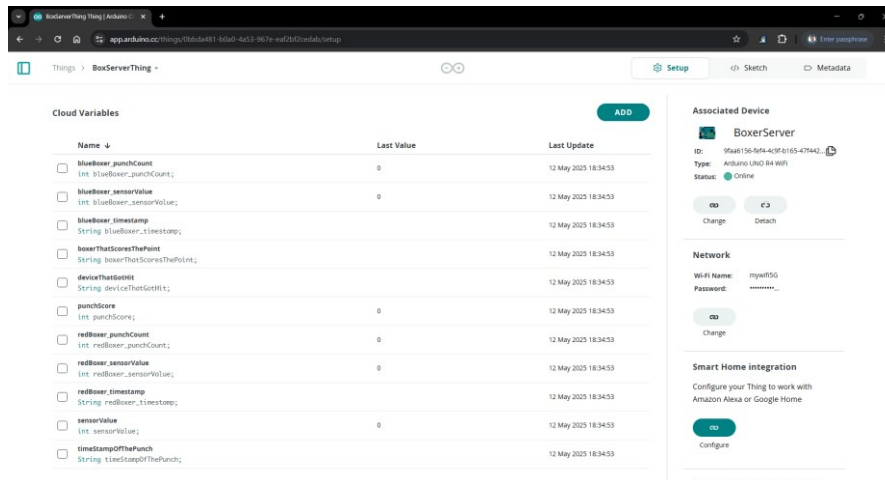
Διαμόρφωση Δικτύου Wi-Fi: Τα διαπιστευτήρια (SSID και κωδικός πρόσβασης) του τοπικού ασύρματου δικτύου καταχωρούνται στο αρχείο `arduino_secrets.h`, επιτρέποντας στον μικροελεγκτή την πρόσβαση στο διαδίκτυο. Αυτό γίνεται μέσω του Arduino IoT Cloud και έτσι τα στοιχεία αυτά δεν φαίνονται στον κώδικα (Εικόνα 61).



Εικόνα 56. Το Thing που δημιουργήσα στο Arduino IoT Cloud.

Ρύθμιση Πλατφόρμας Arduino IoT Cloud: Διασφαλίζεται ότι το "Thing" (ψηφιακή οντότητα της συσκευής) έχει διαμορφωθεί ορθά στην πλατφόρμα Arduino IoT Cloud. Αυτό περιλαμβάνει τον ορισμό όλων των απαραίτητων "Cloud Variables" (π.χ., `deviceThatGotHit`, `punchScore`, `timeStampOfThePunch`, `blueBoxer_punchCount` κ.λπ.) και τη σύνδεσή τους με τις αντίστοιχες μεταβλητές στον κώδικα του μικροελεγκτή (μέσω του `thingProperties.h`) εικόνα 62.

Ενεργοποίηση: Ο Arduino Uno R4 Wi-Fi τίθεται σε λειτουργία. Εδώ προτείνεται να χρησιμοποιήσουμε ένα power bank κινητού τηλεφώνου όταν χρειάζεται να μεταφερθεί ο μικροελεγκτής.



Εικόνα 57. Οι μεταβλητές που συνδέουν τον κώδικα του thing με το dashboard για να εμφανίσει τις τιμές.

Προετοιμασία Ενδιάμεσης Συσκευής (Κινητό Τηλέφωνο με Εφαρμογή "BoxSensors"):

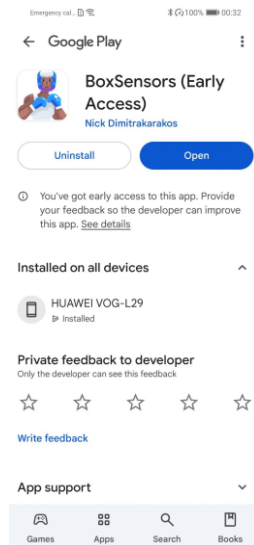
Εγκατάσταση και Ρύθμιση Εφαρμογής: Η εφαρμογή "BoxSensors", η οποία λειτουργεί ως ενδιάμεσος για τη συλλογή δεδομένων από τους ESP32 και την αποστολή τους στον Uno R4, είναι εγκατεστημένη και ρυθμισμένη στο κινητό τηλέφωνο του χρήστη.

Ενεργοποίηση Bluetooth: Η λειτουργία Bluetooth του κινητού τηλεφώνου είναι ενεργοποιημένη για την επικοινωνία με τους μικροελεγκτές. Όταν η εφαρμογή δεν είναι εγκατεστημένη, μπορούμε να την εγκαταστήσουμε από το Google Play Store όπως φαίνεται στην εικόνα 63. Την εφαρμογή μπορούν να την κατεβάσουν οι χρήστες από τον σύνδεσμο του Google Play Store⁷⁶. Υπάρχει και κανάλι στο Youtube⁷⁷ που εξηγεί την χρήση της εφαρμογής και του πληροφοριακού συστήματος και αποθετήριο στο site Github⁷⁸ που περιέχει πληροφορίες για την πτυχιακή και τον κώδικα που χρησιμοποιήθηκε.

⁷⁶ BoxSensors App, https://play.google.com/store/apps/details?id=com.sanguinarypc.box_sensors

⁷⁷ Youtube κανάλι, <https://www.youtube.com/@%CE%9D%CE%99%CE%9A%CE%9F%CE%9B%CE%91%CE%9F%CE%A3%CE%94%CE%97%CE%9C%CE%97%CE%A4%CE%A1%CE%91%CE%9A%CE%91%CE%A1%CE%91%CE%9A%CE%9F%CE%A3>

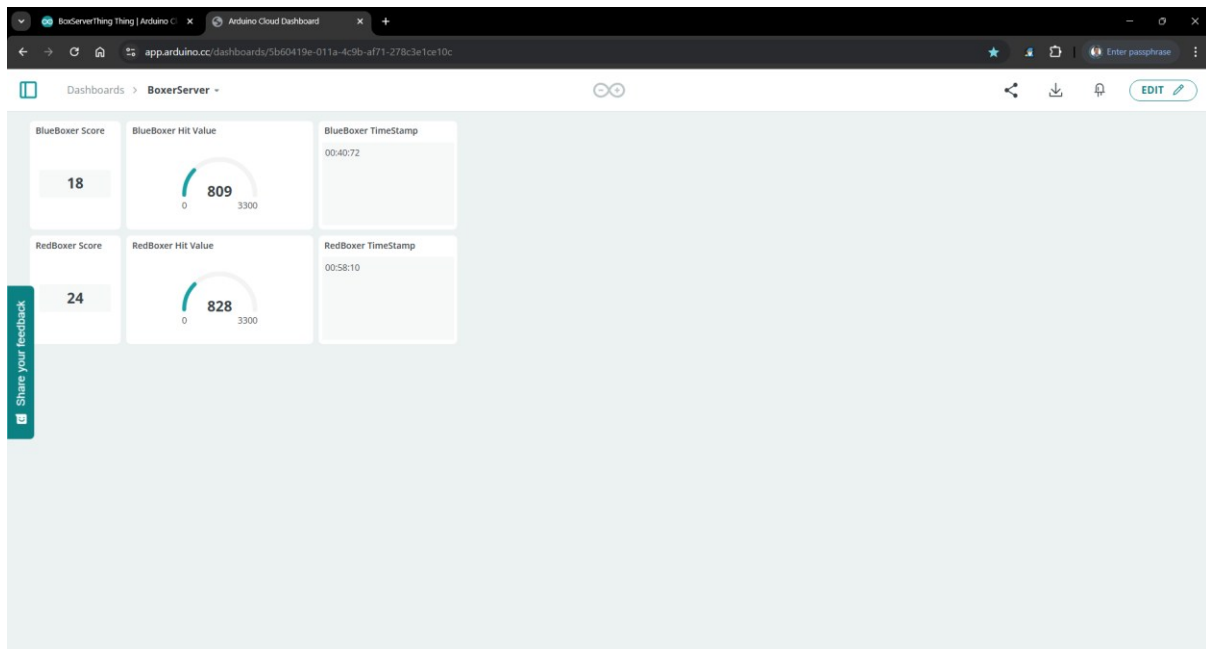
⁷⁸ Github, <https://github.com/sanguinarypc/BoxingGymSensorsProject>



Εικόνα 58. Εγκατάσταση της εφαρμογής σε κινητό από το Google Play Store.

Προετοιμασία Πίνακα ανακοινώσεων (Arduino Dashboard):

Ο πίνακας ελέγχου (dashboard) στην πλατφόρμα Arduino IoT Cloud έχει σχεδιαστεί εκ των προτέρων. Περιλαμβάνει τα κατάλληλα γραφικά στοιχεία (widgets) τα οποία είναι συνδεδεμένα με τις "Cloud Variables" του "Thing" του Arduino Uno R4 Wi-Fi, έτοιμα να απεικονίσουν δυναμικά τα δεδομένα του αγώνα (εικόνα 64).



Εικόνα 59. Το Dashboard που δημιούργησα στο Arduino IoT Cloud.

5.2 Εκτέλεση

Με την ολοκλήρωση των προπαρασκευαστικών βημάτων, το σύστημα είναι έτοιμο για την καταγραφή και παρακολούθηση μιας πυγμαχικής δραστηριότητας. Η τυπική ροή λειτουργίας περιλαμβάνει τα ακόλουθα στάδια (Σχήμα 9. Υλοποίηση συστήματος):

Εκκίνηση και Συνδέσεις Συσκευών:

Με την ενεργοποίησή τους, οι μικροελεγκτές ESP32-C6 ξεκινούν τη διαδικασία αναγγελίας (advertising) μέσω Bluetooth Low Energy, καθιστώντας τους ανιχνεύσιμους με τις ταυτότητες "BlueBoxer" και "RedBoxer".

Ο Arduino Uno R4 Wi-Fi συνδέεται αυτόματα στο προκαθορισμένο δίκτυο Wi-Fi, εγκαθιστά την επικοινωνία με την πλατφόρμα Arduino IoT Cloud, συγχρονίζει την τοπική ώρα με τον χρόνο του παρέχει ένας NTP server, παράλληλα ενεργοποιεί τη δική του υπηρεσία BLE, αναγγέλλοντας τον εαυτό του ως "BoxerServer".

Αλληλεπίδραση μέσω της Εφαρμογής "BoxSensors":

Ο χρήστης (π.χ., προπονητής) εκκινεί την εφαρμογή "BoxSensors" στο κινητό τηλέφωνο. Η εφαρμογή πραγματοποιεί σάρωση για κοντινές συσκευές BLE και συνδέεται διαδοχικά ή ταυτόχρονα με τις μονάδες "BlueBoxer", "RedBoxer" (τους ESP32-C6) και τον "BoxerServer" (τον Arduino Uno R4 Wi-Fi). Η σύνδεση με τους ESP32 επιτρέπει στην εφαρμογή να αποστέλλει εντολές ελέγχου (π.χ., έναρξη/λήξη γύρου, ρύθμιση παραμέτρων αισθητήρων) και να λαμβάνει δεδομένα χτυπημάτων. Η σύνδεση με τον "BoxerServer" χρησιμοποιείται για την προώθηση των συγκεντρωτικών δεδομένων.

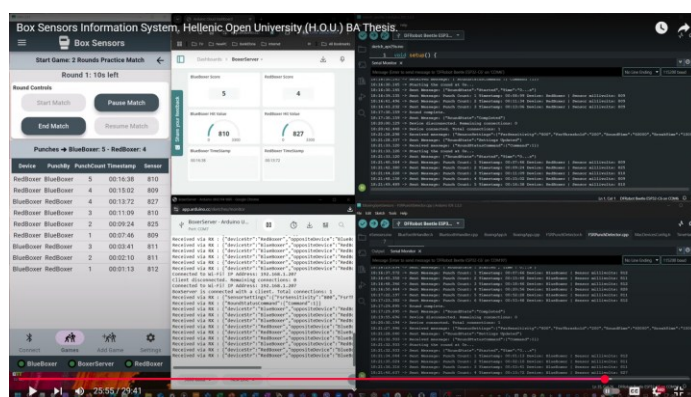
Διεξαγωγή Γύρου και Καταγραφή Χτυπημάτων:

Μέσω της διεπαφής της εφαρμογής "BoxSensors", ο χρήστης δίνει την εντολή για την έναρξη ενός γύρου. Η εντολή αυτή μεταδίδεται στις μονάδες ESP32-C6.

Οι ESP32-C6 ενεργοποιούν την παρακολούθηση των αισθητήρων FSR. Καθώς οι αθλητές εκτελούν χτυπήματα, οι αισθητήρες καταγράφουν τις κρούσεις. Η λογική στην κλάση FSRPunchDetector κάθε ESP32 αναλύει τα σήματα και, εφόσον ένα χτύπημα υπερβεί τα προκαθορισμένα όρια ευαισθησίας και κατωφλίου και ικανοποιεί τα κριτήρια απόρριψης πολλαπλών καταγραφών (debouncing), καταγράφεται ως έγκυρο. Στοιχεία όπως ο αριθμός

των χτυπημάτων, η μέγιστη τιμή τάσης του αισθητήρα (σε millivolts) κατά την κρούση και η χρονική στιγμή του χτυπήματος (σε σχέση με την έναρξη του γύρου) αποθηκεύονται προσωρινά.

Κάθε μονάδα ESP32 ("BlueBoxer" ή "RedBoxer") αποστέλλει τα δεδομένα των ανιχνευμένων χτυπημάτων, μορφοποιημένα (π.χ., ως συμβολοσειρά JSON), στην εφαρμογή "BoxSensors" μέσω της ενεργής σύνδεσης BLE (εικόνα 65).

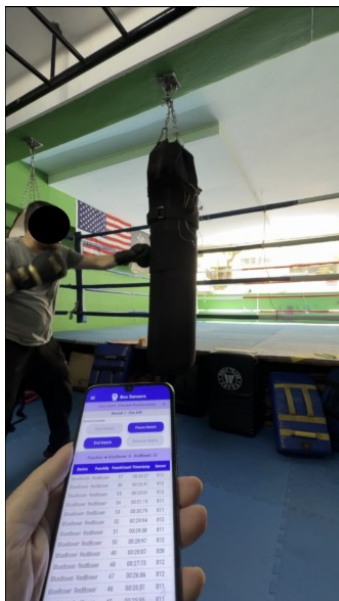


Εικόνα 60. Το πληροφοριακό σύστημα στην πράξη.

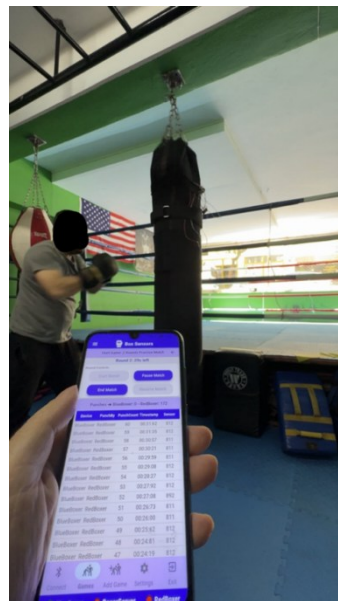
5.3 Αποτελέσματα

Στο γυμναστήριο έκανα δοκιμές μ' έναν αθλητή της πυγμαχίας πάνω σε σάκο, με αισθητήρες του εμπορίου και με αισθητήρες που κατασκεύασα εγώ ίδιος. Οι δοκιμές ήταν αρκετά επιτυχημένες. Μεγαλύτερη επιτυχία υπήρξε με τους αισθητήρες του εμπορίου. Ο μικρότερος αισθητήρας που κατασκεύασα είχε αρκετά ικανοποιητική απόδοση ενώ ο μεγαλύτερος απογοητευτική. Είναι πάρα πολύ σημαντικό να μην παρουσιάζονται διακοπές στο κύκλωμα. Με την παραμικρή διακοπή το κύκλωμα σταματά να καταγράφει σωστά τα χτυπήματα, επομένως η κατασκευή πρέπει να είναι ανθεκτική στις καταπονήσεις και να μην τραυματίζει τους αθλητές αν φορεθεί πάνω τους. Όταν η επιφάνεια του αισθητήρα είναι μεγάλη και δεν είναι εργοστασιακού τύπου ζαρώνει και τσαλακώνεται εύκολα με αποτέλεσμα να μην παρέχει σωστή πληροφόρηση ή καθόλου πληροφόρηση. Αντιθέτως κάτι τέτοιο δε συμβαίνει στους εργοστασιακούς, δεν βρήκα ωστόσο στο εμπόριο αισθητήρες με μεγάλη επιφάνεια ώστε να μπορέσουν να καλύψουν ένα σάκο χωρίς να χρειάζεται να χρησιμοποιηθούν πάρα πολλοί για να δημιουργήσουν την επιφάνεια που θα καταμετρά το χτύπημα. Όταν ένα τέτοιο σύστημα χρειάζεται πολλούς τέτοιους αισθητήρες προκύπτουν προβλήματα άλλου είδους. Οι wearable συσκευές (μικροελεγκτές) δεν έχουν απεριόριστο αριθμό αναλογικών εισόδων που χρειάζεται

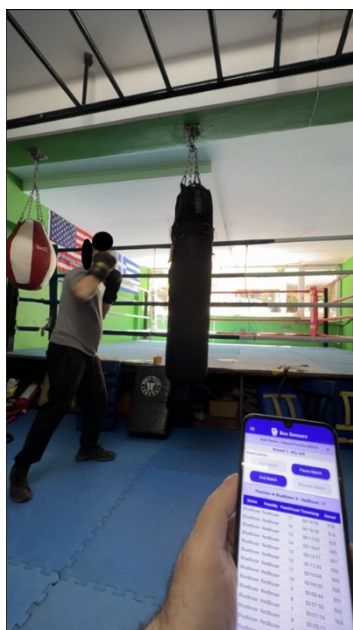
ο συγκεκριμένος αισθητήρας που χρησιμοποίησα για την πτυχιακή. Οπότε θα χρειαζόταν να καταφύγω σε κάποιον πολυπλέκτη (multiplexer). Ιδανικά θα πρέπει να κατασκευαστεί κάποιο ολοκληρωμένο κύκλωμα που έχει πάνω του όλες τις ιδέες του δημιουργού ώστε να μην παρουσιάζονται διακοπές. Χρειάζεται δηλαδή να δημιουργήσει ένα τυπωμένο κύκλωμα (PCB printed circuit board). Ο μικρότερος αισθητήρας που κατασκεύασα δεν παρουσίασε προβλήματα αντίστοιχα. Μειονέκτημα του ήταν ότι έδινε τιμές μεγαλύτερες σε κατάσταση αδράνειας (κατάσταση μηδενικής πίεσης) σε σχέση με τους εργοστασιακούς. Το εύρος τιμών που έδινε ήταν (0-300 millivolt). Η καταγραφή των χτυπημάτων ήταν ικανοποιητική. Μειονέκτημα των εργοστασιακών και των κατασκευασμένων με το χέρι είναι η τρομερή ευαισθησία τους στην καταπόνηση. Ένας τρόπος για να αποφύγεις τη μεγάλη καταπόνηση είναι να τους βάλεις εσωτερικά μέσα σε σάκο. Στις παρακάτω τέσσερις εικόνες βλέπουμε τον αθλητή την ώρα που χτυπά τον σάκο με τους αισθητήρες. Στην πρώτη (εικόνα 66) και δεύτερη φωτογραφία (εικόνα 67) είναι οι αισθητήρες του εμπορίου καλυμμένοι με ένα ύφασμα μαύρο και στη τρίτη φωτογραφία (εικόνα 68) ο μικρότερος ο οποίος λειτούργησε ικανοποιητικά. Οι μετρήσεις με τους εργοστασιακούς ήταν απολύτως ικανοποιητικές και δεν έδειχναν να χάνουν τα χτυπήματα. Ο μικρότερος που κατασκεύασα υπολειπόταν σε σχέση με τους εργοστασιακούς αλλά δεν ήταν απογοητευτική η συλλογή των χτυπημάτων. Απογοητευτική ήταν η συλλογή των χτυπημάτων μόνο από τον μεγάλο σε μέγεθος αισθητήρα που κατασκεύασα ιδιόχειρα (εικόνα 69). Θεωρώ ότι οι αισθητήρες του εμπορίου καλύπτουν απολύτως την καταγραφή των χτυπημάτων.



Εικόνα 61. Κατά την διάρκεια των δοκιμών.



Εικόνα 62. Κατά την διάρκεια των δοκιμών.

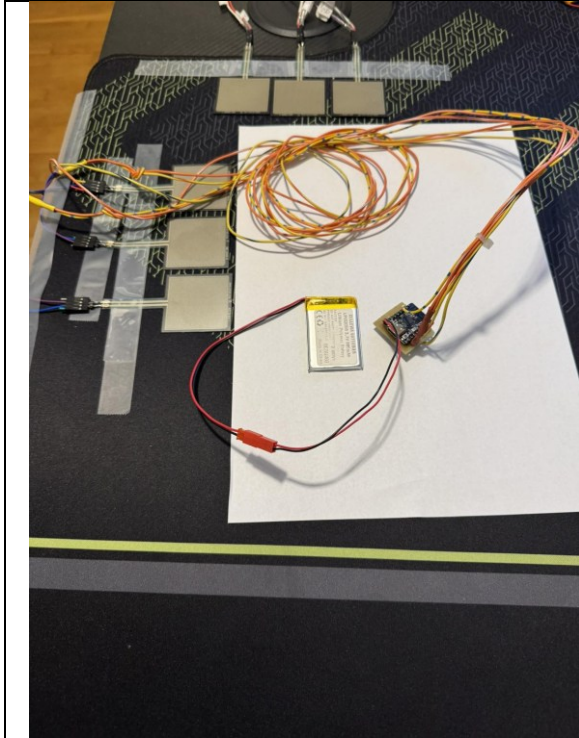


Εικόνα 63. Κατά την διάρκεια των δοκιμών.

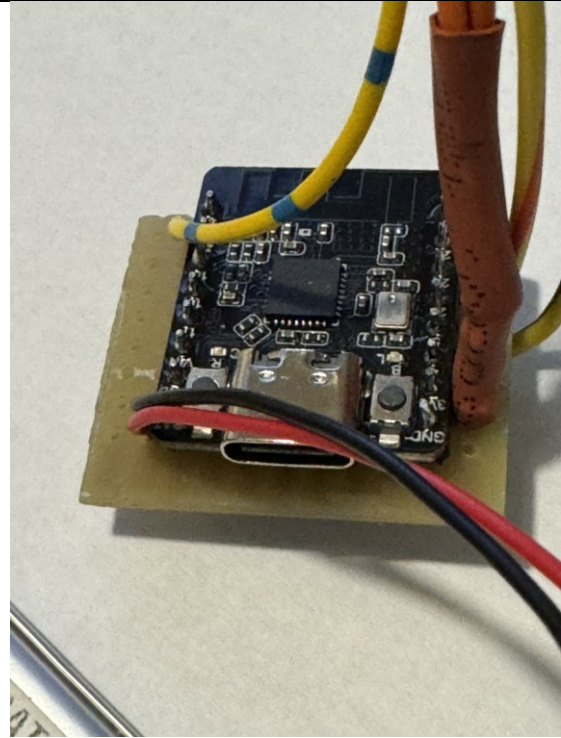


Εικόνα 64. Κατά την διάρκεια των δοκιμών.

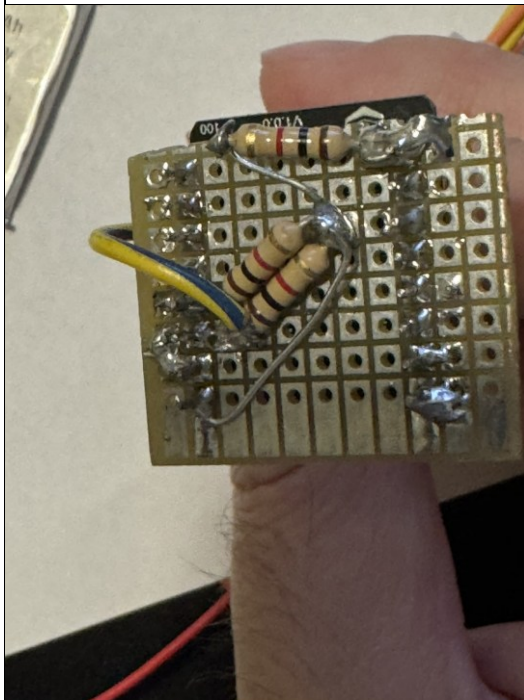
Ο μικροελεγκτής που χρησιμοποιήθηκε για τις δοκιμές με τους τρεις εργοστασιακούς αισθητήρες και το σχεδιάγραμμα στο Tinker Cad (εικόνες 70, 71, 72) , (σχήμα 22).



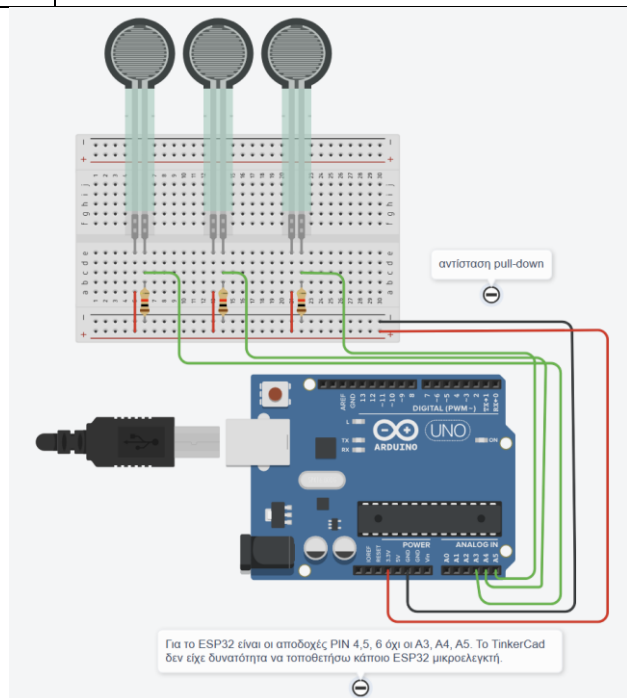
Εικόνα 70. Το κύκλωμα με τους αισθητήρες



Εικόνα 65. Μπροστινή όψη.



Εικόνα 66. Πίσω όψη.



Σχήμα 22. Σχεδιάγραμμα Tinker Cad.

6. Επίλογος

Η παρούσα πτυχιακή εργασία, μέσα από την ανάπτυξη και τις δοκιμές του συστήματος καταγραφής, ανέδειξε τόσο τις δυνατότητές του όσο και συγκεκριμένους τομείς που επιδέχονται περαιτέρω βελτίωση. Ειδικότερα, η ενίσχυση της μηχανικής αντοχής της κατασκευής, με έμφαση στην αυξημένη ανθεκτικότητα των αισθητήρων έναντι των καταπονήσεων, πιθανώς μέσω της διερεύνησης εναλλακτικών υλικών, προβάλλει ως σημαντική μελλοντική προοπτική. Η υλοποίηση τέτοιων προηγμένων χαρακτηριστικών, αν και συχνά υπερβαίνει τους διαθέσιμους πόρους και το χρονικό πλαίσιο μιας πτυχιακής εργασίας, υπογραμμίζει το αξιόλογο δυναμικό του εγχειρήματος για μελλοντική εξέλιξη. Στο πλαίσιο αυτό, η ακόλουθη ενότητα επικεντρώνεται στις παρατηρήσεις που προέκυψαν και στους περιορισμούς που εντοπίστηκαν κατά την τρέχουσα φάση υλοποίησης.

Παρατηρήσεις και Περιορισμοί:

Είναι σημαντικό να αναγνωριστεί ότι, ενώ το σύστημα παρέχει πολύτιμες ενδείξεις, η μέτρηση της έντασης των χτυπημάτων μέσω των αισθητήρων FSR είναι σχετική και δεν αντιστοιχεί σε βαθμονομημένη μέτρηση δύναμης σε φυσικές μονάδες, εκτός αν πραγματοποιηθεί εξειδικευμένη διαδικασία βαθμονόμησης. Επιπλέον, η ακρίβεια του "πραγματικού χρόνου" στην απεικόνιση των δεδομένων υπόκειται στις συνήθεις καθυστερήσεις των ασύρματων δικτύων (Bluetooth, Wi-Fi) και της επεξεργασίας στην πλατφόρμα Cloud. Τέλος, η αξιοπιστία των επικοινωνιών BLE και Wi-Fi μπορεί να επηρεαστεί από περιβαλλοντικές παρεμβολές.

Παρά τους όποιους περιορισμούς, το ολοκληρωμένο σύστημα προσφέρει ένα ισχυρό εργαλείο για τη συλλογή και ανάλυση δεδομένων στην πυγμαχία, με δυνατότητες που μπορούν να ενισχύσουν την προπονητική διαδικασία και την κατανόηση της αγωνιστικής απόδοσης. Η αξία αυτή θα μπορούσε να επανζηθεί σημαντικά μελλοντικά, μέσω της ενσωμάτωσης και άλλων τύπων αισθητήρων όπως επιταχυνσιόμετρα, γυροσκόπια (ή συνδυασμένες αδρανειακές μονάδες μέτρησης - IMU), πιεζοηλεκτρικοί αισθητήρες, αισθητήρες κάμψης, μικρόφωνα και αισθητήρες απόστασης. Η ενσωμάτωση αυτών των προηγμένων αισθητήρων, ωστόσο, εισάγει ένα νέο σύνολο σημαντικών προκλήσεων που αφορούν τόσο τη στοχευμένη ανίχνευση των πυγμαχικών γεγονότων όσο και την πρακτική τους εφαρμογή στο απαιτητικό περιβάλλον του αθλήματος.

Η χρήση αδρανειακών μονάδων (IMUs), για παράδειγμα, για την καταγραφή της κινηματικής των χτυπημάτων, όπως η ταχύτητα της γροθιάς ή η περιστροφή του καρπού, απαιτεί την ανάπτυξη εξελιγμένων αλγορίθμων ικανών να απομονώσουν την κίνηση ενδιαφέροντος από τις συνολικές κινήσεις του σώματος του αθλητή. Παράλληλα, η μηχανική τους ενσωμάτωση στα γάντια ή τον εξοπλισμό πρέπει να είναι εξαιρετικά σταθερή, εργονομική και ανθεκτική στους κραδασμούς, χωρίς να επηρεάζει την απόδοση ή την ασφάλεια. Αντίστοιχα, οι πιεζοηλεκτρικοί αισθητήρες, παρότι προσφέρουν άμεση απόκριση στην κρούση, παρουσιάζουν την πρόκληση της αποτελεσματικής κάλυψης μιας ευρείας επιφάνειας κρούσης και την ανάγκη για υψηλή ανθεκτικότητα σε επαναλαμβανόμενα, ισχυρά χτυπήματα χωρίς υποβάθμιση της απόδοσής τους ή μηχανική αστοχία. Οι αισθητήρες κάμψης, που θα μπορούσαν να ανιχνεύσουν τη διαμόρφωση της γροθιάς ή την κίνηση συγκεκριμένων αρθρώσεων, αντιμετωπίζουν ζητήματα που σχετίζονται με τη μηχανική κόπωση του υλικού τους από τους συνεχείς κύκλους κάμψης, καθώς και την πρόκληση της αξιόπιστης συσχέτισης της μετρούμενης κάμψης με συγκεκριμένα και ουσιαστικά πυγμαχικά γεγονότα.

Η ηχητική ανίχνευση των κρούσεων μέσω μικροφώνων, αν και θεωρητικά ελκυστική, προσκρούει στην πρακτική δυσκολία της αποτελεσματικής απομόνωσης του ήχου του χτυπήματος από τον έντονο περιβάλλοντα θόρυβο ενός γυμναστηρίου ή αγωνιστικού χώρου, καθώς και από τις μηχανικές δονήσεις που μεταδίδονται στον αισθητήρα. Τέλος, οι αισθητήρες απόστασης, οι οποίοι θα μπορούσαν να παρέχουν δεδομένα για την τακτική των πυγμάχων και τη διαχείριση της απόστασης, απαιτούν συνεχή και ανεμπόδιστη οπτική ή ακουστική επαφή με τον στόχο, μια συνθήκη που είναι δύσκολο να διασφαλιστεί αδιάλειπτα κατά τη διάρκεια των γρήγορων, δυναμικών και συχνά απρόβλεπτων κινήσεων της πυγμαχίας.

Σε όλες αυτές τις δυνητικές επεκτάσεις, η θεμελιώδης και διαρκής πρόκληση παραμένει η ευφυής μηχανική σχεδίαση και η προσεκτική, εργονομική ενσωμάτωση των αισθητήρων στο απαιτητικό και δυναμικό περιβάλλον του αθλήματος. Αυτό περιλαμβάνει την εξασφάλιση της ανθεκτικότητας των αισθητήρων και των συνοδευτικών τους καλωδιώσεων έναντι των ακραίων μηχανικών καταπονήσεων και των περιβαλλοντικών συνθηκών (όπως ο ιδρώτας), με απώτερο στόχο την αξιοπιστία των μετρήσεων και τη μακροζωία του εξοπλισμού. Εξίσου, αν όχι περισσότερο, κρίσιμη είναι η εγγύηση ότι η οποιαδήποτε ενσωμάτωση αισθητήρων δεν θα αποτελεί πηγή κινδύνου ή τραυματισμού για τον αθλητή ή τον αντίπαλό του, δεν θα παρεμποδίζει την ελευθερία των κινήσεών του και την τεχνική του απόδοση, και ταυτόχρονα

θα επιτρέπει τη συλλογή αξιόπιστων, επαναλήψιμων και ουσιαστικών δεδομένων. Η αποτελεσματική αντιμετώπιση αυτών των σύνθετων μηχανικών, ηλεκτρονικών και εργονομικών προκλήσεων είναι καθοριστικής σημασίας για την πρακτική εφαρμοσιμότητα και την πραγματική προστιθέμενη αξία οποιουδήποτε τεχνολογικού συστήματος ανάλυσης στην πυγμαχία.

7. Βιβλιογραφία

Μαλούτα, Θ. Β., & Μίχου, Ε. Ζ. (2019). Το Διαδίκτυο των Πραγμάτων (IoT): Συγκριτική παρουσίαση και αξιολόγηση των σημαντικότερων τεχνολογιών υλοποίησης. Διπλωματική εργασία, Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών. Διαθέσιμο στο: <https://pergamos.lib.uoa.gr/uoa/dl/frontend/file/lib/default/data/2866409/theFile>

Παπάζογλου, Π. και Λιωνής, Σ.-Π., 2020. Ανάπτυξη Εφαρμογών με το Arduino. 3η έκδοση. Εκδόσεις Τζιόλα, ISBN: 978-960-418-937-3.

Ala Al-Fuqaha, M., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M., 2015.

Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. IEEE Communications Surveys & Tutorials, 17(4). DOI: 10.1109/COMST.2015.2444095.

Ali, B., 2016. Internet of Things based Smart Homes: Security Risk Assessment and Recommendations. Master's Thesis in Information Security Department of Computer Science, Electrical and Space Engineering - Luleå University of Technology.

Alsaadi, E., & Tubaishat, E., 2015. Internet of Things: Features, Challenges, and Vulnerabilities. International Journal of Advanced Computer Science and Information Technology (IJACSIT), 4(1).

Atzori, L., Iera, A., & Morabito, G., 2010.

The Internet of Things: A survey. Computer Networks. DOI: 10.1016/j.comnet.2010.05.010.

Barrett, S.F. and Pack, D.J. (2006). Microcontrollers Fundamentals for Engineers and Scientists. Synthesis Lectures on Digital Circuits and Systems, [online] 1(1), pp.1–124. Available at: https://www.academia.edu/34455343/Microcontrollers_Fundamentals_for_Engineers_and_Scientists.

Bisinfotech (2020). Top 10 Microcontrollers (MCU) Manufacturers for 2020. [online] Bisinfotech. Available at: <https://www.bisinfotech.com/top-10-microcontrollers-mcu-manufacturers-2020/>.

Bisdikian, N., 2001. An Overview of the Bluetooth Wireless Technology. IEEE Communications Magazine, 39(12), pp.86-94. DOI: 10.1109/35.968817.

Chang, L., Yang, Z., Guo, X., & Zhang, D. (2021). A Novel Flexible Force Sensing Resistor with High Sensitivity and Linear Response over a Wide Force Range. Sensors, (2018), 6109. DOI: 10.3390/s21186109 Available at: <https://www.mdpi.com/1424-8220/21/18/6109>

Channelfutures.com. (2016). The Four Internet of Things Connectivity Models Explained. [online] Available at: <https://www.channelfutures.com/regulation-compliance/the-four-internet-of-things-connectivity-models-explained>.

Chi, E.H., Jin Dong Song and Corbin, G. (2004). "Killer App" of Wearable Computing: Wireless Force Sensing Body Protectors for Martial Arts,. UIST '04: Proceedings of the 17th annual ACM symposium on User interface software and technology, vol. 6, issue 2, Pages 277 – 285, DOI:10.1109/MPRV.2005.67.

Chi E. H., "Introducing wearable force sensors in martial arts," στο IEEE Pervasive Computing, vol. 4, no. 3, pp. 47-53, July-Sept. 2005, DOI: 10.1109/MPRV.2005.67.

Dhondge, K., 2016. Internet Of Things (IoT) Applications With Diverse Direct Communication Methods. A Dissertation in Telecommunications and Computer Networking And Computer Science - University of Missouri.

ESP Boards (2023). Exploring the upgrade path of ESP32 microcontrollers series. [online] www.espboards.dev. Available at: <https://www.espboards.dev/blog/esp32-soc-options/>.

Freedman, M., Freedman, T. R., Freedman, S. M., & Freedman, B. S. (2021). Impact Force Measurement in Boxing Using a Force Sensing Resistor Array. IEEE Sensors Journal, 21(9), 10253–10261. DOI: 10.1109/JSEN.2021.3053412

Available at: <https://ieeexplore.ieee.org/document/9333695>

R. Bannatyne and G. Viot (2002). Introduction to microcontrollers. I. [online] pp.350–360. doi:<https://doi.org/10.1109/wescon.1998.716623>.

Gadre, D. V., 2001. Programming and customizing the AVR microcontroller. 1st ed. New York: McGraw-Hill.

Giovanelli, D., Farella, E., & Castellini, C. (2021). A Velostat-Based Pressure-Sensitive Mat for Center-of-Pressure Measurement. International Journal of Environmental Research and Public Health, 18(11). DOI: 10.3390/ijerph18115958.

Interlink Electronics. (2021). FSR® Integration Guide & Evaluation Parts Catalog. (Εναλλακτικά, PDF) Διαθέσιμο σε: https://cdn.sparkfun.com/assets/e/3/b/3/8/force_sensing_resistor_guide.pdf

International Telecommunication Union, 2012. ITU - T Recommendation database. [online] Available at: <https://www.itu.int/ITU-T/recommendations/rec.aspx?rec=y.2060>.

Introduction, 2018. Retrieved December 11, 2020, Available at: <https://www.arduino.cc/en/guide/introduction>.

Jang, W.-J., Lee, K.-K., Lee, W.-J. and Lim, S.-H. (2022). Development Development of an Inertial Sensor Module for Categorizing Anomalous Kicks in Taekwondo and Monitoring the Level of Impact, στο Sensors 2022, vol.22. issue 7, 2591; <https://doi.org/10.3390/s22072591>.

GeeksforGeeks (2024) Protocols in application layer, GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/protocols-application-layer/> (Accessed: 05 June 2025).

GeeksforGeeks (2024) Types of internet protocols, GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/types-of-internet-protocols/> (Accessed: 05 June 2025).

Minew (2024) Completely introduction of IOT networks in 2024: What is it, types, and usage: Minew, Minew . Available at: <https://www.minew.com/iot-networks-types-and-usage/> (Accessed: 05 June 2025).

Giusto, D., 2010. The Internet of Things: 20th Tyrrhenian workshop on digital communications. New York: Springer.

Hobbs, A., 2018. Five ways the Internet of Things is transforming businesses today. Retrieved December 10, 2020, Available at: <https://internetofbusiness.com/5-ways-the-internet-of-things-is-transforming-businesses-today/>.

Ki, J.-S., Jeong, D.-H. and Lee, H.-J. (2019). Research on Impact Sensors for Developing the Electronic Body Protector of Taekwondo. Journal of the Korea Academia-Industrial cooperation Society, [online] 20(4), pp.648–655. doi:<https://doi.org/10.5762/kais.2019.20.4.648>.

- Kim, Y. & Park, S. (2018). High-Speed Force Sensing for Impact Measurements in Combat Sports. *IEEE Transactions on Instrumentation and Measurement*, 67(10), 2382–2391. DOI: 10.1109/TIM.2018.2804282 Available at: <https://ieeexplore.ieee.org/document/8283567>
- Lethaby, N., 2017. Wireless connectivity for the Internet of Things, One size does not fit all (Rev. A). Texas Instruments. [online] Available at: <http://www.tij.co.jp/jp/lit/wp/swry010a/swry010a.pdf>.
- Li, S., Xu, L.D., & Zhao, S., 2015. The Internet of Things: A Survey. *Inf Syst Front* 17, 243–259. DOI: 10.1007/s10796-014-9492-7.
- Madakam, Somayya., R & Tripathi, Siddharth., (2015). *Internet of Things (IoT): A Literature Review*. [online] Available at: https://www.researchgate.net/publication/280527542_Internet_of_Things_IoT_A_Literature_Review.
- Madlmayr, G., Langer, J., Kantner, C., & Scharinger, J., (2015), Near Field Communication (NFC) Technology: A Survey. *International Journal on Cybernetics & Informatics*, 4(2), pp.133-144. DOI: 10.5121/ijci.2015.4213. Metoree, 2025. Microcontroller Manufacturers. [online] Available at: <https://us.metoree.com/categories/5287/#manufacturers>
- Metoree. (2025). *29 Microcontroller (MCU's) Manufacturers in 2025 | Metoree*. [online] Available at: <https://us.metoree.com/categories/5287/#manufacturers> [Accessed 5 Jun. 2025].
- Minew (2024) Completely introduction of IOT networks in 2024: What is it, types, and usage. Available at: <https://www.minew.com/iot-networks-types-and-usage/> (Accessed: 05 June 2025).
- Park, E., Kim, H., & Jeong, S. (2020). Development of a Force-Sensitive Resistor (FSR)-Based Sensor for Monitoring Pressure Distribution. *Sensors*, 20(24), 7162. DOI: 10.3390/s20247162 Available at: <https://www.mdpi.com/1424-8220/20/24/7162>
- Qureshi, F. and Krishnan, S. (2022). Design and Analysis of Electronic Head Protector for Taekwondo Sports, στο *Sensors* 2022, vol. 22, issue 4, 1415; Available at: <https://doi.org/10.3390/s22041415>.
- Seyfried, J. & Schmalz, T. (2019). Reliability and validity of the FSR sensor for measuring plantar pressure in gait analysis. *Gait & Posture*, 67, 251–255.

DOI: 10.1016/j.gaitpost.2018.10.010

Available at: <https://www.sciencedirect.com/science/article/pii/S0966636218308131>

Smith, J., 2022. Microcontrollers: A Comprehensive Overview and Comparative Analysis of Diverse Types. Engrxiv.org. [online] Available at: <https://engrxiv.org/preprint/view/3228> [Accessed 21 January 2025].

Song, Y., Jeon, Y., Park, G., An, H., Hwang, T., Lee, H. and Lee, S. (2010). Development of Taekwondo Trainer System for Training on Electronic Protector with Hitting Target Indicator. IJCSNS International Journal of Computer Science and Network Security, [online] 10(6), p.51. Available at: http://paper.ijcsns.org/07_book/201006/20100607.pdf [Accessed 4 Jun. 2025].

Sorna Mugi Viswanathan, E-Textile and its Applications
DOI:10.1002/adma.200990013.PMID

Tama Birkocak, D., Gomes, P. and Carvalho, H. (2023). Flexible Pressure Sensors for Integration into Karate Body Protector, στο Sensors, volume 23, issue 14, 6524; <https://doi.org/10.3390/s23146524>.

8. Παράρτημα Α: «Πηγαίος κώδικας Arduino IoT Cloud.»

Αρχείο: BoxServerThing_feb20a.ino

```
/**
 * @file BoxServerThing_feb20a.ino // (Η το πραγματικό όνομα του .ino αρχείου σας)
 * @author [Nick Dimitrakarakos / 83899]
 * @brief Κύριο αρχείο σκίτσου Arduino για το IoT Cloud Boxing Game Project.
 *
 * Αυτό το σκίτσο διαχειρίζεται:
 * 1. Σύνδεση στο Arduino IoT Cloud.
 * 2. Αρχικοποίηση και διαχείριση των Cloud Variables (μέσω του thingProperties.h).
 * 3. Επικοινωνία μέσω Bluetooth Low Energy (BLE) χρησιμοποιώντας την κλάση
BluetoothHandler.
 * 4. Λήψη μηνυμάτων JSON μέσω BLE και επεξεργασία τους χρησιμοποιώντας την ενσωματωμένη
κλάση JsonHandler.
 * 5. Ενημέρωση των Cloud Variables με βάση τα επεξεργασμένα δεδομένα JSON.
 * 6. Συγχρονισμό της τοπικής ώρας με το Arduino IoT Cloud.
 * 7. Εμφάνιση πληροφοριών κατάστασης και debugging μέσω της σειριακής κονσόλας.
 *
 * @version 1.0
 * @date 2025-05-14
 *
 * @copyright Copyright (c) 2025
 */

// Βασικές βιβλιοθήκες Arduino και του project
#include <Arduino.h>
#include "thingProperties.h" // Αυτόματα παραγόμενο αρχείο από το Arduino IoT Cloud.
// Περιέχει τους ορισμούς των Cloud Variables και τη
συνάρτηση initProperties().
// Το περιβάλλον του IoT Cloud το συγχωνεύει κατά τη
μεταγλώττιση.
#include "BluetoothHandler.h" // Προσαρμοσμένη βιβλιοθήκη για τη διαχείριση της
επικοινωνίας BLE.
#include <Timelib.h> // Βιβλιοθήκη για τη διαχείριση και μετατροπή του χρόνου.
#include <ArduinoJson.h> // Βιβλιοθήκη για την αποτελεσματική επεξεργασία (parsing και
δημιουργία) δεδομένων JSON.

/**
 * @class JsonHandler
 * @brief Ενσωματωμένη βοηθητική (utility) κλάση που περιέχει μια στατική μέθοδο
για την επεξεργασία (parsing) εισερχόμενων δεδομένων σε μορφή JSON.
 *
 * Η κλάση αυτή δεν χρειάζεται δημιουργία αντικειμένου καθώς η μέθοδός της είναι στατική.
 * Χρησιμοποιείται για την ανάλυση των μηνυμάτων που λαμβάνονται μέσω BLE.
 */
class JsonHandler {
public:
    /**
     * @brief Επεξεργάζεται (parses) μια εισερχόμενη συμβολοσειρά που αναμένεται να είναι σε
μορφή JSON.
     *
     * Αποσειριοποιεί τη συμβολοσειρά JSON χρησιμοποιώντας τη βιβλιοθήκη ArduinoJson.
     */
};
```

```
* Ενημερώνει τις καθολικές Cloud Variables ανάλογα με το περιεχόμενο του JSON.
* Υποστηρίζει δύο κύριες δομές JSON:
* 1. "RoundStatusCommand": Για εντολές επαναφοράς των μεταβλητών του παιχνιδιού.
* 2. Δεδομένα χτυπήματος: Περιέχει πληροφορίες για ένα χτύπημα (ποιος χτύπησε, ποιος
χτυπήθηκε, σκορ, κ.λπ.).
*
* @param incoming Η συμβολοσειρά (String) που περιέχει τα δεδομένα JSON προς
επεξεργασία.
*/
static void parseIncoming(const String &incoming) {
    // Δημιουργία ενός στατικού JSON document. Το μέγεθος (256 bytes) πρέπει να είναι
    // επαρκές για το μεγαλύτερο αναμενόμενο JSON μήνυμα.
    // Αν τα JSON μηνύματα είναι μεγαλύτερα, αυτό το μέγεθος πρέπει να αυξηθεί.
    StaticJsonDocument<256> doc;

    // Αποσειριοποίηση της εισερχόμενης συμβολοσειράς JSON.
    DeserializationError error = deserializeJson(doc, incoming);

    // Έλεγχος για σφάλματα κατά την αποσειριοποίηση.
    if (error) {
        Serial.print(F("JsonHandler - JSON parse failed: ")); // Χρήση F() για εξοικονόμηση
        Serial.println(error.f_str());
        return; // Έξοδος από τη συνάρτηση αν η αποσειριοποίηση απέτυχε.
    }

    // Έλεγχος αν το JSON περιέχει το κλειδί "RoundStatusCommand".
    // Αυτό υποδεικνύει ένα μήνυμα εντολής για την κατάσταση του γύρου (π.χ., reset).
    if (doc.containsKey("RoundStatusCommand")) {
        // Εξαγωγή της τιμής "Command" από το αντικείμενο "RoundStatusCommand".
        // Η χρήση `| 0` παρέχει μια προεπιλεγμένη τιμή (0) αν το κλειδί "Command" δεν
        υπάρχει ή δεν είναι αριθμός.
        int cmd = doc["RoundStatusCommand"]["Command"] | 0;

        if (cmd == 1) { // Εντολή 1: Επαναφορά όλων των μεταβλητών του παιχνιδιού.
            Serial.println(F("JsonHandler - Received RoundStatusCommand (cmd=1): Resetting all
game variables."));

            // Επαναφορά των γενικών Cloud Variables που αφορούν το τελευταίο χτύπημα.
            deviceThatGotHit = "";
            boxerThatScoresThePoint = "";
            punchScore = 0;
            timeStampOfThePunch = "";
            sensorValue = 0;

            // Επαναφορά των Cloud Variables για τον μπλε πυγμάχο.
            blueBoxer_punchCount = 0;
            blueBoxer_timestamp = "";
            blueBoxer_sensorValue = 0;

            // Επαναφορά των Cloud Variables για τον κόκκινο πυγμάχο.
            redBoxer_punchCount = 0;
            redBoxer_timestamp = "";
            redBoxer_sensorValue = 0;
        }
    }
}
```

```
// Οι αλλαγές στις Cloud Variables θα συγχρονιστούν με το cloud στην επόμενη κλήση
ArduinoCloud.update().
} else {
    Serial.print(F("JsonHandler - Received RoundStatusCommand with unknown command
value: "));
    Serial.println(cmd);
}
}
// Αν δεν είναι "RoundStatusCommand", τότε επεξεργασία ως κανονικό μήνυμα δεδομένων
χτυπήματος.
else {
    Serial.println(F("JsonHandler - Received punch data JSON. Parsing..."));

    // Εξαγωγή των τιμών από τα πεδία του JSON.
    // Χρήση `doc["key"]` για πρόσβαση στις τιμές.
    // Η συντομογραφία `var ? val_if_true : val_if_false` χρησιμοποιείται για έλεγχο
    nullptr.
    const char* devStr    = doc["deviceStr"];        // Η συσκευή που δέχτηκε το χτύπημα
    (π.χ., "RedBoxer")
    const char* oppDev    = doc["oppositeDevice"];    // Η συσκευή που έκανε το χτύπημα (ο
    αντίπαλος)
    const char* punchStr  = doc["punchCount"];        // Ο τρέχων αριθμός χτυπήματος (ως
    string)
    const char* timeStr   = doc["timestamp"];         // Η χρονοσφραγίδα του χτυπήματος
    const char* sensor    = doc["sensorValue"];       // Η τιμή του αισθητήρα (ως string)

    // Εκχώρηση των τιμών στις γενικές Cloud μεταβλητές.
    // Έλεγχος για nullptr (μέσω της τριαδικής συνθήκης) για αποφυγή σφαλμάτων
    // αν κάποιο πεδίο λείπει από το JSON. Αν λείπει, εκχωρείται προεπιλεγμένη τιμή.
    deviceThatGotHit      = devStr    ? String(devStr)    : "";
    boxerThatScoresThePoint = oppDev  ? String(oppDev)    : "";
    punchScore           = punchStr ? atoi(punchStr)      : 0; // Μετατροπή string σε
    integer
    timeStampOfThePunch   = timeStr   ? String(timeStr)   : "";
    sensorValue           = sensor    ? atoi(sensor)      : 0; // Μετατροπή string σε
    integer

    // Debugging output για τις γενικές μεταβλητές
    /*
    Serial.print(F("JsonHandler - Parsed Generic Data: deviceThatGotHit="));
    Serial.print(deviceThatGotHit);
    Serial.print(F(", boxerThatScoresThePoint=")); Serial.print(boxerThatScoresThePoint);
    Serial.print(F(", punchScore=")); Serial.print(punchScore);
    Serial.print(F(", timeStampOfThePunch=")); Serial.print(timeStampOfThePunch);
    Serial.print(F(", sensorValue=")); Serial.println(sensorValue);
    */

    // Ειδική λογική για την ενημέρωση των μεταβλητών του κάθε πυγμάχου.
    // Η λογική είναι: αν το `deviceStr` (η συσκευή που χτυπήθηκε) είναι ο "RedBoxer",
    // τότε ο "BlueBoxer" είναι αυτός που πέτυχε το χτύπημα, και αντίστροφα.
    String deviceString = devStr ? String(devStr) : "";

    if (deviceString == "RedBoxer") {
        // Το χτύπημα καταγράφηκε στον RedBoxer, άρα ο BlueBoxer το προκάλεσε.
        // Ενημέρωση των Cloud Variables του BlueBoxer με τα δεδομένα του χτυπήματος.
```

```
        blueBoxer_punchCount = punchScore; // Ο αριθμός χτυπήματος που πέτυχε ο μπλε.  
        blueBoxer_timestamp = timeStampOfThePunch; // Η χρονοσφραγίδα του χτυπήματος του  
μπλε.  
        blueBoxer_sensorValue = sensorValue;          // Η τιμή του αισθητήρα από το χτύπημα  
του μπλε.  
        Serial.println(F("JsonHandler - Data attributed to BlueBoxer (hit on RedBoxer)."));  
    }  
    else if (deviceString == "BlueBoxer") {  
        // Το χτύπημα καταγράφηκε στον BlueBoxer, άρα ο RedBoxer το προκάλεσε.  
        // Ενημέρωση των Cloud Variables του RedBoxer με τα δεδομένα του χτυπήματος.  
        redBoxer_punchCount = punchScore; // Ο αριθμός χτυπήματος που πέτυχε ο κόκκινος.  
        redBoxer_timestamp = timeStampOfThePunch; // Η χρονοσφραγίδα του χτυπήματος του  
κόκκινου.  
        redBoxer_sensorValue = sensorValue;          // Η τιμή του αισθητήρα από το χτύπημα  
του κόκκινου.  
        Serial.println(F("JsonHandler - Data attributed to RedBoxer (hit on BlueBoxer)."));  
    } else if (devStr != nullptr) { // Αν το devStr υπάρχει αλλά δεν είναι "RedBoxer" ή  
"BlueBoxer"  
        Serial.print(F("JsonHandler - Unknown deviceStr for boxer-specific logic: "));  
        Serial.println(deviceString);  
    } else { // Αν το devStr είναι nullptr  
        Serial.println(F("JsonHandler - deviceStr is null, cannot determine specific boxer  
logic."));  
    }  
}  
}  
};  
  
// Δημιουργία καθολικού αντικειμένου για τη διαχείριση της λειτουργίας Bluetooth.  
BluetoothHandler bleHandler;  
  
// Δηλώσεις συναρτήσεων (Function prototypes) που ορίζονται παρακάτω στο αρχείο.  
void waitForValidTime();  
void printCurrentTime();  
String formatTimestamp(unsigned long timestamp);  
  
/**  
 * @brief Συνάρτηση αρχικοποίησης. Εκτελείται μία φορά κατά την εκκίνηση ή το reset του  
Arduino.  
 *  
 * Αρχικοποιεί:  
 * - Τη σειριακή επικοινωνία για debugging.  
 * - Τη σύνδεση με το Arduino IoT Cloud και τις Cloud μεταβλητές.  
 * - Τον BLE server μέσω του αντικειμένου bleHandler.  
 * - Προσπαθεί να συγχρονίσει την τοπική ώρα με αυτή του IoT Cloud.  
 */  
void setup() {  
    // Έναρξη σειριακής επικοινωνίας για debugging και μηνύματα κατάστασης.  
    Serial.begin(9600);  
    // Μικρή καθυστέρηση για να προλάβει να αρχικοποιηθεί η σειριακή οθόνη (αν χρειάζεται).  
    delay(2000); // 2 δευτερόλεπτα  
  
    Serial.println(F("\n[SETUP] Booting up device..."));
```

```
// Έναρξη σύνδεσης με το Arduino IoT Cloud χρησιμοποιώντας την προτιμώμενη μέθοδο
// σύνδεσης (π.χ., Wi-Fi).
ArduinoCloud.begin(ArduinoIoTPreferredConnection);
// Αρχικοποίηση όλων των Cloud μεταβλητών που ορίστηκαν στο thingProperties.h.
// Αυτή η συνάρτηση καλεί επίσης τις onXXXChange συναρτήσεις για τις μεταβλητές που έχουν
// οριστεί ως READ_WRITE.
initProperties();
Serial.println(F("[SETUP] Initialized Thing Properties.));

Serial.println(F("[SETUP] Attempting to connect to Arduino IoT Cloud...));
// Ορισμός χρονικού ορίου για την προσπάθεια σύνδεσης στο IoT Cloud (30 δευτερόλεπτα).
unsigned long cloudConnectTimeout = millis() + 30000;
// Βρόχος αναμονής για τη σύνδεση στο IoT Cloud.
while (!ArduinoCloud.connected() && millis() < cloudConnectTimeout) {
    ArduinoCloud.update(); // Απαραίτητη κλήση για την επεξεργασία των λειτουργιών του
    cloud.
    delay(500); // Μικρή καθυστέρηση μεταξύ των προσπαθειών.
    Serial.print(F("."));
}
Serial.println(); // Νέα γραμμή μετά τις τελείες της αναμονής.

// Έλεγχος αποτελέσματος σύνδεσης στο IoT Cloud.
if (!ArduinoCloud.connected()) {
    Serial.println(F("[SETUP] WARNING: Failed to connect to IoT Cloud within timeout.));
    // Εδώ θα μπορούσε να προστεθεί λογική για εναλλακτική λειτουργία ή ένδειξη σφάλματος.
} else {
    Serial.println(F("[SETUP] Successfully connected to Arduino IoT Cloud!));
}

Serial.println(F("[SETUP] Waiting for initial IoT Cloud Time Sync (5s delay)...));
// Μικρή καθυστέρηση για να δοθεί χρόνος στο Cloud να είναι έτοιμο για συγχρονισμό ώρας.
delay(5000);

// Έναρξη του BLE server με το όνομα "BoxerServer".
// Η μέθοδος `begin` της `bleHandler` αρχικοποιεί το BLE stack και διαφημίζει τη συσκευή.
bleHandler.begin("BoxerServer"); // Το όνομα που θα φαίνεται στις BLE scans.
Serial.println(F("[SETUP] BLE server started. Advertising as 'BoxerServer'.));

// Εμφάνιση κατάστασης Wi-Fi και τοπικής IP (αν η σύνδεση είναι μέσω Wi-Fi).
if (WiFi.status() == WL_CONNECTED) {
    Serial.println(F("[SETUP] Connected to Wi-Fi.));
    Serial.print(F("[SETUP] Local IP Address: "));
    Serial.println(WiFi.localIP());
} else {
    Serial.println(F("[SETUP] WARNING: Wi-Fi not connected. IoT Cloud functionality might
be affected if Wi-Fi is the preferred connection.));
}

// Καθυστέρηση για σταθεροποίηση πριν τον τελικό συγχρονισμό ώρας.
delay(2000);
// Κλήση της συνάρτησης για συγχρονισμό της τοπικής ώρας με το IoT Cloud.
waitForValidTime();

Serial.println(F("[SETUP] Device setup complete. Entering main loop.));
}
```

```
/**
 * @brief Κύρια συνάρτηση βρόχου. Εκτελείται επαναληπτικά μετά την ολοκλήρωση του setup().
 *
 * Κύριες λειτουργίες εντός του loop:
 * - Ενημέρωση της κατάστασης του Arduino IoT Cloud (κρίσιμο για συγχρονισμό).
 * - Επεξεργασία γεγονότων BLE (νέα μηνύματα, συνδέσεις/αποσυνδέσεις).
 * - Ανάγνωση και επεξεργασία εισερχόμενων μηνυμάτων JSON από το BLE.
 */
void loop() {
    // Ενημέρωση της κατάστασης και των λειτουργιών του Arduino IoT Cloud.
    // Αυτή η κλήση είναι κρίσιμη για τη διατήρηση της σύνδεσης, τον συγχρονισμό
    // των Cloud μεταβλητών και την εκτέλεση των onXXXChange callbacks.
    ArduinoCloud.update();

    // Επεξεργασία των λειτουργιών του BLE (π.χ., έλεγχος για νέα μηνύματα, διαχείριση
    // συνδέσεων).
    // Η `poll()` πρέπει να καλείται τακτικά.
    bleHandler.poll();

    // Ανάγνωση τυχόν εισερχόμενου μηνύματος από τη συνδεδεμένη BLE client συσκευή.
    String incomingMessage = bleHandler.readMessage();

    // Έλεγχος αν υπάρχει κάποιο νέο, μη κενό μήνυμα.
    if (incomingMessage.length() > 0) {
        Serial.print(F("[LOOP] Received raw BLE message: "));
        Serial.println(incomingMessage);

        // Καθαρισμός του buffer του εισερχόμενου μηνύματος στον BluetoothHandler
        // ώστε να είναι έτοιμος να δεχτεί το επόμενο μήνυμα.
        bleHandler.clearMessage();

        // Έλεγχος αν το εισερχόμενο μήνυμα πιθανόν να είναι ένα αντικείμενο JSON
        // (απλός έλεγχος αν ξεκινά με '{').
        // Μια πιο ανθεκτική προσέγγιση θα μπορούσε να περιλαμβάνει και έλεγχο για `[`
        // αν υποστηρίζονται JSON arrays ως κύρια δομή μηνύματος.
        if (incomingMessage.startsWith("{")) {
            Serial.println(F("[LOOP] Message starts with '{', attempting JSON parse..."));
            // Κλήση της στατικής μεθόδου parseIncoming της κλάσης JsonHandler
            // για την επεξεργασία του JSON και την ενημέρωση των Cloud Variables.
            JsonHandler::parseIncoming(incomingMessage);

            // (Προαιρετικό) Επιβεβαίωση των τιμών των Cloud Variables στη σειριακή οθόνη μετά το
            // parsing.
            // Αφαιρέστε τα σχόλια για ενεργοποίηση.
            /*
            Serial.println(F("[LOOP] Current Cloud Variable values after parse:"));
            Serial.print(F("  deviceThatGotHit: ")); Serial.println(deviceThatGotHit);
            Serial.print(F("  boxerThatScoresThePoint: "));
            Serial.println(boxerThatScoresThePoint);
            Serial.print(F("  punchScore: ")); Serial.println(punchScore);
            Serial.print(F("  timeStampOfThePunch: ")); Serial.println(timeStampOfThePunch);
            Serial.print(F("  sensorValue: ")); Serial.println(sensorValue);
            Serial.print(F("  blueBoxer_punchCount: ")); Serial.println(blueBoxer_punchCount);
            Serial.print(F("  redBoxer_punchCount: ")); Serial.println(redBoxer_punchCount);
            */
        }
    }
}
```



```
*/
} else {
    Serial.println(F("[LOOP] Incoming message does not start with '{'. Not treated as
JSON."));
}
}
// Μικρή καθυστέρηση για αποφυγή υπερβολικής χρήσης CPU και για σταθερότητα, αν
χρειάζεται.
// delay(10); // Για παράδειγμα, 10ms. Προσαρμόστε ανάλογα με τις απαιτήσεις απόκρισης.
}

/**
 * @brief Μετατρέπει ένα Unix timestamp (δευτερόλεπτα από την Epoch 1/1/1970)
 * σε μια μορφοποιημένη συμβολοσειρά ημερομηνίας και ώρας.
 *
 * @param timestamp Το Unix timestamp (unsigned long) προς μετατροπή.
 * @return String Η μορφοποιημένη ημερομηνία και ώρα σε μορφή "DD/MM/YYYY HH:MM:SS".
 */
String formatTimestamp(unsigned long timestamp) {
    tmElements_t tm; // Δομή από τη TimeLib.h για την αποθήκευση των στοιχείων της ώρας.
    breakTime(timestamp, tm); // Συνάρτηση από τη TimeLib.h για τη μετατροπή του timestamp σε
tmElements_t.

    // Buffer για τη μορφοποιημένη συμβολοσειρά. Πρέπει να είναι αρκετά μεγάλος
    // για να χωρέσει την ημερομηνία/ώρα και τον null terminator (π.χ., "DD/MM/YYYY
HH:MM:SS\0" -> 20 chars).
    char buffer[20];
    // Μορφοποίηση της ημερομηνίας και ώρας.
    // Το tm.Year είναι έτη από το 1970, οπότε προσθέτουμε 1970 για το πλήρες έτος.
    sprintf(buffer, "%02d/%02d/%04d %02d:%02d:%02d",
        tm.Day, tm.Month, tm.Year + 1970,
        tm.Hour, tm.Minute, tm.Second);
    return String(buffer); // Επιστροφή ως αντικείμενο String του Arduino.
}

/**
 * @brief Αναμένει μέχρι η τοπική ώρα να συγχρονιστεί με το Arduino IoT Cloud.
 *
 * Προσπαθεί να λάβει την ώρα από το cloud για ένα συγκεκριμένο χρονικό διάστημα (timeout).
 * Εμφανίζει κατάλληλα μηνύματα για την επιτυχία ή αποτυχία του συγχρονισμού.
 * Η `ArduinoCloud.getLocalTime()` επιστρέφει Unix timestamp (δευτερόλεπτα από 1/1/1970)
 * ή 0 αν η ώρα δεν είναι ακόμα διαθέσιμη.
 */
void waitForValidTime() {
    Serial.print(F("[TIME] Waiting for IoT Cloud time sync..."));
    // Ορισμός χρονικού ορίου για την προσπάθεια συγχρονισμού ώρας (30 δευτερόλεπτα).
    unsigned long timeSyncTimeout = millis() + 30000;

    // Βρόχος αναμονής μέχρι να ληφθεί έγκυρη ώρα (διαφορετική του 0) ή να λήξει το χρονικό
    όριο.
    while (ArduinoCloud.getLocalTime() == 0 && millis() < timeSyncTimeout) {
        Serial.print(F("."));
        delay(1000); // Αναμονή ενός δευτερολέπτου.
        ArduinoCloud.update(); // Απαραίτητο για την ενημέρωση της ώρας από το cloud.
    }
}
```

```
Serial.println(); // Νέα γραμμή μετά τις τελείες.  
  
// Έλεγχος αποτελέσματος συγχρονισμού ώρας.  
if (ArduinoCloud.getLocalTime() == 0) {  
    Serial.println(F("[TIME] ERROR: IoT Cloud time sync failed after timeout!"));  
    // Εδώ θα μπορούσε να προστεθεί λογική για χρήση μιας προεπιλεγμένης ώρας,  
    // ή ένδειξη ότι η ώρα δεν είναι συγχρονισμένη.  
} else {  
    Serial.print(F("[TIME] IoT Cloud time sync successful! "));  
    printCurrentTime(); // Εκτύπωση της συγχρονισμένης ώρας.  
}  
}
```

Αρχείο : BluetoothHandler.h

```
/**  
 * @file BluetoothHandler.h  
 * @author [Nick Dimitrakarakos / 83899]  
 * @brief Ορισμός της κλάσης BluetoothHandler για τη διαχείριση της επικοινωνίας  
 * Bluetooth Low Energy (BLE) σε μια συσκευή Arduino.  
 * Παρέχει λειτουργίες για τη δημιουργία ενός περιφερειακού BLE (server)  
 * με μια προσαρμοσμένη υπηρεσία τύπου UART (Nordic UART Service - NUS compatible)  
 * για αποστολή και λήψη δεδομένων.  
 * @version 1.0  
 * @date 2025-05-14  
 *  
 * @copyright Copyright (c) 2025  
 */  
  
#ifndef BLUETOOTH_HANDLER_H  
#define BLUETOOTH_HANDLER_H  
  
#include <Arduino.h> // Βασική βιβλιοθήκη Arduino.  
#include <ArduinoBLE.h> // Βιβλιοθήκη για τη λειτουργία Bluetooth Low Energy (BLE) της  
Arduino.  
  
// --- Προσαρμοσμένα UUIDs για την υπηρεσία και τα χαρακτηριστικά BLE (Nordic UART Service  
// - NUS) ---  
// Αυτά τα συγκεκριμένα UUIDs είναι τα τυπικά για την υπηρεσία Nordic UART Service (NUS),  
// η οποία επιτρέπει την αποστολή και λήψη συμβολοσειρών μέσω BLE, μιμούμενη μια σειριακή  
// σύνδεση.  
// Είναι σημαντικό τα ίδια UUIDs να χρησιμοποιούνται και από την κεντρική συσκευή (π.χ.,  
// κινητό τηλέφωνο)  
// για την επικοινωνία.  
  
/**  
 * @brief UUID για την υπηρεσία Nordic UART Service (NUS).  
 * Η υπηρεσία αυτή ομαδοποιεί τα χαρακτηριστικά RX και TX.  
 */  
#define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"  
  
/**  
 * @brief UUID για το χαρακτηριστικό RX (Receive) της υπηρεσίας NUS.
```

```
* Η κεντρική συσκευή (π.χ. κινητό) γράφει (WRITE) σε αυτό το χαρακτηριστικό για να στείλει
δεδομένα στο Arduino.
* Από την πλευρά του Arduino (peripheral), αυτό είναι το "receive" χαρακτηριστικό.
* Ιδιότητες (Properties): WRITE, WRITE_WITHOUT_RESPONSE.
*/
#define CHARACTERISTIC_UUID_RX    "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"

/**
 * @brief UUID για το χαρακτηριστικό TX (Transmit) της υπηρεσίας NUS.
 * Το Arduino (peripheral) γράφει σε αυτό το χαρακτηριστικό για να στείλει δεδομένα (μέσω
NOTIFY) στην κεντρική συσκευή.
* Από την πλευρά του Arduino, αυτό είναι το "transmit" χαρακτηριστικό.
* Ιδιότητες (Properties): NOTIFY (και συνήθως READ).
*/
#define CHARACTERISTIC_UUID_TX    "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

/**
 * @class BluetoothHandler
 * @brief Διαχειρίζεται τη λειτουργικότητα του Bluetooth Low Energy (BLE) για τη συσκευή
Arduino,
* λειτουργώντας ως περιφερειακό (peripheral/server) με την υπηρεσία Nordic UART Service
(NUS).
*
* Η κλάση αυτή αρχικοποιεί το BLE stack, δημιουργεί την υπηρεσία NUS
* και τα αντίστοιχα χαρακτηριστικά RX/TX, διαχειρίζεται τις συνδέσεις
* από κεντρικές συσκευές (centrals/clients), και επιτρέπει την αποστολή και λήψη
μηνυμάτων.
*/
class BluetoothHandler {
public:
    /**
     * @brief Κατασκευαστής της κλάσης BluetoothHandler.
     * Αρχικοποιεί ορισμένες μεταβλητές μέλη στις προεπιλεγμένες τους τιμές.
     */
    BluetoothHandler();

    /**
     * @brief Αρχικοποιεί το BLE stack, ρυθμίζει την υπηρεσία NUS και τα χαρακτηριστικά της,
     * και ξεκινά τη διαφήμιση (advertising) της συσκευής.
     * @param deviceName Το όνομα με το οποίο η συσκευή θα εμφανίζεται στις σαρώσεις BLE από
άλλες συσκευές.
     * Πρέπει να είναι μια C-style συμβολοσειρά (const char*).
     */
    void begin(const char* deviceName);

    /**
     * @brief Πρέπει να καλείται τακτικά (συνήθως σε κάθε επανάληψη της `loop()`)
     * για την επεξεργασία των γεγονότων του BLE stack.
     * Διαχειρίζεται τις συνδέσεις, τις αποσυνδέσεις, και τη λήψη δεδομένων.
     */
    void poll();

    /**
     * @brief Αποστέλλει ένα μήνυμα μέσω του χαρακτηριστικού TX της υπηρεσίας NUS.
     * Το μήνυμα θα σταλεί (μέσω NOTIFY) σε όλες τις συνδεδεμένες κεντρικές συσκευές

```

```
* που έχουν ενεργοποιήσει τις ειδοποιήσεις (notifications) για το χαρακτηριστικό TX.
* @param message Η συμβολοσειρά (Arduino String) που θα αποσταλεί.
*/
void sendMessage(const String &message);

/**
 * @brief Ελέγχει αν υπάρχει τουλάχιστον μία κεντρική συσκευή (central/client)
συνδεδεμένη
 * με αυτή την περιφερειακή συσκευή (peripheral/server).
 * @return true Εάν τουλάχιστον μία κεντρική συσκευή είναι συνδεδεμένη, false
διαφορετικά.
 */
bool isDeviceConnected();

/**
 * @brief Διαβάζει το τελευταίο πλήρες μήνυμα που έχει ληφθεί μέσω του χαρακτηριστικού
RX.
 * Η μέθοδος αυτή επιστρέφει το μήνυμα που έχει αποθηκευτεί εσωτερικά από την callback
`onRxWrite`.
 * @return String Το περιεχόμενο του τελευταίου ληφθέντος μηνύματος.
 * Επιστρέφει μια κενή συμβολοσειρά αν δεν υπάρχει νέο μήνυμα ή αν έχει ήδη καθαριστεί.
 */
String readMessage();

/**
 * @brief Καθαρίζει το buffer του τελευταίου ληφθέντος μηνύματος (receivedMessage).
 * Αυτό πρέπει να καλείται μετά την επεξεργασία ενός μηνύματος για να επιτρέψει
 * την ανίχνευση και αποθήκευση του επόμενου νέου μηνύματος.
 */
void clearMessage();

private:
// Αντικείμενα της βιβλιοθήκης ArduinoBLE για την υπηρεσία και τα χαρακτηριστικά.
BLEService service; ///< Το αντικείμενο που αντιπροσωπεύει την υπηρεσία
NUS.
BLECharacteristic rxCharacteristic; ///< Το χαρακτηριστικό για τη λήψη δεδομένων (RX) από
τον client.
BLECharacteristic txCharacteristic; ///< Το χαρακτηριστικό για την αποστολή δεδομένων
(TX) προς τον client.

String receivedMessage; ///< Buffer για την προσωρινή αποθήκευση του
τρέχοντος εισερχόμενου μηνύματος από το RX.

const char* _deviceName; ///< Το όνομα της συσκευής BLE όπως διαφημίζεται.
Αποθηκεύεται κατά την κλήση της `begin()`.

// Οι παρακάτω μεταβλητές είναι για επίδειξη ή πιθανό debugging και μπορούν να αφαιρεθούν
αν δεν είναι απαραίτητες.
String lastSentMessage; ///< Αποθηκεύει το τελευταίο μήνυμα που στάλθηκε
(π.χ., για αποφυγή διπλοτύπων αποστολών).
int duplicateMessageCount; ///< Μετρητής για διπλότυπα μηνύματα που επιχειρήθηκε
να σταλούν (για debugging).

int connectionCount; ///< Μετρητής για τον αριθμό των ενεργών συνδέσεων
από κεντρικές συσκευές.
```

```
/**
 * @brief Στατικός δείκτης στο μοναδικό αντικείμενο (instance) της κλάσης
BluetoothHandler.
 * Απαιτείται ώστε οι στατικές συναρτήσεις callback του BLE API (όπως onRxWrite,
onCentralConnect)
 * να μπορούν να έχουν πρόσβαση στα μη στατικά μέλη και μεθόδους της κλάσης.
 * Αυτό είναι ένα κοινό μοτίβο σχεδίασης (singleton-like access for callbacks) σε C++ για
Arduino.
 */
static BluetoothHandler* instance;

// --- Στατικές Συναρτήσεις Callback για Γεγονότα BLE ---
// Αυτές οι συναρτήσεις δηλώνονται ως static γιατί περνούν ως δείκτες συναρτήσεων
// στο API της βιβλιοθήκης ArduinoBLE, το οποίο δεν μπορεί να χειριστεί απευθείας non-
static member functions.

/**
 * @brief Συνάρτηση callback που καλείται από το BLE stack όταν μια κεντρική συσκευή
(client)
 * γράφει δεδομένα στο χαρακτηριστικό RX (`rxCharacteristic`).
 * @param central Η αντικείμενο BLEDevice που αντιπροσωπεύει την κεντρική συσκευή που
έγραψε τα δεδομένα.
 * @param characteristic Το αντικείμενο BLECharacteristic στο οποίο έγινε η εγγραφή (θα
πρέπει να είναι το `rxCharacteristic`).
 */
static void onRxWrite(BLEDevice central, BLECharacteristic characteristic);

/**
 * @brief Συνάρτηση callback που καλείται από το BLE stack όταν μια νέα κεντρική συσκευή
(client)
 * συνδέεται επιτυχώς με αυτή την περιφερειακή συσκευή (server).
 * @param central Η αντικείμενο BLEDevice που αντιπροσωπεύει την κεντρική συσκευή που
μόλις συνδέθηκε.
 */
static void onCentralConnect(BLEDevice central);

/**
 * @brief Συνάρτηση callback που καλείται από το BLE stack όταν μια συνδεδεμένη κεντρική
συσκευή (client)
 * αποσυνδέεται.
 * @param central Η αντικείμενο BLEDevice που αντιπροσωπεύει την κεντρική συσκευή που
μόλις αποσυνδέθηκε.
 */
static void onCentralDisconnect(BLEDevice central);
};

#endif // BLUETOOTH_HANDLER_H
```

Αρχείο : BluetoothHandler.cpp

```
/**
 * @file BluetoothHandler.cpp
 * @author [Nick Dimitrakarakos / 83899]
 * @brief Υλοποίηση της κλάσης BluetoothHandler για τη διαχείριση της επικοινωνίας
 * Bluetooth Low Energy (BLE) σε μια συσκευή Arduino, χρησιμοποιώντας την υπηρεσία Nordic
 * UART Service (NUS).
 * @version 1.0
 * @date 2025-05-14
 *
 * @copyright Copyright (c) 2025
 */

#include "BluetoothHandler.h" // Ορισμός της κλάσης BluetoothHandler.
#include <WiFi.h> // Περιλαμβάνεται για την εμφάνιση της IP διεύθυνσης στα
// μηνύματα σύνδεσης/αποσύνδεσης BLE.
// Αυτό μπορεί να είναι χρήσιμο για debugging, για να υπάρχει
// γενικότερο πλαίσιο της κατάστασης του δικτύου.

// Αρχικοποίηση του στατικού δείκτη 'instance'.
// Αυτός ο δείκτης θα δείχνει στο μοναδικό αντικείμενο (instance) της κλάσης
// BluetoothHandler.
// Χρησιμοποιείται από τις στατικές συναρτήσεις callback (onRxWrite, onCentralConnect,
// onCentralDisconnect)
// για να έχουν πρόσβαση στα μη στατικά μέλη και μεθόδους του αντικειμένου.
BluetoothHandler* BluetoothHandler::instance = nullptr;

/**
 * @brief Κατασκευαστής της κλάσης BluetoothHandler.
 * Αρχικοποιεί τα αντικείμενα της υπηρεσίας BLE (NUS) και των χαρακτηριστικών της (RX, TX),
 * καθώς και τις μεταβλητές μέλη της κλάσης στις προεπιλεγμένες τους τιμές.
 */
BluetoothHandler::BluetoothHandler()
    // Λίστα αρχικοποίησης μελών (Member Initializer List):
    // Αρχικοποιεί την υπηρεσία 'service' με το SERVICE_UUID.
    // Αρχικοποιεί το 'rxCharacteristic' με το CHARACTERISTIC_UUID_RX,
    // ορίζοντας τις ιδιότητες BLEWrite (επιτρέπει εγγραφή από client) και
    // BLEWriteWithoutResponse (επιτρέπει εγγραφή χωρίς απόκριση από το peripheral).
    // Το '512' ορίζει το μέγιστο μέγεθος (σε bytes) του buffer για αυτό το χαρακτηριστικό.
    : service(SERVICE_UUID),
      rxCharacteristic(CharacteristicUuidRx, BLEWrite | BLEWriteWithoutResponse, 512),
    // Αρχικοποιεί το 'txCharacteristic' με το CHARACTERISTIC_UUID_TX,
    // ορίζοντας τις ιδιότητες BLERead (επιτρέπει ανάγνωση από client) και
    // BLENotify (επιτρέπει την αποστολή ειδοποιήσεων στον client όταν η τιμή αλλάζει).
    // Το '512' ορίζει το μέγιστο μέγεθος του buffer.
      txCharacteristic(CharacteristicUuidTx, BLERead | BLENotify, 512)
{
    // Αρχικοποίηση των υπόλοιπων μεταβλητών μέλη.
    receivedMessage = ""; // Το buffer για τα εισερχόμενα μηνύματα είναι αρχικά
    // κενό.
    lastSentMessage = ""; // Το τελευταίο απεσταλμένο μήνυμα (για έλεγχο
    // διπλοτύπων) είναι αρχικά κενό.
    duplicateMessageCount = 0; // Ο μετρητής διπλοτύπων απεσταλμένων μηνυμάτων
    // αρχικοποιείται στο 0.
}
```

```
    connectionCount      = 0;          // Ο μετρητής ενεργών συνδέσεων από κεντρικές συσκευές
    αρχικοποιείται στο 0.
}

/**
 * @brief Αρχικοποιεί το BLE stack, ρυθμίζει την υπηρεσία NUS και τα χαρακτηριστικά της,
 * ορίζει το όνομα της συσκευής, προσθέτει τις απαραίτητες συναρτήσεις callback για
 * γεγονότα,
 * και ξεκινά τη διαφήμιση (advertising) της συσκευής ώστε να είναι ανιχνεύσιμη.
 * @param deviceName Το όνομα (C-style string) με το οποίο η συσκευή θα διαφημίζεται μέσω
 * BLE.
 */
void BluetoothHandler::begin(const char* deviceName) {
    // Ο στατικός δείκτης 'instance' τίθεται να δείχνει σε αυτό το τρέχον αντικείμενο (this).
    // Αυτό επιτρέπει στις static callback συναρτήσεις να έχουν πρόσβαση στα μέλη και τις
    // μεθόδους της κλάσης.
    instance = this;
    _deviceName = deviceName; // Αποθήκευση του ονόματος της συσκευής.

    // Έναρξη της βιβλιοθήκης ArduinoBLE.
    if (!BLE.begin()) {
        Serial.println(F("[BLE Handler] Failed to initialize ArduinoBLE library!"));
        // Κρίσιμο σφάλμα: αν η αρχικοποίηση του BLE αποτύχει, η συσκευή εισέρχεται σε ατέρμονα
        // βρόχο.
        // Σε μια πιο ανθεκτική εφαρμογή, θα μπορούσε να γίνει προσπάθεια επανεκκίνησης,
        // να ενεργοποιηθεί μια κατάσταση σφάλματος (π.χ., LED), ή να συνεχίσει χωρίς BLE αν
        // είναι δυνατό.
        while (1);
    }

    // Ορισμός του τοπικού ονόματος της συσκευής BLE (αυτό που φαίνεται στις σαρώσεις).
    BLE.setLocalName(_deviceName);
    // Ορισμός της υπηρεσίας που θα διαφημίζεται (η υπηρεσία NUS).
    BLE.setAdvertisedService(service);

    // Προσθήκη των χαρακτηριστικών RX και TX στην προσαρμοσμένη υπηρεσία NUS.
    service.addCharacteristic(rxCharacteristic);
    service.addCharacteristic(txCharacteristic);

    // Ορισμός της συνάρτησης callback (event handler) για το χαρακτηριστικό RX.
    // Η συνάρτηση `onRxWrite` θα καλείται αυτόματα κάθε φορά που μια κεντρική συσκευή
    // (client)
    // γράφει δεδομένα στο χαρακτηριστικό RX.
    rxCharacteristic.setEventHandler(BLEWritten, onRxWrite);

    // Προσθήκη της υπηρεσίας NUS στο BLE stack της συσκευής.
    BLE.addService(service);
    // Έναρξη της διαφήμισης (advertising). Η συσκευή γίνεται ανιχνεύσιμη από άλλες BLE
    // συσκευές.
    BLE.advertise();

    // Ορισμός των συναρτήσεων callback (event handlers) για τα γεγονότα σύνδεσης και
    // αποσύνδεσης κεντρικών συσκευών.
    BLE.setEventHandler(BLEConnected, onCentralConnect);
    BLE.setEventHandler(BLEDisconnected, onCentralDisconnect);
}
```



```
Serial.print(F("[BLE Handler] BLE stack initialized. Advertising as: "));
Serial.println(_deviceName);
}

/**
 * @brief Επεξεργάζεται τα εισερχόμενα γεγονότα και τις εργασίες του BLE stack.
 * Αυτή η συνάρτηση πρέπει να καλείται συχνά και τακτικά μέσα στην κύρια συνάρτηση `loop()`
 * του Arduino σκίτσου για τη σωστή λειτουργία του BLE (π.χ., διαχείριση συνδέσεων,
 * λήψη δεδομένων, αποστολή ειδοποιήσεων).
 */
void BluetoothHandler::poll() {
    // Καλεί την αντίστοιχη `poll()` της βιβλιοθήκης ArduinoBLE.
    BLE.poll();
}

/**
 * @brief Αποστέλλει ένα μήνυμα (String) μέσω του χαρακτηριστικού TX της υπηρεσίας NUS.
 * Το μήνυμα αποστέλλεται σε όλες τις συνδεδεμένες κεντρικές συσκευές (clients)
 * που έχουν ενεργοποιήσει τις ειδοποιήσεις (notifications) για το TX χαρακτηριστικό.
 * @param message Η συμβολοσειρά (Arduino String) προς αποστολή.
 */
void BluetoothHandler::sendMessage(const String &message) {
    // Έλεγχος αν υπάρχει τουλάχιστον μία συνδεδεμένη κεντρική συσκευή.
    // Αν όχι, η αποστολή παραλείπεται για εξοικονόμηση πόρων και αποφυγή σφαλμάτων.
    if (!isDeviceConnected()) {
        // Serial.println(F("[BLE Handler] No BLE client connected. Message not sent.)); //
        Προαιρετικό μήνυμα
        return;
    }

    // Παράδειγμα λογικής για την παρακολούθηση διπλοτύπων μηνυμάτων (κυρίως για debugging ή
    ειδικές περιπτώσεις).
    // Αυτή η λογική μπορεί να αφαιρεθεί ή να τροποποιηθεί ανάλογα με τις απαιτήσεις της
    εφαρμογής.
    if (message == lastSentMessage) {
        duplicateMessageCount++;
    } else {
        duplicateMessageCount = 0; // Επαναφορά του μετρητή αν το μήνυμα είναι νέο.
    }
    lastSentMessage = message; // Αποθήκευση του τρέχοντος μηνύματος ως το τελευταίο
    απεσταλμένο.

    // Η μορφοποίηση του μηνύματος για να περιλαμβάνει τον αριθμό των διπλοτύπων είναι
    // σε σχόλια, καθώς μπορεί να μην είναι επιθυμητή στην τελική εφαρμογή.
    // String formattedMessage = message + " | Duplicates: " + String(duplicateMessageCount);
    String formattedMessage = message; // Χρήση του αρχικού μηνύματος για αποστολή.

    // Εγγραφή της τιμής (μηνύματος) στο χαρακτηριστικό TX.
    // Η κεντρική συσκευή θα λάβει αυτό το μήνυμα μέσω ειδοποίησης (notification) αν την έχει
    ενεργοποιήσει.
    // Το μήνυμα μετατρέπεται σε πίνακα από bytes (const uint8_t*) χρησιμοποιώντας τη μέθοδο
    c_str() του String.
    txCharacteristic.writeValue((const uint8_t*)formattedMessage.c_str(),
                                formattedMessage.length());
```

```
Serial.print(F("[BLE Handler] Sent TX: "));
Serial.println(formattedMessage);
}

/**
 * @brief Ελέγχει αν υπάρχει τουλάχιστον μία ενεργή σύνδεση BLE με κεντρική συσκευή.
 * @return true Αν υπάρχει τουλάχιστον μία συνδεδεμένη κεντρική συσκευή, false διαφορετικά.
 */
bool BluetoothHandler::isDeviceConnected() {
    // Η μέθοδος `BLE.connected()` της βιβλιοθήκης ArduinoBLE επιστρέφει `true`
    // αν υπάρχει τουλάχιστον μία ενεργή σύνδεση, και `false` διαφορετικά.
    return BLE.connected();
}

/**
 * @brief Επιστρέφει το τελευταίο πλήρες μήνυμα που λήφθηκε μέσω του χαρακτηριστικού RX
 * και αποθηκεύτηκε στην εσωτερική μεταβλητή `receivedMessage`.
 * @return String Το περιεχόμενο του τελευταίου ληφθέντος μηνύματος.
 * Επιστρέφει κενή συμβολοσειρά αν δεν έχει ληφθεί νέο μήνυμα ή αν έχει καθαριστεί.
 */
String BluetoothHandler::readMessage() {
    return receivedMessage;
}

/**
 * @brief Καθαρίζει (κάνει κενή) την εσωτερική μεταβλητή `receivedMessage` που αποθηκεύει
 * το τελευταίο ληφθέν μήνυμα.
 * Αυτό πρέπει να καλείται από τον κώδικα της εφαρμογής μετά την επεξεργασία ενός
 * μηνύματος,
 * ώστε να είναι δυνατή η ανίχνευση και αποθήκευση του επόμενου νέου μηνύματος.
 */
void BluetoothHandler::clearMessage() {
    receivedMessage = "";
}

/**
 * @brief Στατική συνάρτηση callback. Καλείται αυτόματα από το BLE stack όταν μια κεντρική
 * συσκευή (client)
 * γράφει δεδομένα στο χαρακτηριστικό RX (`rxCharacteristic`) αυτής της περιφερειακής
 * συσκευής.
 * @param central Η αντικείμενο BLEDevice που αντιπροσωπεύει την κεντρική συσκευή που
 * έγραψε τα δεδομένα.
 * @param characteristic Το αντικείμενο BLECharacteristic στο οποίο έγινε η εγγραφή
 * (αναμένεται να είναι το `rxCharacteristic`).
 */
void BluetoothHandler::onRxWrite(BLEDevice central, BLECharacteristic characteristic) {
    // Έλεγχος αν ο στατικός δείκτης 'instance' έχει αρχικοποιηθεί (δείχνει σε ένα έγκυρο
    αντικείμενο BluetoothHandler).
    if (instance) {
        // Λήψη των ανεπεξέργαστων δεδομένων (raw bytes) και του μήκους τους από το
        χαρακτηριστικό.
        const uint8_t* data = characteristic.value();
        unsigned int length = characteristic.valueLength(); // Χρήση unsigned int για το μήκος
        για συμβατότητα.
    }
}
```

```
String incoming; // Δημιουργία ενός Arduino String για την αποθήκευση του μηνύματος.  
incoming.reserve(length); // Προ-δέσμευση μνήμης για το String για πιθανή βελτίωση  
απόδοσης.  
  
// Μετατροπή των bytes σε Arduino String.  
for (unsigned int i = 0; i < length; i++) {  
    incoming += (char)data[i];  
}  
incoming.trim(); // Αφαίρεση τυχόν κενών χαρακτήρων (whitespace) από την αρχή ή το  
τέλος του μηνύματος.  
  
// Αποθήκευση του ληφθέντος και επεξεργασμένου μηνύματος στο μέλος 'receivedMessage'  
// του αντικειμένου BluetoothHandler, μέσω του στατικού δείκτη 'instance'.  
instance->receivedMessage = incoming;  
  
Serial.print(F("[BLE Handler] Received RX from [");  
Serial.print(central.address()); // Εκτύπωση της διεύθυνσης MAC της κεντρικής συσκευής  
για debugging.  
Serial.print(F("]: "));  
Serial.println(incoming);  
}  
}  
  
/**  
 * @brief Στατική συνάρτηση callback. Καλείται αυτόματα από το BLE stack όταν μια νέα  
κεντρική συσκευή (client)  
 * συνδέεται επιτυχώς με αυτή την περιφερειακή συσκευή (server).  
 * @param central Η αντικείμενο BLEDevice που αντιπροσωπεύει την κεντρική συσκευή που μόλις  
συνδέθηκε.  
 */  
void BluetoothHandler::onCentralConnect(BLEDevice central) {  
    if (!instance) return; // Έλεγχος αν ο δείκτης 'instance' είναι έγκυρος.  
  
    instance->connectionCount++; // Αύξηση του μετρητή ενεργών συνδέσεων.  
  
    // Εμφάνιση πληροφοριών σύνδεσης στη σειριακή οθόνη.  
    // Η εμφάνιση της IP εδώ είναι για γενικότερο logging context, καθώς η σύνδεση είναι BLE.  
    if (WiFi.status() == WL_CONNECTED) {  
        Serial.print(F("[BLE Handler] Wi-Fi Connected. IP: "));  
        Serial.println(WiFi.localIP());  
    } else {  
        Serial.println(F("[BLE Handler] Wi-Fi not connected."));  
    }  
    Serial.print(F("[BLE Handler] Client [");  
    Serial.print(central.address()); // Εκτύπωση της διεύθυνσης MAC του client.  
    Serial.print(F("] connected. Total connections: "));  
    Serial.println(instance->connectionCount);  
}  
  
/**  
 * @brief Στατική συνάρτηση callback. Καλείται αυτόματα από το BLE stack όταν μια  
συνδεδεμένη  
 * κεντρική συσκευή (client) αποσυνδέεται από αυτή την περιφερειακή συσκευή.
```

```
* @param central Η αντικείμενο BLEDevice που αντιπροσωπεύει την κεντρική συσκευή που μόλις
αποσυνδέθηκε.
*/
void BluetoothHandler::onCentralDisconnect(BLEDevice central) {
    if (!instance) return; // Έλεγχος αν ο δείκτης 'instance' είναι έγκυρος.

    // Μείωση του μετρητή ενεργών συνδέσεων, διασφαλίζοντας ότι δεν γίνεται αρνητικός.
    if (instance->connectionCount > 0) {
        instance->connectionCount--;
    }

    // Εμφάνιση πληροφοριών αποσύνδεσης στη σειριακή οθόνη.
    if (WiFi.status() == WL_CONNECTED) {
        Serial.print(F("[BLE Handler] Wi-Fi Status: Connected. IP: "));
        Serial.println(WiFi.localIP());
    } else {
        Serial.println(F("[BLE Handler] Wi-Fi Status: Not connected.));
    }
    Serial.print(F("[BLE Handler] Client ["));
    Serial.print(central.address()); // Εκτύπωση της διεύθυνσης MAC του client.
    Serial.print(F("] disconnected. Remaining connections: "));
    Serial.println(instance->connectionCount);
}
```

Αρχείο : thingProperties.h

```
/**
 * @file thingProperties.h
 * @author Arduino IoT Cloud Generator & [Nick Dimitrakarakos / 83899] (for
review/comments)
 * @brief Αυτό το αρχείο παράγεται αυτόματα από το Arduino IoT Cloud.
 * Ορίζει τις Cloud Variables που συγχρονίζονται μεταξύ της συσκευής Arduino
 * και του Arduino IoT Cloud, καθώς και τις ρυθμίσεις σύνδεσης δικτύου.
 *
 * @warning **ΜΗΝ ΕΠΕΞΕΡΓΑΣΤΕΙΤΕ ΑΠΕΥΘΕΙΑΣ ΑΥΤΟ ΤΟ ΑΡΧΕΙΟ ΕΚΤΟΣ ΑΝ ΓΝΩΡΙΖΕΤΕ ΤΙ ΚΑΝΕΤΕ.**
 * Οι αλλαγές ενδέχεται να αντικατασταθούν από το Arduino IoT Cloud.
 * Είναι προτιμότερο να διαχειρίζεστε τις ιδιότητες (properties) μέσω του web interface του
Arduino IoT Cloud.
 *
 * @version Auto-Generated (plus manual comments v1.0)
 * @date 2025-05-14 (Date of comment addition)
 *
 * @copyright Copyright (c) Arduino
 */

// Code generated by Arduino IoT Cloud, DO NOT EDIT. (Original comment from generator)
#ifndef THINGPROPERTIES_H
#define THINGPROPERTIES_H

// Συμπερίληψη του αρχείου arduino_secrets.h.
// Αυτό το αρχείο περιέχει ευαίσθητες πληροφορίες όπως τα διαπιστευτήρια Wi-Fi (SSID και
κωδικός πρόσβασης).
```

```
// Οι σταθερές SECRET_SSID & SECRET_OPTIONAL_PASS ορίζονται εκεί.
// Το αρχείο αυτό συνήθως δεν ανεβαίνει σε δημόσια repositories (π.χ., προστίθεται
στο .gitignore).
#include "arduino_secrets.h"
// Βασική βιβλιοθήκη για τη λειτουργία του Arduino IoT Cloud.
#include <ArduinoIoTCloud.h>
// Βιβλιοθήκη για τη διαχείριση των συνδέσεων δικτύου (π.χ., Wi-Fi) για το IoT Cloud.
#include <Arduino_ConnectionHandler.h>

// --- Διαπιστευτήρια Σύνδεσης Δικτύου ---
// Ορισμός των σταθερών SSID και PASS για τη σύνδεση Wi-Fi.
// Οι τιμές τους λαμβάνονται από τις αντίστοιχες SECRET_SSID και SECRET_OPTIONAL_PASS
// που ορίζονται στο αρχείο 'arduino_secrets.h'.
const char SSID[] = SECRET_SSID; // Το όνομα του δικτύου Wi-Fi (Service Set
Identifier).
const char PASS[] = SECRET_OPTIONAL_PASS; // Ο κωδικός πρόσβασης για το δίκτυο Wi-Fi.

// --- Δήλωση των Cloud Variables ---
// Αυτές είναι οι μεταβλητές που θα συγχρονίζονται με το Arduino IoT Cloud.
// Οι τύποι τους (CloudString, CloudInt, κ.λπ.) παρέχονται από τη βιβλιοθήκη
ArduinoIoTCloud.

// Γενικές μεταβλητές που περιγράφουν το τελευταίο καταγεγραμμένο χτύπημα.
CloudString deviceThatGotHit; // Αναγνωριστικό της συσκευής/πυγμαχού που δέχτηκε το
χτύπημα (π.χ., "RedBoxer", "BlueBoxer").
CloudString boxerThatScoresThePoint; // Αναγνωριστικό του πυγμαχού που πέτυχε το χτύπημα.
CloudInt punchScore; // Το σκορ ή η ισχύς του συγκεκριμένου χτυπήματος.
CloudString timeStampOfThePunch; // Η χρονοσφραγίδα (σε μορφή String) του πότε συνέβη
το χτύπημα.
CloudInt sensorValue; // Η ακατέργαστη ή επεξεργασμένη τιμή από τον
αισθητήρα που κατέγραψε το χτύπημα.

// Μεταβλητές που αφορούν συγκεκριμένα τον Μπλε Πυγμαχό (Blue Boxer).
CloudInt blueBoxer_punchCount; // Ο συνολικός αριθμός επιτυχημένων χτυπημάτων από τον
Μπλε Πυγμαχό.
CloudString blueBoxer_timestamp; // Η χρονοσφραγίδα του τελευταίου χτυπήματος του Μπλε
Πυγμαχού.
CloudInt blueBoxer_sensorValue; // Η τιμή του αισθητήρα από το τελευταίο χτύπημα του
Μπλε Πυγμαχού.

// Μεταβλητές που αφορούν συγκεκριμένα τον Κόκκινο Πυγμαχό (Red Boxer).
CloudInt redBoxer_punchCount; // Ο συνολικός αριθμός επιτυχημένων χτυπημάτων από τον
Κόκκινο Πυγμαχό.
CloudString redBoxer_timestamp; // Η χρονοσφραγίδα του τελευταίου χτυπήματος του
Κόκκινου Πυγμαχού.
CloudInt redBoxer_sensorValue; // Η τιμή του αισθητήρα από το τελευταίο χτύπημα του
Κόκκινου Πυγμαχού.

/**
 * @brief Αρχικοποιεί τις ιδιότητες (Cloud Variables) και τις συνδέει με το Arduino IoT
Cloud.
 *
 * Αυτή η συνάρτηση καλείται μία φορά μέσα στο `setup()` του κύριου σκίτσου Arduino.
 * Για κάθε Cloud Variable, ορίζεται:
```

```
* - Η ίδια η μεταβλητή.  
* - Τα δικαιώματα πρόσβασης (permission):  
* - `READWRITE`: Η μεταβλητή μπορεί να διαβαστεί και να γραφτεί τόσο από τη συσκευή όσο και από το Cloud dashboard.  
* - `READONLY`: Η μεταβλητή μπορεί μόνο να γραφτεί από τη συσκευή και να διαβαστεί από το Cloud.  
* - Η πολιτική ενημέρωσης (update policy):  
* - `ON_CHANGE`: Η τιμή συγχρονίζεται με το Cloud μόνο όταν αλλάξει.  
* - `PERIODIC`: Η τιμή συγχρονίζεται περιοδικά, ανεξάρτητα από το αν έχει αλλάξει (π.χ., κάθε Χ δευτερόλεπτα).  
* - Μια προαιρετική συνάρτηση callback που θα εκτελεστεί όταν η τιμή της μεταβλητής αλλάξει από το Cloud.  
* Εδώ, το `NULL` υποδεικνύει ότι δεν έχει οριστεί κάποια συγκεκριμένη callback συνάρτηση για αυτές τις μεταβλητές  
* σε αυτό το επίπεδο (αν και μπορεί να οριστεί στο κύριο σκίτσο ή μέσω του Cloud interface).  
*/  
void initProperties() {  
    // Προσθήκη κάθε Cloud Variable στο σύστημα διαχείρισης του ArduinoIoTCloud.  
    // Όλες οι παρακάτω μεταβλητές ορίζονται ως READWRITE και ενημερώνονται ON_CHANGE,  
    // χωρίς συγκεκριμένη callback συνάρτηση κατά την προσθήκη τους εδώ.  
  
    // Γενικές μεταβλητές χτυπήματος  
    ArduinoCloud.addProperty(boxerThatScoresThePoint, READWRITE, ON_CHANGE, NULL);  
    ArduinoCloud.addProperty(deviceThatGotHit, READWRITE, ON_CHANGE, NULL);  
    ArduinoCloud.addProperty(punchScore, READWRITE, ON_CHANGE, NULL);  
    ArduinoCloud.addProperty(timestampOfThePunch, READWRITE, ON_CHANGE, NULL);  
    ArduinoCloud.addProperty(sensorValue, READWRITE, ON_CHANGE, NULL);  
  
    // Μεταβλητές Μπλε Πυγμάχου  
    ArduinoCloud.addProperty(blueBoxer_punchCount, READWRITE, ON_CHANGE, NULL);  
    ArduinoCloud.addProperty(blueBoxer_timestamp, READWRITE, ON_CHANGE, NULL);  
    ArduinoCloud.addProperty(blueBoxer_sensorValue, READWRITE, ON_CHANGE, NULL);  
  
    // Μεταβλητές Κόκκινου Πυγμάχου  
    ArduinoCloud.addProperty(redBoxer_punchCount, READWRITE, ON_CHANGE, NULL);  
    ArduinoCloud.addProperty(redBoxer_timestamp, READWRITE, ON_CHANGE, NULL);  
    ArduinoCloud.addProperty(redBoxer_sensorValue, READWRITE, ON_CHANGE, NULL);  
}  
  
// --- Διαχείριση Σύνδεσης Δικτύου ---  
// Δημιουργία ενός αντικειμένου Connection Handler.  
// Το `WiFiConnectionHandler` είναι μια κλάση που παρέχεται από τη βιβλιοθήκη  
// `Arduino_ConnectionHandler`.  
// Διαχειρίζεται τη διαδικασία σύνδεσης στο δίκτυο Wi-Fi χρησιμοποιώντας τα παρεχόμενα SSID και PASS.  
// Το `ArduinoIoTPreferredConnection` είναι το όνομα του αντικειμένου που θα χρησιμοποιηθεί  
// από το `ArduinoCloud.begin()` για την εγκαθίδρυση της σύνδεσης.  
WiFiConnectionHandler ArduinoIoTPreferredConnection(SSID, PASS);  
#endif // THINGPROPERTIES_H
```

arduino_secrets.h

```
#define SECRET_OPTIONAL_PASS ""  
#define SECRET_SSID ""
```

sketch.json

```
{  
  "cpu": {  
    "fqbn": "arduino:renesas_uno:unor4wifi",  
    "name": "",  
    "type": "serial"  
  },  
  "secrets": [  
    {  
      "name": "SECRET_OPTIONAL_PASS",  
      "value": ""  
    },  
    {  
      "name": "SECRET_SSID",  
      "value": ""  
    }  
  ],  
  "included_libs": []  
}
```


9. Παράρτημα Β: «Πηγαίος κώδικας ESP32.»

Αρχείο: EBoxingGymSensors.ino

```
#include "BoxingApp.h"

BoxingApp boxingApp;

void setup() {
    boxingApp.setup();
}

void loop() {
    boxingApp.loop();
}
```

Αρχείο: BluetoothHandler.h

```
#ifndef BLUETOOTH_HANDLER_H
#define BLUETOOTH_HANDLER_H

#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
#include <vector>

#define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

class BluetoothHandler {
public:
    BluetoothHandler();
    void begin(const char* deviceName);
    void sendMessage(const String& message);
    String readMessage();
    void clearMessage();
    bool isDeviceConnected();

private:
    BLEServer* pServer;
    BLECharacteristic* pTxCharacteristic;
    BLECharacteristic* pRxCharacteristic;
    std::vector<uint16_t> connectedClients;
    String receivedMessage;

    class ServerCallbacks : public BLEServerCallbacks {
        BluetoothHandler* parent;
    public:
        ServerCallbacks(BluetoothHandler* parentInstance);
        void onConnect(BLEServer* pServer) override;
    };
};
```

```
void onDisconnect(BLEServer* pServer) override;
};

class RxCallbacks : public BLECharacteristicCallbacks {
    BluetoothHandler* parent;
public:
    RxCallbacks(BluetoothHandler* parentInstance);
    void onWrite(BLECharacteristic* pCharacteristic) override;
};

ServerCallbacks serverCallbacks;
RxCallbacks rxCallbacks;
void cleanDisconnectedClients();
};

#endif // BLUETOOTH_HANDLER_H
```

Αρχείο: BluetoothHandler.cpp

```
#include "BluetoothHandler.h"
#include "esp_gap_ble_api.h" // For security parameters if needed
#include <string.h>

BluetoothHandler::BluetoothHandler()
: pServer(nullptr), pTxCharacteristic(nullptr), pRxCharacteristic(nullptr),
  serverCallbacks(this), rxCallbacks(this) {}

void BluetoothHandler::begin(const char* deviceName) {
    // Initialize BLE with the given device name.
    BLEDevice::init(deviceName);

    // Set MTU to a value more compatible with Android devices.
    BLEDevice::setMTU(247);

    pServer = BLEDevice::createServer();
    pServer->setCallbacks(&serverCallbacks);

    BLEService* pService = pServer->createService(SERVICE_UUID);

    // Create TX characteristic with notify property.
    pTxCharacteristic = pService->createCharacteristic(
        CHARACTERISTIC_UUID_TX, BLECharacteristic::PROPERTY_NOTIFY);
    BLE2902* p2902 = new BLE2902();
    p2902->setAccessPermissions(ESP_GATT_PERM_READ | ESP_GATT_PERM_WRITE);
    pTxCharacteristic->addDescriptor(p2902);

    // Create RX characteristic with write and read properties.
    pRxCharacteristic = pService->createCharacteristic(
        CHARACTERISTIC_UUID_RX, BLECharacteristic::PROPERTY_WRITE |
        BLECharacteristic::PROPERTY_READ);
    pRxCharacteristic->setAccessPermissions(ESP_GATT_PERM_READ | ESP_GATT_PERM_WRITE);
    pRxCharacteristic->setCallbacks(&rxCallbacks);

    pService->start();
```

```
// Enable scan response to include the device name.
BLEAdvertising* pAdvertising = pServer->getAdvertising();
pAdvertising->setScanResponse(true);
pAdvertising->start();
Serial.println("Waiting for client connections...");
}

void BluetoothHandler::sendMessage(const String& message) {
    pTxCharacteristic->setValue(message.c_str());
    pTxCharacteristic->notify(); // Using notify without connection id.
    Serial.println("Sent Message: " + message); // Print the message that is sending in
terminal
}

String BluetoothHandler::readMessage() {
    return receivedMessage;
}

void BluetoothHandler::clearMessage() {
    receivedMessage = "";
}

bool BluetoothHandler::isDeviceConnected() {
    return !connectedClients.empty();
}

void BluetoothHandler::cleanDisconnectedClients() {
    connectedClients.erase(
        std::remove_if(connectedClients.begin(), connectedClients.end(),
            [&](uint16_t handle) {
                return pServer->getConnectedCount() == 0;
            }),
        connectedClients.end());
}

// ----- Server Callbacks -----

BluetoothHandler::ServerCallbacks::ServerCallbacks(BluetoothHandler* parentInstance)
    : parent(parentInstance) {}

void BluetoothHandler::ServerCallbacks::onConnect(BLEServer* pServer) {
    // You may not have direct access to a connection ID with this signature.
    // For now, we simply note a connection has been made.
    parent->connectedClients.push_back(1); // Dummy ID; adjust as needed.
    Serial.println("Device connected. Total connections: " +
String(parent->connectedClients.size()));
    parent->pTxCharacteristic->setValue("");
    parent->pTxCharacteristic->notify();
}

void BluetoothHandler::ServerCallbacks::onDisconnect(BLEServer* pServer) {
    parent->connectedClients.clear();
    delay(150);
}
```

```
Serial.println("Device disconnected. Remaining connections: " +  
String(parent->connectedClients.size()));  
pServer->startAdvertising();  
}  
  
// ----- RX Callbacks -----  
  
BluetoothHandler::RxCallbacks::RxCallbacks(BluetoothHandler* parentInstance)  
: parent(parentInstance) {}  
  
void BluetoothHandler::RxCallbacks::onWrite(BLECharacteristic* pCharacteristic) {  
parent->receivedMessage = pCharacteristic->getValue().c_str();  
parent->receivedMessage.trim();  
Serial.println("Received message: " + parent->receivedMessage);  
}
```

Αρχείο: BoxingApp.h

```
#ifndef BOXING_APP_H  
#define BOXING_APP_H  
  
#include <Arduino.h>  
#include "BluetoothHandler.h"  
#include "FSRPunchDetector.h"  
#include "TimeHandler.h"  
// #include "SleepHandler.h"  
  
class BoxingApp {  
private:  
    BluetoothHandler* bluetoothHandler;  
    FSRPunchDetector* fsrHandler;  
    TimeHandler* timeHandler;  
  
    // App configuration  
    bool startReading;  
    long roundTime;  
    long breakTime;  
    int fsrSensitivity;  
    int fsrThreshold;  
    bool isPaused;  
    unsigned long elapsedTime;  
    int command;  
    bool roundActive;  
  
    // added for debugging messages  
    String lastSentPunch; // Stores the last punch details to detect duplicates  
    int duplicatePunchCount; // Counts how many times the same punch was detected  
  
    void handleCommands();  
public:  
    BoxingApp();  
    void setup();
```

```
void loop();  
void sendPunchData(); // Sends punch data with duplicate prevention  
};  
  
#endif // BOXING_APP_H
```

Αρχείο: BoxingApp.cpp

```
#include "BoxingApp.h"  
#include <ArduinoJson.h>  
#include <BLEDevice.h>  
#include "MacDevicesConfig.h" // mac address store header  
  
String DEVICE_NAME = ""; // Global variable for device name  
  
BoxingApp::BoxingApp()  
: startReading(false),  
  roundTime(180000),  
  breakTime(60000),  
  fsrSensitivity(800),  
  fsrThreshold(200),  
  isPaused(false),  
  elapsedTime(0),  
  command(0),  
  roundActive(false),  
  lastSentPunch(""),  
  duplicatePunchCount(0) // Initialize duplicate counter  
{  
  bluetoothHandler = new BluetoothHandler();  
  // fsrHandler = new FSRPunchDetector(6, fsrSensitivity);  
  // new: watch pins 4, 5, and 6  
  fsrHandler = new FSRPunchDetector({ 4, 5, 6 },  
                                     fsrSensitivity,  
                                     fsrThreshold);  
  
  timeHandler = new TimeHandler();  
}  
  
void BoxingApp::setup() {  
  Serial.begin(115200);  
  
  // Initialize BLE and get BLE address  
  BLEDevice::init("");  
  String bleAddress = BLEDevice::getAddress().toString().c_str();  
  Serial.print("BLE Address: ");  
  Serial.println(bleAddress.c_str());  
  
  // Default device name; adjust as needed by checking your MAC lists  
  DEVICE_NAME = "UnknownDevice";  
  for (int i = 0; i < NUM_BLUEBOXER_MACS; i++) {  
    if (bleAddress.equalsIgnoreCase(BLUEBOXER_MACS[i])) {  
      DEVICE_NAME = "BlueBoxer";  
      break;  
    }  
  }  
}
```

```
}
if (DEVICE_NAME.equals("UnknownDevice")) {
    for (int i = 0; i < NUM_REDBOXER_MACS; i++) {
        if (bleAddress.equalsIgnoreCase(REDBOXER_MACS[i])) {
            DEVICE_NAME = "RedBoxer";
            break;
        }
    }
}

DEVICE_NAME.trim(); // Remove extra spaces
BLEDevice::deinit(); // Reset BLE stack
BLEDevice::init(DEVICE_NAME.c_str()); // Reinitialize with device name

bluetoothHandler->begin(DEVICE_NAME.c_str()); // Start BLE with your device name
fsrHandler->setup();
Serial.print(DEVICE_NAME);
Serial.print(" BLE Device is Ready ");
Serial.println("BoxingApp is ready... ");
Serial.println("Waiting for client connections...");
}

void BoxingApp::loop() {
    handleCommands();

    // If the round is active and not paused, check for punches
    if (roundActive && !isPaused) {
        if (fsrHandler->checkPunch()) {
            sendPunchData(); // Process and send punch data
        }

        // End the round when time elapses
        if (timeHandler->getElapsedMilliseconds() >= roundTime) {
            Serial.println("Round complete.");
            roundActive = false;
            bluetoothHandler->sendMessage("{\"RoundState\":\"Completed\"}");
            timeHandler->reset();
        }
    }
}

void BoxingApp::handleCommands() {
    String incomingMessage = bluetoothHandler->readMessage();
    if (incomingMessage.length() > 0) {

        StaticJsonDocument<256> jsonDoc;
        DeserializationError error = deserializeJson(jsonDoc, incomingMessage);
        if (error) {
            Serial.print("JSON Parsing Failed: ");
            Serial.println(error.c_str());
            return;
        }

        // Handle sensor settings update
```

```
if (jsonDoc.containsKey("SensorSettings")) {
    JsonObject settings = jsonDoc["SensorSettings"];
    fsrSensitivity = settings["FsrSensitivity"];
    fsrThreshold = settings["FsrThreshold"];
    roundTime = settings["RoundTime"];
    breakTime = settings["BreakTime"];
    fsrHandler->setSensitivity(fsrSensitivity);
    fsrHandler->setThreshold(fsrThreshold);
    // Serial.println("Sensor settings updated.");
    // Serial.println(fsrHandler->getSensitivity());
    // Serial.println(fsrHandler->getThreshold());
    bluetoothHandler->sendMessage("{\"RoundState\":\"Settings Updated\"}");
    bluetoothHandler->clearMessage();
}

// Handle round commands
if (jsonDoc.containsKey("RoundStatusCommand")) {
    int commandValue = jsonDoc["RoundStatusCommand"]["Command"];
    unsigned long elapsedSeconds = timeHandler->getElapsedSeconds();

    switch (commandValue) {
        case 1: // Start round
            Serial.println("Starting the round at " + String(elapsedSeconds) + "s...");
            fsrHandler->resetPunchCount();
            timeHandler->reset();
            timeHandler->start();
            roundActive = true;
            isPaused = false;
            bluetoothHandler->sendMessage("{\"RoundState\":\"Started\", \"Time\":\"\" +
String(elapsedSeconds) + "s\"}");
            bluetoothHandler->clearMessage();
            break;
        case 2: // Pause round
            Serial.println("Pausing the round at " + String(elapsedSeconds) + "s...");
            timeHandler->pause();
            isPaused = true;
            bluetoothHandler->sendMessage("{\"RoundState\":\"Paused\", \"Time\":\"\" +
String(elapsedSeconds) + "s\"}");
            bluetoothHandler->clearMessage();
            break;
        case 3: // Resume round
            Serial.println("Resuming the round at " + String(elapsedSeconds) + "s...");
            timeHandler->resume();
            isPaused = false;
            bluetoothHandler->sendMessage("{\"RoundState\":\"Resumed\", \"Time\":\"\" +
String(elapsedSeconds) + "s\"}");
            bluetoothHandler->clearMessage();
            break;
        case 4: // Reset round
            Serial.println("Resetting the round at " + String(elapsedSeconds) + "s...");
            fsrHandler->resetPunchCount();
            timeHandler->reset();
            timeHandler->start();
            roundActive = true;
            isPaused = false;
```



```

        bluetoothHandler->sendMessage("{\"RoundState\":\"Reset\",\"Time\":\"0s\"}");
        bluetoothHandler->clearMessage();
        break;
    case 5: // End round
        Serial.println("Ending the round at " + String(elapsedSeconds) + "s...");
        roundActive = false;
        bluetoothHandler->sendMessage("{\"RoundState\":\"Ended\",\"FinalTime\":\"" +
String(elapsedSeconds) + "s\"}");
        timeHandler->reset();
        fsrHandler->resetPunchCount();
        bluetoothHandler->clearMessage();
        break;
    default:
        Serial.println("Unknown Command Received.");
        bluetoothHandler->sendMessage("{\"Error\":\"Unknown Command\"}");
        bluetoothHandler->clearMessage();
        break;
    }
}
}
}

void BoxingApp::sendPunchData() {
    unsigned long elapsedMilliseconds = timeHandler->getElapsedMilliseconds();

    if (fsrHandler->getFsrValue() > fsrHandler->getThreshold()) {
        fsrHandler->increasePunch();
        String punchDetails = fsrHandler->getPunchDetails(elapsedMilliseconds);
        //Serial.println(punchDetails); // print the message in terminal <-----
        -----
        bluetoothHandler->sendMessage(punchDetails);
    }
}
}

```

Αρχείο: FSRPunchDetector.h

```

// FSRPunchDetector.h
#ifndef FSR_PUNCH_DETECTOR_H
#define FSR_PUNCH_DETECTOR_H

#include <Arduino.h>
#include <vector>
#include <initializer_list>

class FSRPunchDetector {
private:
    std::vector<int> fsrPins;
    int fsrSensitivity;
    int fsrThreshold;
    bool isPressed;
    int punchCount;
    int punchPower;
    float sensorVoltage;
    int fsrValue;

```

```
    unsigned long lastPunchTime;

public:
    // Now takes a list of pins + sensitivity + threshold
    FSRPunchDetector(const std::initializer_list<int>& pins,
                     int sensitivity,
                     int threshold);

    void setup();
    bool checkPunch();

    float getSensorVoltage();
    int getFsrValue();

    void setSensitivity(int value);
    int getSensitivity();
    void setThreshold(int value);
    int getThreshold();

    void increasePunch();
    int getPunchCount();
    void resetPunchCount();
    String getPunchDetails(unsigned long elapsedMilliseconds);

    // Internals for result formatting
    void calculateResults(int fsrSensorValue,
                         float R_FSR,
                         unsigned long punchTimestamp,
                         String &outputMessage);
};

#endif // FSR_PUNCH_DETECTOR_H
```

Αρχείο: FSRPunchDetector.cpp

```
// FSRPunchDetector.cpp
#include "FSRPunchDetector.h"
#include <Arduino.h>

// Declare that DEVICE_NAME is defined in another file (boxingApp.cpp)
extern String DEVICE_NAME;

FSRPunchDetector::FSRPunchDetector(const std::initializer_list<int>& pins,
                                     int sensitivity,
                                     int threshold)
```

```
: fsrPins(pins.begin(), pins.end()),
  fsrSensitivity(sensitivity),
  fsrThreshold(threshold),
  isPressed(false),
  punchCount(0),
  punchPower(0),
  sensorVoltage(0.0f),
  fsrValue(0),
  lastPunchTime(0UL) {}

void FSRPunchDetector::setup() {
  for (auto pin : fsrPins) {
    pinMode(pin, INPUT);
  }
  Serial.println("FSR Punch Detector Initialized.");
}

bool FSRPunchDetector::checkPunch() {
  const unsigned long debounceDelay = 200; // ms between punches
  unsigned long now = millis();

  // Read all sensors and track the maximum reading
  int maxReading = 0;
  for (auto pin : fsrPins) {
    int mv = analogReadMilliVolts(pin);
    if (mv > maxReading) {
      maxReading = mv;
    }
  }
  fsrValue = maxReading;

  // If we detect a new hit above sensitivity
  if (maxReading > fsrSensitivity && !isPressed) {
    if (now - lastPunchTime > debounceDelay) {
      isPressed = true;
      lastPunchTime = now;
      return true;
    }
  }
  // Reset when reading drops below threshold
  else if (maxReading < fsrThreshold && isPressed) {
    isPressed = false;
  }

  return false;
}

String FSRPunchDetector::getPunchDetails(unsigned long elapsedMilliseconds) {
  String details = "";

  sensorVoltage = fsrValue / 1000.0;
  float R_FSR = 0.0; // Optional: calculate resistance if needed

  calculateResults(fsrValue, R_FSR, elapsedMilliseconds, details);
  details = "Punch Count: " + String(punchCount) + " " + details;
}
```

```
    return details;
}

void FSRPunchDetector::calculateResults(int fsrSensorValue,
                                         float R_FSR,
                                         unsigned long punchTimestamp,
                                         String& outputMessage) {
    unsigned long minutes = punchTimestamp / 60000;
    unsigned long seconds = (punchTimestamp % 60000) / 1000;
    unsigned long hundredths = (punchTimestamp % 1000) / 10;

    char timestampFormatted[12];
    sprintf(timestampFormatted, "%02lu:%02lu:%02lu", minutes, seconds, hundredths);

    outputMessage += "Timestamp: " + String(timestampFormatted)
                   + " Device: " + DEVICE_NAME
                   + " | Sensor millivolts: " + String(fsrSensorValue);
}

float FSRPunchDetector::getSensorVoltage() {
    return sensorVoltage;
}

void FSRPunchDetector::setSensitivity(int value) {
    fsrSensitivity = value;
}

void FSRPunchDetector::setThreshold(int value) {
    fsrThreshold = value;
}

int FSRPunchDetector::getFsrValue() {
    return fsrValue;
}

int FSRPunchDetector::getSensitivity() {
    return fsrSensitivity;
}

int FSRPunchDetector::getThreshold() {
    return fsrThreshold;
}

void FSRPunchDetector::increasePunch() {
    punchCount++;
}

int FSRPunchDetector::getPunchCount() {
    return punchCount;
}

void FSRPunchDetector::resetPunchCount() {
    punchCount = 0;
}
```

Αρχείο: TimeHandler.h

```
#ifndef TIME_HANDLER_H
#define TIME_HANDLER_H

#include <Arduino.h>

class TimeHandler {
private:
    unsigned long startCountTime;
    unsigned long elapsedTime;
    bool isRunning;

    unsigned long pausedTime;
    unsigned long resumedTime;

public:
    TimeHandler();
    void start();
    void pause();
    void reset();
    void resume();
    void restart(); // New method to reset and start the timer

    unsigned long getElapsedMilliseconds();
    unsigned long getElapsedSeconds();
    unsigned long getPausedTime();
    unsigned long getResumedTime();
    unsigned long getTimeSinceStart(); // New method to get time since start
};

#endif // TIME_HANDLER_H
```

Αρχείο: TimeHandler.cpp

```
#include "TimeHandler.h"

// Constructor
TimeHandler::TimeHandler()
    : startCountTime(0), elapsedTime(0), isRunning(false), pausedTime(0), resumedTime(0) {}

// Start the timer (or resume from where it was paused)
void TimeHandler::start() {
    if (!isRunning) {
        startCountTime = millis() - elapsedTime; // Adjust to resume if paused
        isRunning = true;
    }
}
```

```
}

// Pause the timer
void TimeHandler::pause() {
    if (isRunning) {
        elapsedTime = millis() - startCountTime; // Record the elapsed time
        pausedTime = elapsedTime / 1000;         // Record paused time in seconds
        isRunning = false;
    }
}

// Resume the timer
void TimeHandler::resume() {
    if (!isRunning) {
        startCountTime = millis() - elapsedTime; // Adjust to resume from where paused
        resumedTime = elapsedTime / 1000;        // Record resumed time in seconds
        isRunning = true;
    }
}

// Reset the timer (elapsed time set to 0, timer stopped)
void TimeHandler::reset() {
    startCountTime = 0;
    elapsedTime = 0;
    pausedTime = 0;
    resumedTime = 0;
    isRunning = false;
}

// Restart the timer (reset and start from 0)
void TimeHandler::restart() {
    reset(); // Reset all values
    startCountTime = millis(); // Start counting from current time
    isRunning = true;
}

// Get elapsed time in milliseconds
unsigned long TimeHandler::getElapsedMilliseconds() {
    if (isRunning) {
        return millis() - startCountTime; // Time since the timer started
    } else {
        return elapsedTime; // Return paused elapsed time
    }
}

// Get elapsed time in seconds
unsigned long TimeHandler::getElapsedSeconds() {
    return getElapsedMilliseconds() / 1000;
}

// Get time when paused
unsigned long TimeHandler::getPausedTime() {
    return pausedTime;
}
```

```
// Get time when resumed
unsigned long TimeHandler::getResumedTime() {
    return resumedTime;
}

// Get time since the timer was started
unsigned long TimeHandler::getTimeSinceStart() {
    return getElapsedMilliseconds(); // Same as elapsed time when running
}
```

Αρχείο: MacDevicesConfig.h

```
#ifndef MAC_DEVICES_CONFIG_H
#define MAC_DEVICES_CONFIG_H

#include <Arduino.h>

// Array of MAC addresses for BlueBoxer
static const String BLUEBOXER_MACS[] = {
    "f0:f5:bd:2c:10:72",
    "f0:f5:bd:2c:1a:32",
    "f0:f5:bd:2c:0b:ee",
    "f0:f5:bd:2c:16:3a",
    "54:32:04:3f:e8:8e"
};

static const int NUM_BLUEBOXER_MACS = sizeof(BLUEBOXER_MACS) / sizeof(BLUEBOXER_MACS[0]);

// Array of MAC addresses for RedBoxer
static const String REDBOXER_MACS[] = {
    "f0:f5:bd:2c:15:1a",
    "f0:f5:bd:2c:11:e6",
    "54:32:04:3f:da:ee"
};

static const int NUM_REDBOXER_MACS = sizeof(REDBOXER_MACS) / sizeof(REDBOXER_MACS[0]);

#endif
```


10. Παράρτημα Γ: «Πηγαίος κώδικας Android.»

Επειδή ο πηγαίος κώδικας της εφαρμογής Android είναι πολύ μεγάλος και πιάνει πάρα πολλές σελίδες θα παραθέσω εδώ το αποθετήριο που υπάρχει ο κώδικας στο internet στο GitHub.

Το Link είναι το εξής για όλο project:

<https://github.com/sanguinarypc/BoxingGymSensorsProject>

Σε αυτό το Link βρίσκεται ο κώδικας της Android εφαρμογής:

https://github.com/sanguinarypc/BoxingGymSensorsProject/tree/main/box_sensors

Υπεύθυνη Δήλωση Συγγραφέα:

Δηλώνω ρητά ότι, σύμφωνα με το άρθρο 8 του Ν.1599/1986, η παρούσα εργασία αποτελεί αποκλειστικά προϊόν προσωπικής μου εργασίας, δεν προσβάλλει κάθε μορφής δικαιώματα διανοητικής ιδιοκτησίας, προσωπικότητας και προσωπικών δεδομένων τρίτων, δεν περιέχει έργα/εισφορές τρίτων για τα οποία απαιτείται άδεια των δημιουργών/δικαιούχων και δεν είναι προϊόν μερικής ή ολικής αντιγραφής, οι πηγές δε που χρησιμοποιήθηκαν περιορίζονται στις βιβλιογραφικές αναφορές και μόνον και πληρούν τους κανόνες της επιστημονικής παράθεσης.